



DISEÑO UML

Grupo WHO

Alba Bautista Núñez
Natalia Rodríguez Caballero
Beñat Pérez de Arenaza Eizaguirre
Javier Amado Lázaro
Mario Calvarro Marines
Lucía Alonso Mozo

ÍNDICE

Arquitectura	5
Patrón MVC	6
Vista cliente-servidor	10
Vista despliegue	12
Interfaces	13
Controller	13
PlayerStrategy	13
Factory	13
ObserverUNO y ObservableUNO	13
Diseño y evolución de la interfaz del usuario	15
Sprint 1	16
Sprint 2	17
Sprint 3	18
Sprint 4	19
Sprint 5	28
Sprint 6	31
Diseño y evolución de las clases principales del modelo	40
Diseño y evolución de las cartas	40
Sprint 1	40
Sprint 2	43
Sprint 3	44
Sprint 4	45
Sprint 5	46
Sprint 7	47
Diseño y evolución de las listas de cartas:	48
Sprint 1	48
Sprint 2	49
Sprint 3	49
Sprint 5	50
Diseño y evolución de los jugadores	51
Sprint 1	51
Sprint 2	52
Sprint 3	53
Sprint 4	54
Sprint 5	55
Sprint 7	56
Diseño y evolución de las estrategias	57
Sprint 5	57
Diseño y evolución del Game	59
Sprint 1	59

Sprint 2	60
Sprint 3	62
Sprint 4	63
Sprint 5	65
Sprint 6	67
Sprint 7	68
Diseño y evolución del Controller	70
Sprint 1	70
Sprint 3	70
Sprint 4	71
Sprint 6	72
Diseño y evolución de los comandos	75
Sprint 1	75
Sprint 2	77
Sprint 3	78
Sprint 4	78
Sprint 6	79
Diseño y evolución de las historias de usuario	80
Como usuario quiero jugar al UNO	81
Sprint 1	82
Sprint 2	83
Sprint 4	86
Sprint 5	88
Sprint 7	90
Cómo usuario quiero poder elegir el número de jugadores a participar.	94
Sprint 1	95
Sprint 2	96
Como usuario quiero jugar con las cartas.	97
3.1 Cartas Normales:	97
3.2 Cartas Especiales:	97
Sprint 1	98
Sprint 2	99
Sprint 3	101
Sprint 5	102
Como usuario quiero que las cartas se comporten adecuadamente.	104
Sprint 1	105
Sprint 2	106
Sprint 4	108
Sprint 7	111
Como usuario quiero conocer los nombres de los otros jugadores de la partida.	112
Sprint 2	112
Sprint 5	115

Como usuario quiero poder guardar una partida en curso.	116
Sprint 2	117
Sprint 3	118
Como usuario quiero poder cargar una partida que he guardado con anterioridad.	121
Sprint 2	122
Sprint 3	123
Sprint 4	126
Como usuario quiero poder introducir mediante teclado un comando y que se ejecute en la partida. (ELIMINADA)	129
Sprint 3	129
Sprint 5	131
Como usuario quiero poder jugar contra la máquina en diferentes dificultades.	132
Fácil	132
Medio	132
Difícil	132
Sprint 4	133
Sprint 5	134
Sprint 6	143
Sprint 7	144
Como usuario quiero poder jugar una partida en red.	147
Sprint 6	147
Sprint 7	153
Diagrama de componentes	155
Patrones de diseño utilizados en el proyecto	156
Patrón MVC	156
Aplicación del patrón MVC en nuestro proyecto	156
Patrón Observer	158
Aplicación del patrón Observador en nuestro proyecto	158
Patrón Factory Method	159
Aplicación del patrón Factory Method en nuestro proyecto	159
Patrón Strategy	160
Aplicación del patrón Strategy en nuestro proyecto	160
Patrón Command	161
Aplicación del patrón Command en nuestro proyecto	161
Patrón Singleton	162
Aplicación del patrón Singleton en nuestro proyecto	162
Patrón Adapter	163
Aplicación del patrón Adapter en nuestro proyecto	163
Patrón Polimorfismo	165
Aplicación del patrón Polimorfismo en nuestro proyecto	165
Patrón Bajo Acoplamiento	167

Patrón de Alta Cohesión	167
Sistema resultante	168
Diagrama de componentes	155

Arquitectura

Para desarrollar el software de nuestro proyecto, hemos establecido una arquitectura que consiste en un esquema de organización estructural esencial, formado por subsistemas e interrelaciones entre ellos, para así poder trabajar con atributos de calidad del software como la testabilidad, modificabilidad, usabilidad, seguridad y correctitud.

Hemos centrado nuestros esfuerzos en el desarrollo del patrón Modelo-Vista-Controlador, que separa los datos de lo que es la lógica de negocio de una aplicación de su representación y el módulo encargado de gestionar los eventos y las comunicaciones. Sus ideas principales son la reutilización de código y separación de conceptos, para facilitar el desarrollo de aplicaciones y su posterior mantenimiento. Así como en la conexión en red del juego mediante el patrón cliente-servidor.

Presentamos en este documento todos los aspectos relacionados con la arquitectura del proyecto, Hardware y Software, acompañados de diagramas de clases y secuencias, que irán progresando respecto a un total de 7 sprints, cada uno de dos semanas.

Luego lo hemos dividido en módulos y paquetes según la naturaleza del código. También hemos realizado la conexión de componentes mediante el patrón cliente-servidor. Y representado en un diagrama de despliegue la correlación del Hardware y el software mediante nodos.

Patrón MVC

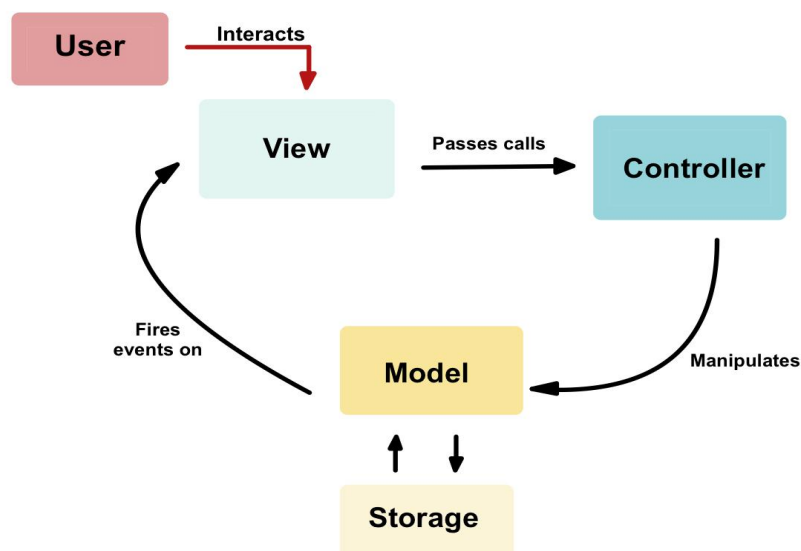
El MVC es un patrón de arquitectura para aplicaciones de software, que busca la reutilización el diseño del código, compuesto por: el modelo (parte lógica del juego), que en nuestro caso se presenta en la clase *Game*, las cartas, los jugadores y las estrategias; la vista (la salida de información para interactuar con el usuario), compuesta por la interfaz gráfica de usuario y la consola; y por último, el controlador (la entrada de información tras la interacción con el usuario), formado por *Controller*, *ControllerLocal* y *ControllerNet*.

La idea principal en que se basa este patrón es que el modelo es independiente de la vista y el controlador, así se podría cambiar el tipo de controlador o de vista sin tener que modificar nada del modelo. Para las conexiones entre las clases, debemos saber que la vista notifica al modelo a través del controlador, y el juego notifica a la vista para realizar cambios en ella.

El modelo presenta una interfaz denominada *ObserverUNO*, que consiste en una interfaz de observador que permitirá al modelo notificar cambios a las vistas o, en el caso de la conexión en red, al servidor. Durante la ejecución del juego, siguen un ciclo:

1. Las vistas ven el controlador y le avisan de la acción realizada por el usuario.
2. El controlador recibe las peticiones de la vista y responde a ellas modificando el modelo.
3. El modelo, después de hacer los cambios, se los notifica a los observadores (que son las vistas o el servidor).
4. Se visualizan los nuevos datos a través de las vistas.

Además, para la lógica del juego de poder guardar y cargar partida, creamos un paquete denominado *storage* que tendrá una conexión en ambos sentidos con el modelo.



Esquema del patrón MVC + Storage

Vista de uso

En este apartado se muestran los diagramas de clases del código del juego y sus correspondientes paquetes, permitiéndonos observar así las conexiones entre ellas.

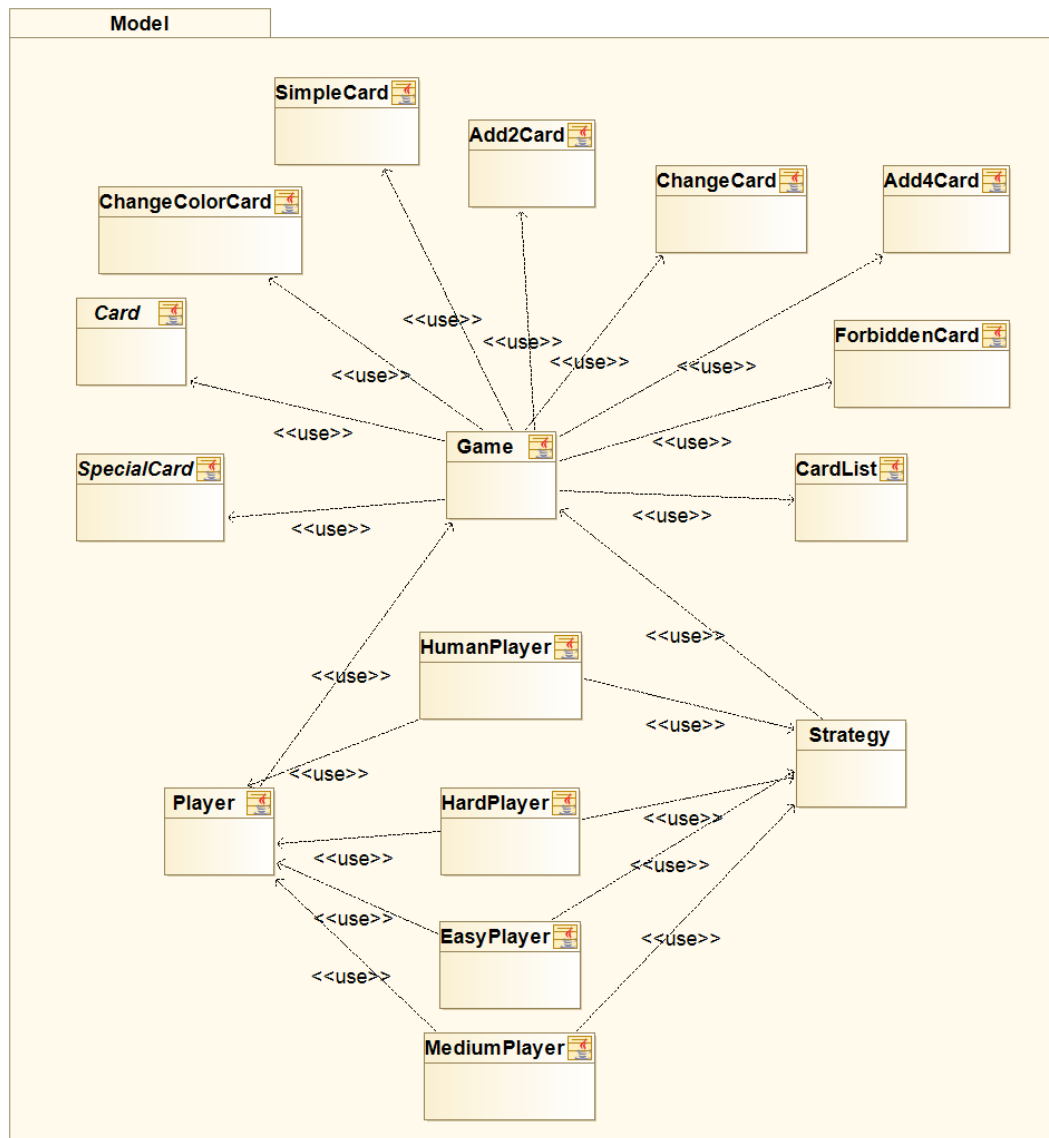
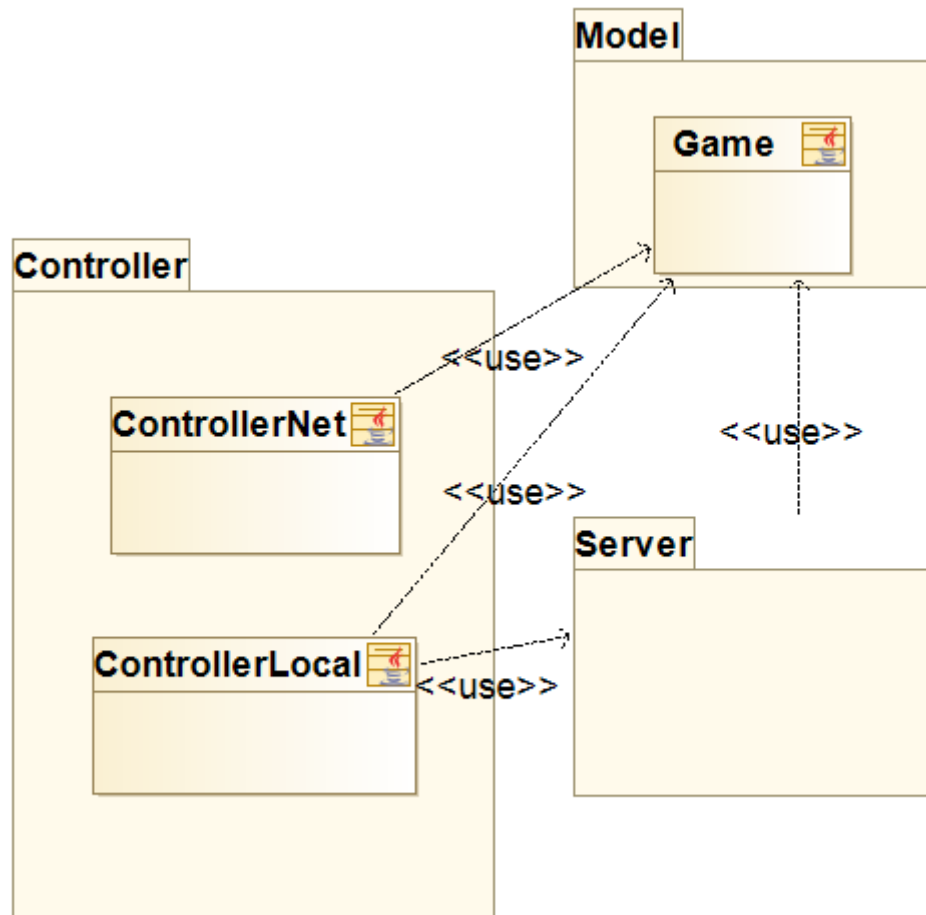
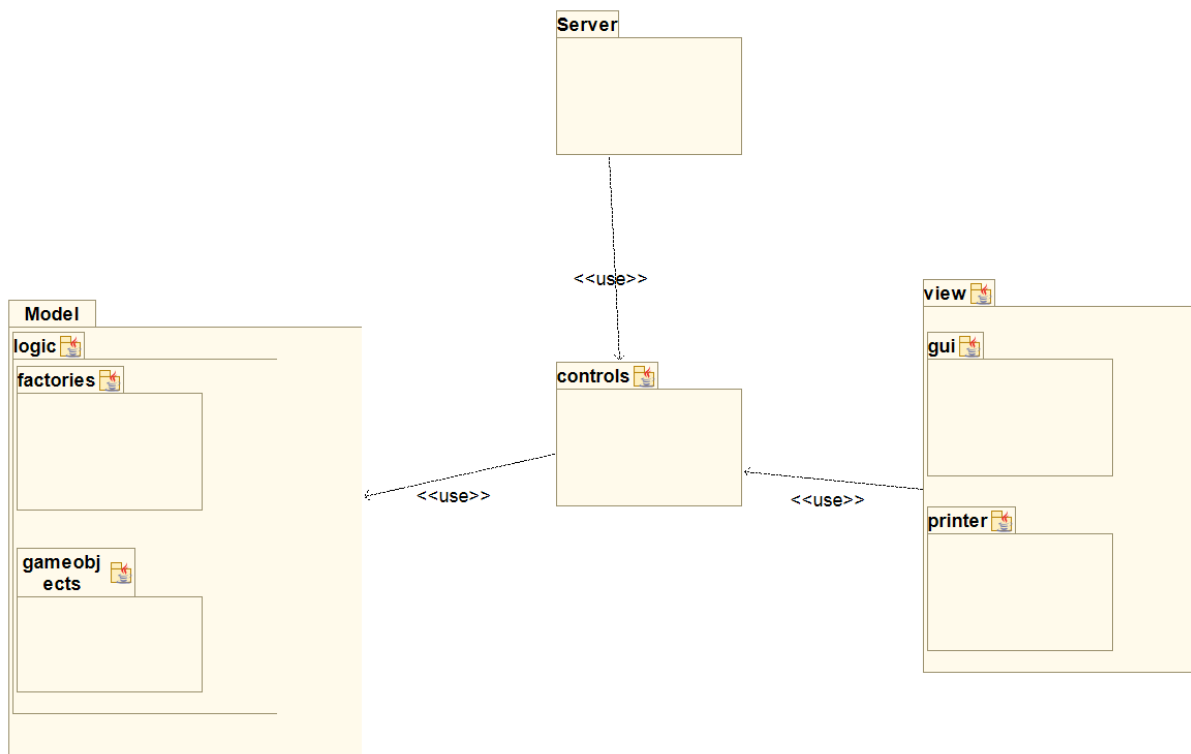


Diagrama de clases del modelo.



Conexión Cliente-Servidor.



Relación entre paquetes principales.

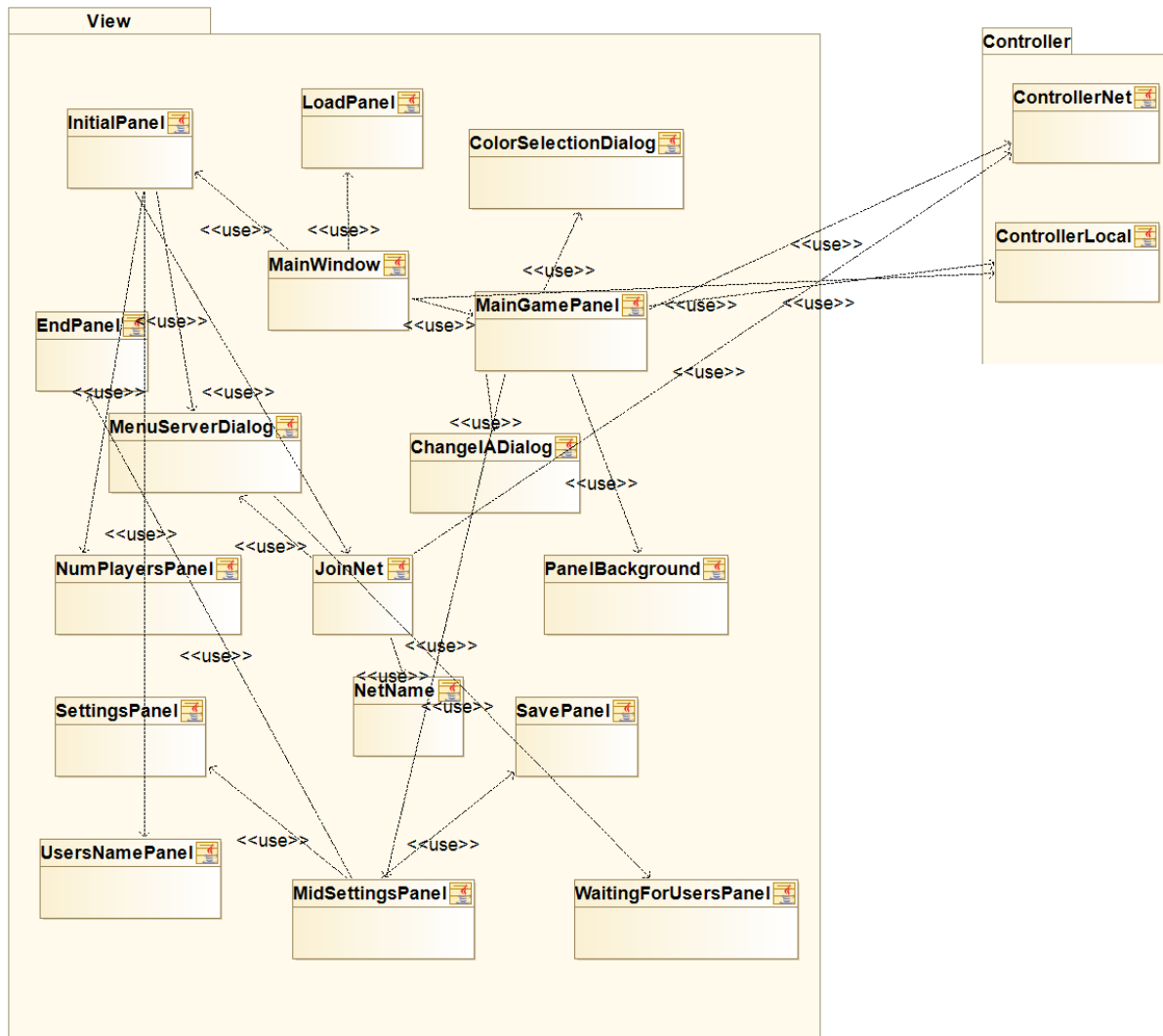
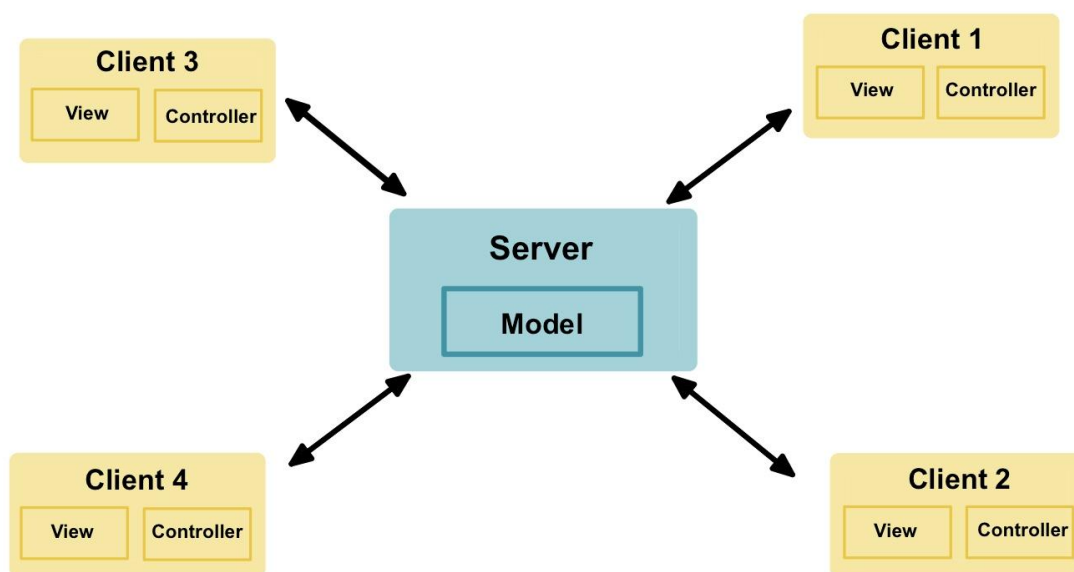


Diagrama de las clases de la vista.

Vista cliente-servidor

Para poder desarrollar la funcionalidad del juego en red, tuvimos que llevar a cabo la realización de las clases y conexiones entre los clientes y el servidor, basándonos en el patrón Cliente-Servidor.

Lo realizamos de forma que el servidor sea la clase que posea el modelo, y los clientes solo dispongan de la información que el servidor mande, así como su propia vista y su propio controlador, permitiendo que cada usuario juegue en un dispositivo, o se muestre la salida/entrada del juego de una forma u otra.

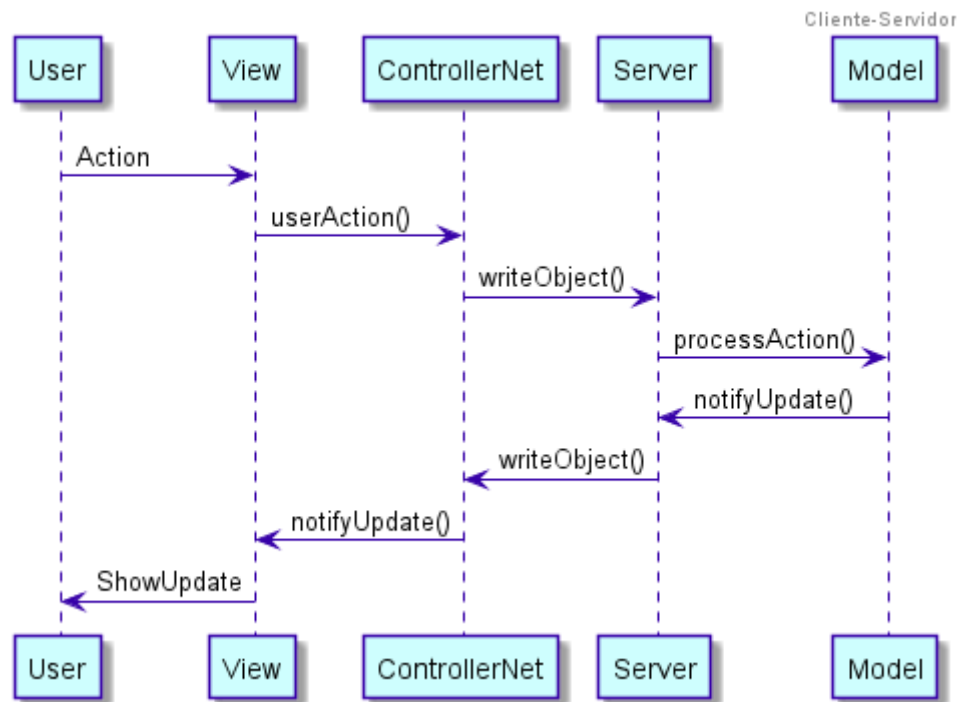


Diseño centralizado del cliente-servidor.

El proceso que sigue el programa consiste en lo siguiente:

1. El modelo notifica a sus observadores, es decir, al servidor, de que ha ocurrido un cambio.
2. El servidor recibe esta notificación y, a su vez, “notifica” a sus clientes. Esto quiere decir que el servidor envía a los clientes un tipo u otro de información dependiendo de qué notificación haya ejecutado el modelo.
3. Los clientes reciben la información del servidor. Gracias a que también se envía a los clientes el tipo de notificación que ha ocasionado este envío, los clientes notifican a su vez a una lista de observadores que poseen. Generalmente, estos observadores serán la vista del cliente que se actualizará respecto a los cambios ocasionados en el modelo.
4. Una vez que los clientes hayan actualizado sus vistas, el cliente con el jugador actual permitirá a su usuario introducir una acción que el cliente enviará al servidor.

5. El servidor recibirá dicha información que será interpretada por su propio controlador que actuará sobre el modelo produciendo un cambio y generando una notificación.



Cabe destacar que el envío de información entre el servidor y los clientes se realiza a través de los sockets en forma de objetos serializables. Estos son simples cadenas de texto o, en casos más complejos, **adapters de otros objetos**.

La lógica encargada del manejo del servidor se encuentra en el paquete main y en la clase Server.

Vista despliegue

Los diagramas de despliegue se utilizan para visualizar los procesadores/nodos/dispositivos de hardware de un sistema, los enlaces de comunicación entre ellos y la colocación de los archivos de software en ese hardware.

Cabe destacar la distinción entre dos nodos:

- **Nodo Cliente:** Tiene su propia vista (GUI), y se conecta al Cliente/Servidor, el host de la partida, que guarda el juego.
- **Nodo Servidor:** Actúa también como cliente. Se encarga de la lógica del juego, y de actualizar el juego tras cada acción de los jugadores.

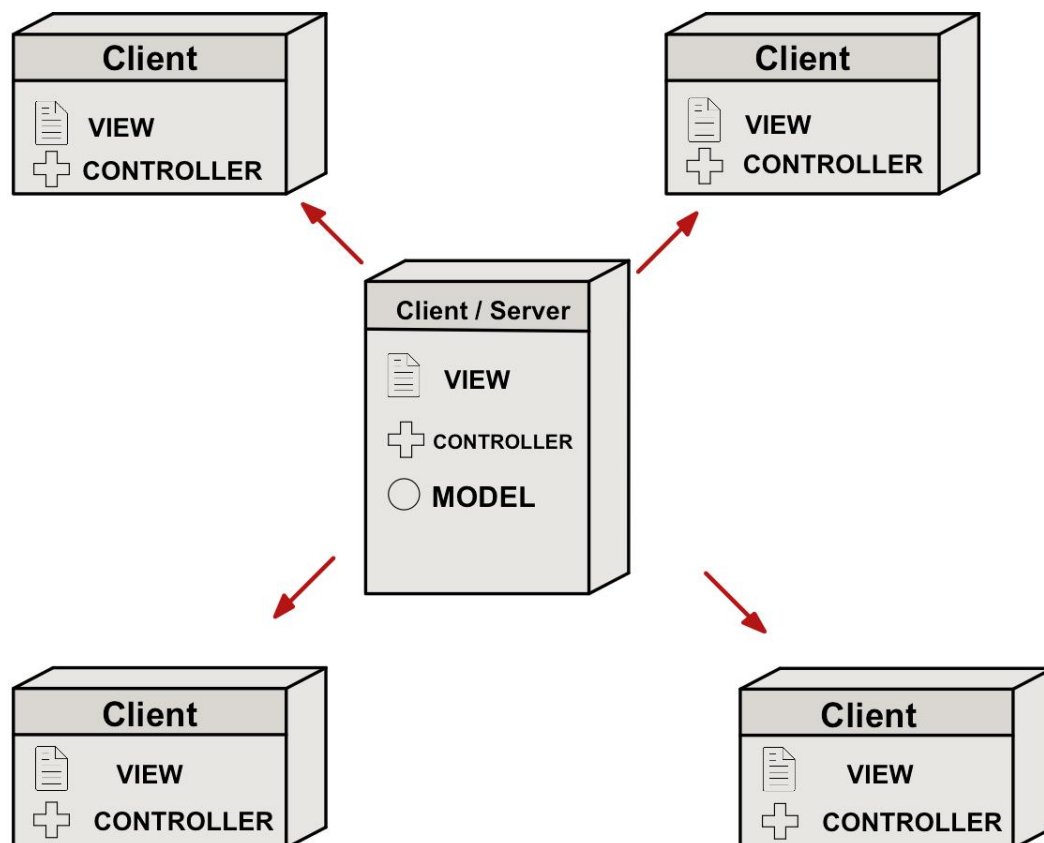


Diagrama de despliegue.

Interfaces

Controller

Esta interfaz nos permite seleccionar el tipo de controlador que se desea usar en la partida. Tenemos dos opciones, un controlador de uso local (*ControllerLocal*) y otro que utilizará conexión en red (*ControllerNet*).

Ambos controladores tienen la misma función, la entrada de datos por el usuario y su gestión: el *ControllerLocal* recibe el comando y lo ejecuta directamente a través del modelo; por otro lado, el *ControllerNet* actúa como cliente en la relación Cliente-Servidor y, por ello, presenta métodos como *connectToServer()* para conectarse al servidor y *receiveData()* para recibir los cambios que han realizado los otros clientes y notificar a su propia vista, a través de la lista de observadores.

PlayerStrategy

Se trata de una interfaz muy sencilla, con un único método al que se le pasa el modelo como parámetro. Es implementada por los cuatro tipos de jugadores: *human*, *easy*, *medium* o *hard*. En el caso de que sea un bot, este método se encarga de gestionar qué carta debe lanzar, dependiendo de su dificultad.

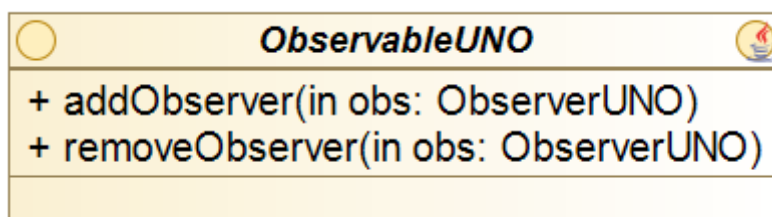
Factory

Genera todos los elementos del juego mediante el patrón de diseño Factoría. Es implementada por *BuilderBasedFactory*, que posee un array de builders, el cual recorrerá para que dado un *JSONObject*, se devuelva la instancia del objeto.

ObserverUNO y *ObservableUNO*

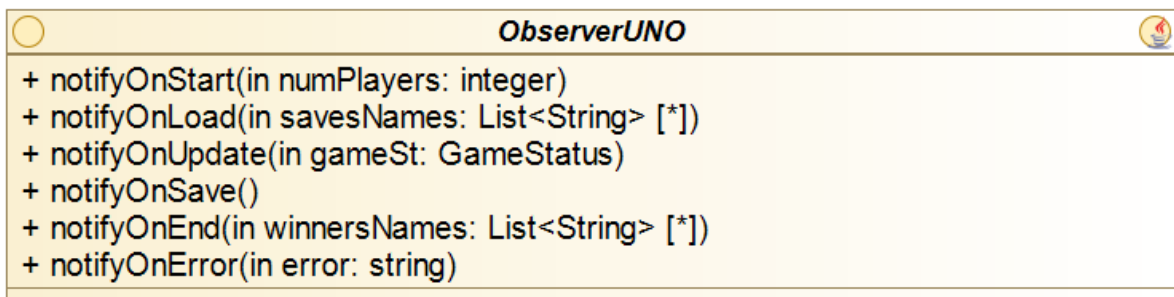
Nos permite diferenciar entre la lógica del programa y la visualización de los datos. Ambas interfaces se complementan para conseguir construir el patrón observador.

La interfaz *Observable* es implementada por *Game*, presenta métodos para añadir y eliminar *ObserversUNO* de la lista de observables. La clase que la implementa es aquella que controla y llama a los observadores con los datos que estos necesitan.



La interfaz `ObserverUNO` presenta todos los métodos que hemos considerado necesarios para notificar a la vista de cambios en el modelo. Veamos el objetivo de cada uno:

- `notifyOnStart()`: se encarga de notificar el comienzo del juego, entre otros, indicar el número de jugadores en la partida.
- `notifyOnLoad()`: envía los nombres de las partidas anteriormente guardadas.
- `notifyOnUpdate()`: se trata del método, por excelencia, que notifica a la vista. Tras cualquier mínimo cambio, envía a la vista un adapter del juego.
- `notifyOnSave()`: notifica a la vista de que el juego se ha guardado.
- `notifyOnEnd()`: su función es enviar a la vista el nombre de los tres jugadores mejor clasificados de la partida. Notifica el final de la partida.
- `notifyOnError()`: su objetivo es notificar a la vista de que ha habido un error.



CardStatus

La clase *Card* implementa esta interfaz. Se trata de un *adapter* para impedir que otras clases puedan modificar la carta, permitiéndoles sólo la lectura de esta.

PlayerStatus

La clase *Player* implementa esta interfaz. Su objetivo es también actuar como *adapter* pero para los jugadores.

GameStatus

La clase *Game* implementa esta interfaz. Sirve de *adapter* para el juego, y principalmente para que la vista sólo pueda visualizar el juego, pero no cambiarlo.

Diseño y evolución de la interfaz del usuario

La vista de nuestro proyecto ha ido evolucionando y mejorando tras el paso de los sprints, a medida que nuestros conocimientos sobre ello aumentaban.

Durante los tres primeros sprints el juego tan solo se mostraba a través de la consola, de forma eficaz, simple, básica y sin atractivo visual. El usuario debía introducir los comandos y acciones a realizar por teclado, resultando en muchas ocasiones, poco intuitivo. Por todo ello, en el sprint 4, el equipo se puso manos a la obra en el desarrollo y creación de una interfaz gráfica funcional para la mejora del diseño del juego, así como la complejidad de interpretarlo y jugarlo.

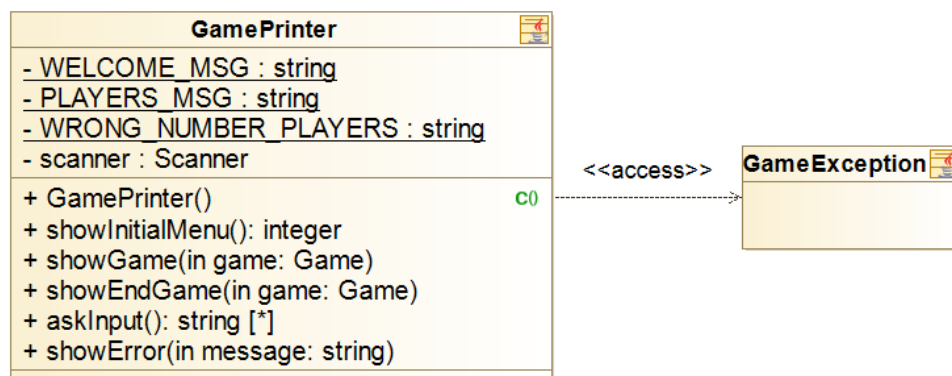
La vista está compuesta por un paquete denominado *view*, formado por dos subpaquetes: *GUI* (Graphical User Interface), que desarrolla e implementa la lógica de la vista por interfaz gráfica; y *printer*, que se encarga de la salida por consola.

Cabe destacar que independientemente de la forma escogida para la visualización del juego, presenta una estructura fija.

1. Se pregunta al usuario si desea crear una nueva partida, unirse a una existente por servidor o continuar con una partida ya guardada.
2. El usuario introduce o selecciona el número de jugadores.
3. Se selecciona el tipo de cada jugador, humano o inteligencia artificial, y un nombre o *username* para cada uno.
4. La partida comienza mostrando las cartas e indicando el turno de cada jugador.
5. Cuando un jugador se queda sin cartas se indica el jugador ganador o el podium de los 3 mejores clasificados.

Sprint 1

Esta fue la primera vista del juego el cual se muestra por consola. Cada carta aparece con su correspondiente símbolo y color, este último gracias a una serie de códigos de escape ANSI. Las cartas del cambio de color como *Add4Card* y *ChangeColorCard* permanecen siempre de color negro. Poseemos la clase *GamePrinter*, la cual se encarga de mostrar los datos y cartas, así como de preguntar al usuario el número de jugadores, o que comando desea realizar.



Durante este Sprint nos encontramos con los siguientes fallos que solucionamos en el siguiente Sprint.

```

Welcome to UNO
How many players are going to play?:
4

PLAYER 1: 8 9 6 +4 +2 +2 6
Todavía se pueden sacar 84 cartas.
Acción > l 8 red
8
PLAYER 2: 3 1 5 0 1 5 3
Todavía se pueden sacar 84 cartas.
Acción >
  
```

Como podemos observar, se mezcla el idioma, posteriormente decidimos que se mostraría todo en inglés. Los identificadores de los jugadores son "Player - número". La primera carta en ser lanzada puede ser cualquiera, la elige el primer jugador.

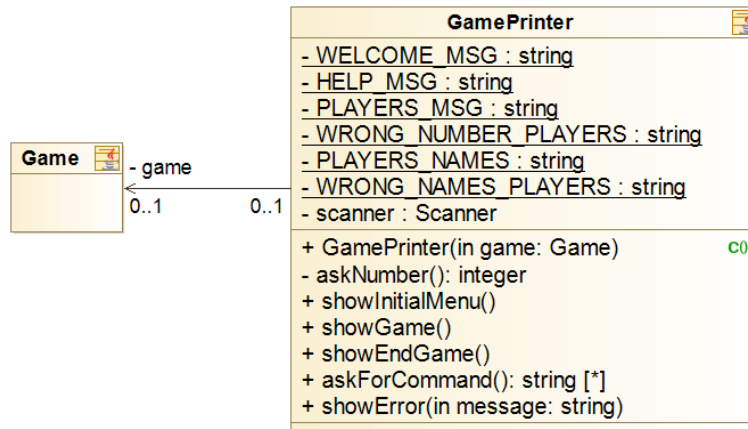
En este ejemplo, el jugador 3 ha seleccionado el +4, con cambio de color al azul. La carta se lanza, el juego interpreta que el color actual es el azul, pero no cambia su color.

```

PLAYER 3: +4 4 3 1 4 7 9
Todavía se pueden sacar 84 cartas.
Acción > l +4 blue
+4
  
```

Sprint 2

El juego continúa mostrándose por consola.



```

How many players are going to play?:
4
Introduce the names of the players:
Juan
Pablo
Maria
Laura
    
```

```

JUAN: 5 8 -> 9 9 -> 0
You can still draw 84 cards.
Action > |
    
```

Al principio de la partida, se pregunta a los usuarios sus nombres. Durante la partida, para mostrar los turnos se usa el nombre. Arreglamos la unanimidad del idioma.

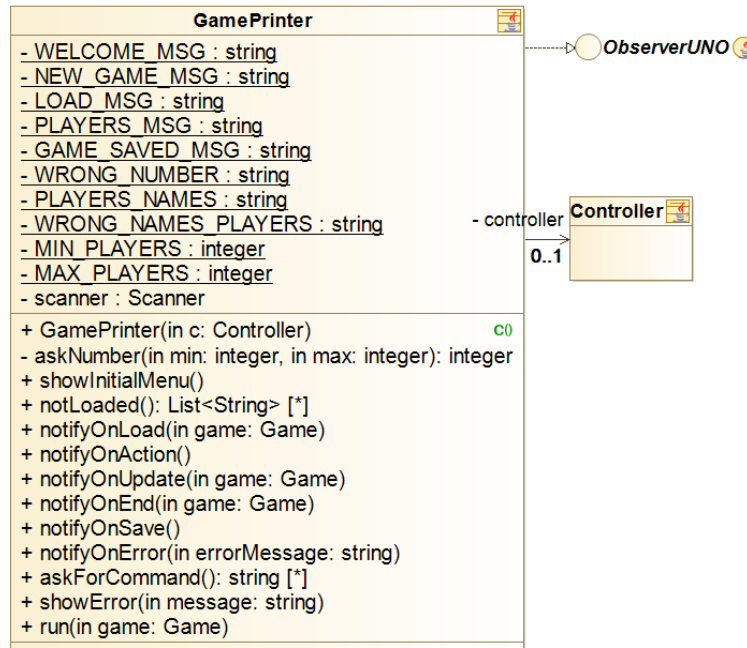
Ampliamos la idea de que cuando el usuario tire una carta que pueda cambiar el color, como las mencionadas anteriormente, la carta se muestre por consola, al siguiente jugador, del color que ha seleccionado como observamos en el ejemplo.

```

MARIA: +2+ 6 C 8 3 2 0 +4+
You can still draw 70 cards.
Action > t +4+ yellow
+4+
    
```

Sprint 3

El juego se continúa mostrando por consola.



Se crea la lógica de cargar partida guardada, y por tanto, se pregunta al usuario si desea cargar partida o empezar una nueva partida, con respuesta: “yes” o “no”. Si desea cargarla, aparecerá la partida desde el mismo punto donde se guardó. Cabe destacar que cualquier otra cosa distinta de “yes” es interpretada como “no” para evitar ejecutar, y modificar, una partida ya empezada.

Welcome to UNO
Do you want to load a save?

Al comienzo de una nueva partida, al contrario que antes, aparece ya una carta en el montón de lanzadas, dejando de ser la que el primer jugador lance.

```

2
LAURA: 0 2 8 2 -> +2+ 0
You can still draw 83 cards.
Action > |
  
```

Sprint 4

Creamos la clase padre *View*, tanto *GamePrinter* como *MainWindow* (clase principal de la interfaz gráfica) serán clases hijas de ella.

La consola permanece igual.

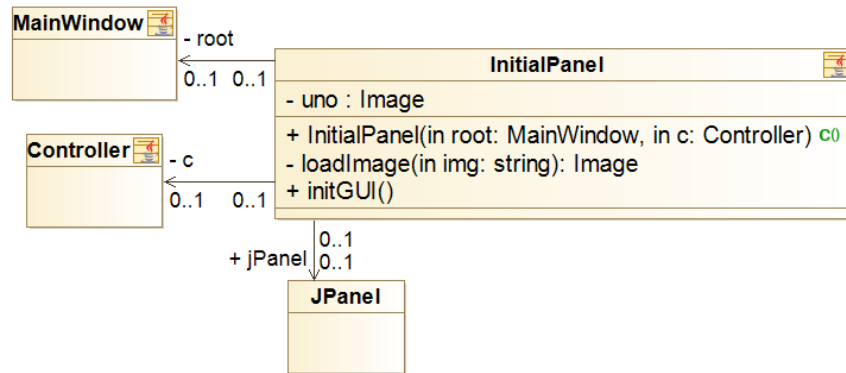
En este sprint, generamos la vista por interfaz gráfica. El primer modelo básico de vista consistía en ver las cartas del jugador actual, el montón de robar cartas, y el montón de cartas lanzadas. Cada carta se interpreta como un botón, así, si el usuario desea lanzar una carta, simplemente deberá seleccionarla. La misma lógica para robar del montón. Con esta vista, el usuario no puede ver el número de cartas restantes de cada jugador.

La forma en que se van alternando los paneles según se van necesitando es crear un panel base con un *CardLayout* al que se le van añadiendo los demás.

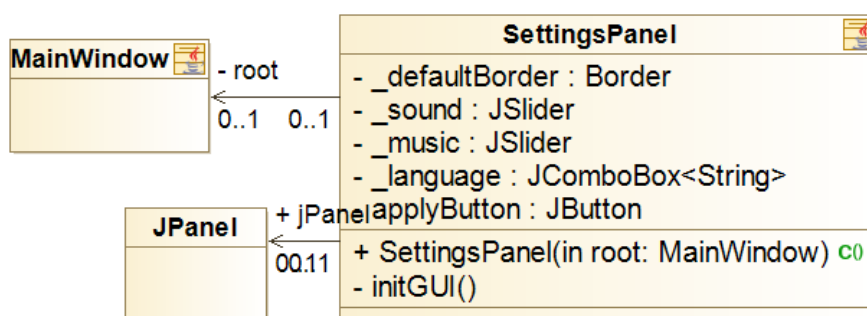
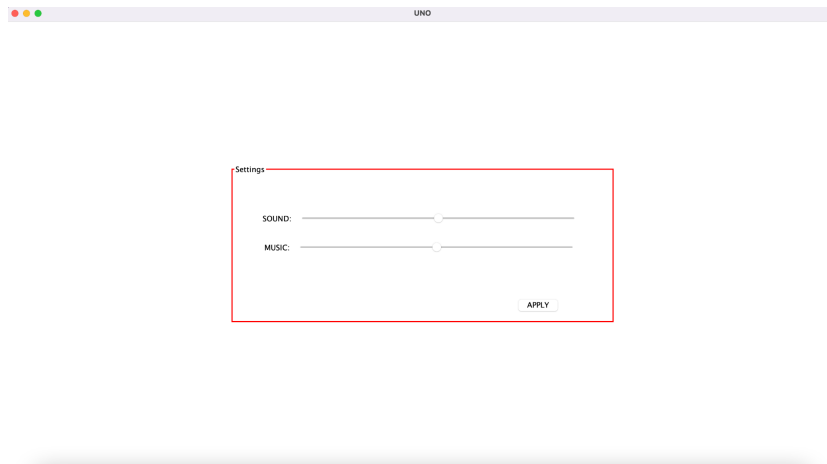
Las clases que forman parte de la interfaz gráfica son:

- *InitialPanel*, que consta de dos opciones, *New Game*, para empezar una nueva partida, y *Load Game*, para cargar una partida ya existente. Si el usuario empieza nueva partida, se irá al *NumPlayersPanel*, y si decide cargar, al *LoadPanel*. Además, el *InitialPanel* presenta un botón de *Settings*, que si se pulsa, mostrará un panel de configuraciones.

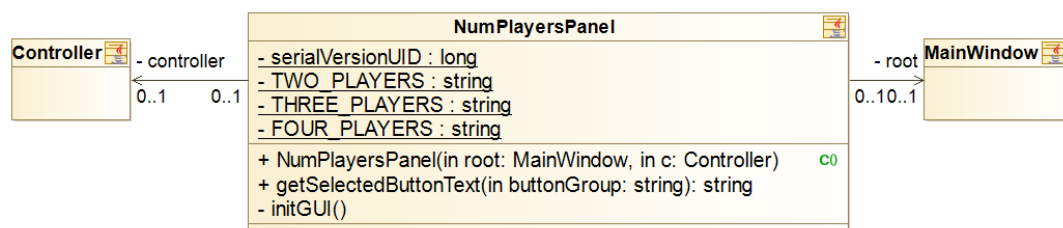
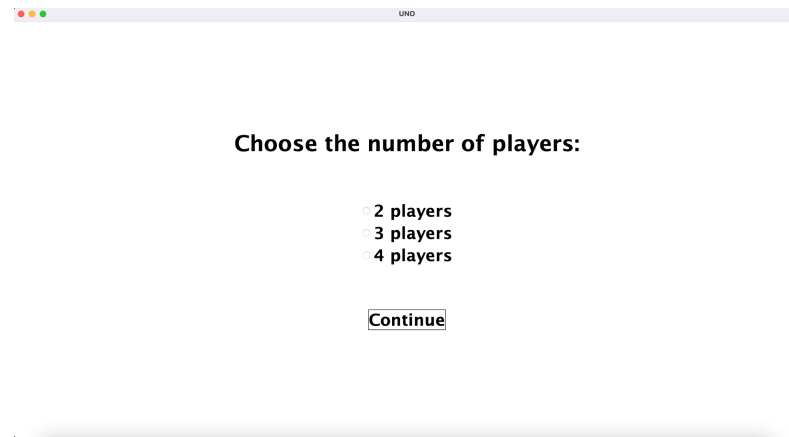




- *SettingsPanel*, panel cuya principal función es configurar aspectos del juego como: el volumen de la música de fondo y el volumen de los sonidos de efectos. En un principio se le añade una opción de cambiar de idioma (inglés, español y chino), pero luego se decide quitar por complejidad y falta de tiempo.

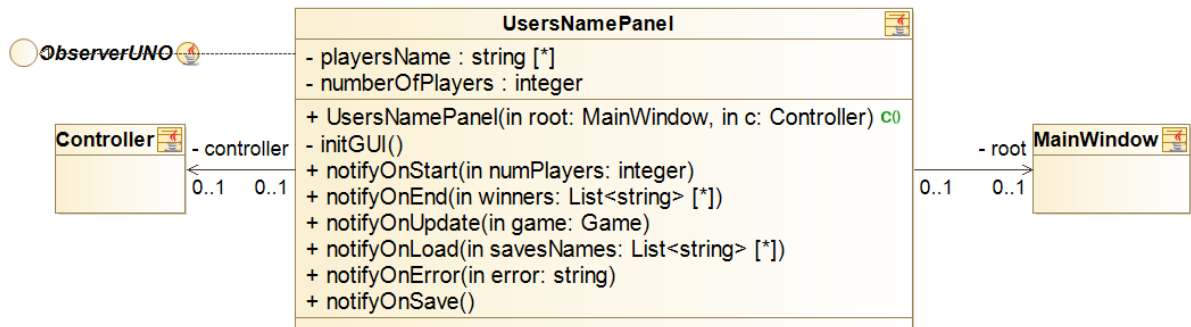


- *NumPlayersPanel*, panel básico y sencillo que se encarga de preguntarle al usuario cuántos jugadores van a jugar (2-4). Se cambia al *UserNamePanel*.

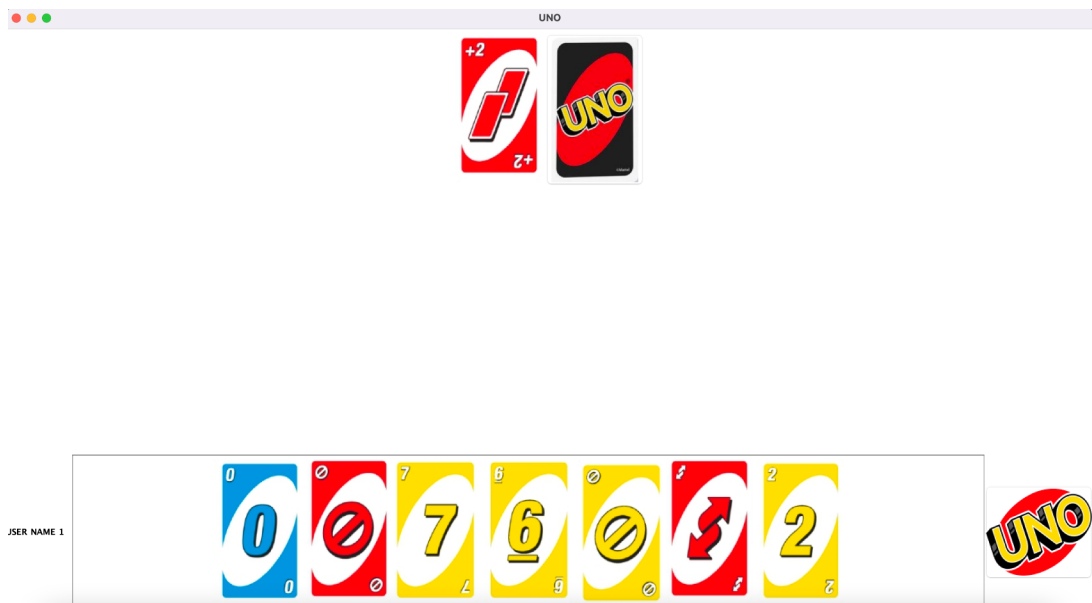


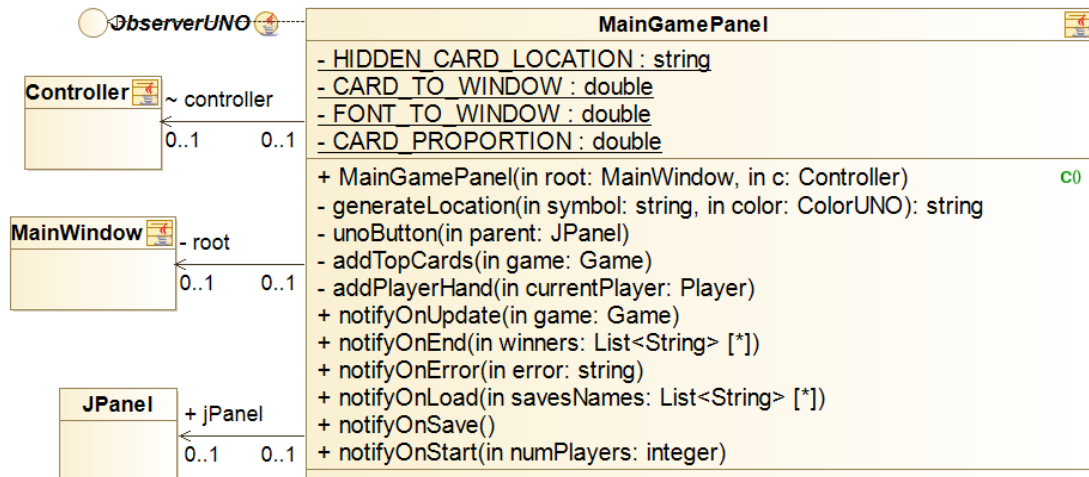
- *UserNamePanel*, que consiste en que los usuarios introduzcan sus nombres, con los que serán identificados durante toda la partida.



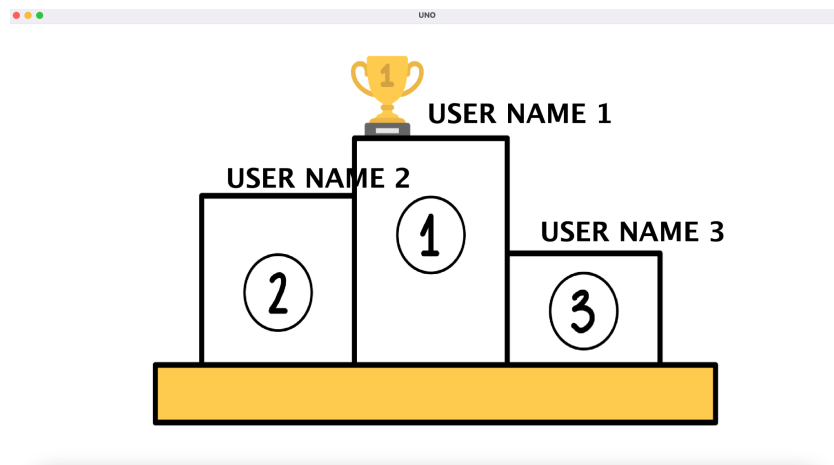


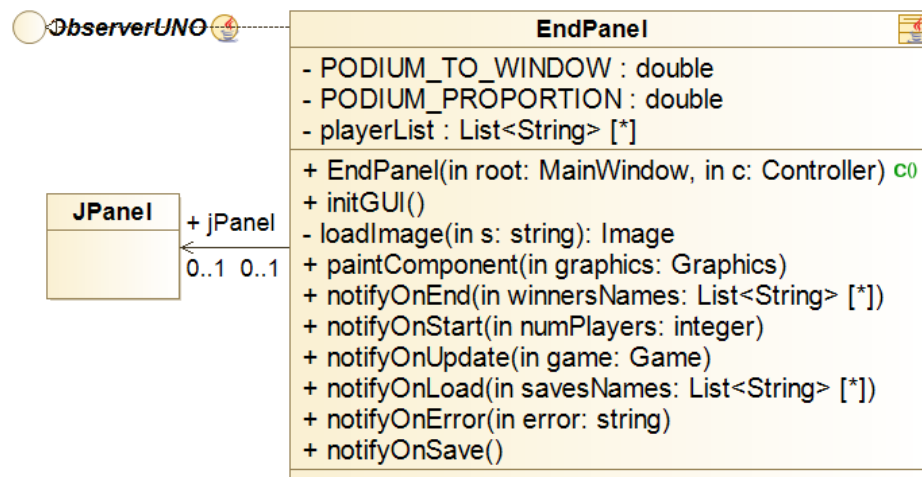
- *MainGamePanel*, es el panel donde se muestra el juego. Se muestran las cartas del jugador del turno actual, el montón de robar cartas, y el montón donde se lanzan las cartas. Además, podemos observar que en la esquina inferior izquierda aparece el nombre del jugador cuyas cartas se están mostrando, así como el botón del UNO, cuya función no está todavía implementada.



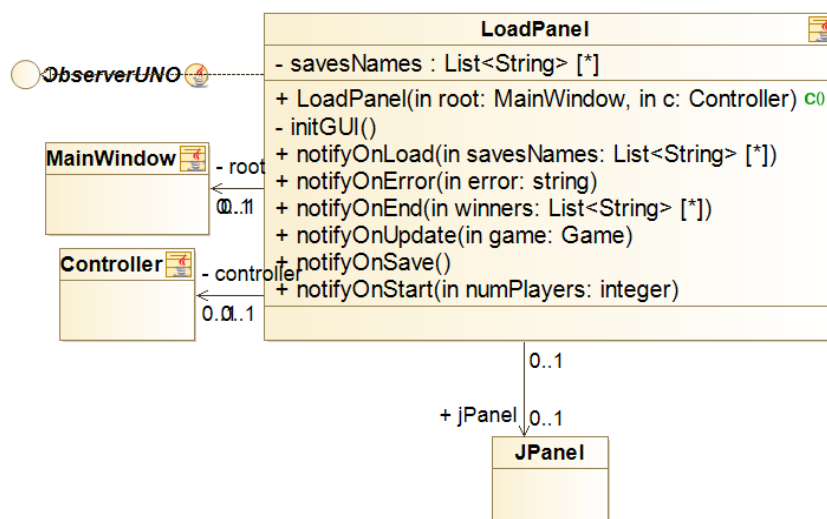


- *EndPanel*, se trata de un panel donde se muestra un pódium con los nombres de usuario de los tres jugadores mejor clasificados en el juego. En el caso de que solo haya 2 jugadores, solo se mostrarán dos jugadores. En el caso de que haya 4, uno quedará fuera del pódium. Los jugadores quedarán posicionados según el número de cartas que tenían en su mano en el momento en el que finalizó la partida. De manera que el que tuviese menos cartas quedaría en la segunda posición y el que más en el cuarto puesto.

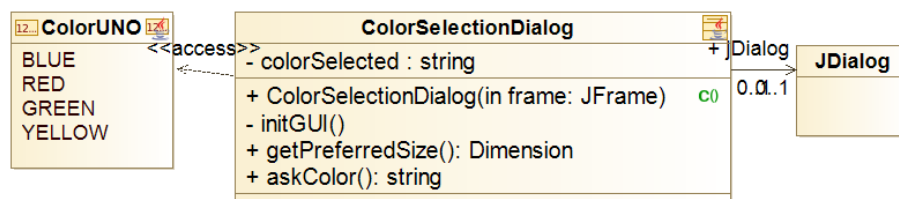




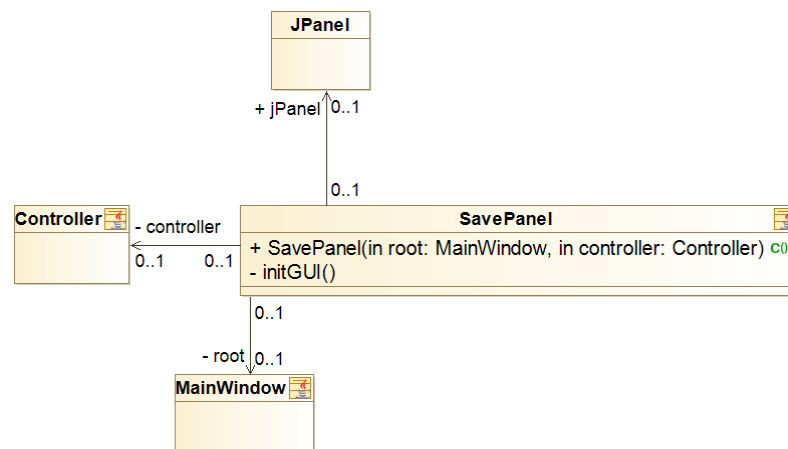
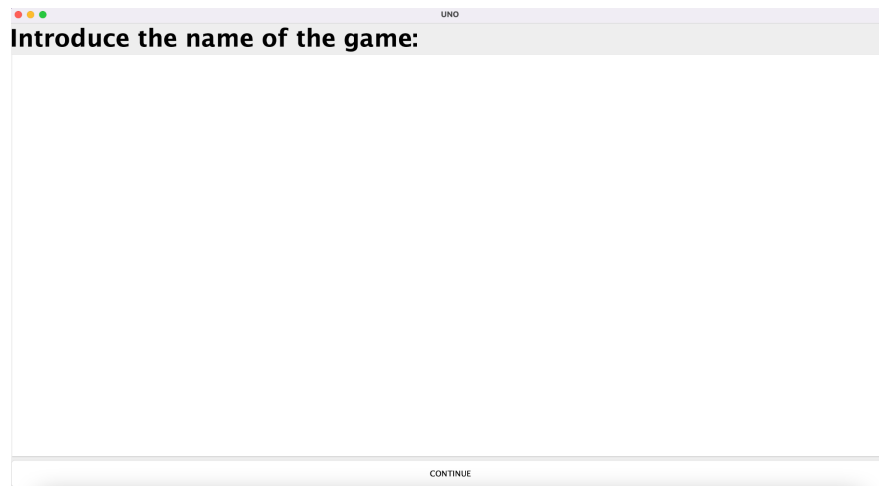
- *LoadPanel*, panel que muestra las posibles partidas a cargar. Una vez el usuario seleccione la que desee, el juego se retomará en el mismo punto que se guardó.



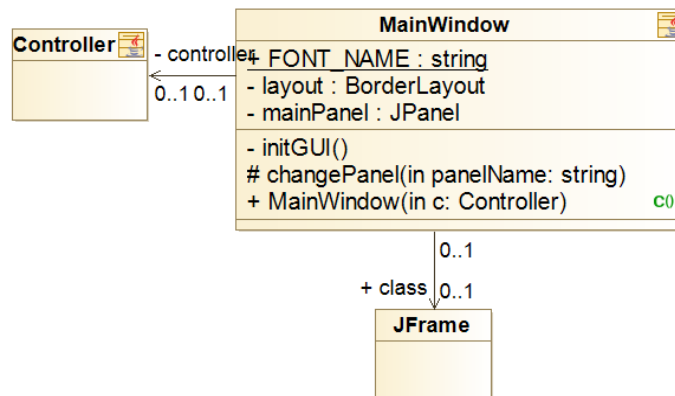
- *ColorSectionDialog*, consiste en una ventana que aparecerá cuando el usuario desee tirar una carta de las que se encargan de cambiar el color actual, es decir, cuando el usuario desee lanzar *Add4Card* o *ChangeColorCard*. Esta pequeña ventana muestra las 4 opciones de colores posibles: “yellow”, “blue”, “red” o “green”.



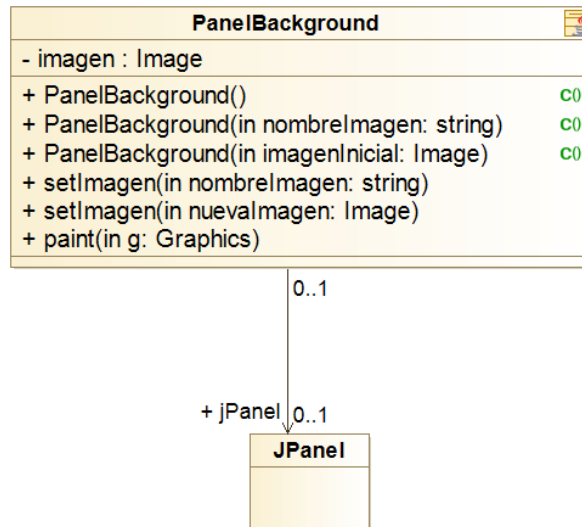
- *SavePanel*, panel que se encarga de guardar la partida actual. Pide al usuario un identificador del juego, y así cuando el usuario desee cargarla pueda identificarlo. En este sprint no se ha creado todavía el botón que te dirija a este panel, y por lo tanto, no se puede guardar una partida todavía. Deberá guardarse por consola.



- *MainWindow*, clase principal que se encarga de manejar los paneles, y de la conexión entre ellos.



- *PanelBackground*, clase que destinamos a poner fondos/backgrounds en el juego.



Los botones en este sprint son básicos. Poseen el siguiente diseño:

- Fuente: Microsoft Yahei.
- Tipo: negrita.
- Tamaño: 30.

Sprint 5

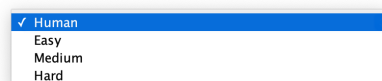
La vista mejora, permitiendo al jugador actual ver el número de cartas del resto de usuarios. Si un jugador posee más de cuatro cartas, mostramos solo cuatro por defecto, pero añadimos un identificador de la forma “+k”, siendo k el número de cartas que faltan por mostrar.

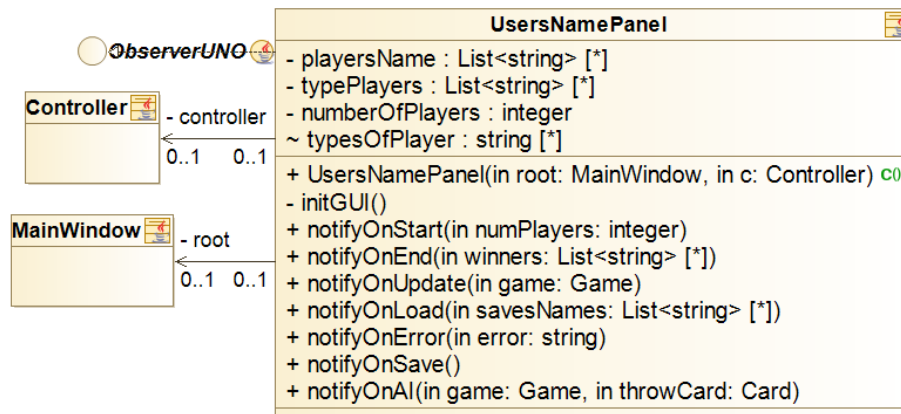


Ejemplo donde cada jugador tiene 25 cartas.

Además, tras implementar la IA, además de preguntar al usuario el nombre de cada jugador, en el `UserNamePanel`, debemos preguntarle si será de tipo: *Easy*, *Medium*, *Hard*, o *Human*.

Choose type of player





También se añadirá a la consola la pregunta de qué tipo de jugador será:

Introduce the names of the players:

Maria

Please, introduce the type of player: Human, Easy, Medium, Hard

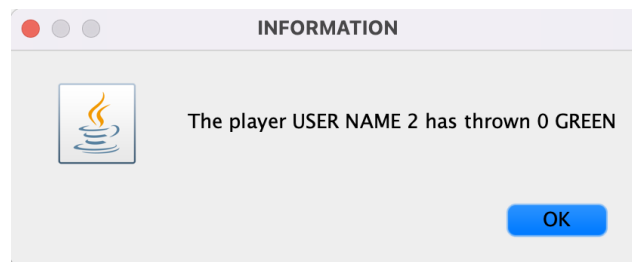
Human

Pepe

Please, introduce the type of player: Human, Easy, Medium, Hard

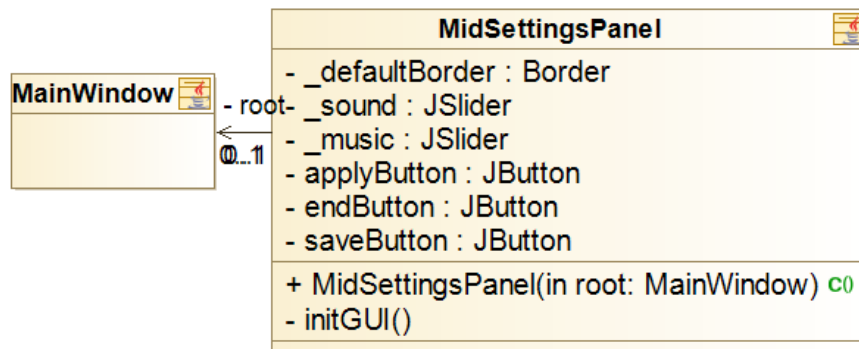
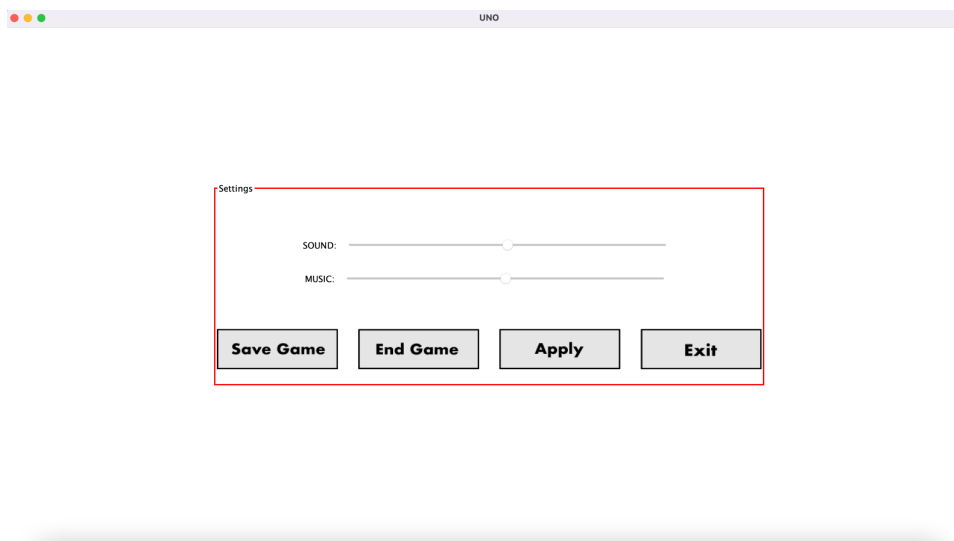
Hard

Además, para poder saber qué carta ha elegido la máquina de IA lanzar, mostraremos un mensaje en cada turno como el siguiente.

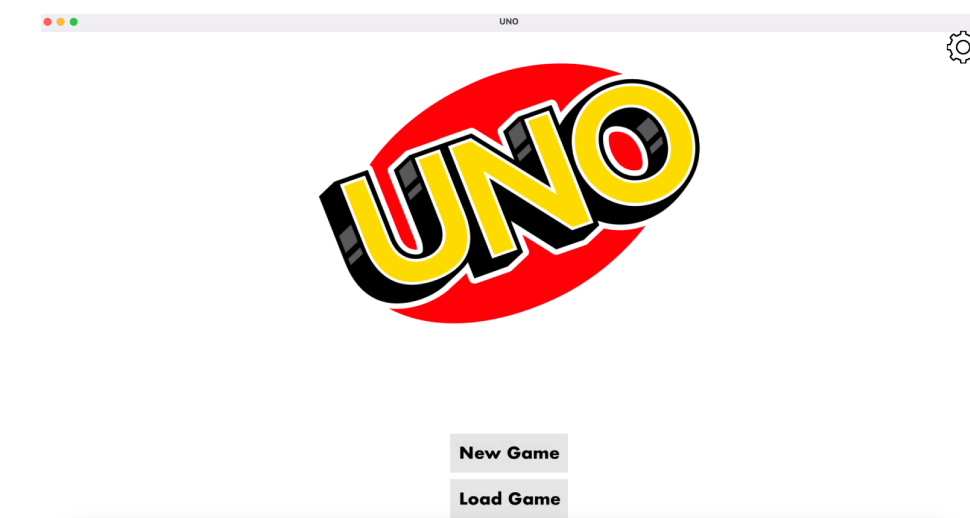


Decidimos eliminar el *PanelBackground*, pues su funcionalidad ya no era necesaria, y añadimos el panel de *MidSettingsPanel*, con el principal objetivo de poder guardar una partida en mitad del juego, subir o bajar el volumen de la música, terminar un juego o empezar uno nuevo.

- El botón de *Save Game* nos dirige al panel *SavePanel*, ya mostrado anteriormente.
- El botón *Exit* cierra la aplicación.
- El botón *End Game* nos retorna al *InitialPanel*.
- El botón *Apply* registra los cambios realizados.

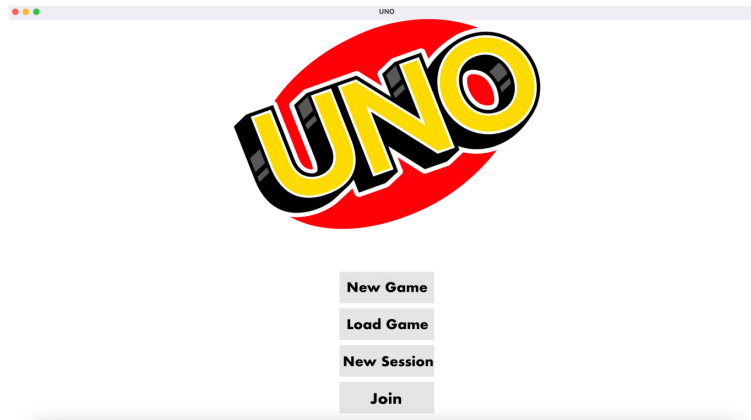


En este sprint, decidimos darle diseño a los botones. Se trata de un diseño sencillo y elegante, un botón gris con letras de color gris. Veamos el *InitialPanel*:

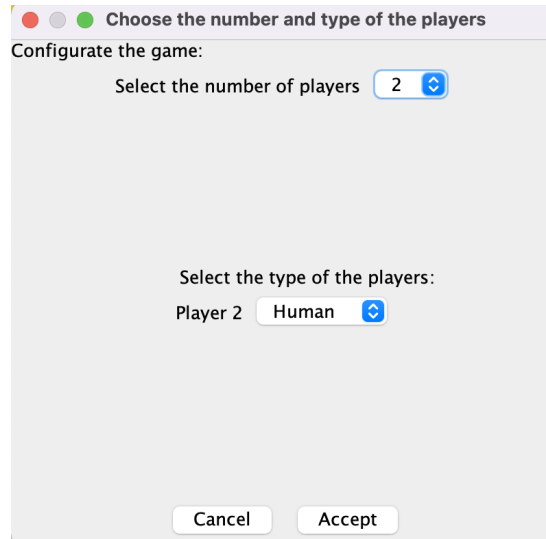


Sprint 6

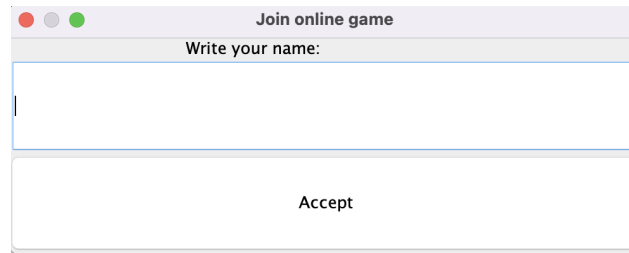
Añadimos al *InitialPanel* un botón *New Session*, cuya funcionalidad es crear una partida en red para que otros usuarios puedan unirse, y a su vez, creamos *Join*, para poder conectarse a una partida ya creada.



Cuando accionamos el botón *New Session* nos aparece el diálogo mostrado a continuación. Se seleccionará el número de jugadores, y el tipo de cada uno, obligando a que el usuario creador de la sesión juegue como humano.

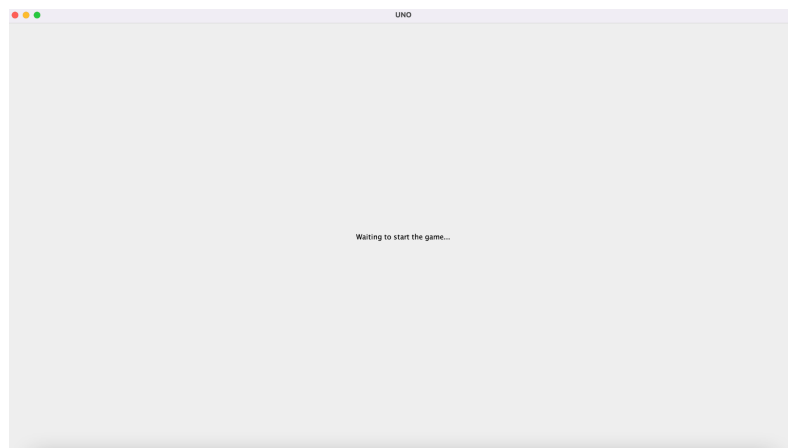


Una vez seleccionado, saldrá una ventana emergente que preguntará al host de la sesión recién creada su nombre de usuario, como se muestra a continuación.

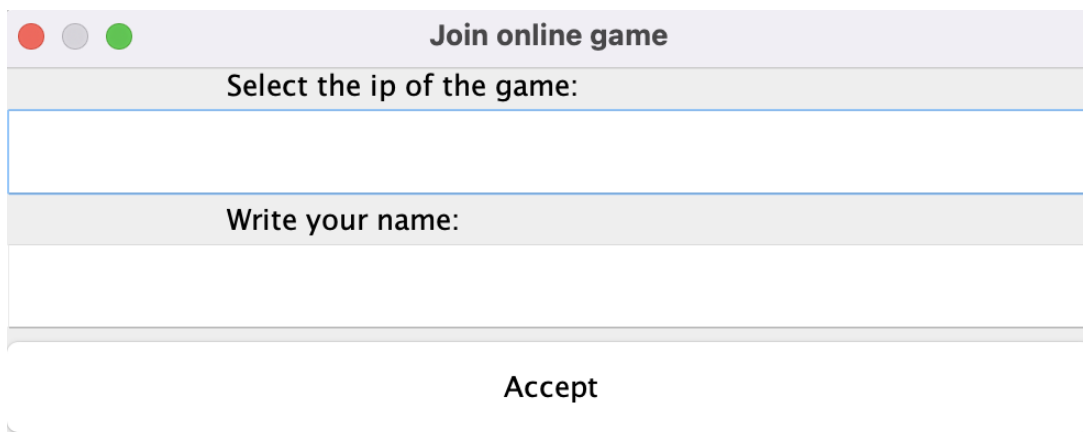


A screenshot of a macOS-style dialog box titled "Join online game". It features a light gray header bar with standard window control buttons (red, yellow, green) on the left. Below the header, the text "Write your name:" is displayed above a large, empty text input field. At the bottom of the dialog, there is a wide, light gray button labeled "Accept".

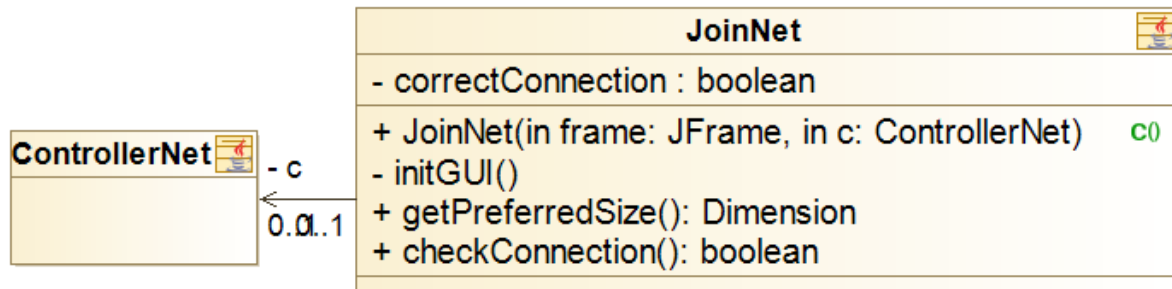
La pantalla del host de la sesión permanece en espera hasta que el número de jugadores seleccionados anteriormente se unan al juego. Una vez estén todos, empezará el juego automáticamente en todas las pantallas de los usuarios.



Por otra parte, si pulsamos el botón de *Join* en el *InitialPanel*, se nos preguntará la dirección IP del ordenador en el que se ha creado la sesión a la que deseamos unirnos, así como nuestro nombre de usuario.

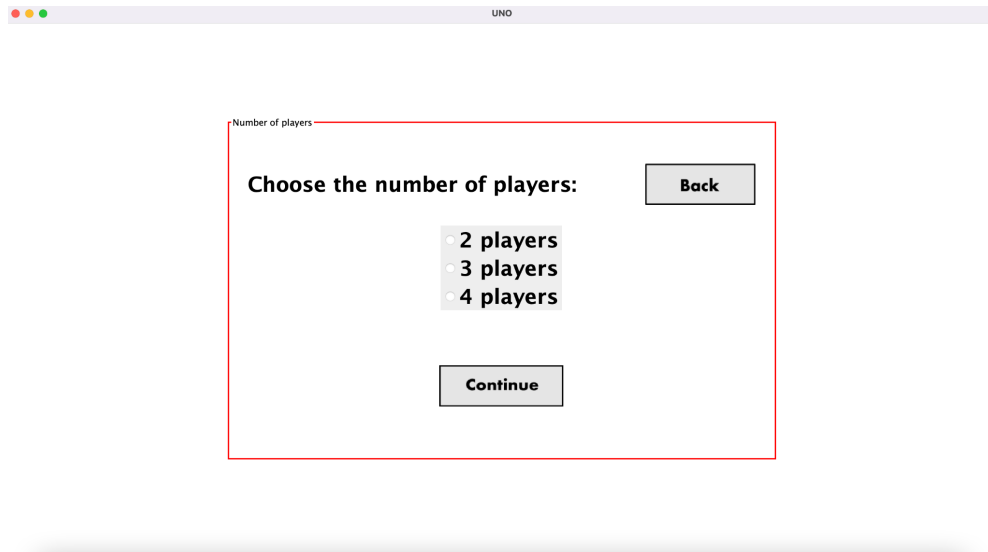


A screenshot of a macOS-style dialog box titled "Join online game". It features a light gray header bar with standard window control buttons (red, yellow, green) on the left. Below the header, the text "Select the ip of the game:" is displayed above a large, empty text input field. Below this field, the text "Write your name:" is displayed above another large, empty text input field. At the bottom of the dialog, there is a wide, light gray button labeled "Accept".

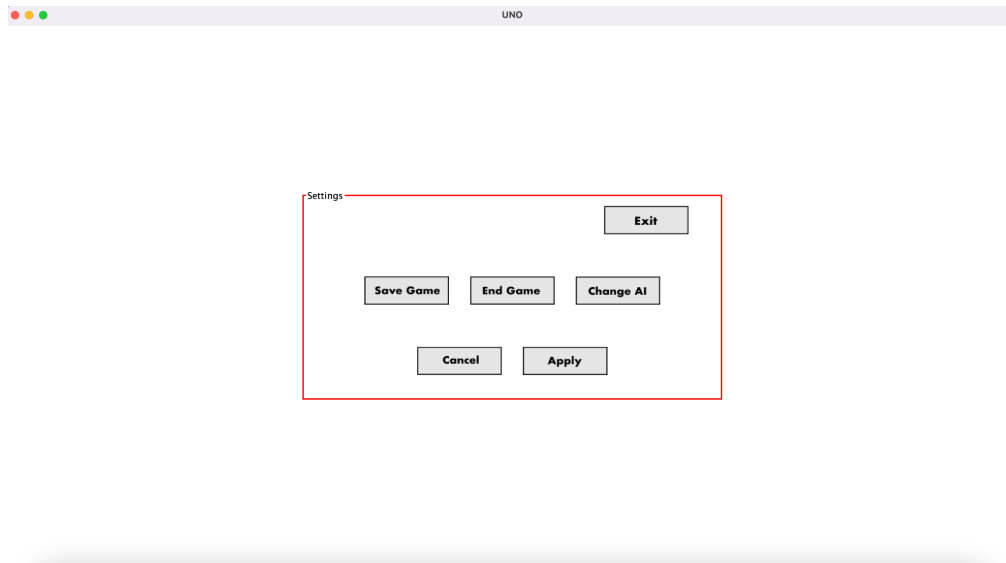


Mejoramos visualmente el *SavePanel*, el *UserNamePanel*, y el *NumPlayersPanel*.

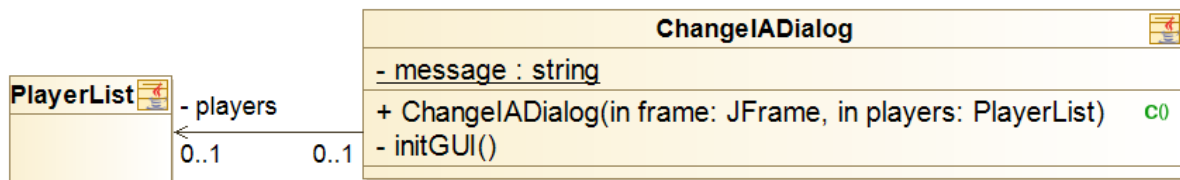
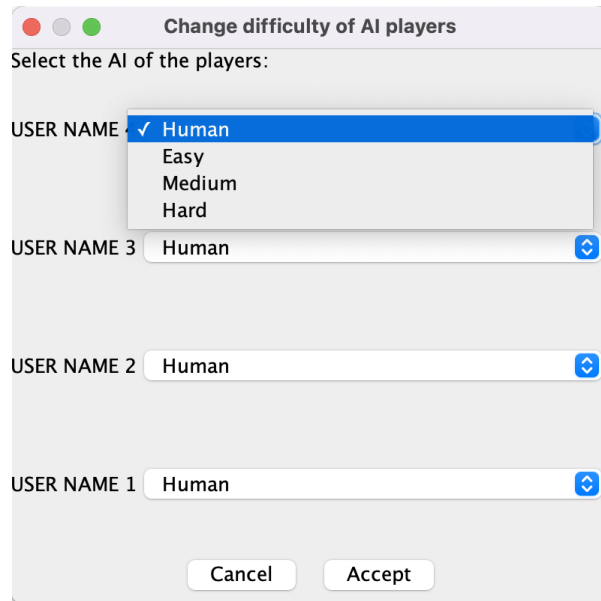




Añadimos nuevas opciones al *MidSettingsPanel* como salir del juego (*Exit*) y cambiar el nivel de inteligencia de los bots (*Change AI*). Por complejidad, decidimos eliminar las opciones de música y sonido en el proyecto.



Añadimos la funcionalidad de poder cambiar la inteligencia artificial de los bots en mitad del juego, para ello creamos el botón de *Change AI* en la pantalla de *MidSettingsPanel*, que al ser accionado genera una ventana emergente en la que el usuario podrá elegir entre *Human*, *Easy*, *Medium* o *Hard*, como se muestra a continuación.



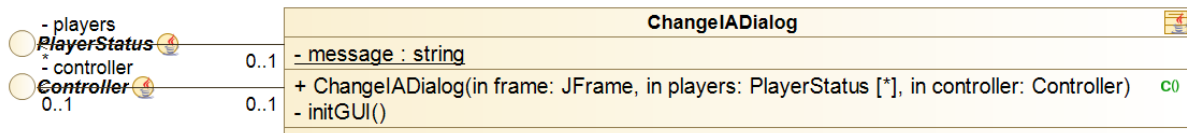
Antes de poner el juego en marcha, el usuario deberá introducir como argumento el modo de juego que desea, si consola, o interfaz gráfica.

El modo elegido por defecto es *Gui*. Si su elección es la consola, deberá escribir “cli”.

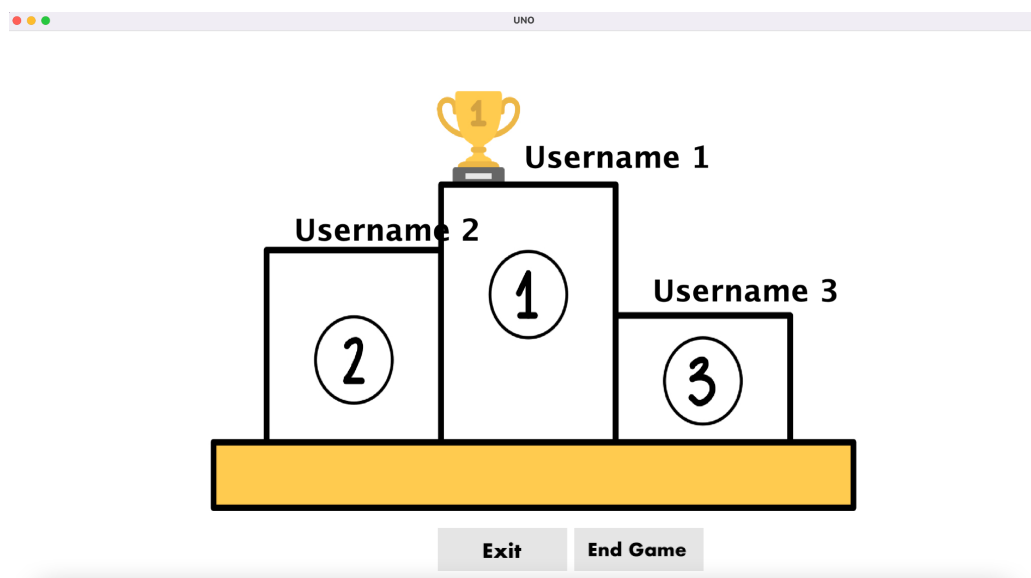
No mode introduced. Default is GUI.

Sprint 7

Durante este Sprint nos dimos cuenta que la forma en la que gestionamos el cambio de inteligencia rompía el diseño, por ello, tuvimos que modificarlo obligando a hacerlo a través del Controller, por ello tenemos que pasar al *ChangeIADialog* el *Controller* como atributo en el constructor.

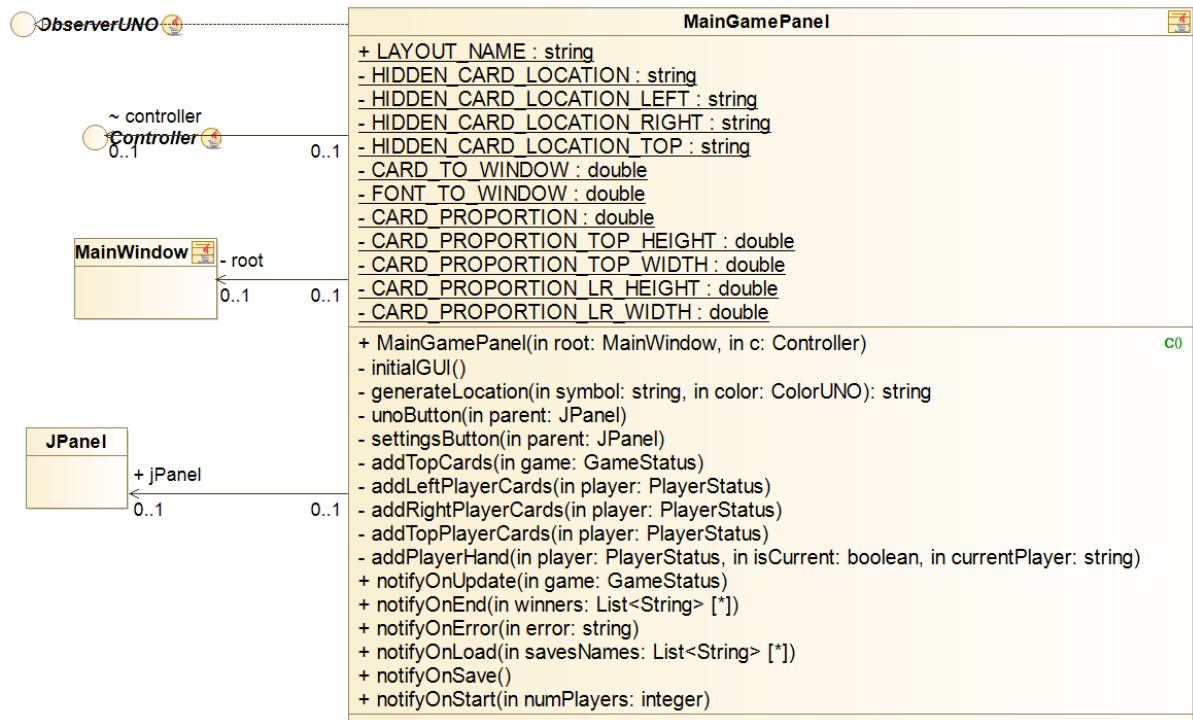


En el *EndPanel* añadimos la funcionalidad de poder volver al *InitialPanel* o salir del juego si el usuario lo desea.



Por otra parte, el *MainGamePanel* deja de poseer como atributo el *Game* y pasa a tener *GameStatus*. En general la vista solo tiene adapters, implicando que los métodos *notify* de *ObserverUNO* pasen como parámetro el *GameStatus*.

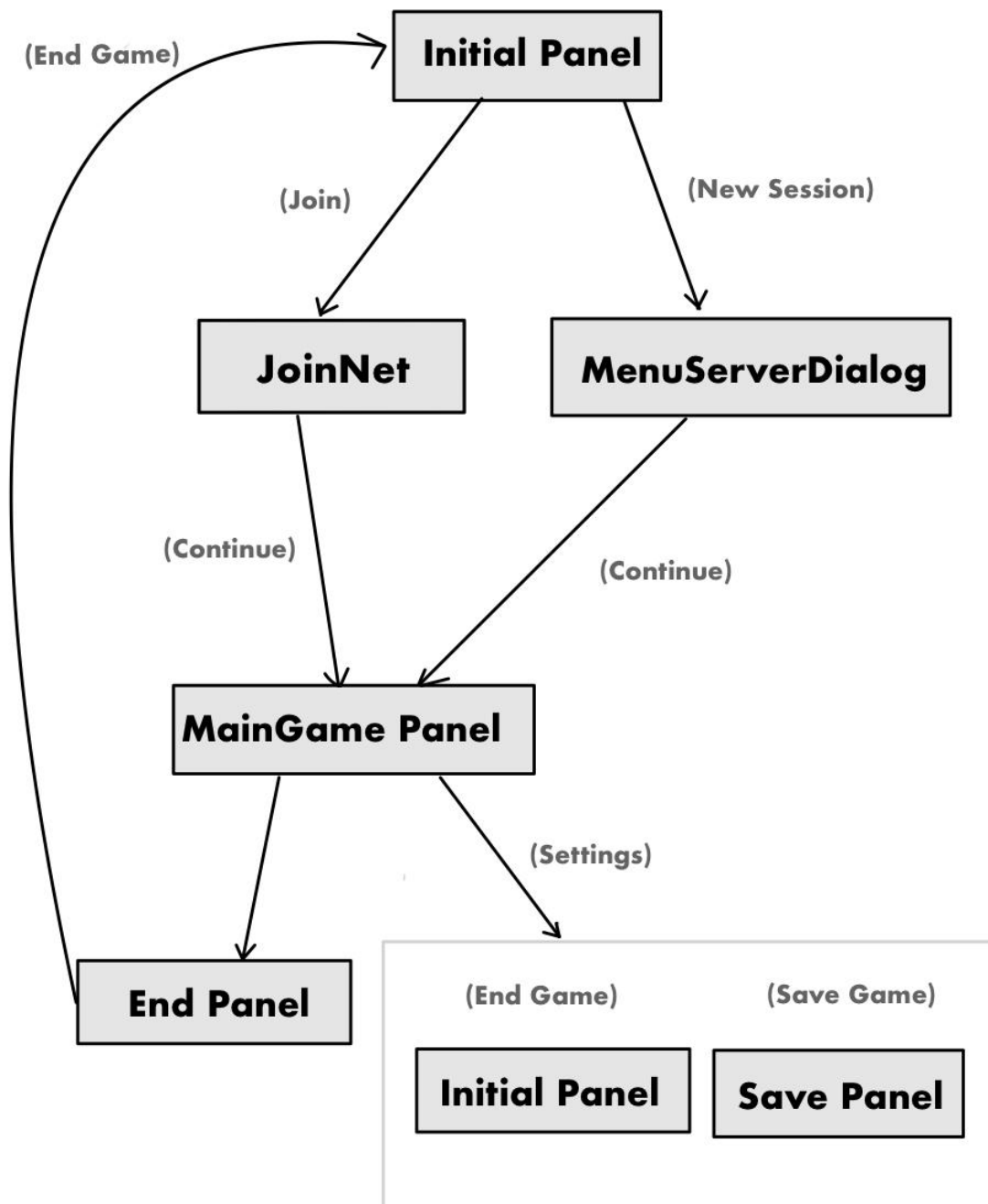
Decidimos borrar la clase padre *View*, pues finalmente *GamePrinter* y *MainWindow* no tienen tantos métodos en común como en un principio se pensaba, trabajarán como clases separadas.



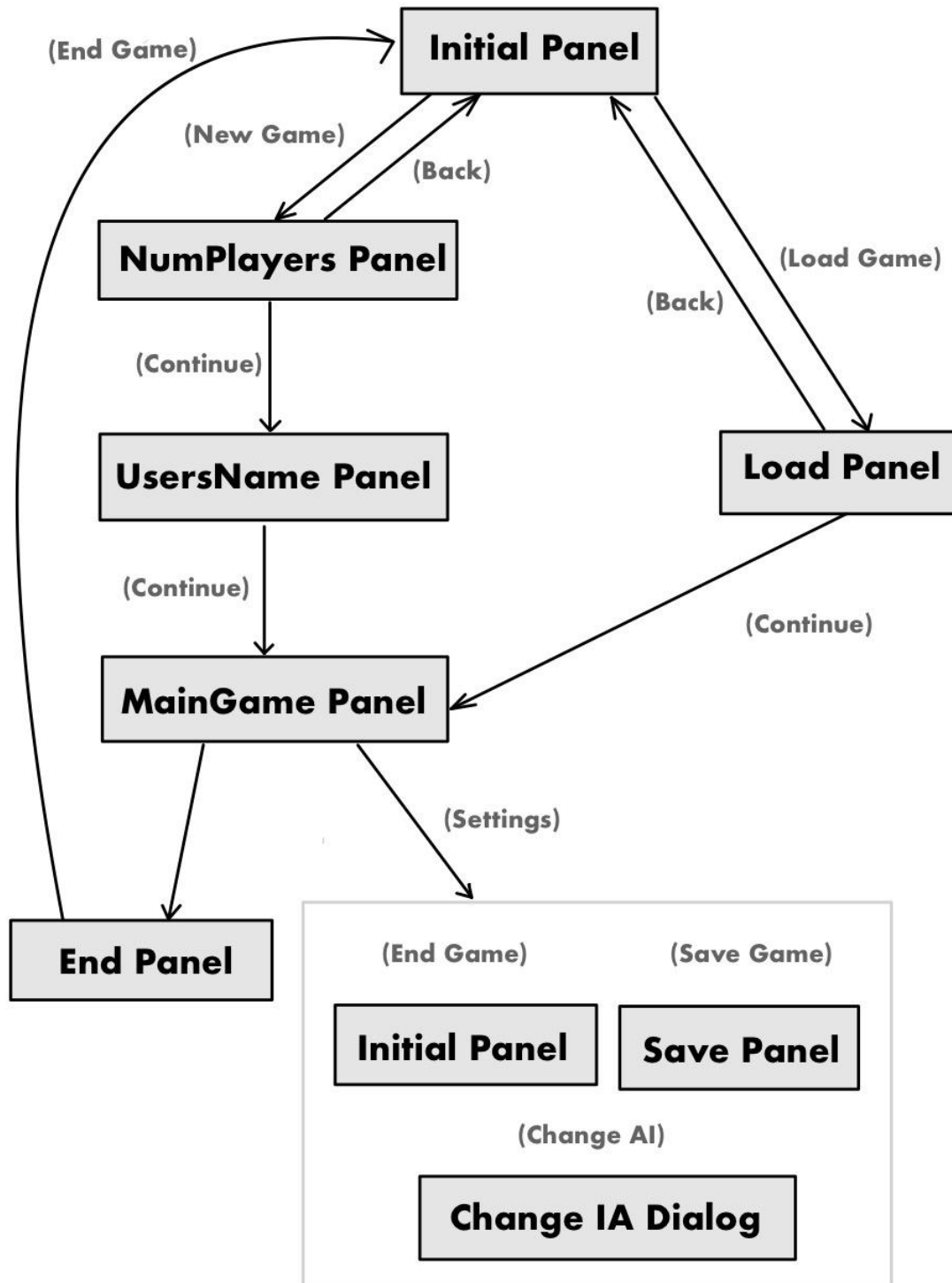
Arreglamos errores relacionados con el *NewSession*, añadimos la gestión de que si al crear una sesión, el host no introduce su nombre o no se ha conseguido crear correctamente la sesión, no permite que el juego continúe y notifica al usuario.

El *SettingsPanel* y el *waitingforUsersPanel* han sido eliminados.

Esquema de la secuencia de conexión en red.



A continuación vamos a mostrar un sencillo esquema de la secuencia de paneles del juego final suponiendo partida local.

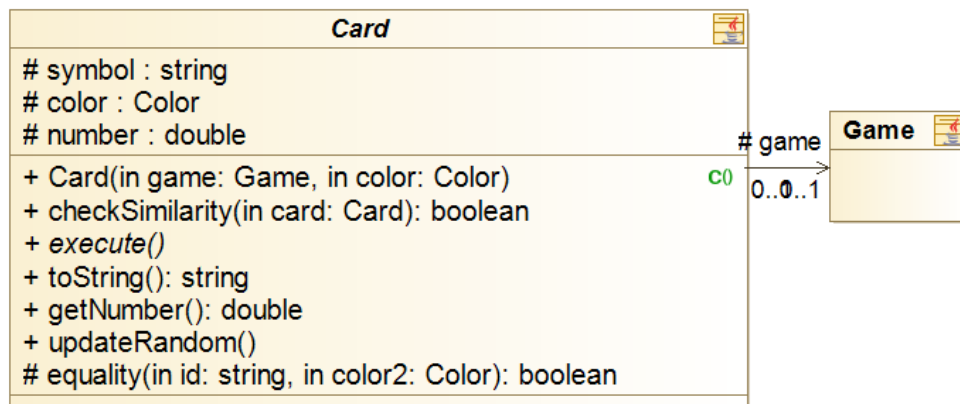


Diseño y evolución de las clases principales del modelo

Diseño y evolución de las cartas

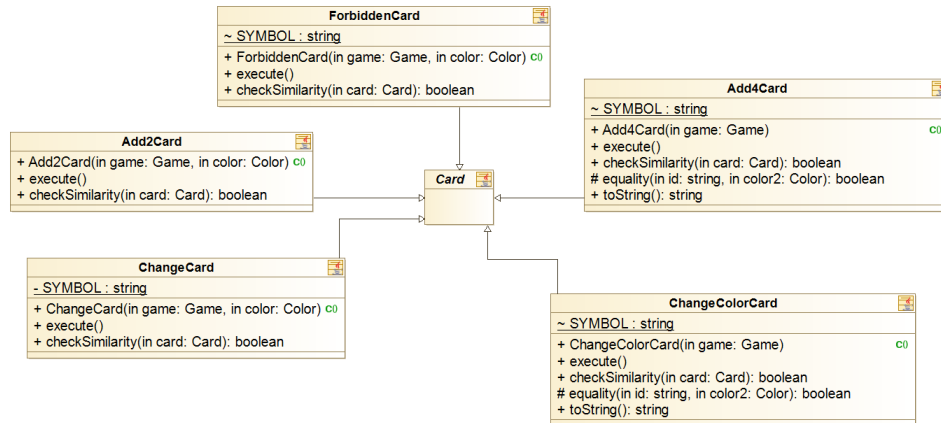
Sprint 1

Se crea la clase padre **Card**, cuyos hijos (**SimpleCard**, **ChangeCard**, **ChangeColorCard**, **ForbiddenCard**, **Add4Card**, **Add2Card**) representan la totalidad de las cartas de la baraja del UNO. Los atributos de esta clase padre son:



- Symbol: string que representa la carta en la consola, para las *SimpleCard* sus respectivos números, *ChangeCard* (->), *ChangeColorCard* (C), *ForbiddenCard* (O), *Add4Card* (+4), y por último, *Add2Card* (+2).
- Color: elemento del tipo enumerado de los 4 colores representativos del juego, *Add4Card* y *ChangeColorCard* tienen este atributo a *null*, pues son las cartas que se encargan de cambiar el color. Los colores posibles son azul, amarillo, verde y rojo.
- Game: Todas las cartas guardan una instancia del juego entero, para que así en su correspondiente *execute*, puedan hacer la funcionalidad de cada carta.
- Number: Este atributo no se corresponde con el número de la *SimpleCard*, si no con el utilizado para el momento de barajar.

Diseño y evolución de las clases principales del modelo



PLAYER 1: 9 6 2 1 2 3 0 3 -> 0 0 5 0 5 3
 PLAYER 2: 5 1 1 -> 8 3 +4 3 -> 8 9 +2 4 7 6

Ejemplo sencillo de cómo se muestran las cartas de los jugadores.

En *player 1*, vemos como casi no se puede diferenciar entre 0 (*SimpleCard* con número 0), y O (*ForbiddenCard*).

Para el manejo del propio número de la carta, la clase *SimpleCard* presenta como atributo un entero *Number*, que en este caso sí cumple la funcionalidad de recoger el número de la carta (0-9). Se trata de la única clase con este.



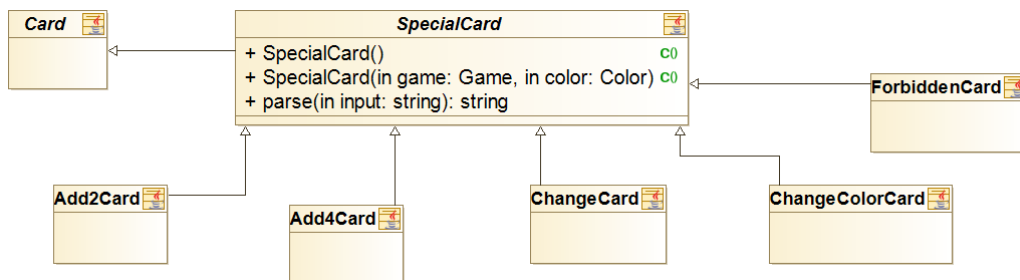
Función de cada carta:

- *ForbiddenCard*: Si un jugador lanza esta carta se salta el turno al siguiente jugador suyo. En el caso de que haya solo dos jugadores, el turno se mantendrá en el jugador que lanzó la carta.
- *ChangeColorCard*: Se encarga de cambiar el color actual en la partida. El jugador que la lance podrá escoger el nuevo color.
- *SimpleCard*: La carta no presenta ninguna función.
- *Add2Card*: Esta carta obligará al jugador del siguiente turno a robar dos cartas del montón de cartas sin saltarle el turno.

- ChangeCard: Se encarga de cambiar el orden de los jugadores, el sentido en el que se gira el turno. Si en un principio el sentido es de las agujas del reloj y se lanza, el nuevo sentido será el antihorario, y viceversa.
- Add4Card: Esta carta obligará al jugador del siguiente turno a robar cuatro cartas del montón de cartas sin saltarle el turno. Además, el jugador que la lance podrá elegir el nuevo color de la partida.

Sprint 2

En este sprint decidimos crear la clase padre **SpecialCard**, para encapsular la funcionalidad de todas las cartas especiales, sus hijas: **Add4Card**, **Add2Card**, **ForbiddenCard**, **ChangeColorCard**, **ChangeCard**; cuya lógica es más sofisticada que la comprobación de si la carta a lanzar coincide en color o número con la última lanzada. Esta clase hereda de Card.



Todas las cartas mantienen los mismos atributos, excepto las clases: *ForbiddenCard*, que por confusión entre el número 0 y la letra O a la hora de lanzar carta, decidimos cambiar su identificador por "X", y por problemas de código (el sistema detectaba los símbolos como enteros), también las clases *Add2Card* y *Add4Card* cambian: de "+2" a "+2+", y de "+4" a "+4+".

LAURA: 6 X 6 6 0 +2+ 6

Como cambio fundamental de este sprint, comentamos la adición de la funcionalidad de guardar partidas, para en el próximo sprint, poder cargarlas. Precisamente por eso, nos hemos visto en la necesidad de crear un método report en la clase *Card* que devuelva un *JSONObject* con la información esencial de las cartas. En esta clase hemos decidido seguir esta estructura de guardar la información: guardamos la información de la carta, donde se incluye el símbolo y el color, y después incluimos el tipo de la carta, tal y como se muestra a continuación:

```

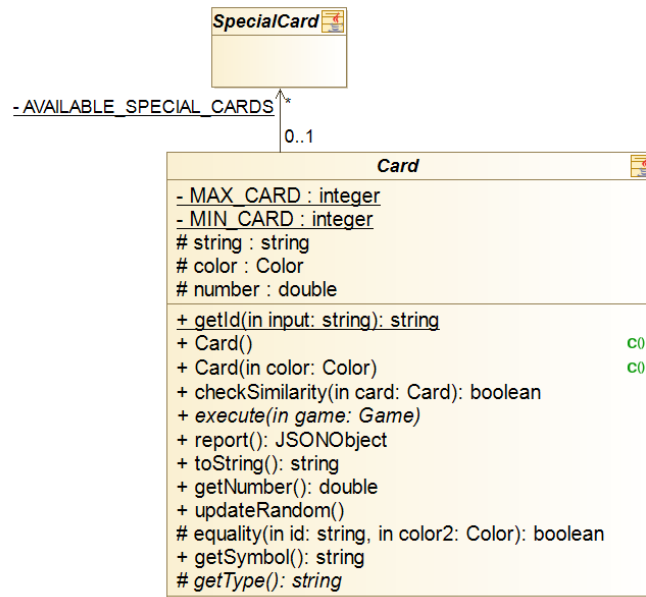
centercard =: {
  "data": {"symbol": "0", "color": "BLUE"},
  "type": "SimpleCard"}
  
```


Sprint 3

En el sprint 3, reflexionamos sobre el hecho de que carta tuviese una referencia a *game*, y llegamos a la conclusión de que, como muchas cartas no la usaban, y otras solo en el método *execute*, la mejor opción era simplemente pasarla como parámetro en este método. Así, las cartas no tendrían por qué conocer el juego.

Además, desde el punto de vista lógico, no veíamos el sentido a que una carta “tuviese” un juego ya que lo razonable es que un juego tenga cartas.

También añadimos el atributo *type* a las cartas para facilitar el guardado y el cargado de las mismas.



Sprint 4

La implementación de las cartas sufre una ligera modificación respecto al anterior sprint ya que se renombra el método *equality* a *equals* para mantener la consistencia con Java y se reorganiza el orden del código de una forma más lógica. Con la introducción de la interfaz gráfica de usuario, las cartas empiezan a tener asociadas a sus símbolos e identificadores de colores imágenes a ordenador, donde cada tipo de carta presenta su propio diseño.

- *ForbiddenCard*: Muestra el símbolo del prohibido en el color correspondiente de su carta.
- *ChangeColorCard*: Se trata de una carta que presenta un óvalo en el centro con los cuatro colores posibles a cambiar.
- *SimpleCard*: Es del color de la carta y presenta su número en el centro.
- *Add2Card*: Aparece un “+2” en la carta. El color de la carta es su propio color.
- *Add4Card*: Se trata de una carta que presenta un óvalo en el centro con los cuatro colores posibles a cambiar y “+4”, simbolizando que el siguiente jugador debe robar 4 cartas.
- *ChangeCard*: Se trata de dos flechas en sentido contrario. El color de la carta es su propio color.



Ejemplo de cada tipo de carta.

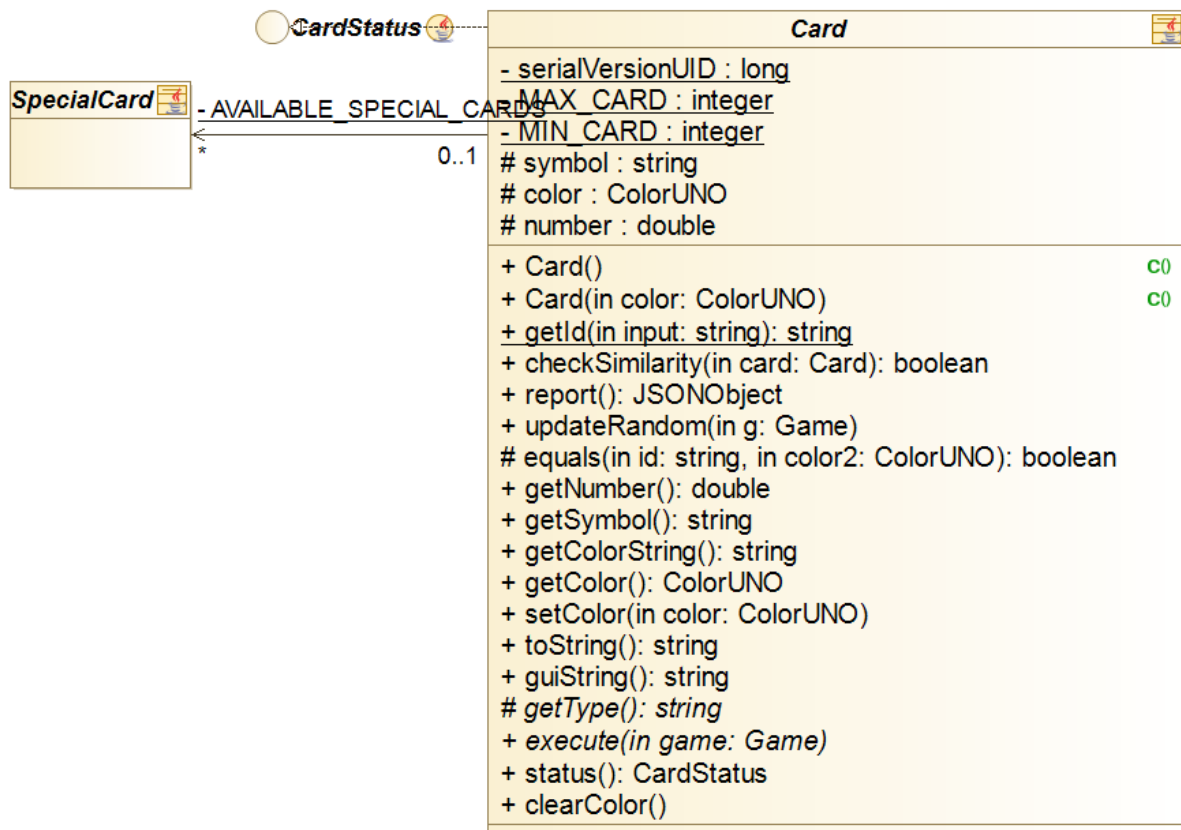
Sprint 5

Los cambios de las cartas en el sprint 5 son prácticamente nulos. Aún así, cabe destacar, la introducción de un método para devolver una cadena de texto que será más legible en la GUI y un método que permite a las IAs cambiar el color de los +4 (*Add4Card*) y de los cambios de color (*ChangeColorCard*).

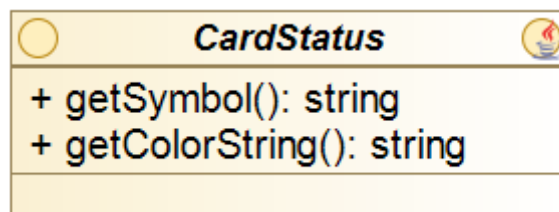
Sprint 7

Nos encontramos con un problema con las cartas de *ChangeColor* y *Add4* ya que cuando se acaba el monton de robar y se repone con el monton de lanzadas, las cartas se quedaban con el color que el usuario había elegido en lugar de volver a pasar a negro.

Por ello añadimos el método abstracto *clearColor* para asegurarnos que las cartas de *ChangeColor* y *Add4* se vuelvan a poner de color “negro”, cuando pasan del montón de lanzadas al montón de robar.



Además, hemos añadido el adapter *CardStatus*.

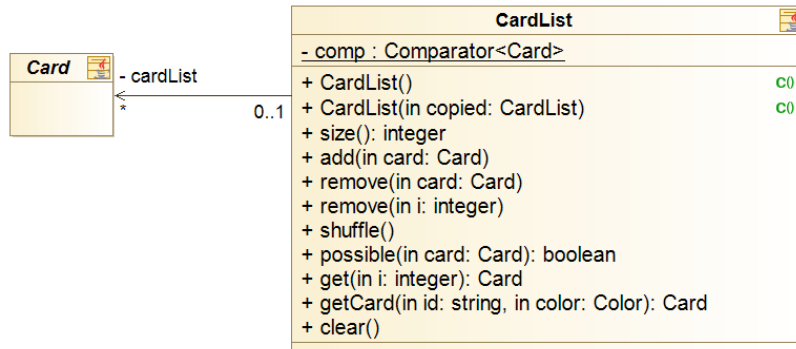


Diseño y evolución de las listas de cartas:

Sprint 1

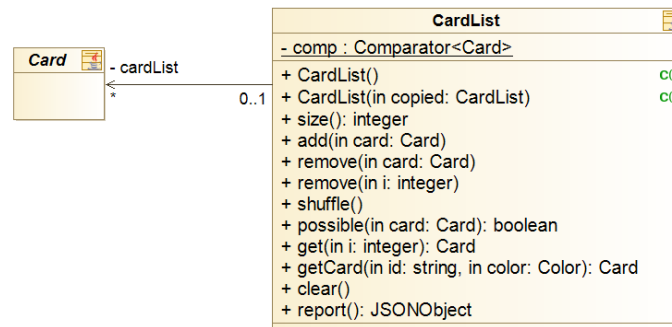
La clase **CardList** recoge las diferentes listas de cartas que puedan ir apareciendo a lo largo del juego, como por ejemplo el montón de robar, la mano de cada uno de los jugadores o la lista de cartas que se han ido lanzando.

Esta clase posee los métodos básicos para una lista: añadir o borrar un elemento, coger el tamaño de la lista o una carta en una posición concreta, pero además, encontramos la implementación del método *shuffle* cuya finalidad es barajar el montón de cartas. Por ejemplo, cuando se terminan las cartas del montón del que se pueden robar cartas, deberemos coger el montón donde se han ido lanzando las cartas, vaciarlo, barajar, llamando a este método, y volver a reponerlo; de esta forma, siempre se podrá robar carta.



Sprint 2

La funcionalidad de *CardList* se mantiene igual.



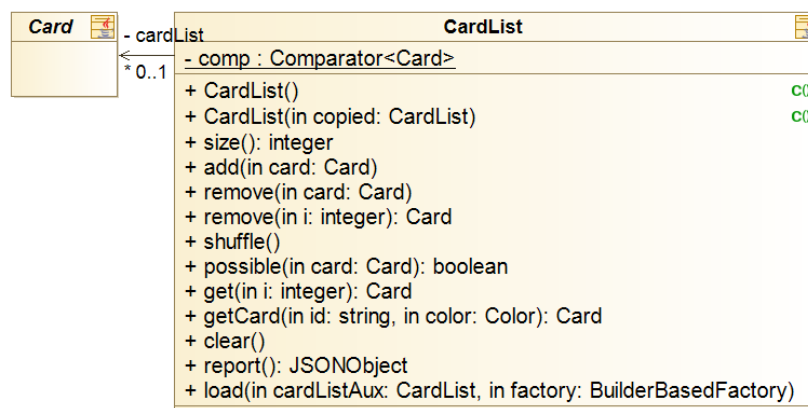
Con la misma finalidad que en la clase *Card*, nos vemos en la necesidad de crear un método *report* en *CardList* para poder guardar los montones de cartas en formato de *JSONObject*. Cada montón posee su identificador, y un *JSONArray* compuesto de las cartas de este, cada carta se guarda con el formato comentado anteriormente. Veamos un ejemplo:

```

{"cardhand":[{"data":{"symbol":"0","color":"BLUE"},"type":"SimpleCard"}, {"data":{"symbol":
:"7","color":"YELLOW"},"type":"SimpleCard"}, {"data":{"symbol":"2","color":"GREEN"},
"type":"SimpleCard"}]}
  
```

Sprint 3

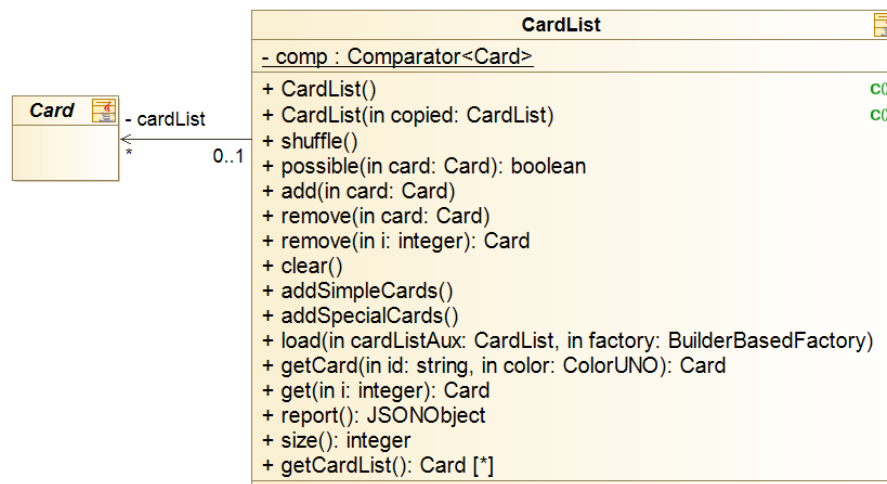
Se añade el método *load*, cuya principal función es poder generar una lista de cartas a partir de un *JSONObject*. Para ello, recorreremos el *JSONArray*, generando para cada posición del array, una carta, y se van añadiendo a la lista.



La funcionalidad de *CardList* se mantiene igual.

Sprint 5

Decidimos distribuir las responsabilidades de la clase *Game*, entre ellas, la generación de las cartas. Ahora, la clase *CardList* se encargará de ello. Como observamos, se añaden los métodos *addSimpleCards()* y *addSpecialCards()*.

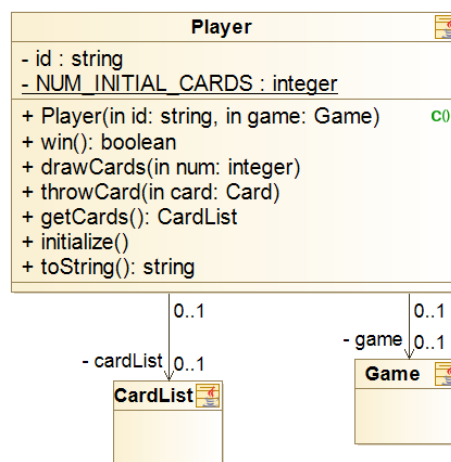


Diseño y evolución de los jugadores

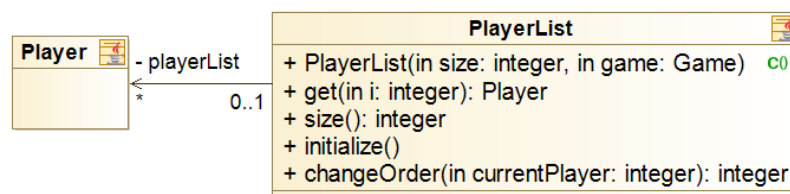
Sprint 1

Se crea la clase **Player** que tiene los atributos:

- *id*: se trata de un string que se utiliza para representar a cada uno de los jugadores, en este sprint, sus identificadores serán simplemente un número del 1 - numJugadores.
- *NUM_INITIAL_CARDS*: es un entero que representa el número de cartas iniciales que tienen todos los jugadores, en nuestro caso, 7.

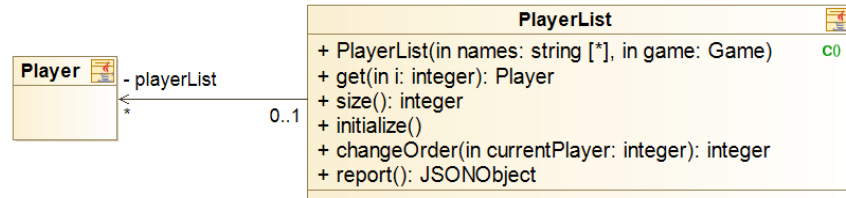


Todos los jugadores están recogidos en una **PlayerList**, esta lista podrá ser reordenada con el objetivo de marcar el sentido de juego que puede cambiar con *ChangeCard* o también para dar el orden en el que se clasifican al final.



Sprint 2

El *id* de cada *Player* pasa a ser el nombre de usuario que el jugador seleccione al inicio de cada partida. No podrá repetirse el identificador entre jugadores de una misma partida.



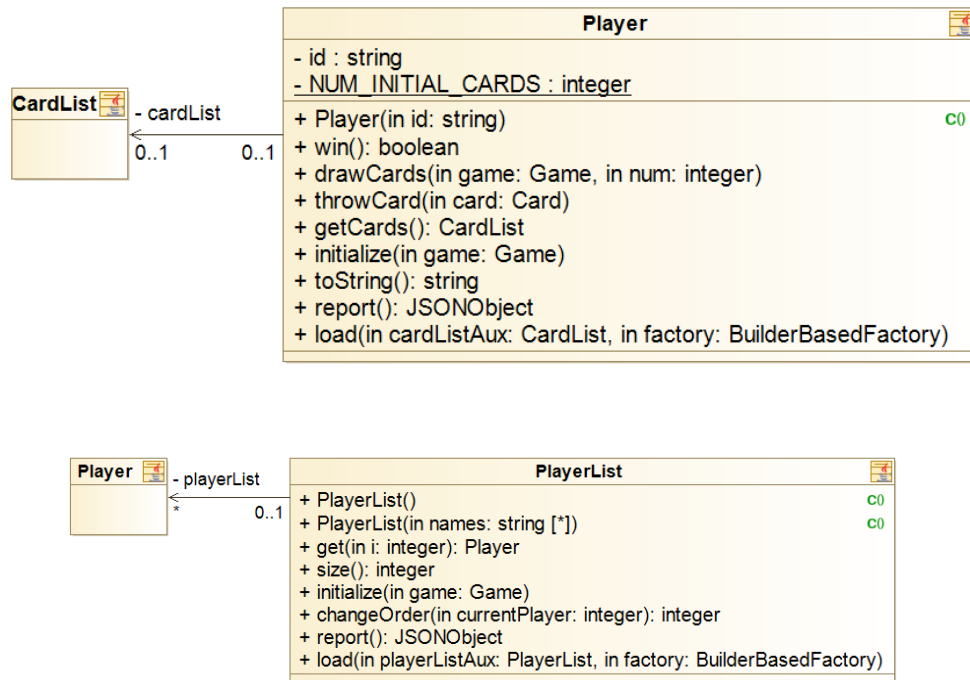
El método *report* de la clase *Player* consiste en guardar el *id* del jugador, y la información de su montón de cartas, tal y como se ha mostrado en el apartado de la clase *CardList* en formato de *JSONObject* de la siguiente manera:

```
[{"cardhand":[{"data":{"symbol":"0","color":"BLUE"},"type":"SimpleCard"}, {"data":{"symbol":"7","color":"YELLOW"},"type":"SimpleCard"}, {"data":{"symbol":"2","color":"GREEN"},"type":"SimpleCard"}], "id":"a"}]
```

Sprint 3

Se añade a las clases *Player* y *PlayerList* una función necesaria para que se realice la parte del comando *load* (para cargar partidas guardadas) que les corresponde.

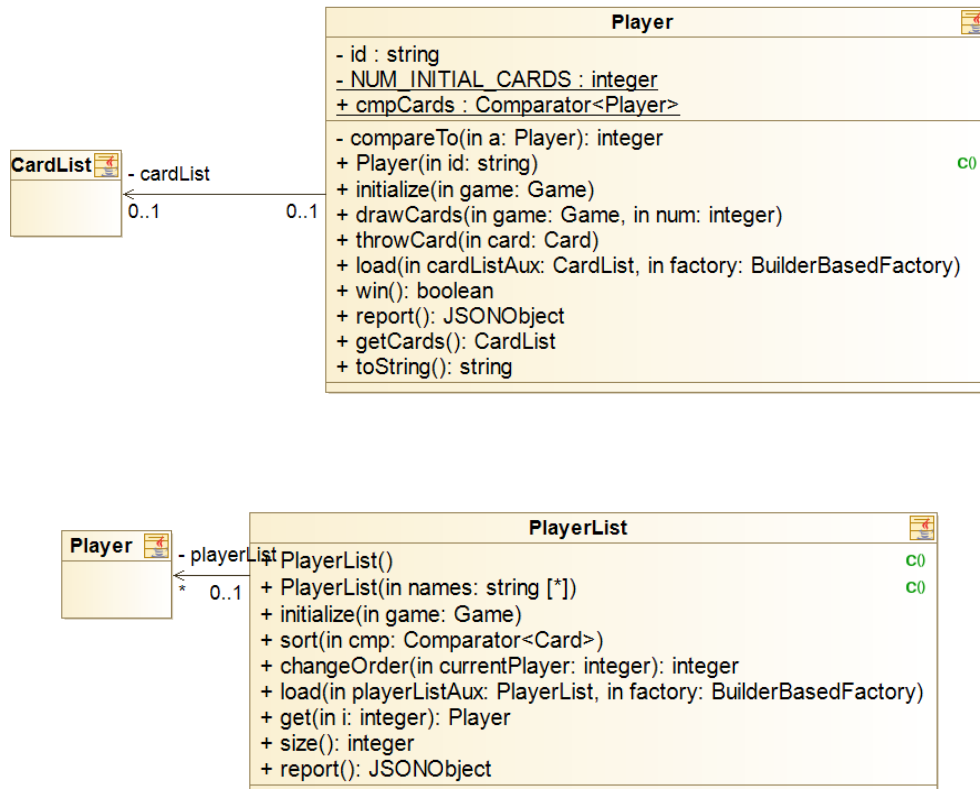
Su funcionalidad no cambia y no se le añaden atributos.



Sprint 4

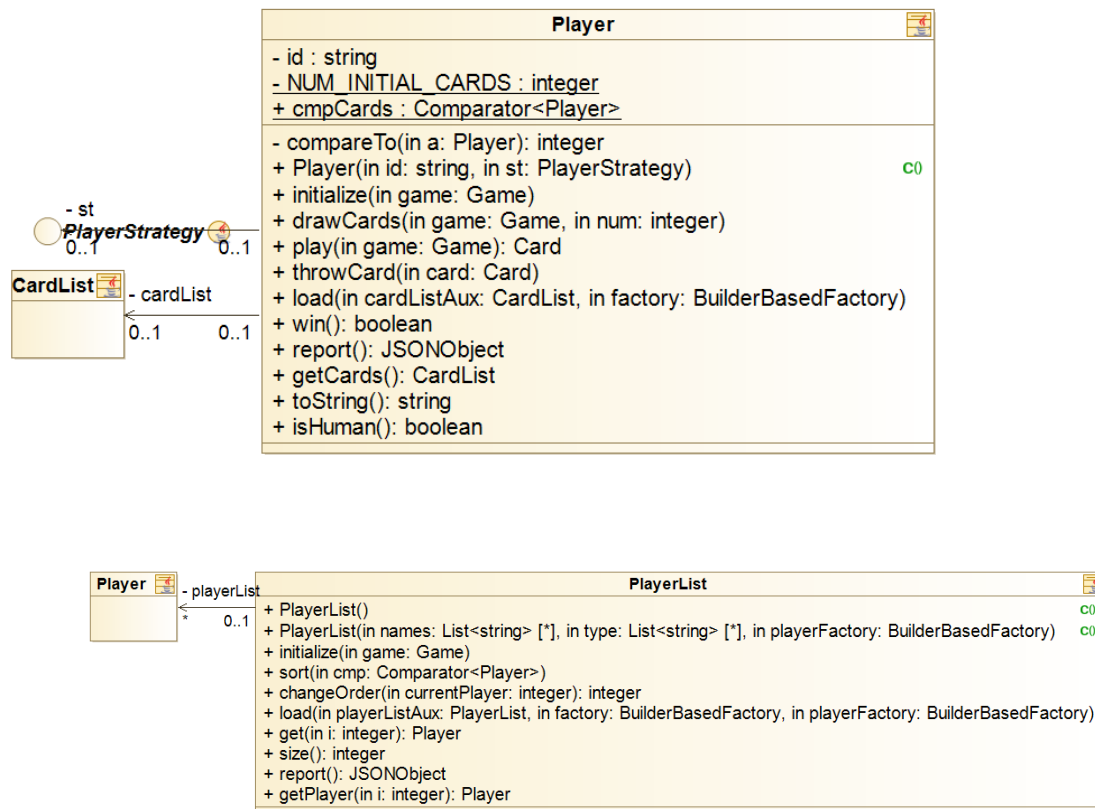
Para poder diferenciar al final del juego el ranking, añadimos a la clase *Player* un comparador, de forma que al finalizar la partida los jugadores quedarán posicionados según el número de cartas de cada uno.

En primer lugar, ganará el usuario que se haya quedado sin cartas, y en los siguientes puestos, el resto de jugadores ordenados por menor número de cartas.



Sprint 5

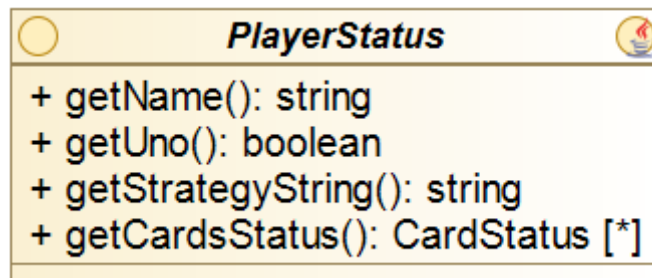
Se añade un atributo a *Player* de tipo *PlayerStrategy*, con opción a ser: *EasyPlayer*, *MediumPlayer*, *HardPlayer* o *HumanPlayer*, dependiendo del tipo de IA que se haya seleccionado al empezar la partida. Cada estrategia implementa el método *play()*, que devolverá la carta que cada jugador/máquina haya decidido lanzar. Se comentará la estrategia que sigue cada jugador en la historia de usuario.



Motivado por la incorporación de las estrategias descritas nos hemos visto obligados a tener que modificar el método *report* de la clase *Player* el cual pasa a tener una nueva componente *type* que indica el tipo de jugador. Este cambio queda reflejado de la siguiente forma: `{"type": "Easy"}` y afecta a todos los métodos *report* que incluyen el *report* de la clase *Player*.

Sprint 7

Se creó el adapter del *Player*, *StatusPlayer*.



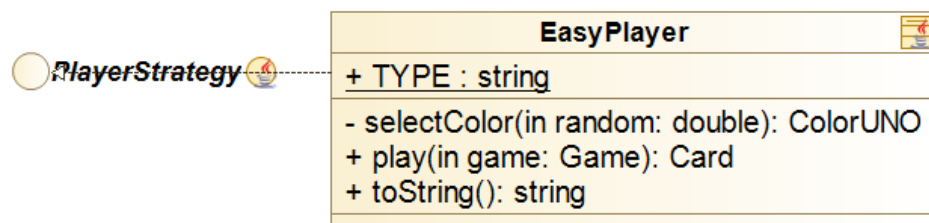
Diseño y evolución de las estrategias

Sprint 5

Creamos primeramente una interfaz **PlayerStrategy** la cual se implementará en las 3 clases destinadas a recoger la lógica de los tres niveles de dificultad: **EasyPlayer**, **MediumPlayer** y **HardPlayer**, éstas tienen como atributo un string que recoge el tipo de estrategia y diferentes funciones que completan su funcionalidad.

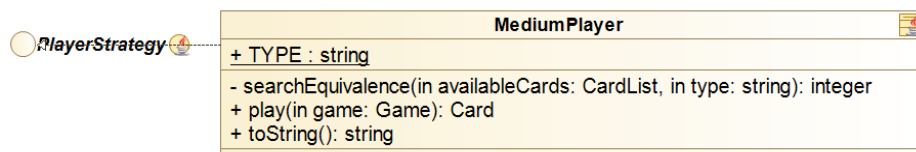
EasyPlayer:

Corresponde con la estrategia de la dificultad fácil. Como ya hemos mencionado, esta clase implementa *Strategy*. Su funcionalidad únicamente se basa en hacer que el bot lance la primera carta que se pueda de la lista, si no puede lanzar ninguna, robará carta hasta que pueda. Atributo de tipo: "Easy".



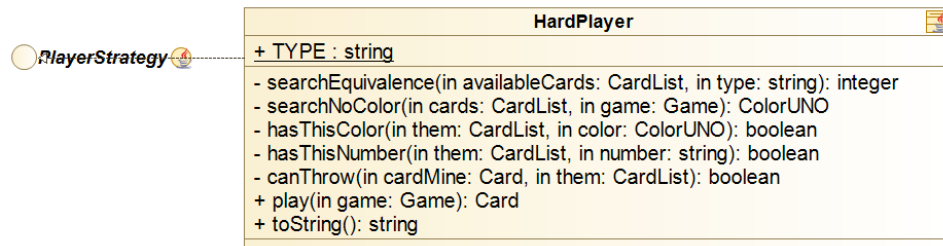
MediumPlayer:

Corresponde con la estrategia de la dificultad media. También implementa *Strategy*. Su funcionalidad es más sofisticada, los bots que implementan esta funcionalidad conocerán el número de cartas del siguiente jugador, por lo que podrán actuar en consecuencia para evitar que ganen cuando le queden pocas. Atributo de tipo: "Medium".



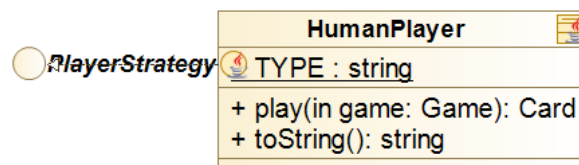
HardPlayer:

Corresponde con la estrategia de la dificultad alta. También implementa *Strategy*. Para hacer realmente difícil este modo de juego, hemos permitido que los bots que implementan esta estrategia, puedan conocer las cartas exactas del siguiente jugador, por lo que lanzarán tratando de conseguir que su contrincante no pueda lanzar. Atributo de tipo: "Hard".



HumanPlayer.

El jugador humano se identificará con el atributo: “Human”. Su funcionalidad no cambiará con respecto a los sprints anteriores.



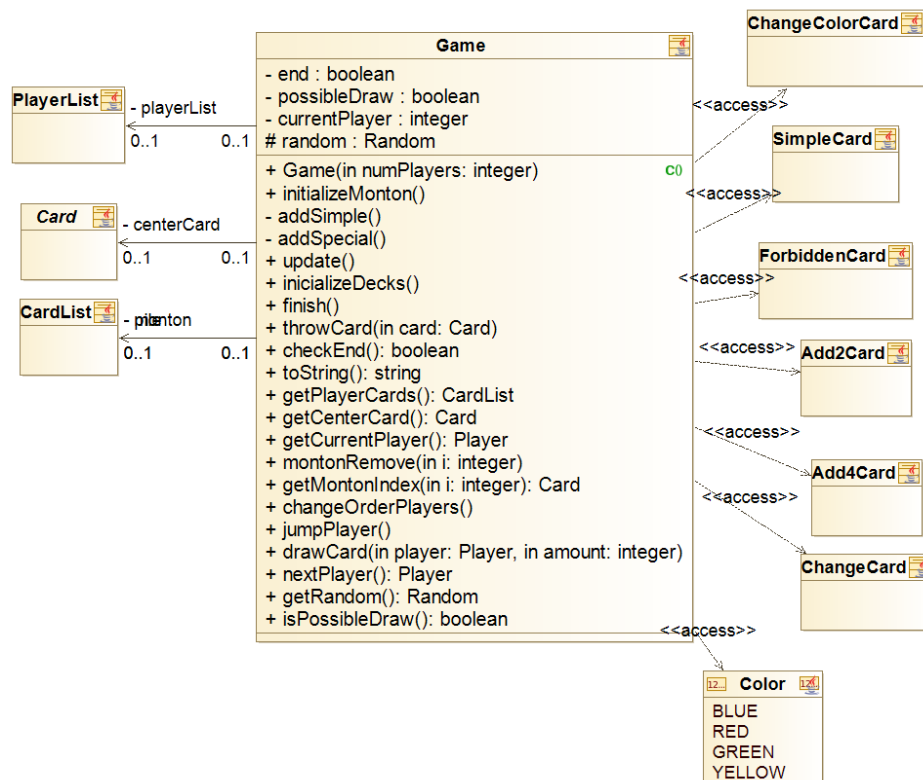
Diseño y evolución del Game

Sprint 1

En la clase *Game* se recogen las principales funcionalidades del juego, donde se conectan las diferentes clases que hemos descrito para realizar acciones útiles para el juego. En cuanto a los atributos del mismo, encontramos los siguientes:

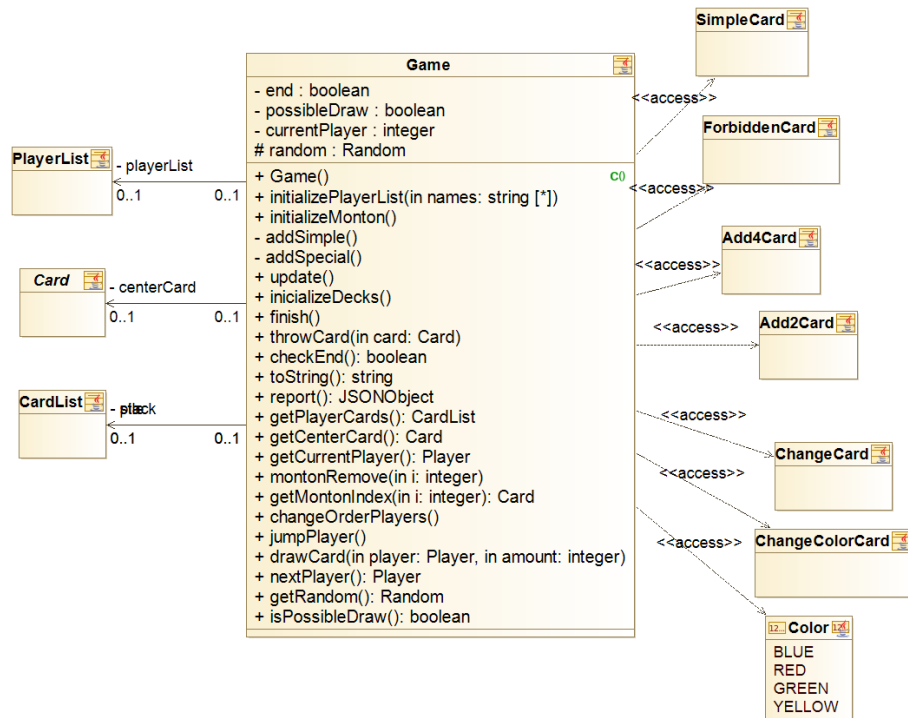
- *end*: se trata de un booleano que indica si el juego se ha terminado o no.
- *possibleDraw*: es un booleano que muestra si el jugador actual puede coger una carta.
- *currentPlayer*: es un entero que indica el jugador actual.
- *random*: es un atributo de tipo *Random* que se usará para barajar las cartas.

En cuanto a los métodos de esta clase, entre los más destacados podemos mencionar los siguientes: por un lado *initializeMonton* se encarga de crear todas las cartas (tanto las normales como las especiales) y barajarlas. *Update* se responsabiliza de que el jugador actual coja o tire cartas y de pasar al siguiente jugador. Además, se encarga de verificar si el juego ha terminado o no. Los métodos *throwCard* y *drawCard* se encargan de tirar y coger cartas respectivamente. Asimismo, *checkEnd* mira si se ha terminado el juego y *finish* lo termina. Además de ciertos *getters*, estos son los métodos más importantes de dicha clase.



Sprint 2

En este sprint las funcionalidades básicas de esta clase se han mantenido intactas. No obstante, se han añadido ciertos métodos que modifican ciertamente la funcionalidad y que ayudan a los cambios del mismo. Entre otros, podemos destacar el método *initializePlayerList* que inicializa el juego con los nombres de los jugadores recibidos.



Como cambio fundamental de este sprint tenemos la funcionalidad de guardar y cargar partidas. Precisamente por eso, nos hemos visto en la necesidad de crear un método *report* en la clase *Game* que devuelva un *JSONObject* con la información esencial del juego.

En esta clase hemos decidido seguir esta estructura de guardar la información: guardamos si el jugador actual puede coger una carta del montón, el identificador del jugador actual, las cartas del montón tal y como se ha descrito en la clase *CardList*, la lista de jugadores, tal y como se ha especificado en la clase *Player*, la pila de cartas tal y como se describe en la clase *CardList*, el identificador de la partida cargada, la carta del centro y si el juego se ha terminado o no, de tal forma que se queda de la siguiente manera:

```

{"possibledraw":true,"currentplayer":0,"stack":[{"data":{"symbol":"X","color":"GREEN"},"type":"ForbiddenCard"}, {"data":{"symbol":"4","color":"RED"},"type":"SimpleCard"}, {"data":{"symbol":"2","color":"GREEN"},"type":"SimpleCard"}, {"data":{"symbol":"0","color":"BLUE"},"type":"SimpleCard"}, {"data":{"symbol":"7","color":"YELLOW"},"type":"SimpleCard"}, {"data":{"symbol":"2","color":"GREEN"},"type":"SimpleCard"}], "players": [{"cardhand": [{"data":{"symbol":"0","color":"BLUE"},"type":"SimpleCard"}, {"data":{"symbol":"7","color":"YELLOW"},"type":"SimpleCard"}, {"data":{"symbol":"2","color":"GREEN"},"type":"SimpleCard"}], "id": "a"}, {"cardhand": [{"data":{"symbol":"4","color":"GREEN"},"type":"SimpleCard"}, {"data":{"symbol":"+4+","color":"BLUE"}]}]}
  
```

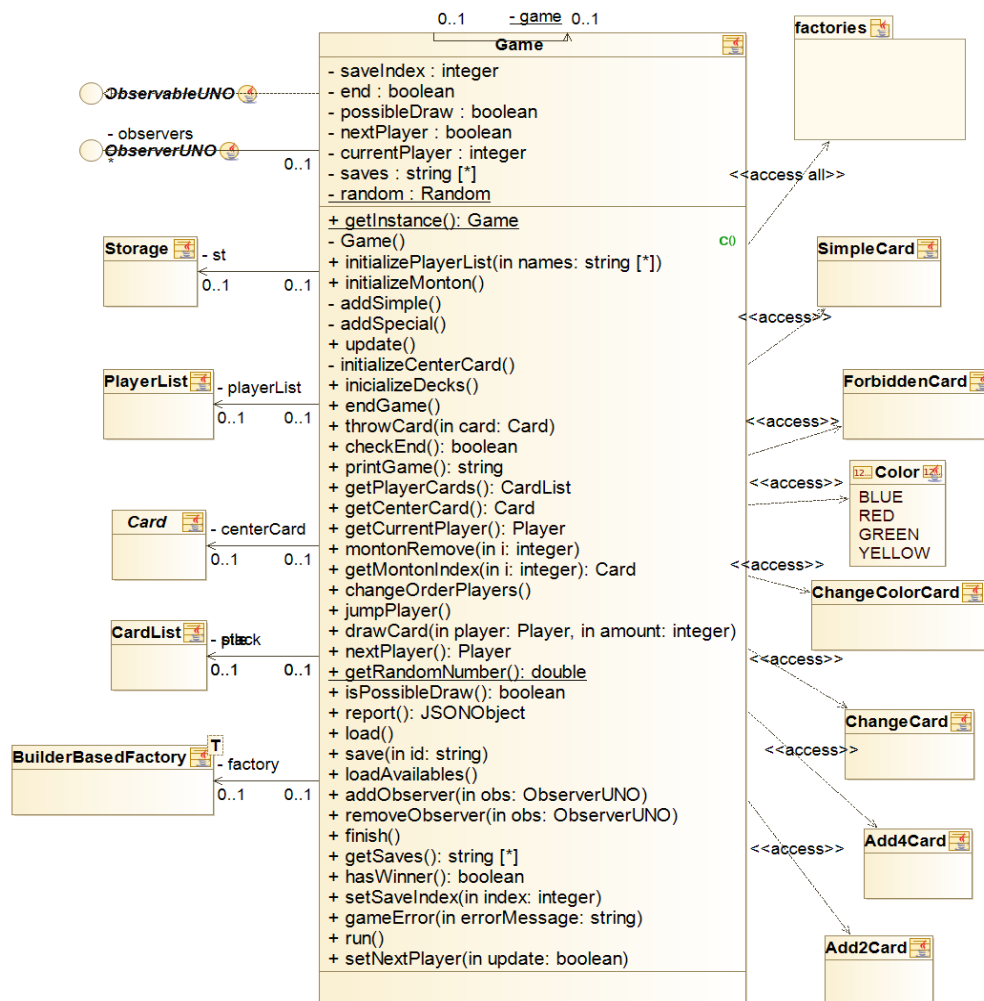
```
ACK"},"type":"Add4Card"},{"data":{"symbol":"1","color":"BLUE"},"type":"SimpleCard"},"id
":"b","type"},"pile":[{"data":{"symbol":"9","color":"GREEN"},"type":"SimpleCard"},"id":"tes
t5","centercard":{"data":{"symbol":"9","color":"GREEN"},"type":"SimpleCard"},"status":fals
e}
```

Sprint 3

En este sprint el cambio más destacado dentro de la clase game es la introducción del atributo (*st: Storage*) y de la introducción de las factorías.

Este primer cambio surge debido a la necesidad de abstraer el guardado y la carga de la partida a través de una clase externa que se encargue de relacionar el almacenamiento interno del equipo del usuario con los *JSON*, formato de almacenamiento de datos, que guardan la información referente al estado de la partida. Gracias a esta abstracción *Storage* es posible cambiar fácilmente la forma en que se almacena el estado de la partida.

En segundo lugar, la introducción de las factorías nos permite utilizar de nuevo una abstracción que permita transformar información en un cierto formato, en nuestro caso *JSON* de nuevo, en los objetos *Cards*. Esto es utilizado a la hora de cargar las partidas guardadas.



Sprint 4

En este sprint se llevaron a cabo cambios con el objetivo de adaptar la estructura del programa al patrón MVC. Por otro lado, también se introdujo la idea del patrón *Singleton* que permitió la existencia de una única instancia de *Game* en todo el programa.

El patrón MVC se basa en gran medida en la existencia del patrón *Observador*, es decir, un elemento del programa llamado *Observable*, aquel que es “observado”, envía “notificaciones” a otros elementos que se llamarán *Observer*, aquellos que “observan”. La idea principal consiste en que debido a que los *Observers* no necesitan más que ver, y no modificar directamente, al *Observable*, este último, a través de las notificaciones, envía la información necesaria para que los *Observers* se actualicen acordemente. Este patrón facilita la comunicación al hacerlo de una manera abstracta a través de interfaces.

En el caso de *Game*, esta clase implementa a la interfaz *Observable* por lo que tendrá una lista de *Observers* cuyos métodos serán llamados en el momento en que ocurra una actualización en el estado del juego para que puedan realizar los cambios que sean necesarios. En un principio, los *Observers* serán los elementos que componen la Vista, pero más adelante se modificará con la introducción del juego en línea.

Por otro lado, el patrón *Singleton*, a pesar de no ser imprescindible, si que actúa como “seguro”. Ya que durante la ejecución de la partida todos los jugadores deben tener la misma información referente a esta, es decir, todos “tienen” el mismo juego consideramos que sería prudente forzar esto. Lo que quiere decir esto es que a través del patrón *Singleton* se imposibilita a todos los elementos del programa la creación de una nueva instancia de *Game* manteniendo su estado en todo momento.

Diseño y evolución de las clases principales del modelo

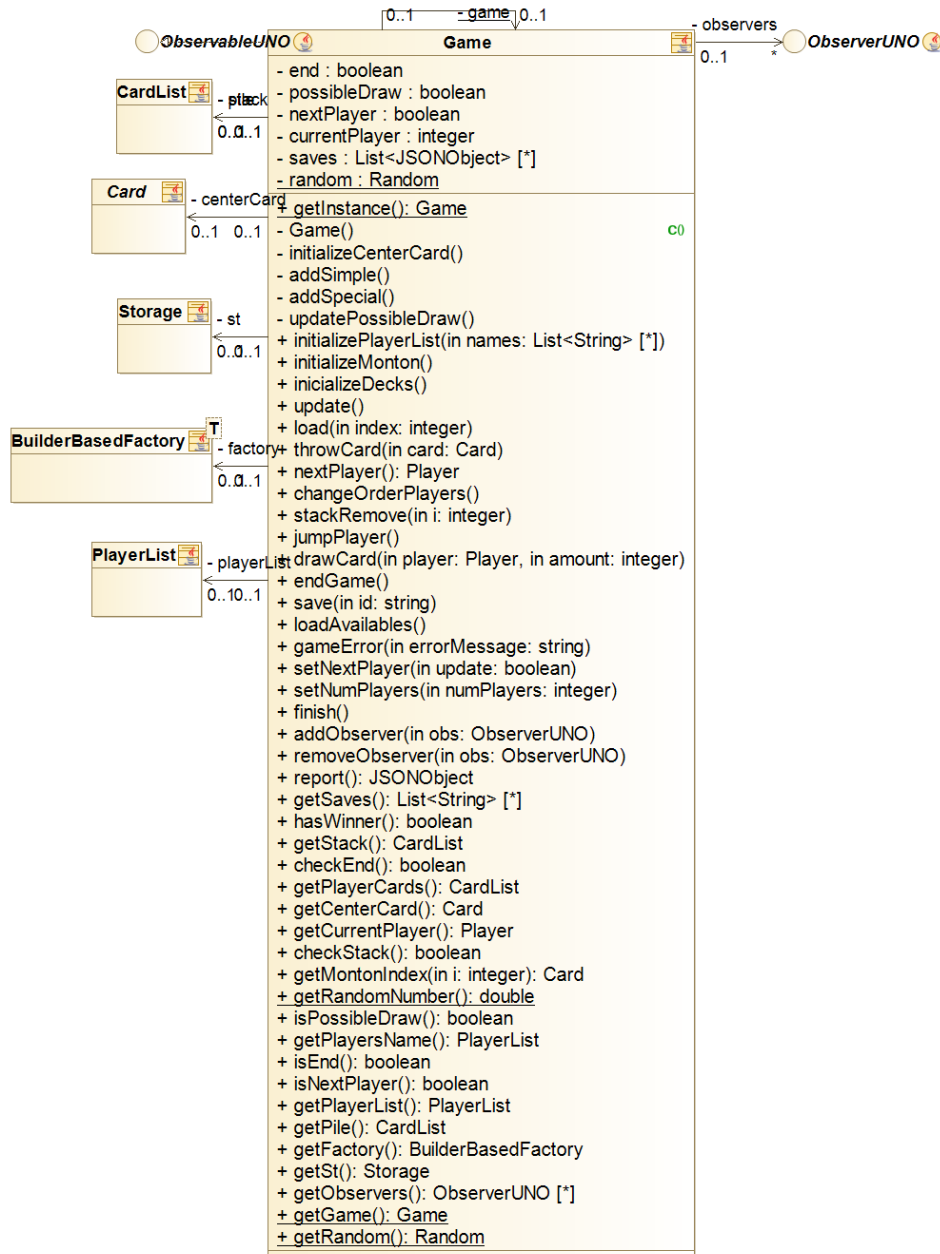


Diagrama de clases del sprint 4 de la clase Game.

Sprint 5

Con este nuevo sprint la clase *Game* sufrió cambios encaminados en dos direcciones principales: la introducción de las Inteligencias Artificiales (IAs) y la rebaja del tamaño de la clase.

Empezando por el final, en este punto del proyecto el trabajo realizado hasta el momento cumplía ampliamente su cometido, sin embargo, ciertas partes realizadas al comienzo no habían sido cuestionadas de nuevo. Este era el caso de ciertos métodos de *Game*.

Desde el inicio del trabajo, una gran parte del peso del modelo había acabado recayendo sobre esta clase, no obstante, tras tantas nuevas características del juego la clase había adquirido un tamaño excesivamente grande como para tener un control absoluto sobre la misma. Es por esta razón que, se decidió repartir ciertas de sus responsabilidades entre otras clases del modelo, principalmente entre los *gameobjects*. Con esto se eliminaron los *addCards*, *initializePlayerList* y ciertos *getters* y *setters*.

El otro gran cambio fue la aparición de las IAs. En este primer momento, nos decidimos por incluir las IAs como parte del modelo y no hacer que actuaran como si fuesen otros usuarios. Es por esto que se debió modificar el método *update* para reflejar esta idea, además de que se añadió una nueva notificación para el caso en que la IA tome una decisión sobre qué jugada realiza.

Finalmente, y en segundo plano, se encuentra la introducción de la factoría de estrategias de jugador que es utilizada al cargarlos.

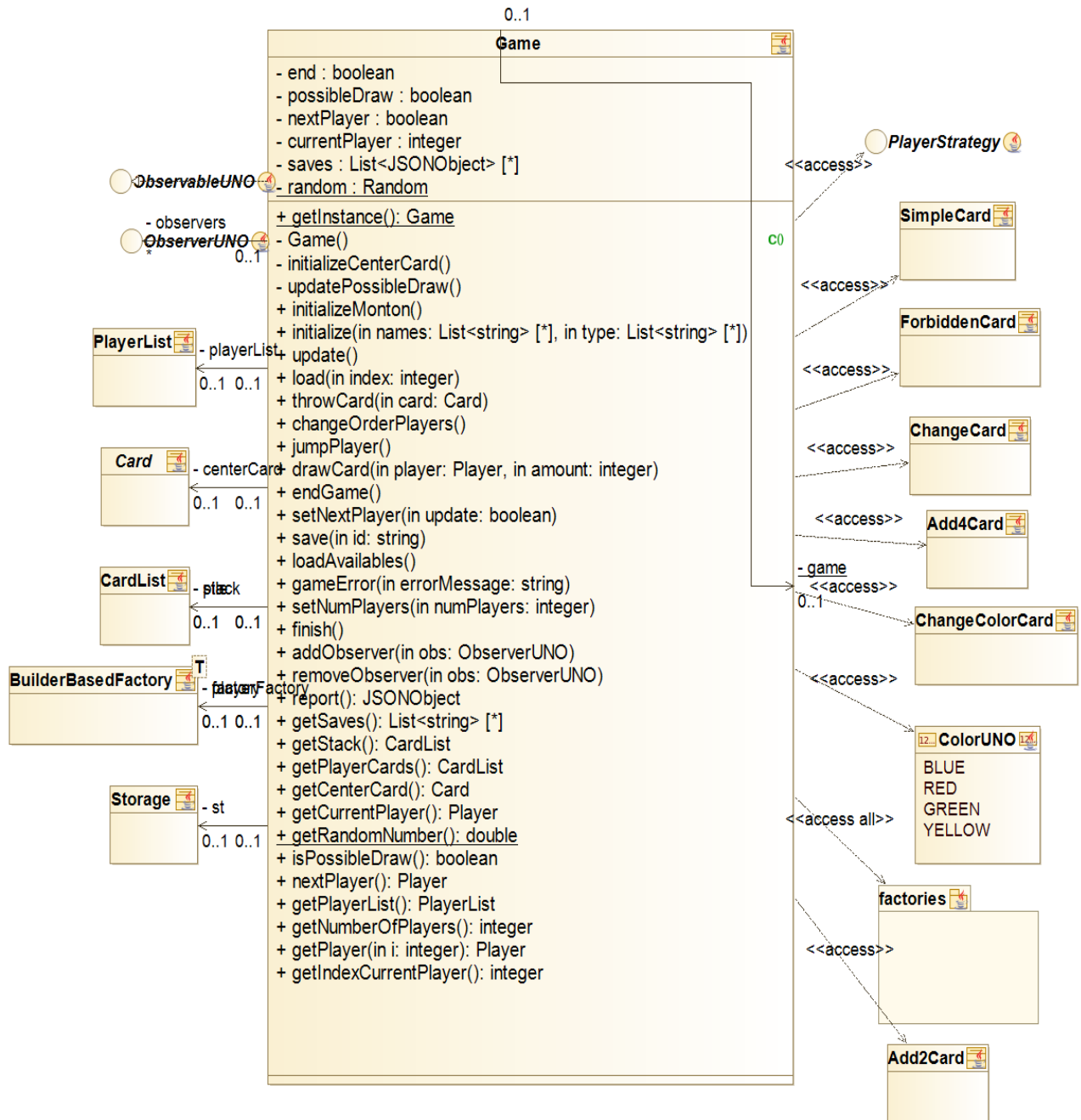


Diagrama de clases del sprint 5 de la clase Game.

Sprint 6

En este sprint la clase se mantuvo en su mayor parte igual al anterior. Solo cabe destacar un par de detalles.

Por un lado, con la introducción del juego en red se hizo necesario que *Game* pudiese ser transmitido del servidor a los clientes. Sin embargo, esta comunicación sólo es posible si el mensaje transmitido es serializable, es decir, se hizo necesario permitir la serialización de esta clase. En su mayor parte esta labor fue trivial, aunque sí que fue necesaria la introducción del atributo *transient*, que deja a dicho atributo fuera de la serialización, en ciertos momentos: con las factorías, el *Storage*, los observadores y la lista de partidas guardadas.

Por otro lado, se introdujo en el juego la funcionalidad del botón UNO por lo que se debieron modificar en cierta medida los métodos *update* y *updatePossibleDraw* para tener en cuenta esta nueva funcionalidad.

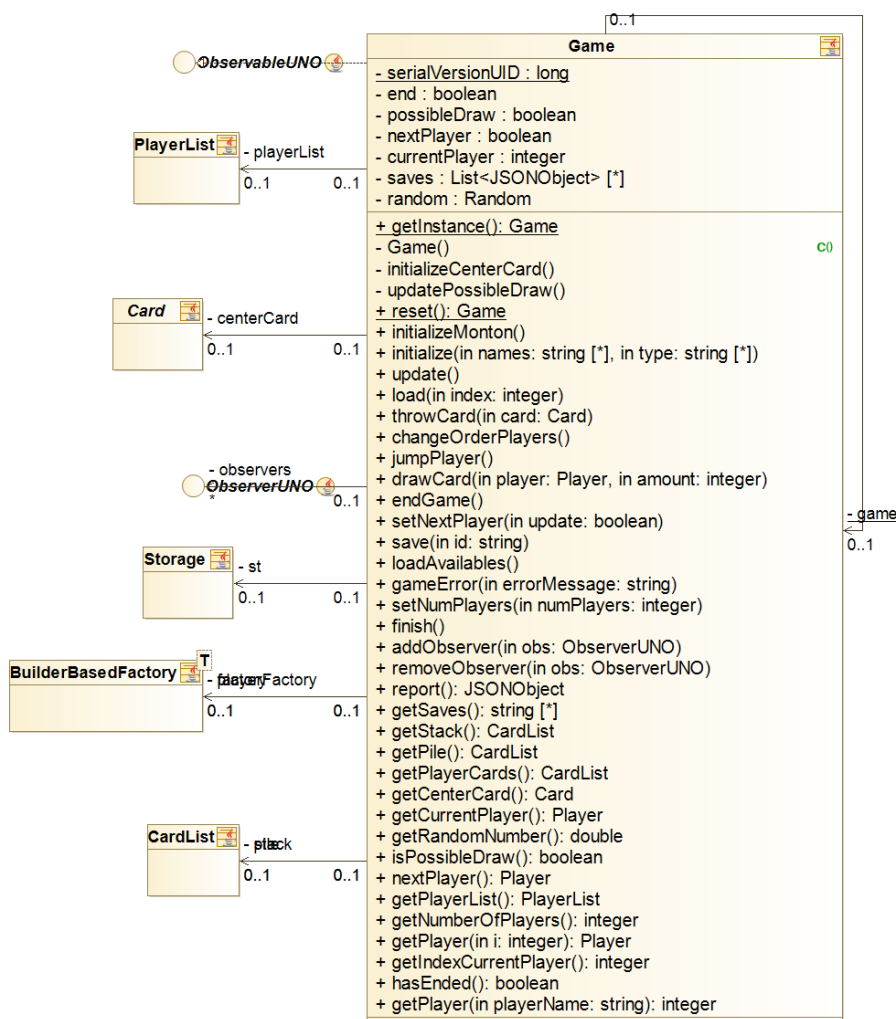


Diagrama de clases del sprint 6 de la clase *Game*.

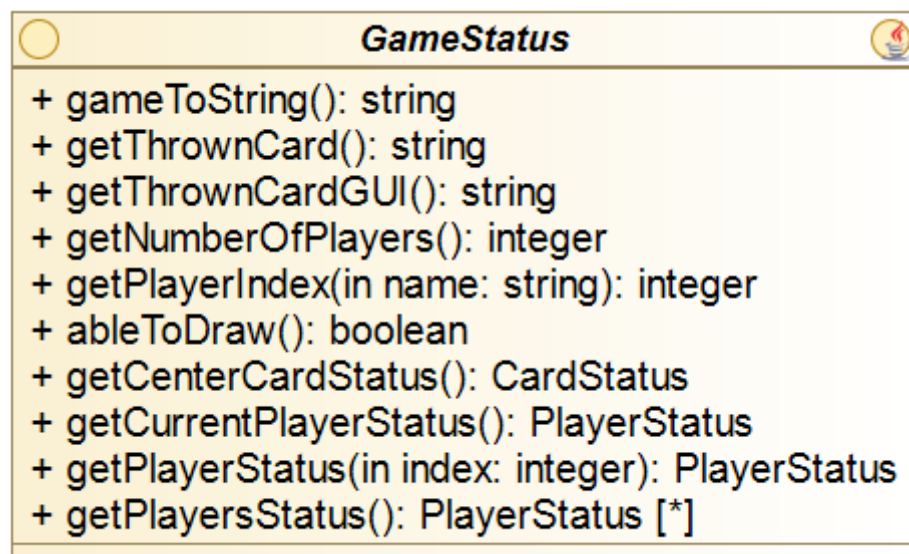
Sprint 7

En la clase `Game`, hemos añadido 2 atributos a destacar: *jumpPlayer*, que se encarga de la nueva gestión tras lanzar una carta encargada de saltar a un jugador, esta gestión es sencilla y consiste en que si se lanza la carta, *jumpPlayer* será cierto, y no saltará al siguiente jugador, si no, al que le toca en dos turnos; por otro lado, el atributo *thrownCard*, para interpretar si el usuario ha lanzado carta o ha robado carta.

Tuvimos que añadir estos atributos tras la adición del método *cycle()*, y corrección del *update()*. Se comentarán estos métodos en la historia de usuario uno.

También añadimos la funcionalidad de que una vez el juego haya terminado se pueda volver a empezar una partida sin necesidad de volver a compilar el juego, y por ello la aparición del método *reset()* del `Game`, que crea una nueva instancia del juego.

Además, hemos creado el adapter del *Game* denominada *GameStatus*, para minimizar las comunicaciones y dependencias de los subsistemas, en concreto, añadir esta interfaz nos permite liberar y descargar la dependencia entre el modelo y la vista.



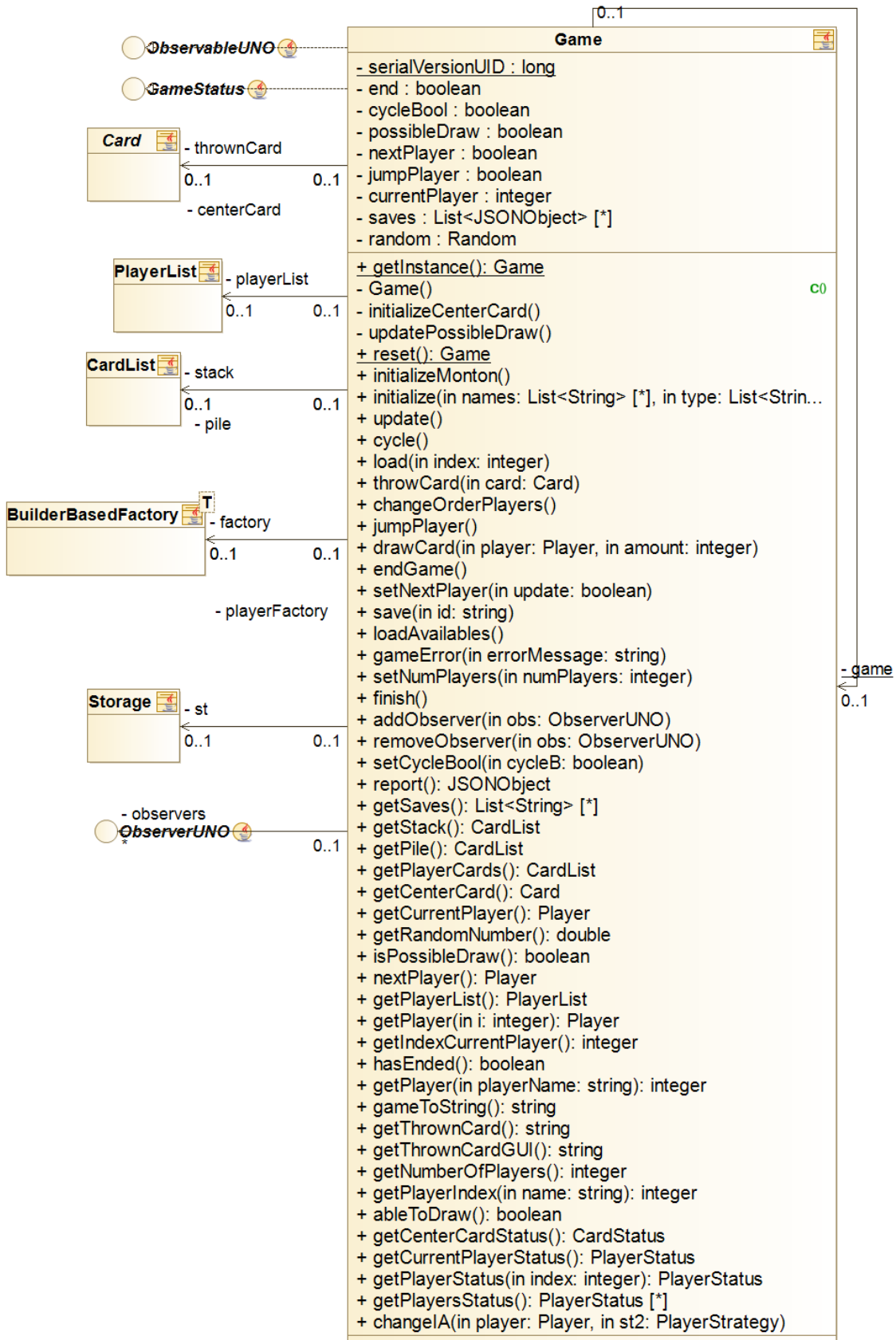
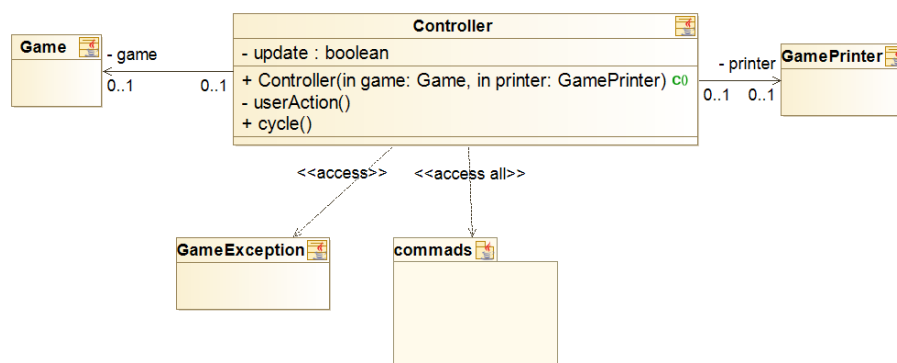


Diagrama de clases del sprint 7 de la clase Game.

Diseño y evolución del Controller

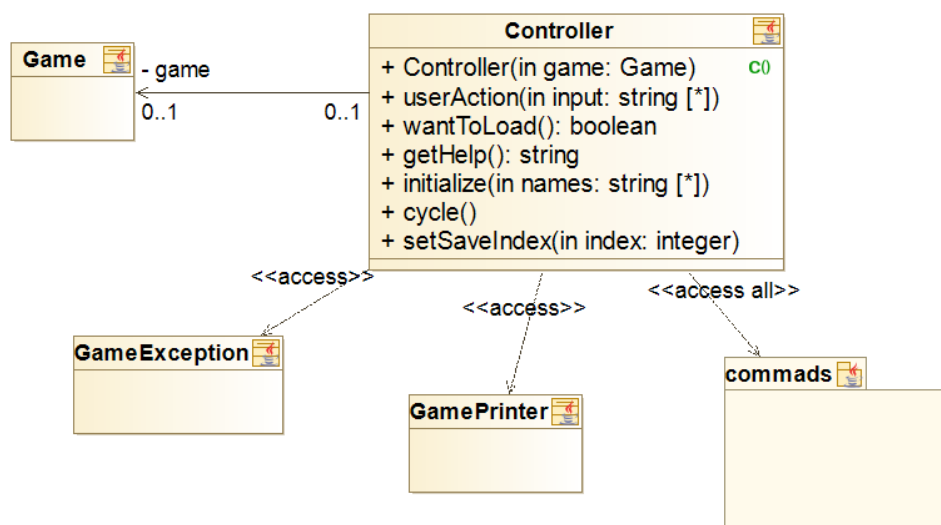
Sprint 1

El *Controller* posee un *Game*, y un *GamePrinter*, es decir, el modelo y la vista. Consiste en un ciclo. En primer lugar, se *inicializa* el juego (las cartas de cada jugador y el montón de robar), y comienza un bucle que no termina hasta que el juego no se haya terminado. Este bucle consiste en preguntarle al usuario que acción desea realizar (comando), interpretarlo, y realizarlo.



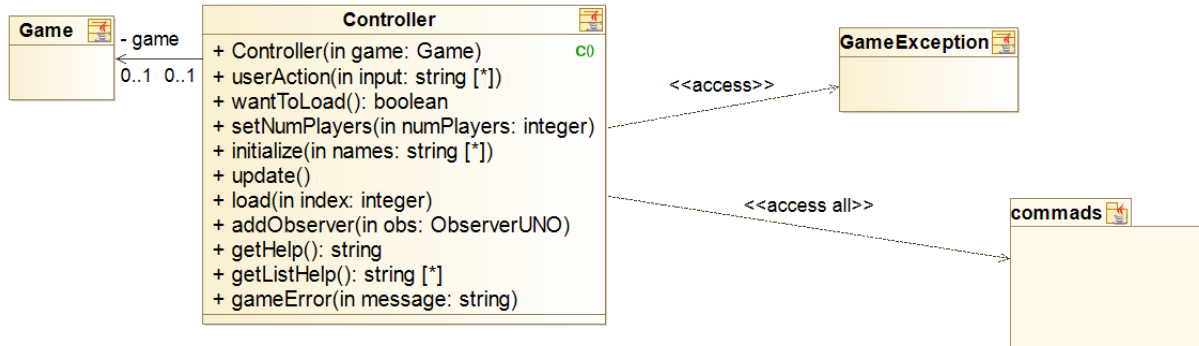
Sprint 3

Se modifican los argumentos de la clase *Controller*, ahora solo tiene un *Game*. Quitamos la lógica del bucle del juego, y lo transportamos a la clase *Game*. Creamos el método *wantToLoad*, cuya principal finalidad es encargarse de guardar partida.



Sprint 4

Debido a la creación de los observadores, añadimos un método que sirva para añadir observadores al Game. En este caso, será o *GamePrinter* o las clases relacionadas con la interfaz gráfica.



Sprint 6

Debido a la introducción del juego en línea el controlador debió sufrir importantes cambios en la búsqueda de dos objetivos: mantener la anterior funcionalidad sin tener que realizar numerosos cambios en el resto del código y ampliarla con la intención de acomodarla al nuevo juego en línea. Por esta razón, nos decidimos por abstraer la idea de controlador a una interfaz *Controller* y crear un nuevo controlador que se encargará de la comunicación con el servidor.

Con la nueva interfaz *Controller* se abstrajeron las principales funcionalidades del controlador, principalmente controlar la acción del usuario y añadir observadores. Gracias a esta abstracción no fue necesario realizar numerosos cambios en los elementos de las vistas lo que permitió generalizar el comportamiento de la aplicación al caso de que fuese un juego en línea o local.

Por otro lado, la nueva clase *ControllerNet* realmente actúa como el cliente del sistema, es decir, se encarga de la comunicación entre el servidor y el programa ejecutado por el usuario. Esta comunicación se lleva a cabo gracias a los sockets que consisten básicamente en dos elementos principales, un flujo de entrada y otro de salida. El servidor posee una de estas parejas mientras que el cliente posee otra inversa. Esto quiere decir que lo que escribe el servidor en su flujo de salida es leído por el cliente en su entrada y viceversa. Debido a que estos flujos simplemente son de información en bits todo objeto serializable es transmisible por este medio. Gracias a esta propiedad ha sido posible que los clientes reciban la información referente a las notificaciones que tienen que enviar a su vista directamente del servidor y hayan podido enviar al servidor la acción del usuario en un formato muy simple.

El cliente, como ya hemos dicho anteriormente, simplemente se encarga de recibir y enviar información con el servidor, pero como el servidor puede enviar información en cualquier momento es de vital importancia que el cliente se encuentre en todo momento “escuchando” por si esto ocurriese. Esto se traduce, irremediablemente, en que el cliente posee dos hilos de ejecución, como mínimo, uno encargado del envío y otro del recibo.

El cliente recibe información únicamente cuando debe actualizar su estado, en otras palabras, cuando debe notificar a sus observadores de un cambio. Por esta razón lo que el servidor envía consiste en dos partes. En primer lugar, se produce un envío del tipo de notificación que ha ocurrido. Tras esto, se envía la información necesaria y referente a dicha notificación para que el cliente pueda actualizarse acordeamente.

Diseño y evolución de las clases principales del modelo

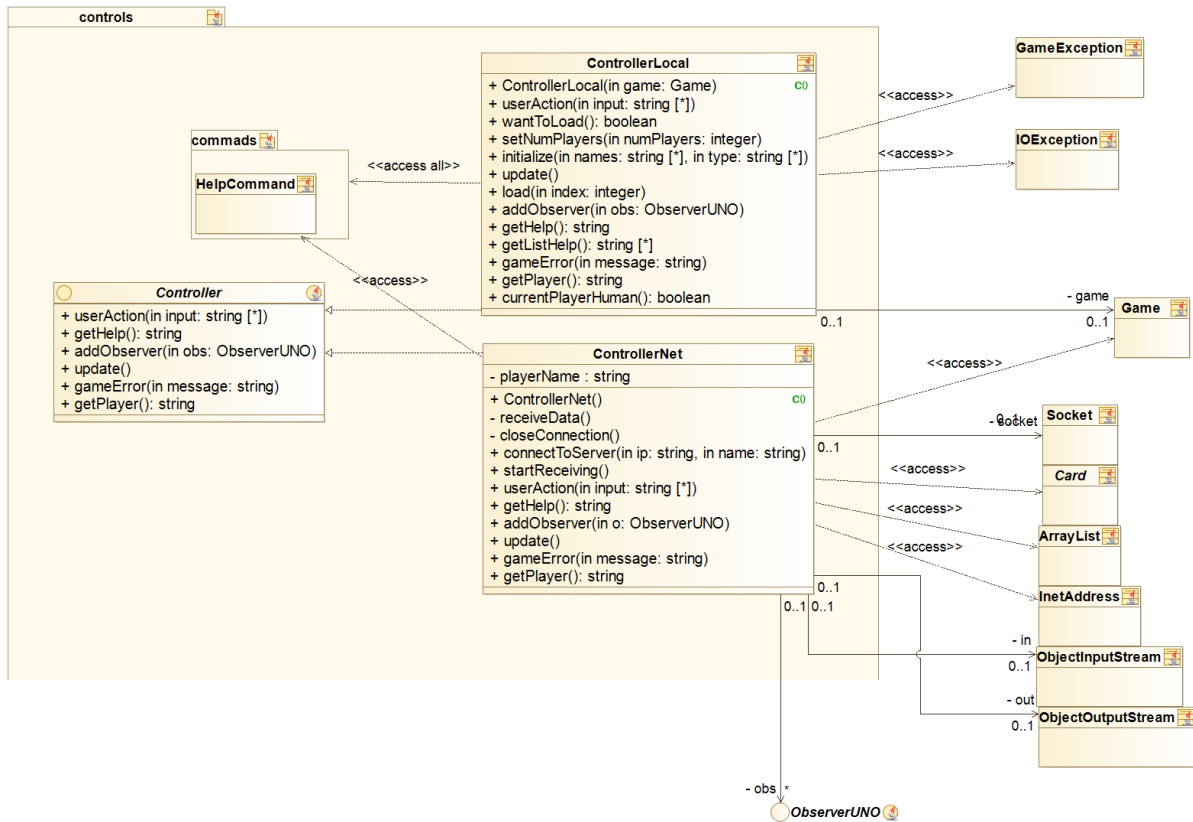
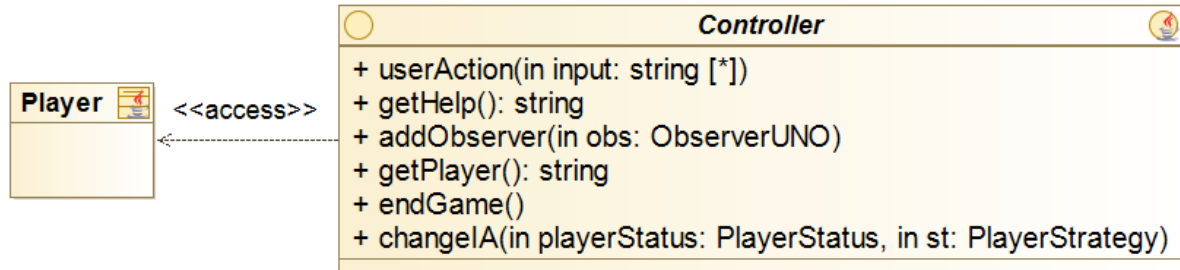


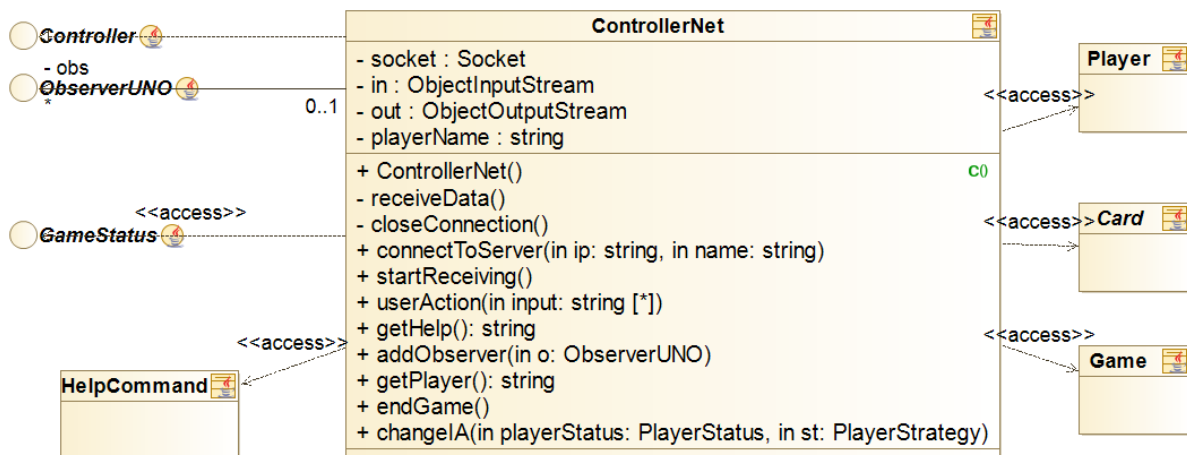
Diagrama de clases del sprint 6 del paquete controls.

Sprint 7

En la interfaz *Controller* se ha eliminado el método *GameError* y se han añadido *EndGame* y *ChangeAI*.



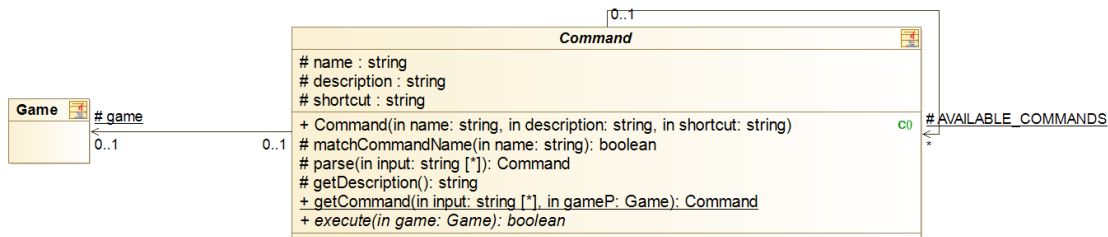
En *ControllerNet* hemos eliminado el *notifyonAI*, pues consideramos que era un error de diseño tener que distinguir entre cómo notificar a las inteligencias artificiales y a los humanos. Mejoramos el código respecto al diseño para que solo haya un único notify.



Diseño y evolución de los comandos

Sprint 1

Poseemos una clase padre abstracta denominada *Command* de la que todos los comandos heredarán.



Cada comando presenta un identificador para así poder ser interpretado. Poseen dos funciones principalmente: parsear el comando introducido por consola y, si es correcto, ejecutarlo. Los comandos implementados en este sprint son:

- *DrawCardCommand*: Sirve para robar una carta del montón, pero solo se puede robar si quedan suficientes. Su identificador es: "d".

```

PLAYER 1: -> 9 9 7 2 3 0 4 0 -> 4 5 8 5 0 C 0 0
Todavía se pueden sacar 78 cartas.
Acción > d
+4
PLAYER 2: 3 6 5 3 8 3 1 +4 2 9 7 0 3 6
Todavía se pueden sacar 77 cartas.
Acción >
  
```

- *EndCommand*: Su funcionalidad es terminar la partida en el estado actual. Su identificador es: "f".

```

PLAYER 2: 3 6 5 3 8 3 1 +4 2 9 7 0 3 6
Todavía se pueden sacar 77 cartas.
Acción > f
El juego ha finalizado.
  
```

- *ThrowCardCommand*: Consiste en lanzar una carta. Para ello, el usuario deberá introducir el identificador de la carta y su correspondiente color. En caso de que sea *ChangeColorCard* o *Add4Card*, en vez de introducir el color, deberá indicar el color al que desea cambiar. Si el jugador no posee cartas que se puedan lanzar, se le obligará a introducir un nuevo comando. Su identificador es: "l".

PLAYER 1: 7 -> 9 9 7 2 3 0 4 0 -> 4 5 8 5

Todavía se pueden sacar 82 cartas.

Acción > l 7 green

|7

PLAYER 2: 3 6 5 3 8 3 1 +4 +4 2 9 7 0 3 6

Todavía se pueden sacar 82 cartas.

Acción > l +4 yellow

|+4

- *HelpCommand*: Este comando tiene como objetivo mostrar un menú de ayuda al usuario, para que así pueda conocer las posibles acciones que puede realizar. Su identificador es: "h".

Acción > h

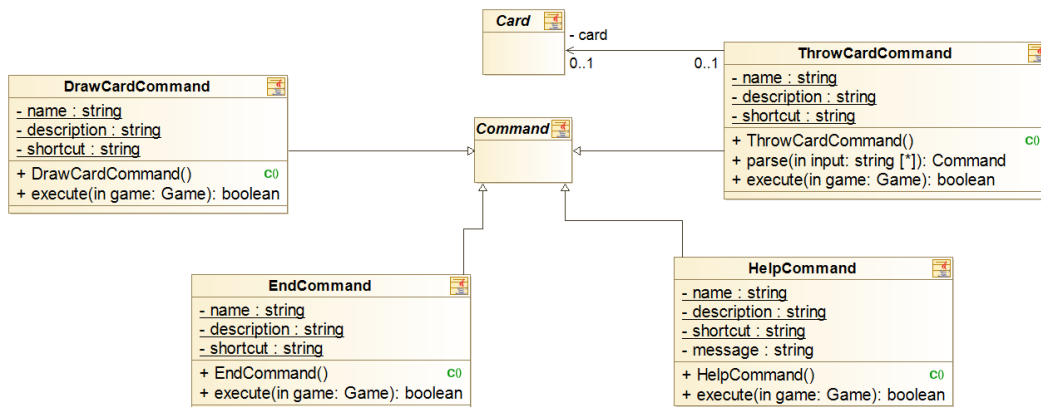
Comandos posibles:

LANZAR [l]: Lanza la carta seleccionada. Argumentos: id color

FINALIZAR [f]: Finaliza la partida en el estado actual

DRAW [d]: Roba una carta del montón

HELP [h]: Enseña el menú de ayuda



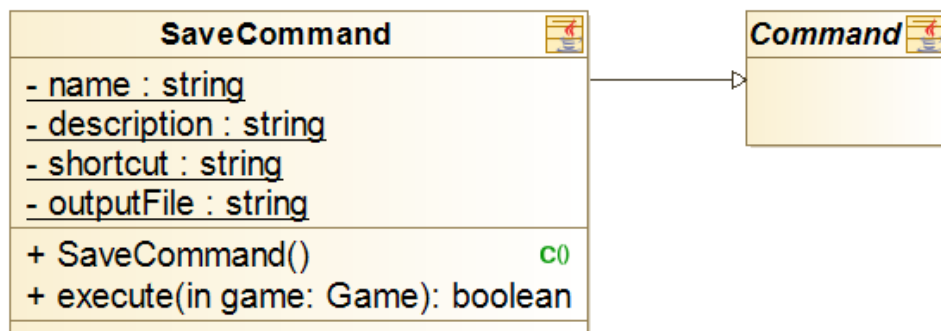
Sprint 2

El comando *ThrowCardCommand* cambia su identificador de “l” (lanzar carta) a “t” (throw Card), para mantener la uniformidad con el idioma (inglés).

```
8
MARIA: +2+ 6 C 8 2 0 3
You can still draw 84 cards.
Action > t 2 yellow
```

Creamos un nuevo comando, denominado *SaveCommand*, que permite al usuario guardar una partida en mitad de la misma, para así poder retomarla en este mismo punto en cualquier momento. Su identificador es: “s”.

```
LAURA: 7 7 7 7 C 9 9
You can still draw 98 cards.
Action > s
7|
```



Se muestra un nuevo menú de ayuda, traducido al inglés, y ampliado con el comando *Save*.

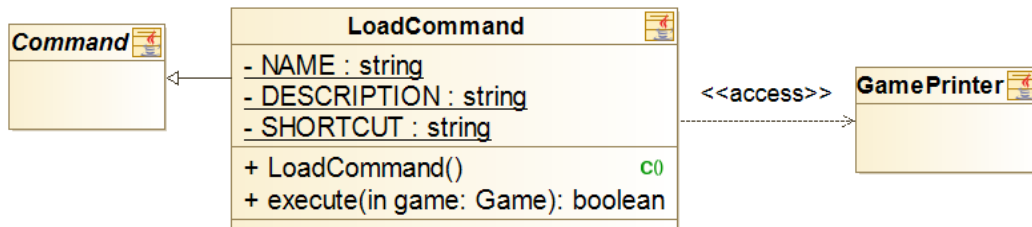
```
Action > h
Possible commands:
THROW [t]: Throws the selected card. Arguments: id
FINISH [f]: Finish the game in the current state
DRAW [d]: Draws a card from the stack
SAVE [s]: Saves the current game
HELP [h]: Shows the help menu
```

Sprint 3

Creamos un nuevo comando, denominado *LoadCommand*, que se trata del inverso del *SaveCommand*, es decir, sirve para cargar una partida anteriormente guardada.

Su funcionalidad no es exactamente igual a la de otros comandos, debido a que solo se puede invocar antes de empezar una partida.

Welcome to UNO
Do you want to load a save?
yes



Sprint 4

Borramos el comando *LoadCommand*. Su lógica se mantiene igual, pero ya no se trata de un comando, pues no actúa como uno de ellos (solo se puede usar al principio de la partida).

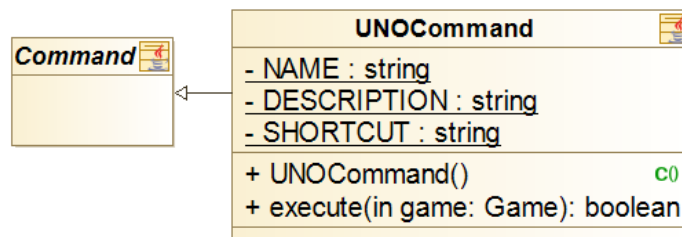
Principalmente, gestionamos esta lógica en *Storage*, una clase creada para gestionar cuando cargamos o guardamos una partida, además de, *Game*, y *Controller*.

El comando *HelpCommand* pasa a tener sentido sólo si se juega en consola, pues por interfaz gráfica, no hay comandos a introducir, solo botones a pulsar.

Sprint 6

En un principio, para implementar la funcionalidad del botón del UNO, codificamos toda la lógica de manera individual, como un botón que simplemente cambiaba un atributo *boolean* de cada jugador que lo pulsase teniendo dos cartas.

Sin embargo, luego nos dimos cuenta de la similitud que tenía esta acción con la de robar cartas del montón central que teníamos como *DrawCardCommand*, por tanto decidimos crear un comando para esta situación, denominado *UNOCommand*.



Diseño y evolución de las historias de usuario

En este apartado del documento se registrarán y explicarán el diseño y la evolución de las historias de usuario por sprints. Además, se comentará el tiempo dedicado a cada historia de usuario, así como si se ha podido completar en el sprint que se esperaba.

Cada historia de usuario presenta: una explicación de la misma; un diagrama de clases, con las clases implicadas en la realización de la misma; y, un diagrama de secuencia, donde veremos la ejecución y la interacción entre los objetos.

1. Como usuario quiero jugar al UNO

Esta historia de usuario es la épica, pues engloba a todo el juego. Consiste en que el usuario pueda jugar al UNO, para ello, debe poseer contrincantes, poder lanzar y robar cartas, y coronarse ganador.

El juego del UNO es muy sencillo de jugar. Cada jugador comienza el juego con 7 cartas.

Debe haber siempre una carta en el centro, y un usuario podrá lanzar una de sus cartas, si esta coincide en color y/o símbolo con la actual del centro. Si la carta a lanzar es un cambio de color o un “chupate” cuatro, el usuario deberá escoger el nuevo color de la partida. Con cualquiera de los chupate, el próximo jugador deberá robar el un número concreto de cartas. Si se lanza el prohibido, se saltará turno a un jugador, mientras que si se lanza el cambio de sentido, se rotará el sentido en el que se pasaba de jugador.

Es importante conocer la funcionalidad del botón UNO, cuando un jugador solo tenga dos cartas y vaya a lanzar, deberá antes presionar el botón y luego lanzar, en caso contrario, robará una carta del montón.

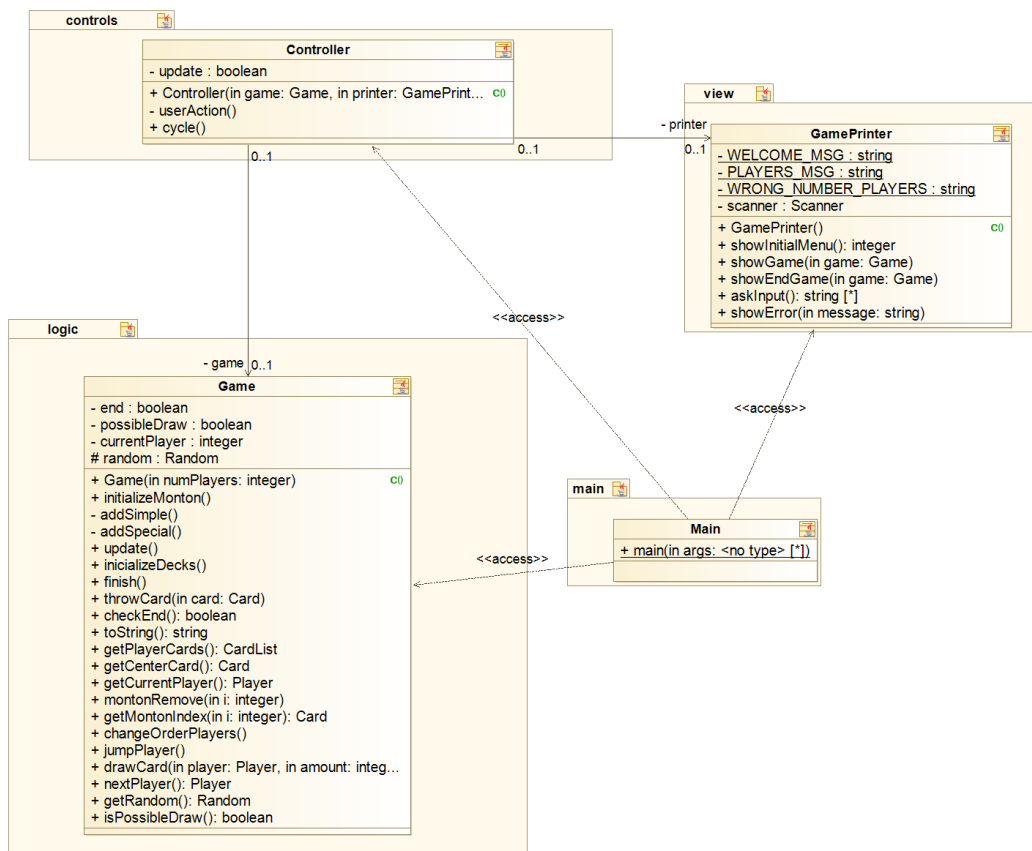
Gana el primer jugador que se quede sin cartas.

- Estimación: Duración del proyecto.
- Priorización: Debe tenerlo.
- Criterios de aceptación: El juego funciona correctamente.

Sprint 1

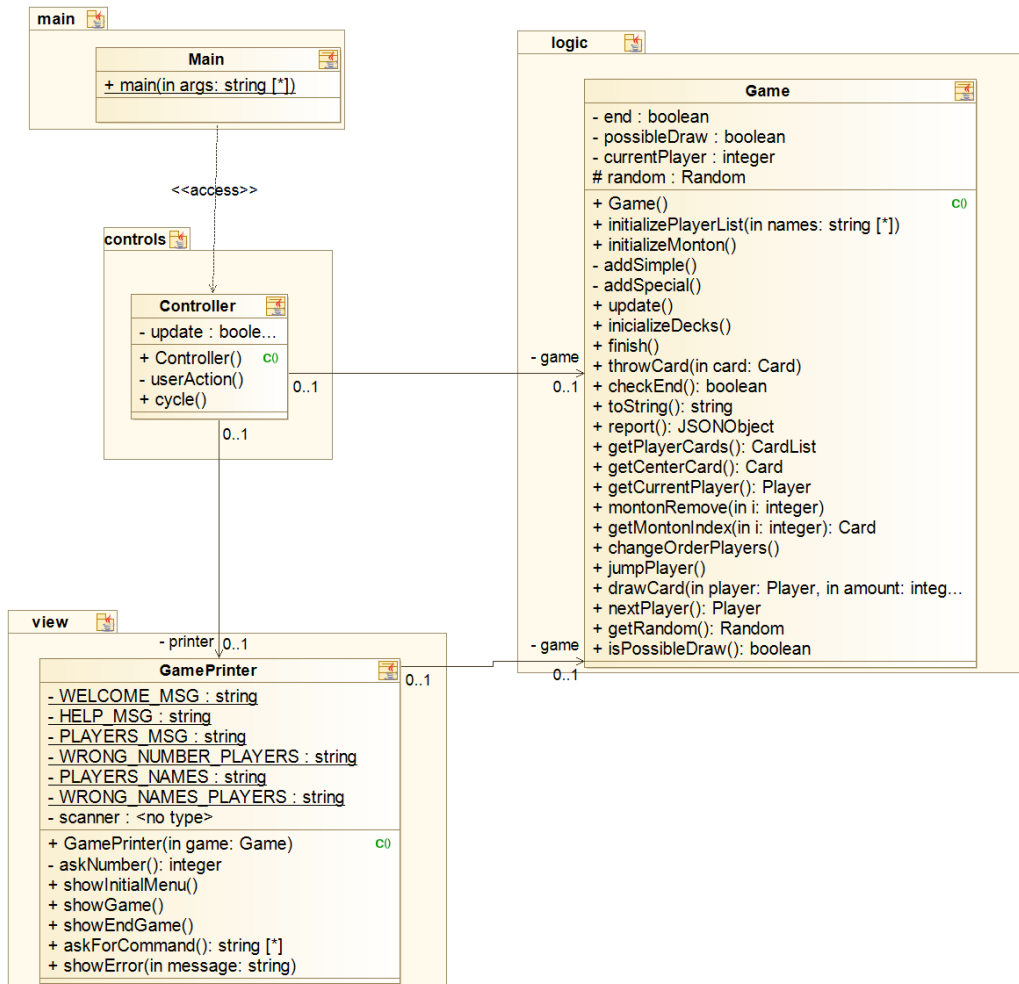
En el primer sprint nos dedicamos a darle la forma básica al juego, consiguiendo que el usuario pudiese ejecutar los comandos necesarios para jugar con las reglas comúnmente conocidas.

Inicialmente la partida se seguía únicamente desde la consola, con símbolos y colores característicos del juego pero sin mucha complejidad en los gráficos. En este sprint aún no teníamos implementado el modelo-vista-controlador.



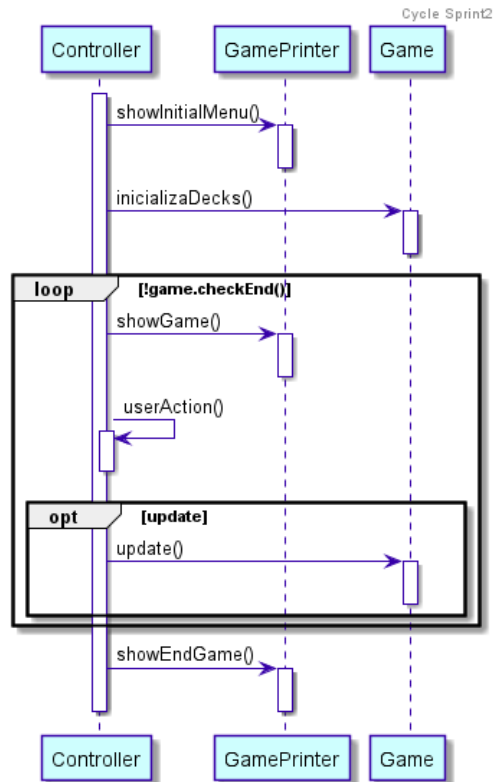
Sprint 2

En el segundo sprint, cambiamos toda la lógica para que el juego se creará en el *Controller*.

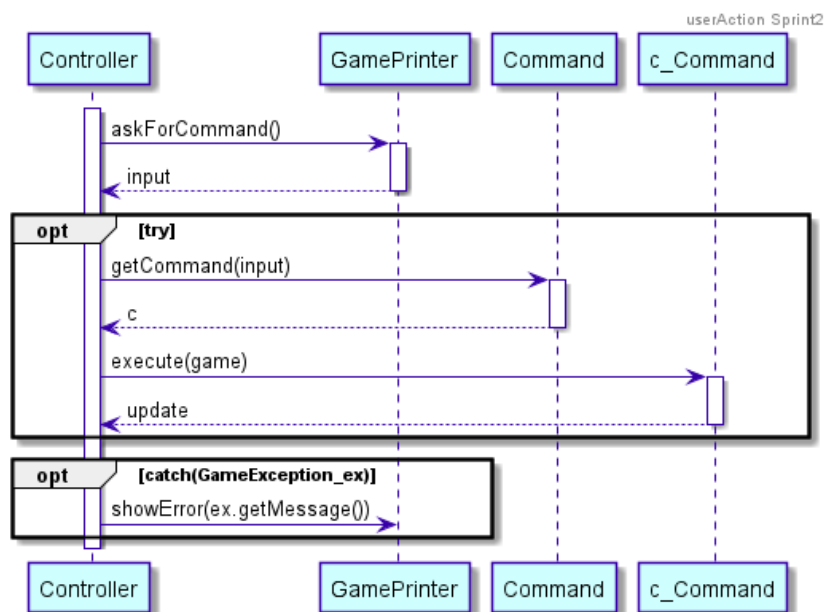


En este sprint, el bucle principal del juego se encargaba de enseñar el menú principal, en el que se establecía el número de jugadores en la partida, posteriormente se inicializaban todos los montones de cartas, (esto se verá de forma detallada más adelante).

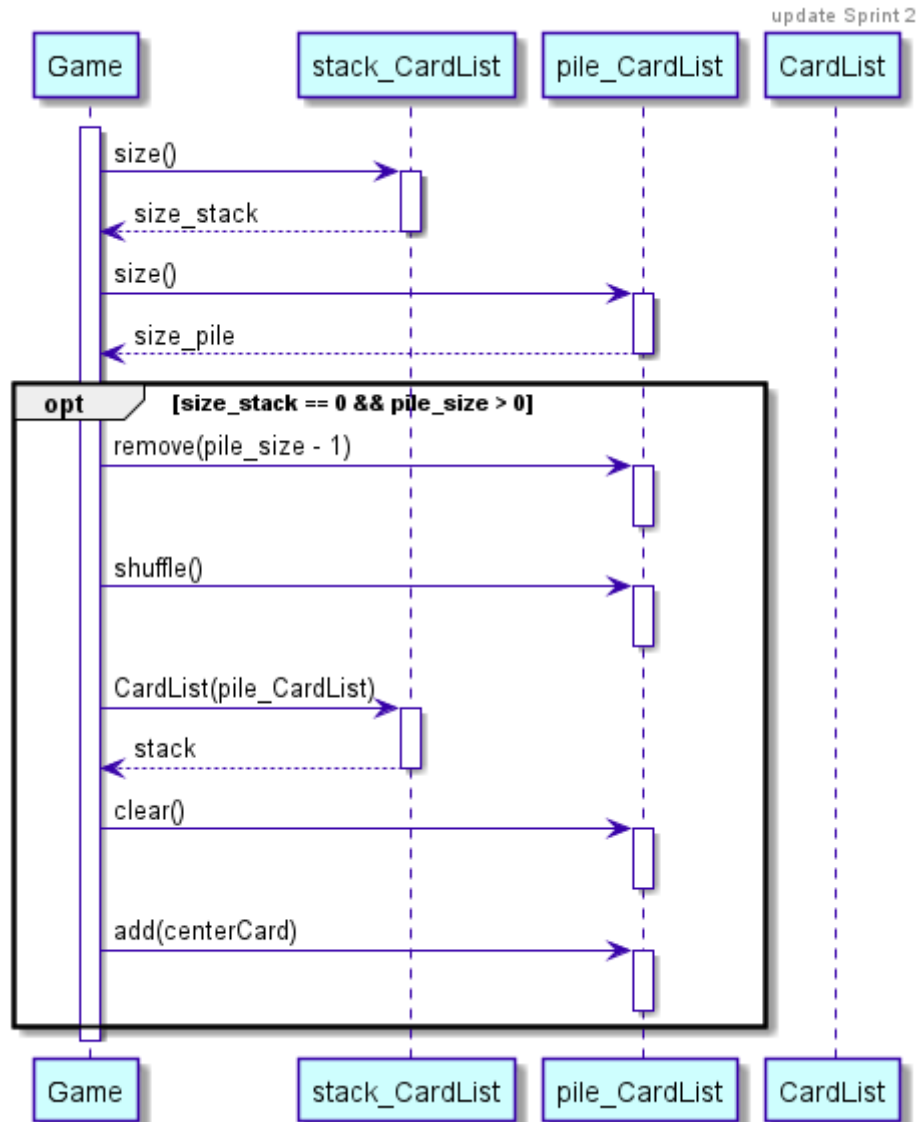
Existía un bucle que acababa cuando o bien alguien hubiese ganado o si el usuario decidía acabar la partida. Mientras esto no ocurriese se enseñaba el juego y el usuario escribía el comando que decidiese hacer en ese momento, todo esto por consola.



Aquí podemos ver con más detalle cómo funcionaba la lógica de los comandos. Se preguntaba al usuario por el comando que desease hacer, se identificaba que comando era a través de la función `getCommand()` como se puede observar en el diagrama. Por último se ejecuta el comando y se actualizará el juego si el comando lo requiere.



A continuación se muestra el funcionamiento del juego si el montón de cartas para robar (*stack*) acaba, nuestra intención era que el *pile* se moviese y pasase a ser el *stack* y así todas las veces que se acabasen las cartas en el *stack*. Sin embargo, no conseguimos que funcionase de manera correcta en este Sprint.

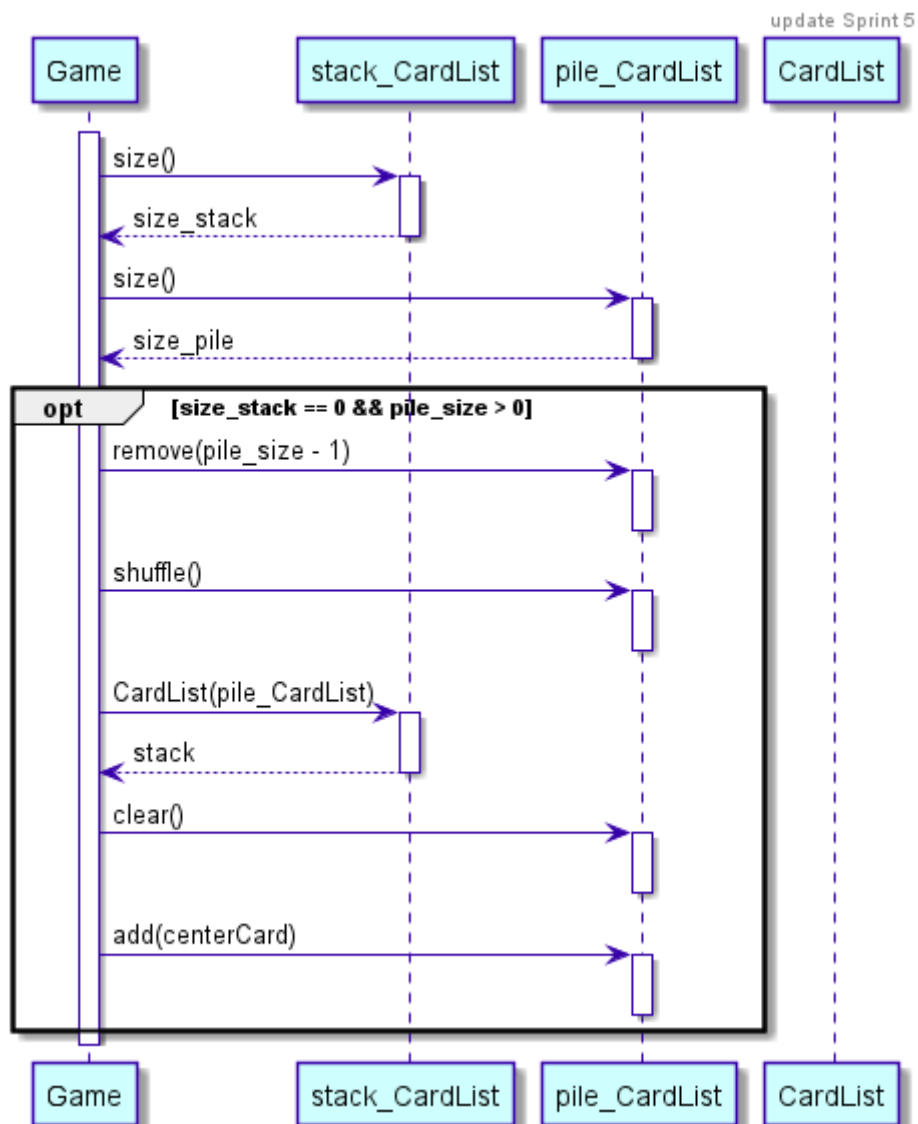


Por ejemplo, en el código que se muestra a continuación nos encontramos en un carta que gestionamos como si fuese un botón, si pulsas una carta significa que quieres tirarla, se puede observar como somos nosotros mismos los que generamos el *input*.

```
String [] input = {"t", c.getSymbol(), colorString};  
controller.userAction(input);  
controller.update();
```

Sprint 5

Aquí conseguimos solucionar el problema expuesto en el Sprint 2, sobre qué ocurría si se acababan las cartas para robar (*stack*).

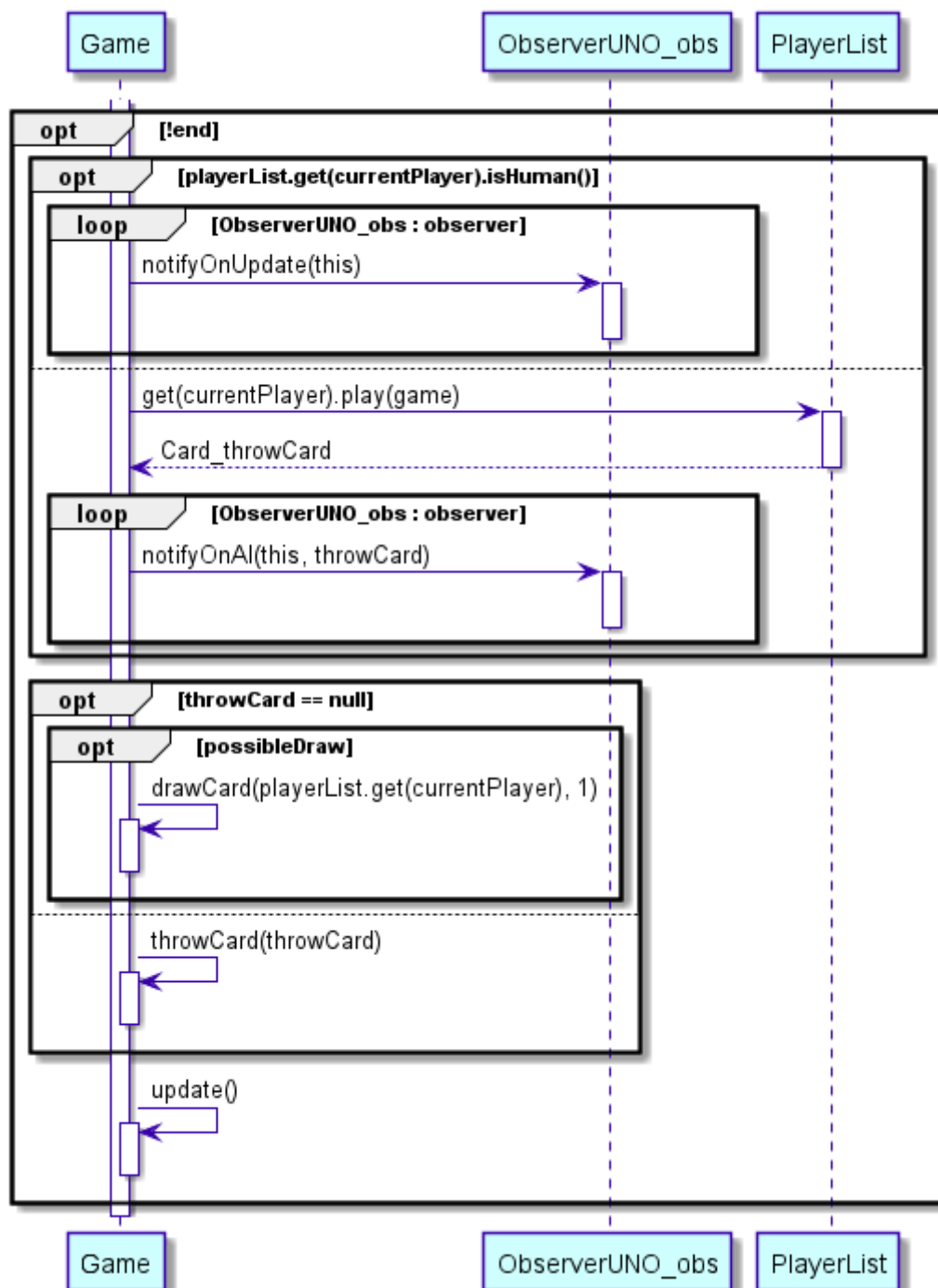


En el Sprint5 implementamos las estrategias *EasyPlayer*, *MediumPlayer*, *HardPlayer* lo que causó un cambio en el funcionamiento cuando se juega con ellas.

Como se puede ver se comprueba primero si el usuario es humano o no, si lo es jugará el usuario como anteriormente, si no jugará la estrategia elegida, implementada en cada una de ellas a través del método *play(game)*.

Al usuario humano se le irá informando de los movimientos realizados por la máquina, es decir, cuando la máquina deba robar se le dará un aviso al usuario de que la máquina ha

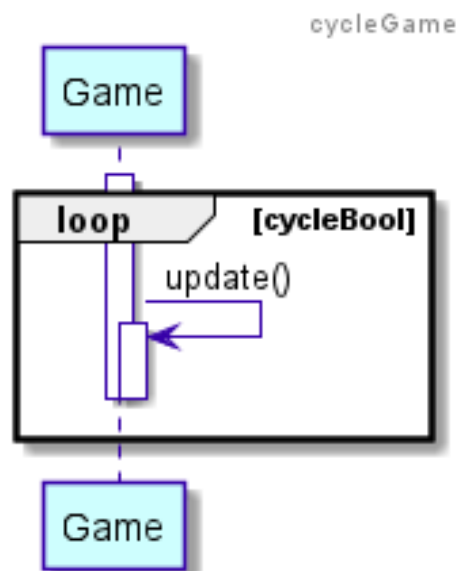
robado y cuando la máquina tira una carta se le informará al usuario de la carta que se ha tirado.

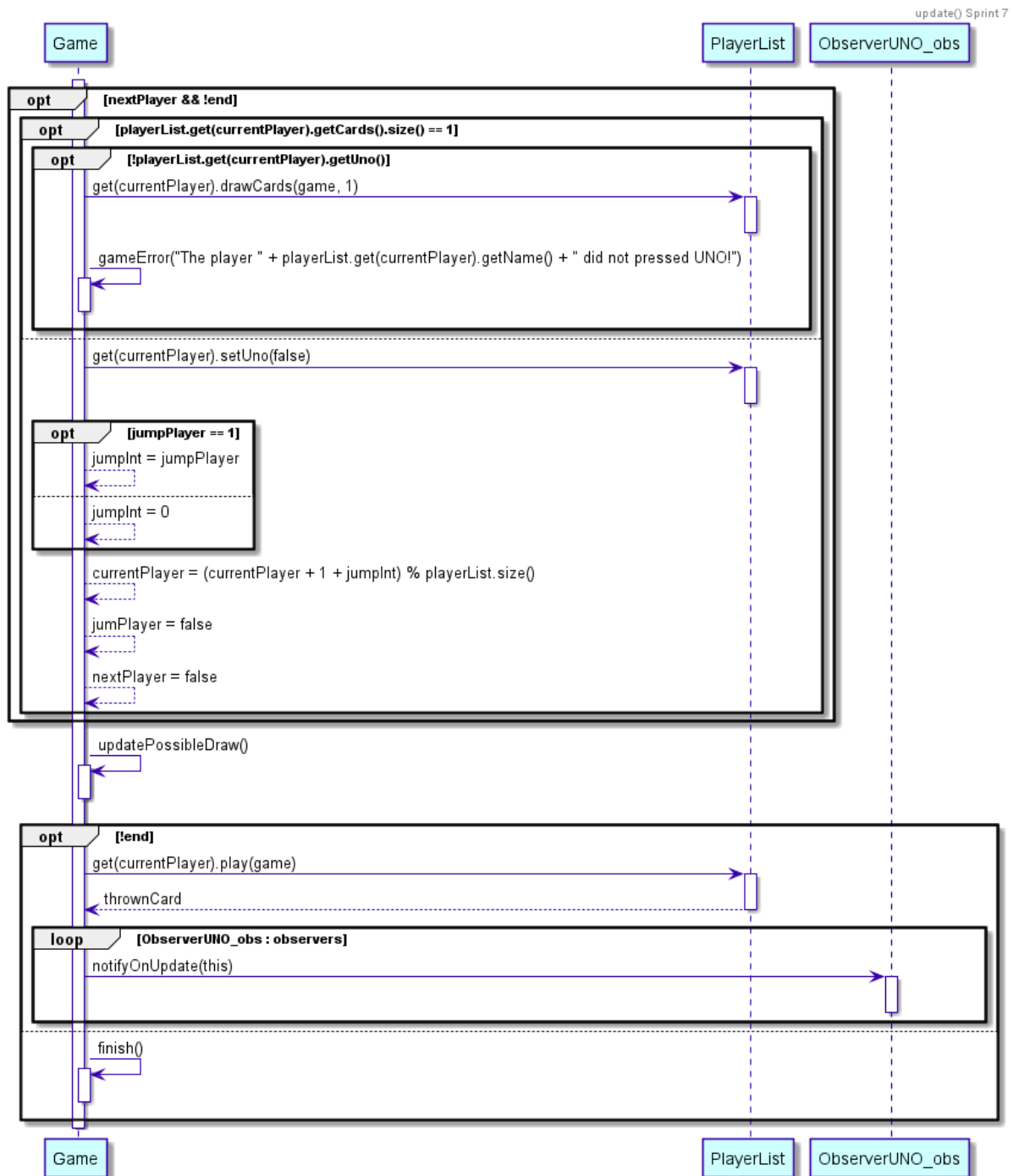


Sprint 7

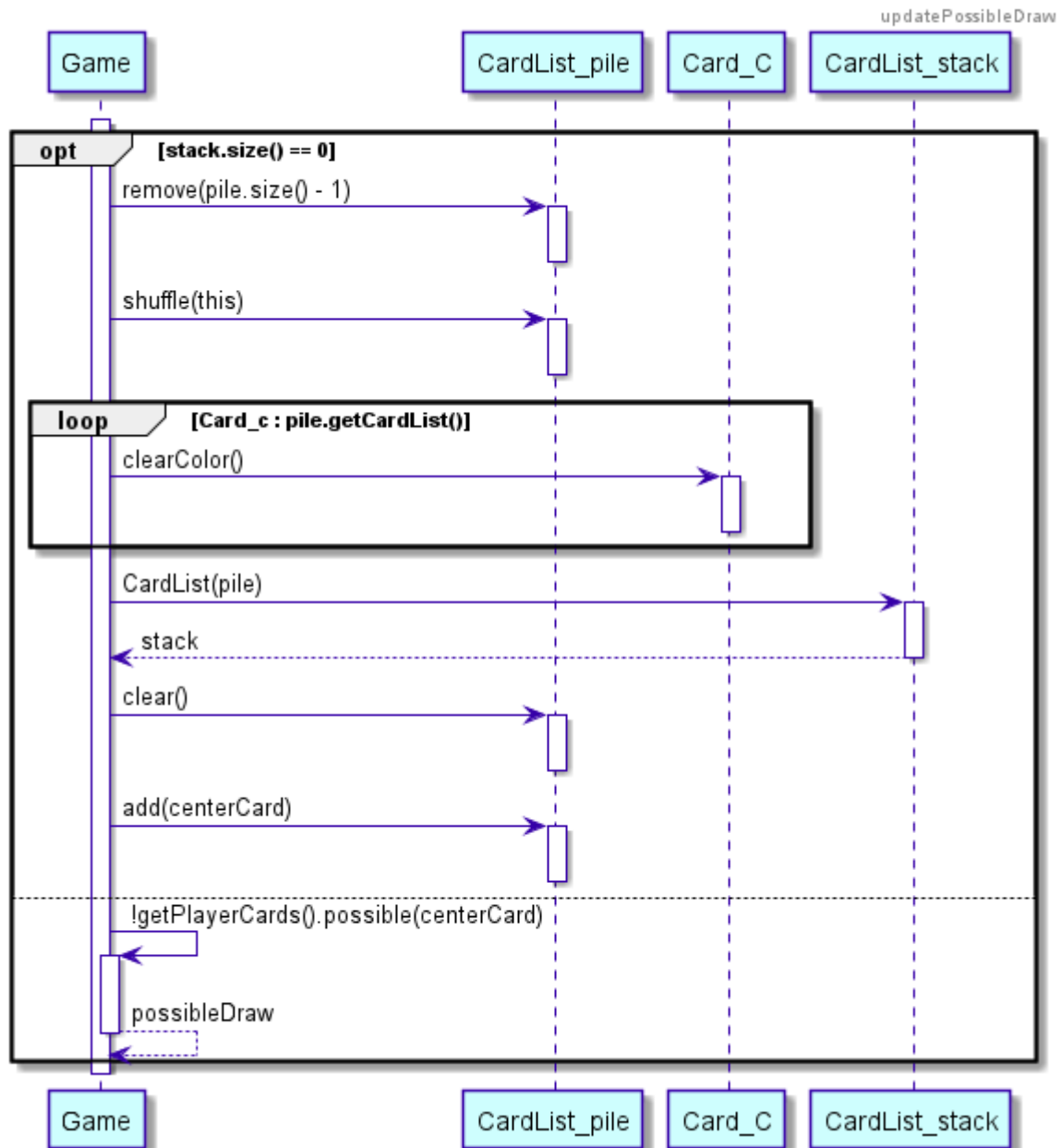
Cambiamos el `update` de *Game*, antes en ocasiones no se acababa la partida cuando no debía. Además hemos eliminado la distinción entre ser humano o máquina de inteligencia artificial (*isHuman()*). Por otra parte, hemos quitado que el *update* sea recursivo, ahora es cíclico, haciendo que el play de los humanos corte el ciclo y se reactive cuando introduzcan una acción, para esto añadimos el método *cycle()*.

El modo en que funciona en este momento el `update` es el siguiente. Un atributo de *Game* controla que el bucle continúe ejecutándose. Este bucle tiene el problema de que en el caso de que le toque a un jugador se debe parar, esperar a que introduzca una acción y, sólo tras esto, continuar, mientras que con la IA el bucle debe continuar directamente ya que la acción de ese tipo de jugador se toma de manera distinta, instantánea. Por esta razón, nos decidimos a hacer que que el play del jugador humano desactivase el atributo que controla el bucle y que lo volviese a activar una vez introdujese una acción mientras que la IA lo mantiene activado.

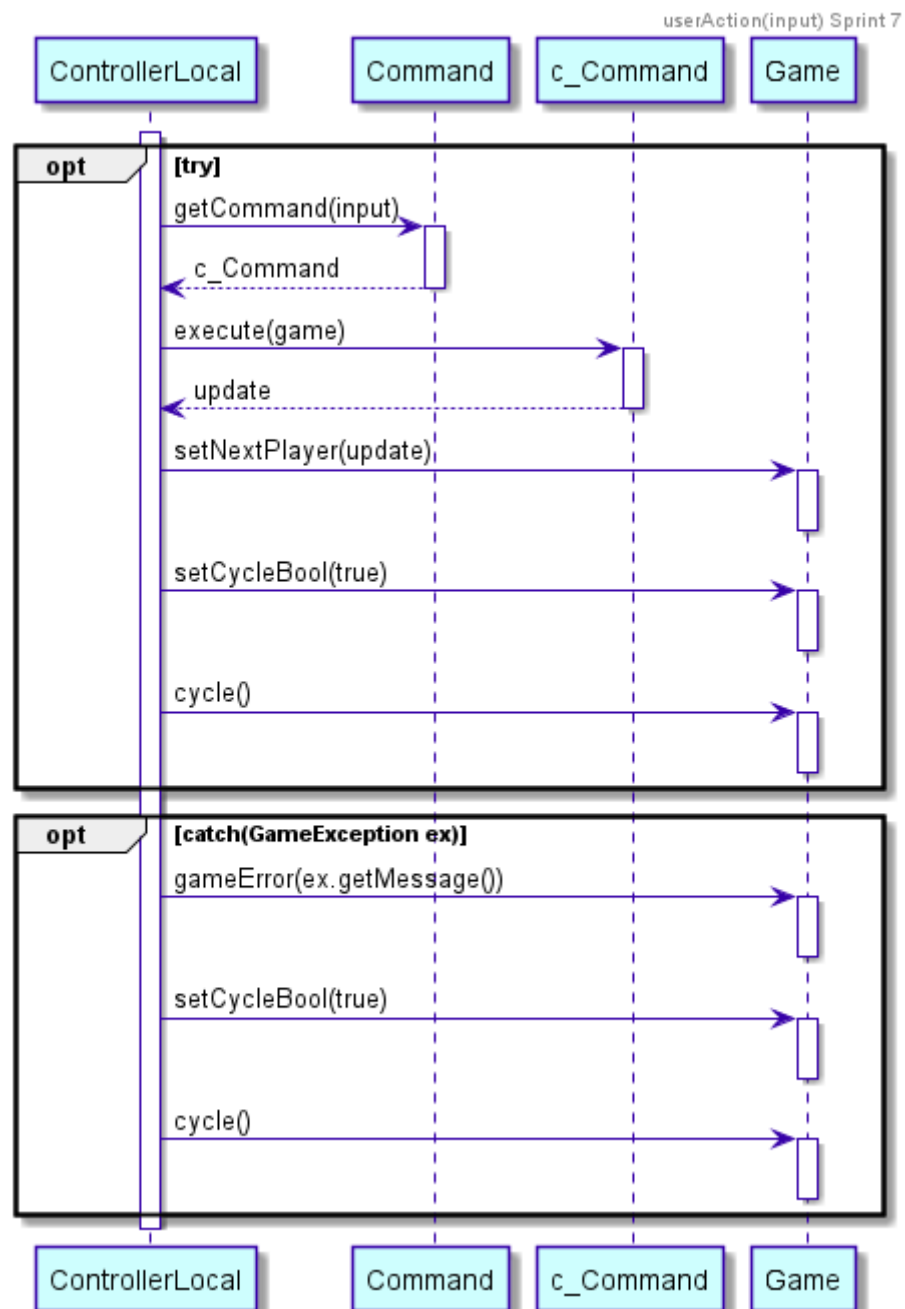




La funcionalidad de volcar las cartas se ha pasado al método *updatePossibleDraw()* , además ahora, antes de pasar las cartas al *stack*, si la carta es un *Add4Card* o un *ChangeColorCard* se le cambia el color de nuevo a “negro” ya que antes al volcar las cartas esto dos tipos aparecían con el color al que habían sido cambiados anteriormente.



Se ha cambiado el *userAction* del *ControllerLocal*, el *update()* por *setCycleBool(true)* y *cycle()*, cuyo diagrama se ha mostrado anteriormente.



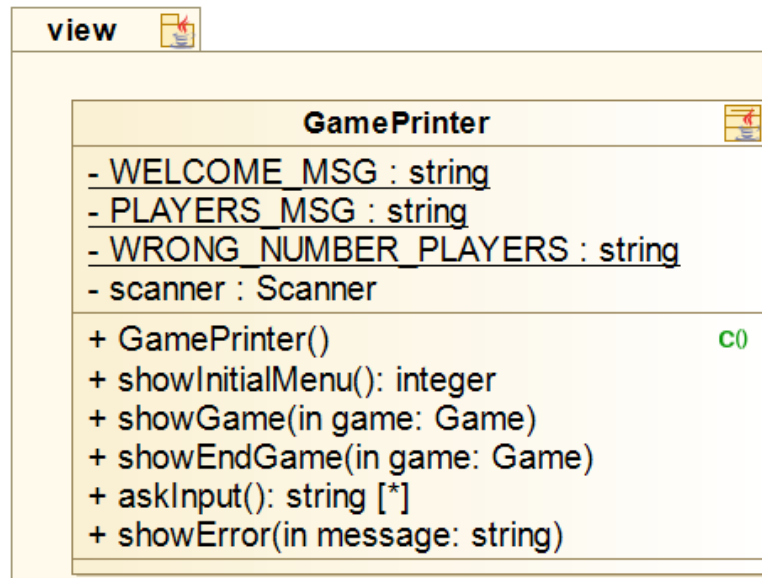
2. Cómo usuario quiero poder elegir el número de jugadores a participar.

Esta historia de usuario consiste en la creación de oponentes para un jugador. Por simplificar, decidimos que el número mínimo de jugadores sería 2, y el número máximo 4. Por ello, al inicio de cada partida, el usuario debe poder escoger el número de jugadores en la partida.

- Estimación: Se estimó que en realizar esta historia de usuario se tardaría 1 semana, sin embargo, aunque la función principal de la misma estuvo preparada en el primer Sprint, se realizaron una serie de cambios para mantener la cohesión con el resto de elementos añadidos.
- Priorización: Debe tenerlo.
- Criterios de aceptación: El usuario puede elegir el número de jugadores dentro de un margen.

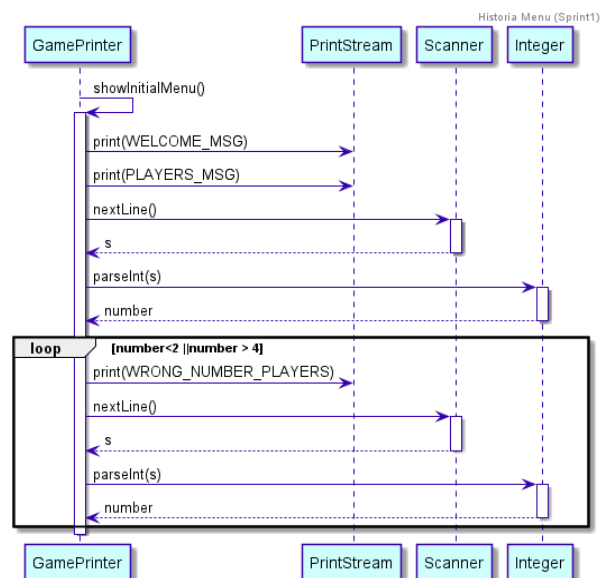
Sprint 1

Basamos la lógica del juego en que fuese independiente del número de jugadores que hubiese, para así no tener que hacer un gran número de cambios en función de cuántas personas jueguen. El número sería de 2 a 4, esto no varía en todo el proyecto.



En un principio, se le pide al usuario que introduzca el número de jugadores por consola. Veamos en el diagrama de secuencia que aparece a continuación el funcionamiento y orden de esta historia.

Para asegurarnos de que el número de jugadores está entre 2 y 4 creamos un bucle el cual si el jugador, introduce un número que está fuera del rango, el juego vuelve a pedir que lo introduzca.



Sprint 2

Como mencionaremos más adelante, en este Sprint esta historia de usuario es fusionada en la historia de usuario 5, por lo que el resto de modificaciones realizadas sobre este tema serán redactadas en el apartado correspondiente.

3. Como usuario quiero jugar con las cartas.

El principal objetivo de esta historia de usuario es que una vez haya jugadores, estos puedan jugar, escogiendo las cartas a lanzar, y haciendo que una vez se lance se produzca el efecto correspondiente.

3.1 Cartas Normales:

Las cartas normales serán números del 0 al 9, con sus respectivos colores (amarillo, azul, rojo, verde). Por cada combinación, habrá dos cartas con el mismo número y color.

- Estimación: Se estima que en realizar esta historia se tardará una semana.
- Priorización: Debe tenerlo.
- Criterios de aceptación: Cada carta tiene su color, número.

3.2 Cartas Especiales:

Las cartas especiales están formadas por: cartas con color: prohibido, cambio de sentido, +2; cartas sin color: +4, cambio de color.

- Estimación: Se estima que en realizar esta historia se tardará una semana.
- Priorización: Debería tenerlo.
- Criterios de aceptación: Cada carta tiene su color (si tiene), y funcionalidad.

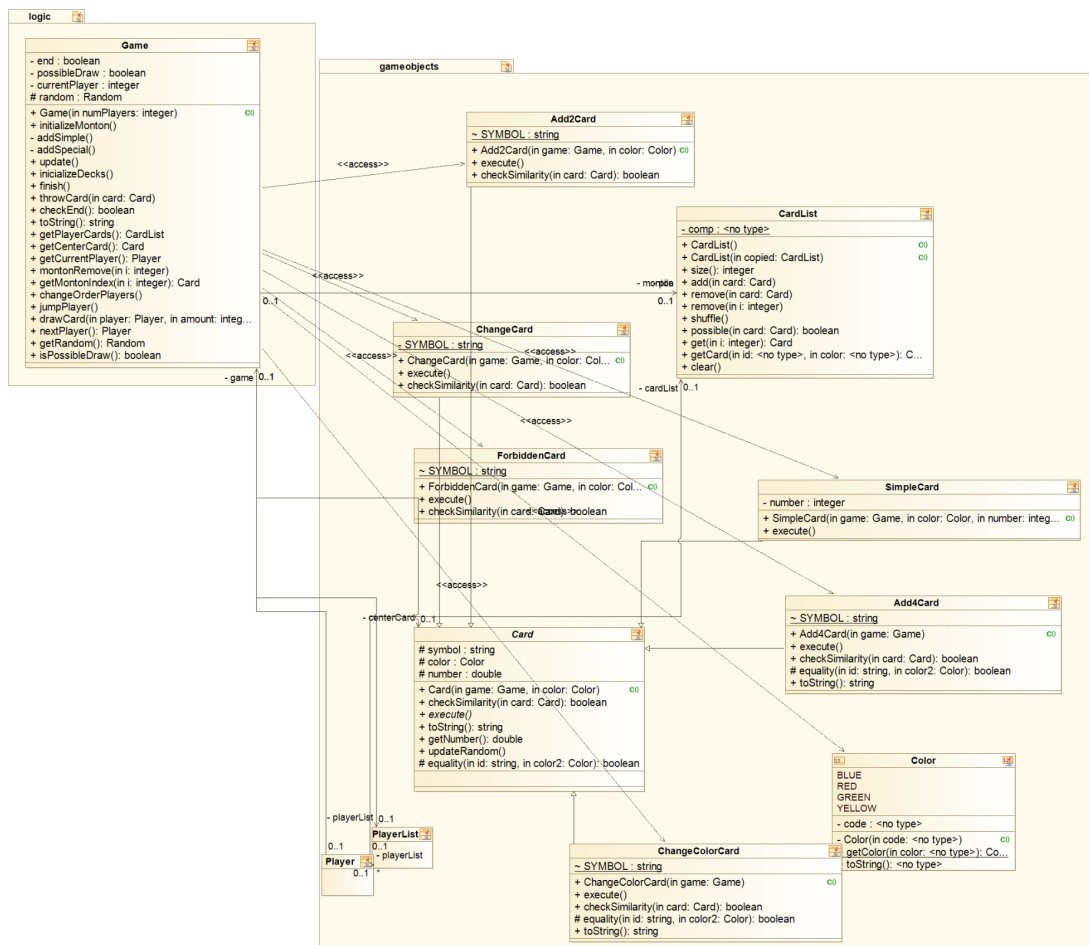
Se consiguió tener los criterios de aceptación conseguidos de las cartas normales en una semana, como se había previsto. Sin embargo, con respecto a las cartas especiales, conseguimos la funcionalidad de todas en una semana, excepto de las relacionadas con los cambios de color.

El problema surgió debido a que las cartas que se encargan de los cambios de color, no poseen un color propio, y por tanto, la lógica del juego nos fallaba. Se soluciona en el siguiente sprint.

Sprint 1

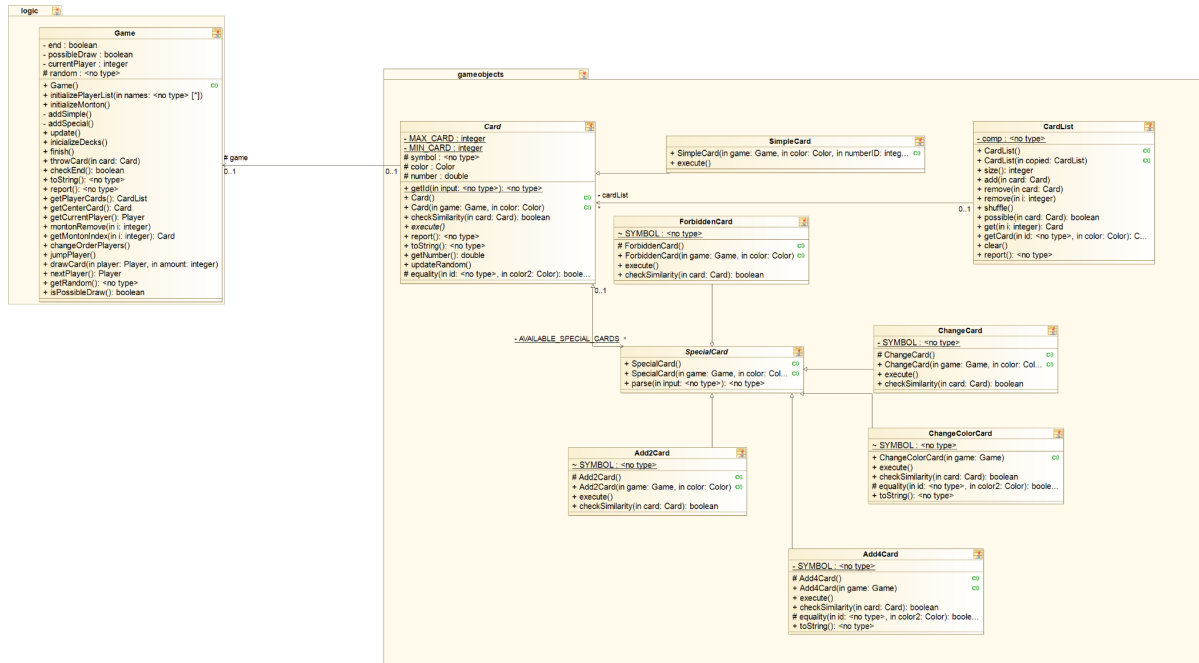
En este sprint se crean las diferentes clases de las cartas usuales del UNO, distinguiendo entre las normales SimpleCard, con tan solo color y número, y el resto por separado (“+4”, “+2”, etc).

Todas ellas se inician al comenzar la partida y se van acumulando en un montón de cartas con el número de cada una de ellas que tendrían en el UNO original. Se le reparte a cada jugador 7 cartas aleatorias.



Sprint 2

Se crea la clase *SpecialCard* para que todas las cartas que no sean número con color, hereden de ella, y así poder simplificar el código y encapsular funcionalidades.



Como se puede observar a continuación de cada carta decidimos meterla dos veces, en un principio hablamos de si meter más de una baraja y finalmente optamos por hacerlo, ya que así quedaba el juego mucho más completo y entretenido.

Primero añadimos las *SimpleCards* y posteriormente las *SpecialCards*, una vez añadidas el montón se barajan con la función *shuffle* para que no estén ordenadas según las metemos en la lista de cartas *stack*.

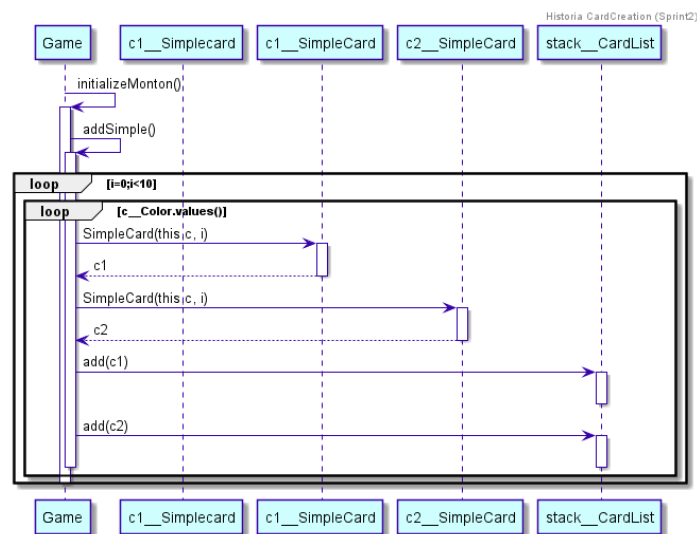


Diagrama que muestra la creación de las cartas simples.

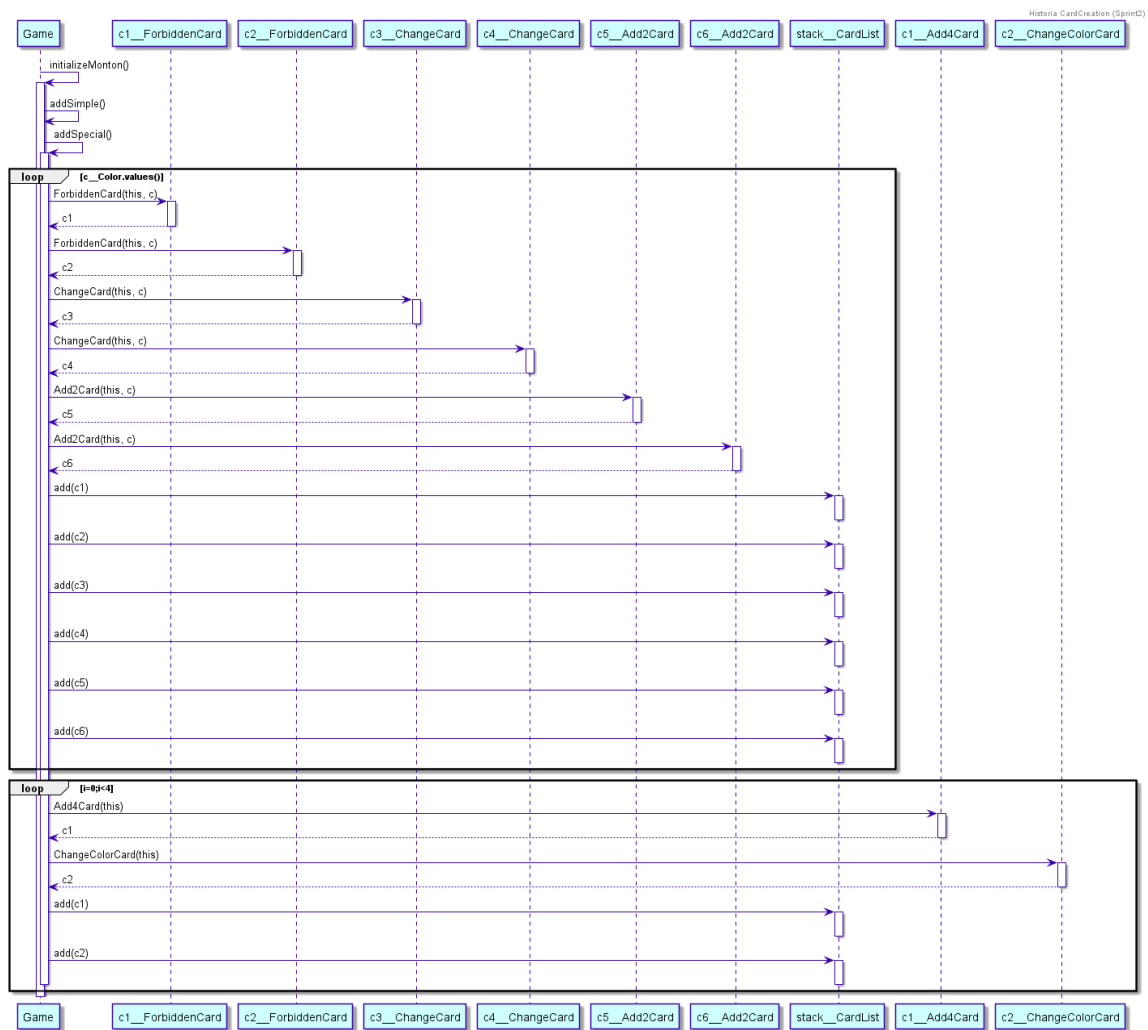


Diagrama que muestra la creación de las cartas especiales.

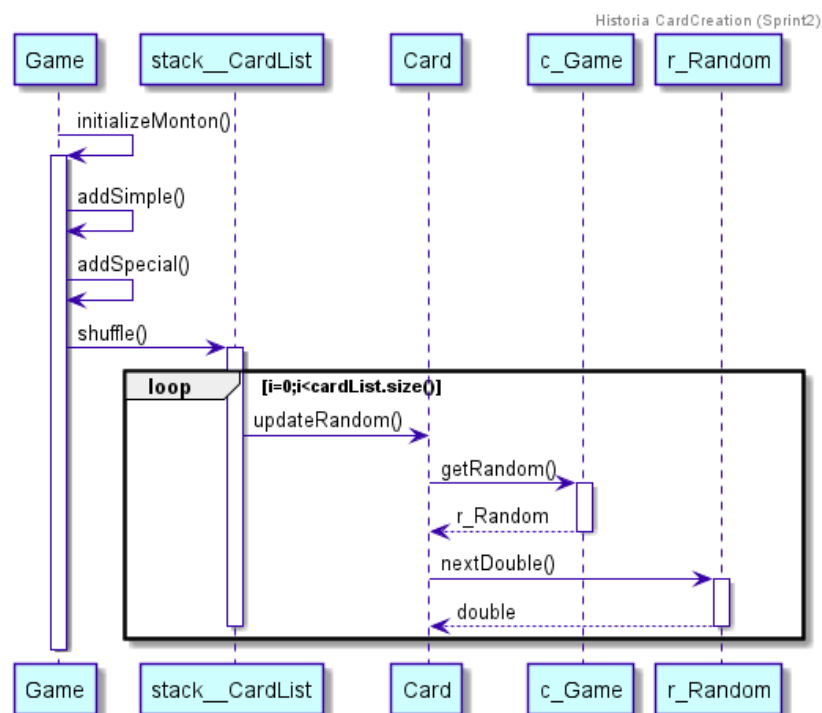
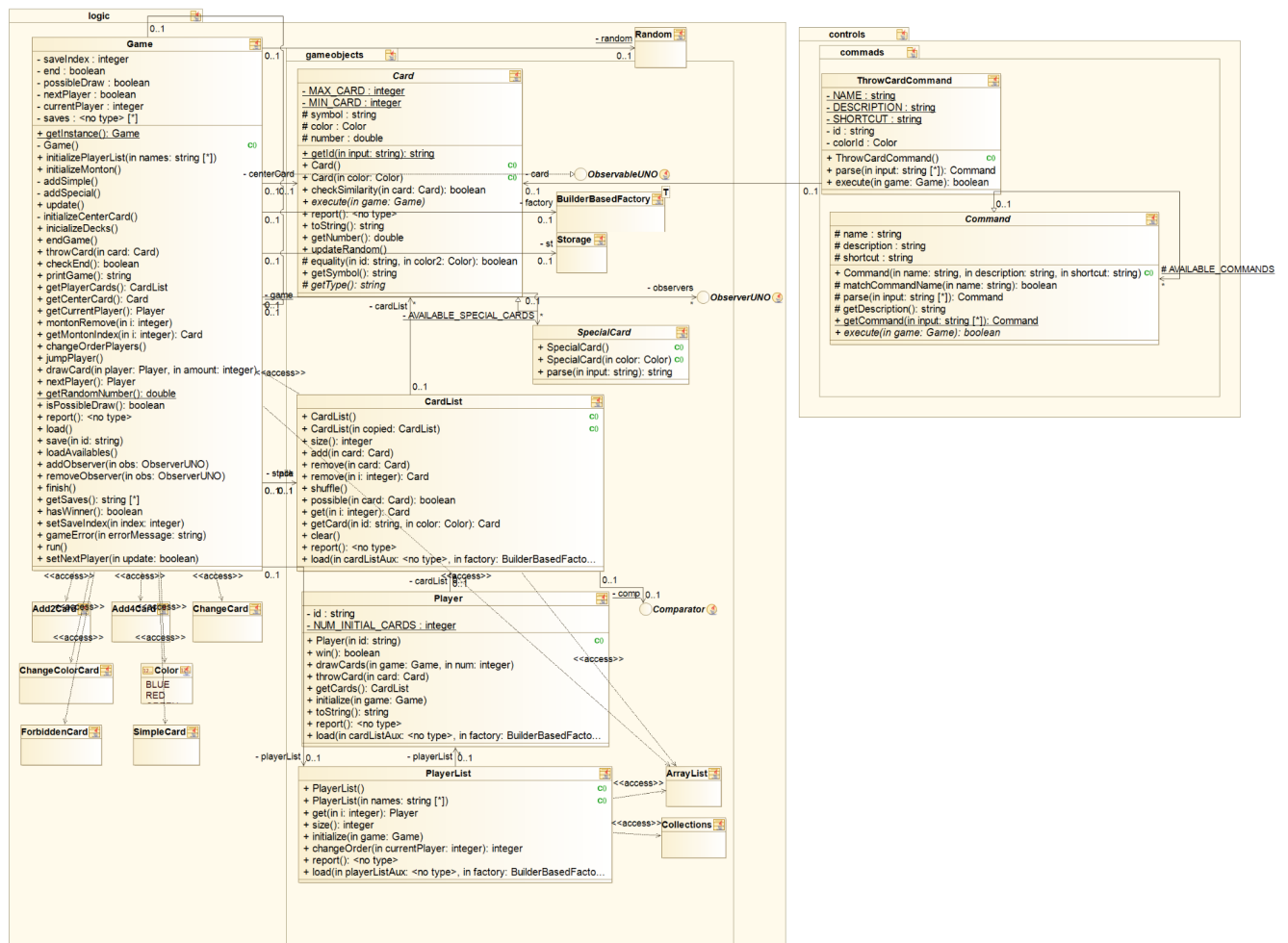


Diagrama que muestra el shuffle.

Sprint 3

Se realizan cambios en el constructor de las cartas según se considere que ayuda a la simplificación del código. Decidimos que no tenía mucho sentido que las cartas tuviesen la lógica de *Game* ya que únicamente la utilizaban las cartas especiales para realizar su funcionalidad única, por tanto se eliminó el *game* del constructor de cada carta y decidimos pasarla como atributo en la función *execute*, una función abstracta que solo influye a las *Specialcards*.



Sprint 5

Nos dimos cuenta de que *Game* tiene una cantidad desorbitada de responsabilidades y decidimos pasar gran parte de la lógica a otras clases. En lo que corresponde a esta historia de usuario, las cartas pasan a crearse en *CardList*, en vez de en *Game*.

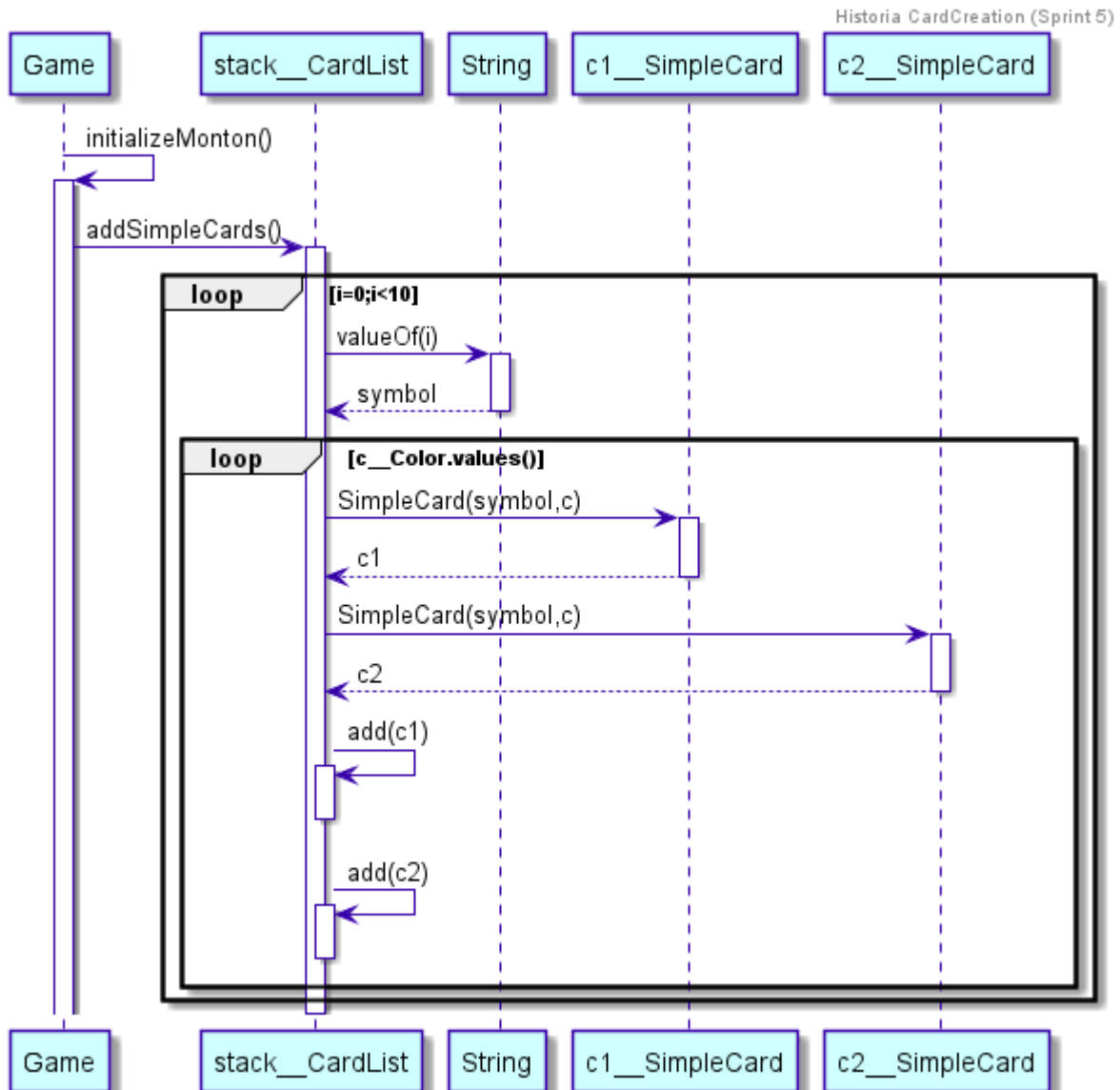
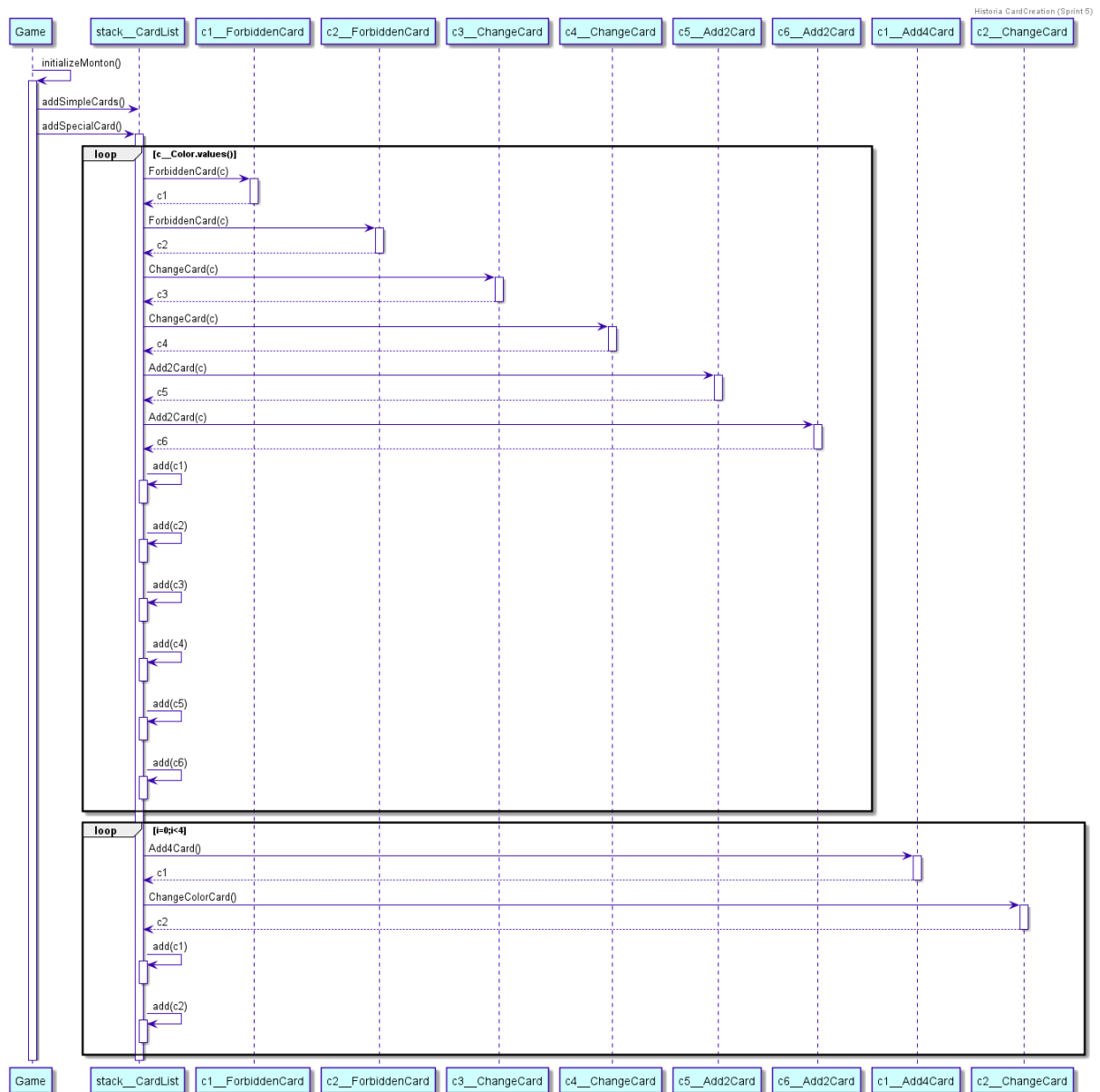


Diagrama de creación de las cartas simples.

Como se puede observar ahora la lógica de la creación de las cartas está en *CardList*, en particular se crean en el *stack*, que será nuestro montón de robar.



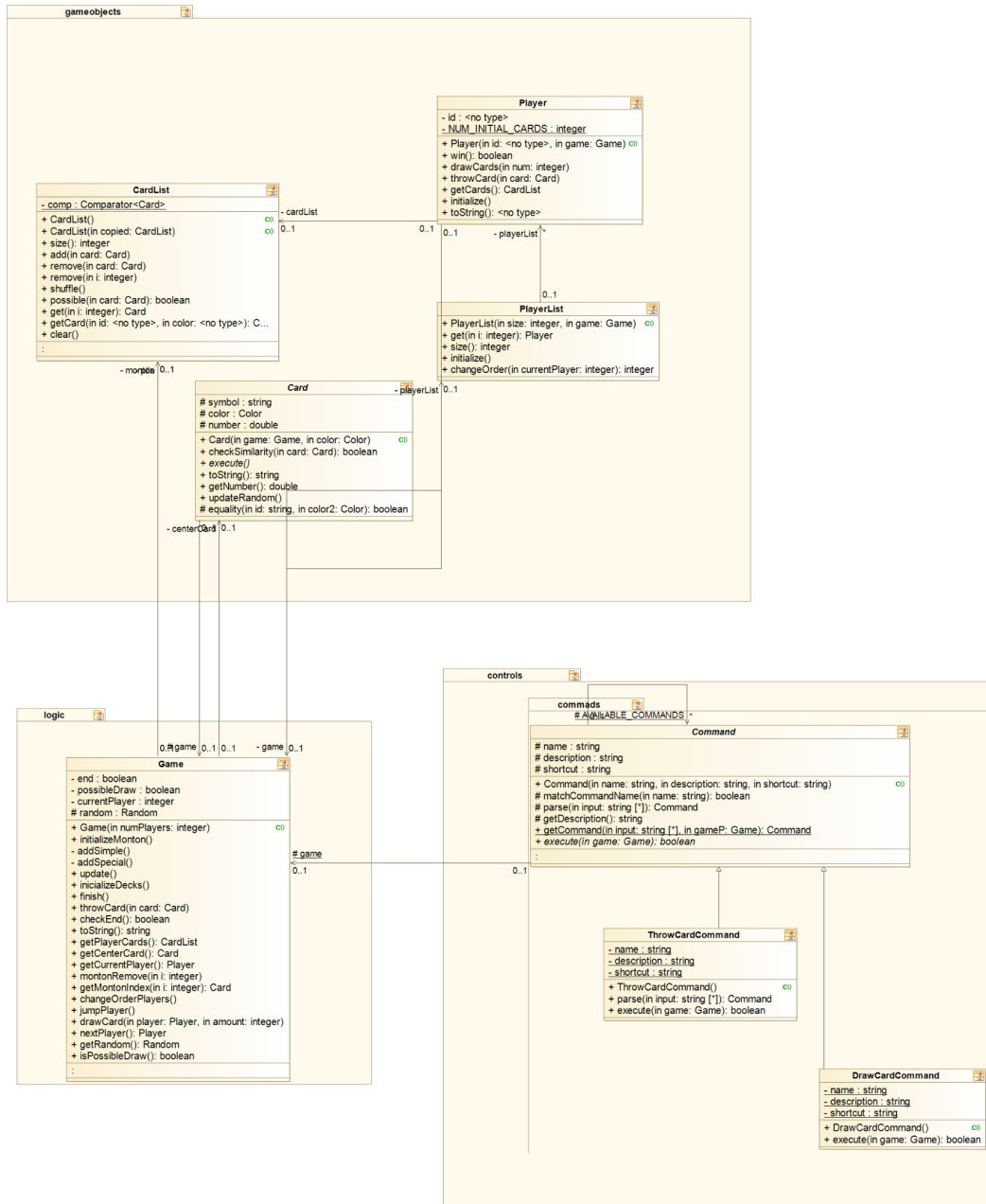
4. Como usuario quiero que las cartas se comporten adecuadamente.

Al tirar una carta esta debe desaparecer de mis cartas, añadirse al montón conjunto, y ocurrir el efecto correspondiente.

- Estimación: Se estima que en realizar esta historia de usuario se tardará 2 semanas. Posteriormente se realizaron cambios.
- Priorización: Debe tenerlo.
- Criterios de aceptación: La carta desaparezca realmente, no solo visualmente. Además, que se aprecie que el montón de cartas de la mano ha disminuido, y el montón de lanzar carta ha aumentado, es decir, la carta lanzada es la primera, y así, el efecto sea el correcto.

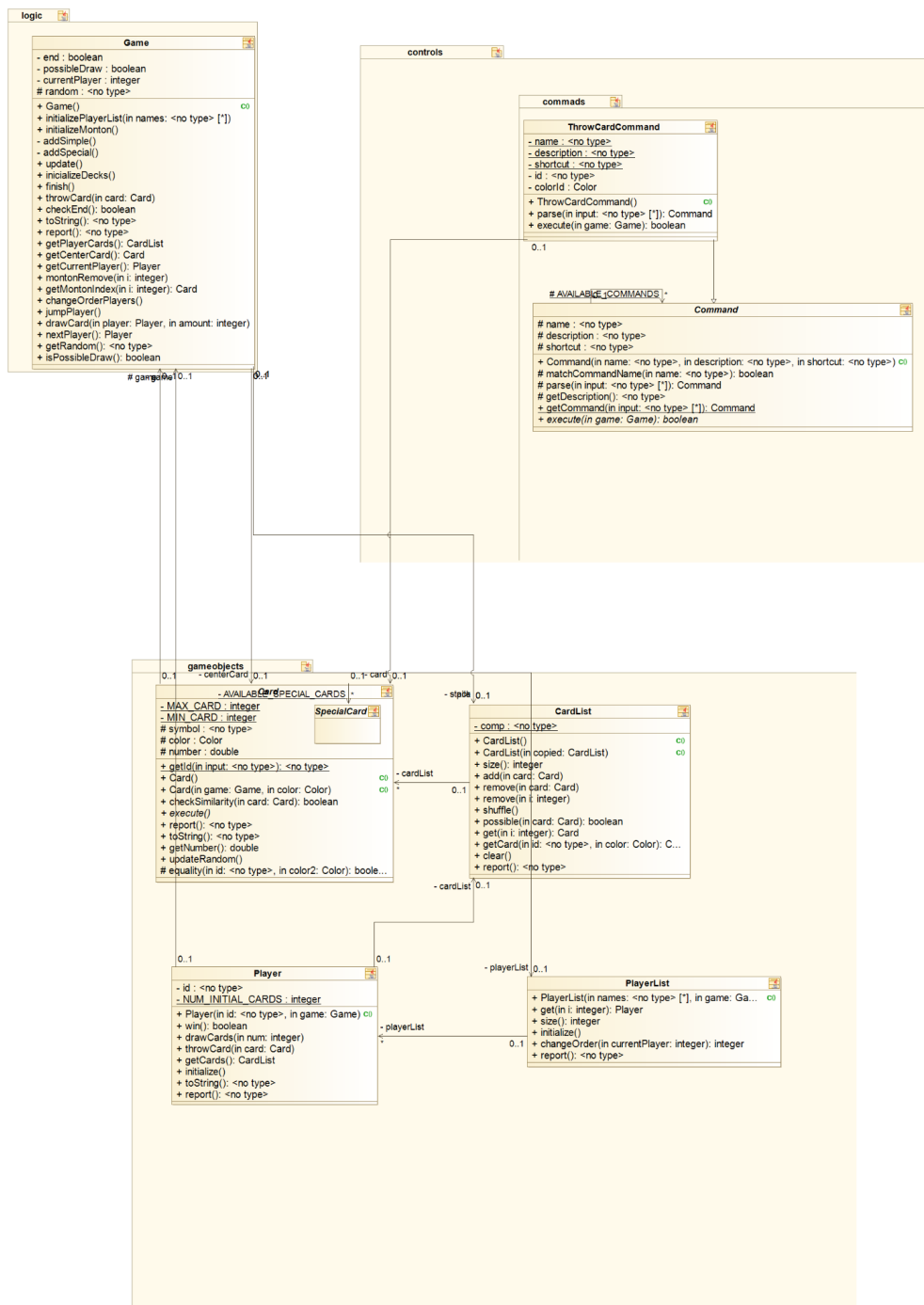
Sprint 1

Desde el principio del proyecto hicimos que las cartas tuviesen el comportamiento habitual del juego, acumulándose en los diferentes montones, para robar o lanzar. Una acción que fue implementada desde un principio pero nos dió problemas fue *shuffle*, que se encargaba de barajar las cartas del centro (las lanzadas) una vez el montón de robar se acabase.



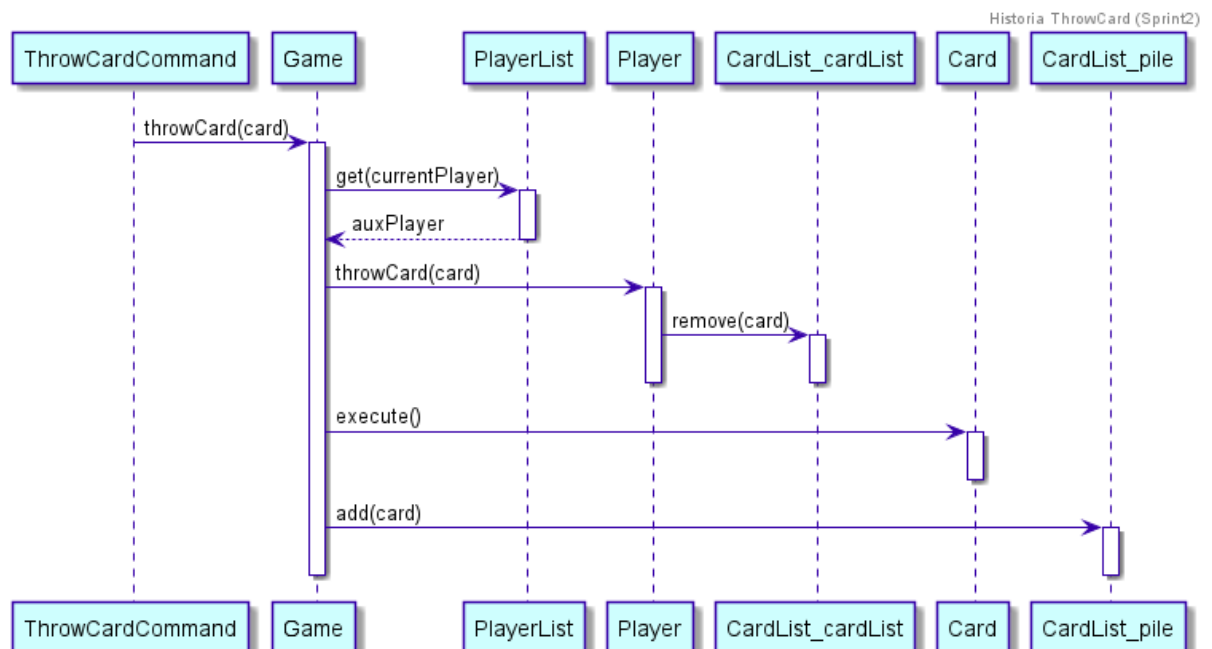
Sprint 2

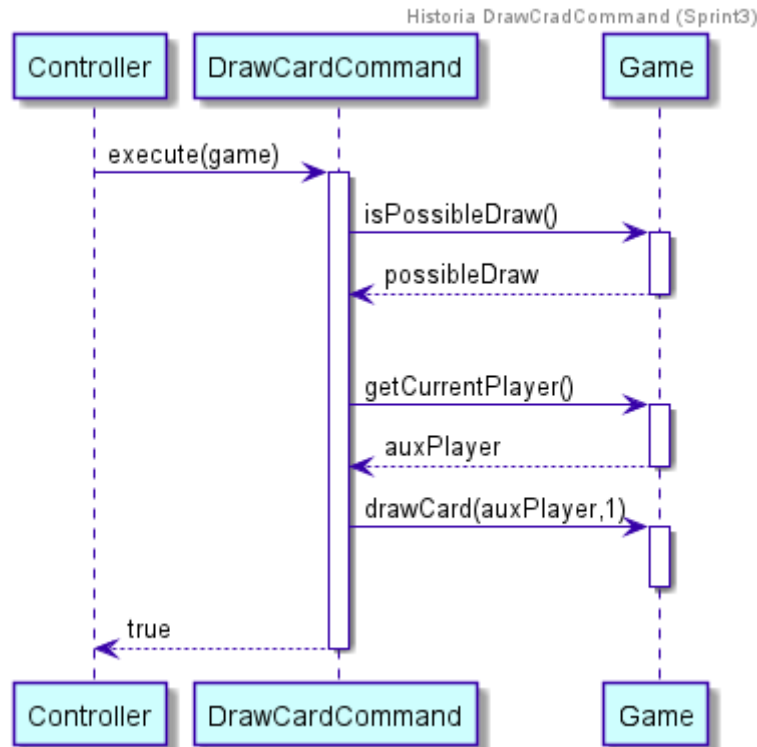
En el sprint anterior, el programa solo avisaba al usuario si la carta que quería lanzar era incorrecta. En este sprint, ampliamos la función de que si no tiene cartas posibles a lanzar, se avise al usuario de que deberá robar carta.



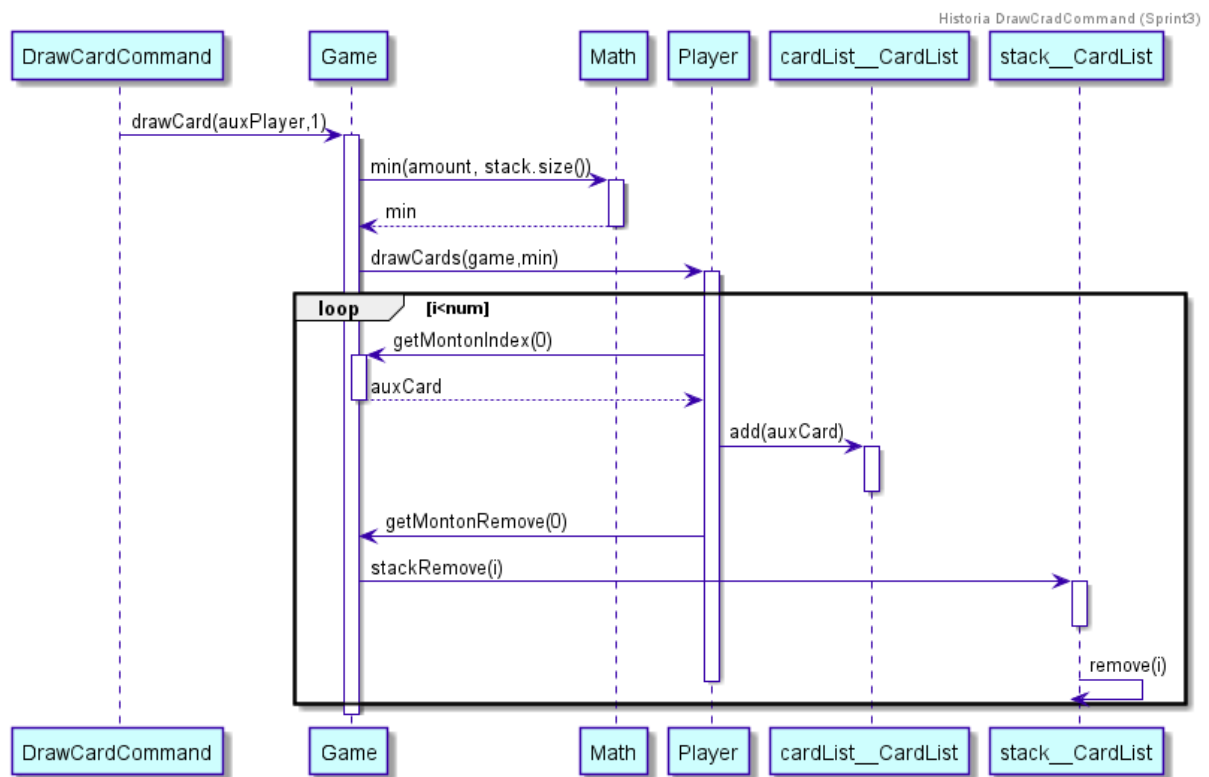


El usuario selecciona la carta que desea tirar mediante la entrada introducida por la consola. Esa carta se elimina de la *CardList* del jugador y se añade al montón de cartas tiradas (*pile*).





Primero se comprueba que es posible robar una carta y posteriormente se llama al método *drawCard* de *Game* para que el jugador correspondiente robe una carta.



Como se puede ver si el número de cartas que el jugador tiene que robar es mayor al número de cartas que quedan en el montón para robar, entonces se robará el mínimo. Este caso solo se dará cuando se tire un *Add2Card* o un *Add4Card*.

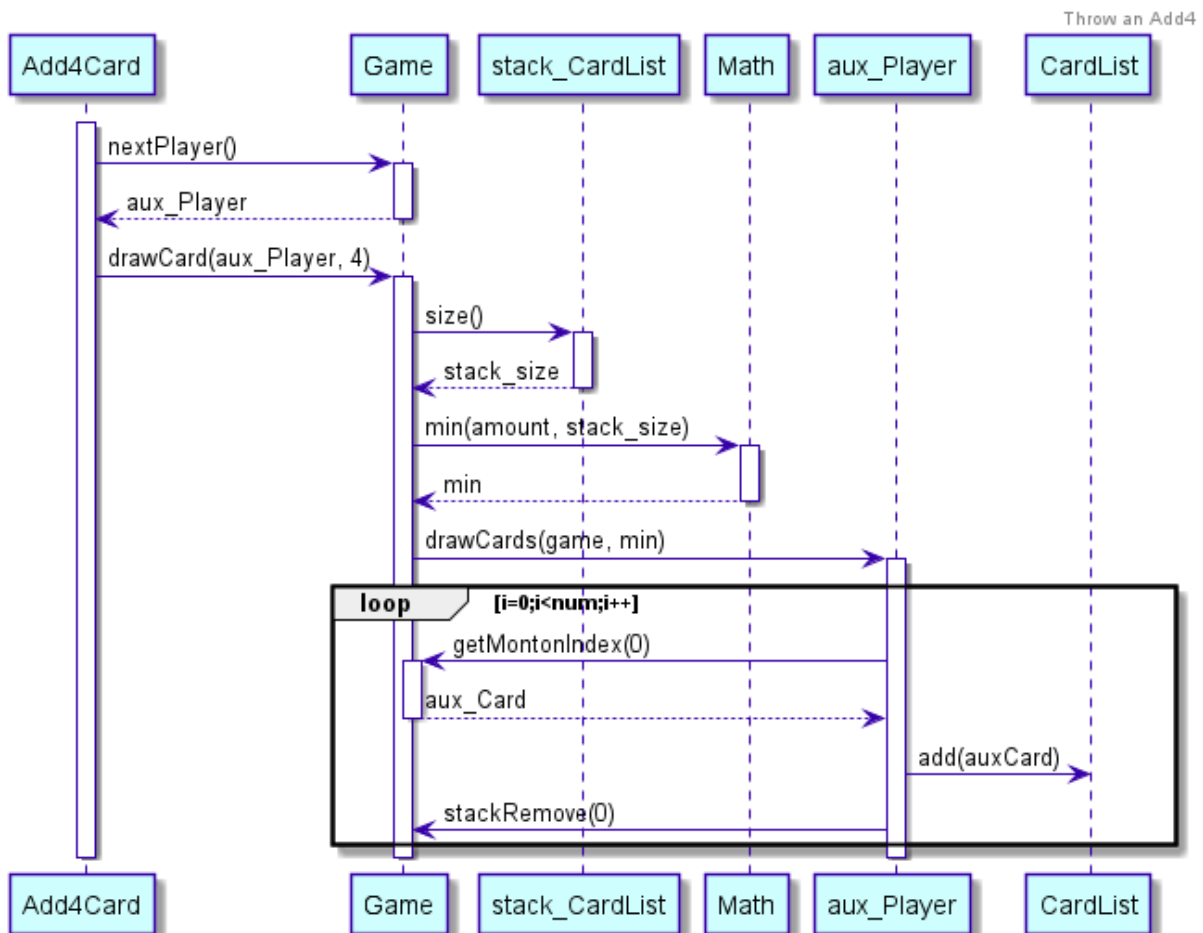
Sprint 4

A pesar de no haber cambios importantes en cuanto a la funcionalidad de que ocurre cuando tiras una carta. Durante este Sprint decidimos que sería interesante ver el funcionamiento de las cartas especiales dentro del juego, hemos querido reflejar esto con varios diagramas de secuencia, simulando que ocurriría si el usuario tirase cada una de las siguientes cartas.

Si el usuario tira un +4 el programa hará lo siguiente. Primero se pregunta al game cual es el siguiente jugador, ya que este será el que tenga que robar las 4 cartas correspondientes. Se llama al *Game* con la función *drawCard* a la que se le pasa el jugador que tiene que robar y el número de cartas.

Como se puede observar hacemos el mínimo entre el número de cartas que tiene que robar y el tamaño del *stack* (montón de cartas para robar) ya que si quedan menos cartas en el *stack* de las que el jugador tiene que robar, no tiene sentido que las robe. Se añaden las cartas a la lista de cartas del jugador y se eliminan del *stack*.

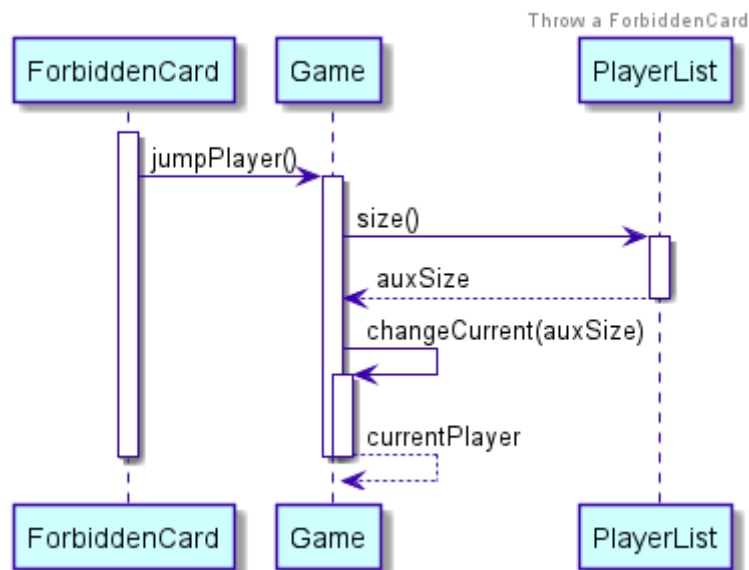
El cambio de color correspondiente, se hará mediante la interfaz grafica, a través de un cuadro de dialogo.



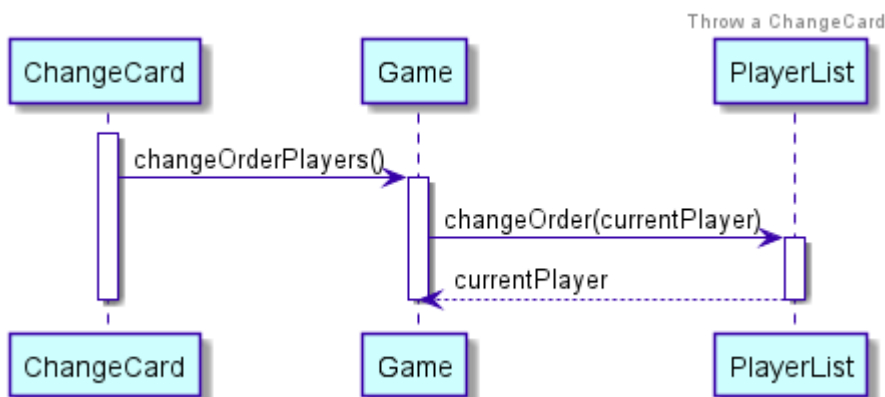
En el caso del +2 el funcionamiento es el mismo, únicamente diferenciándose en que el jugador siguiente debe robar 2 cartas en lugar de cuatro.

Cabe destacar que los jugadores que reciban un +2 o un +4, no tendrán opción a no robar las cartas, es decir, no podrán tirar un +2 o un +4 encima independientemente de que lo tengan en su mano o no.

Si el usuario tira un prohibido, el juego debería saltar al siguiente jugador, esto se hace mediante la función *changeCurrent(size)* en la que se realiza la siguiente operación, $currentPlayer = (currentPlayer + 1) \% size$ siendo size el tamaño de la lista de jugadores en esa partida. Así conseguimos que el siguiente jugador pase a ser el correspondiente debido a la carta de prohibido.

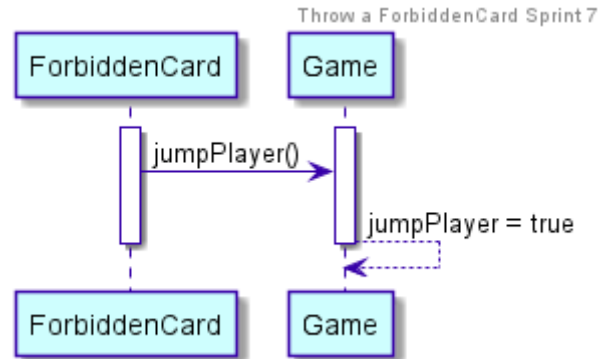


Si el usuario tira un Cambio de Sentido, como tenemos la lista de jugadores ordenada, en la función *changeOrder(currentPlayer)* la invertimos con la función *Collections.reverse(list)* y devolvemos $size - current - 1$ que será el *currentPlayer* una vez invertida la lista.

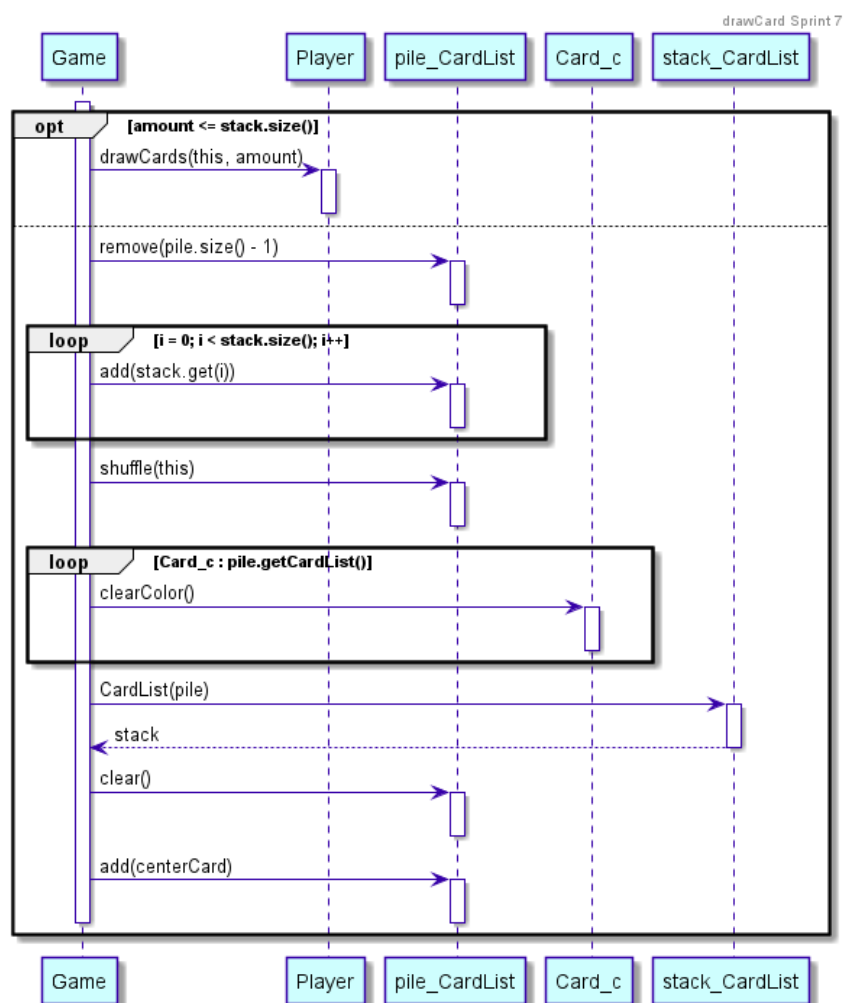


Sprint 7

Hemos cambiado la forma de tirar un prohibido ahora es una variable que suma uno, como se ha visto anteriormente en la Historia de Usuario 1, en el *update* de *Game*.



Hemos cambiado el método *drawCard*, para que no se robe el mínimo entre las cartas que se tienen que robar y las que quedan en el montón, ahora en todos los casos se robará el número correspondiente de cartas.



5. Como usuario quiero conocer los nombres de los otros jugadores de la partida.

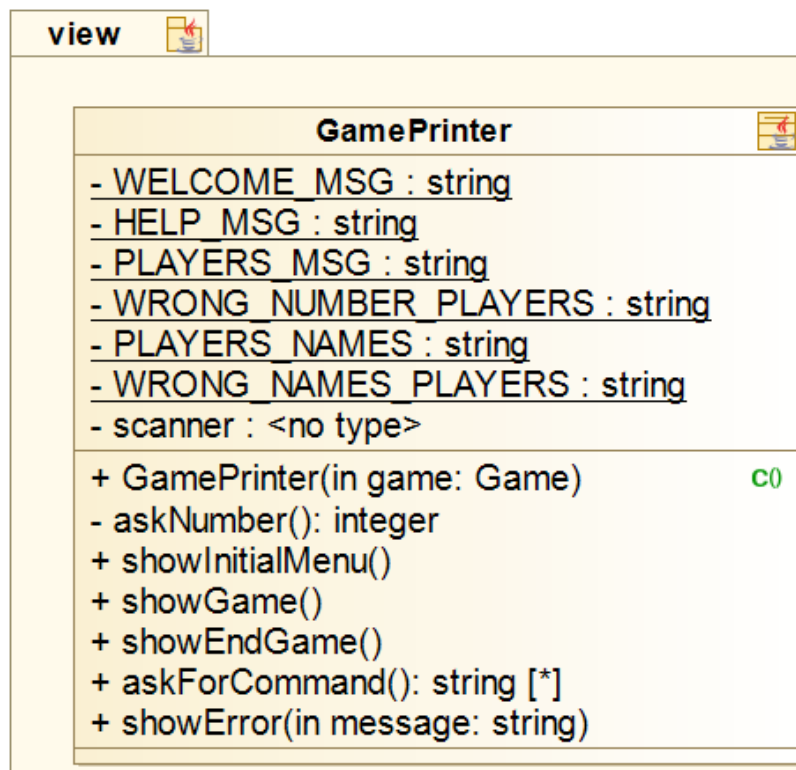
Se mostrará al usuario los nombres del resto de los usuarios que están participando en su misma partida.

- Estimación: Se estima que en realizar esta historia se emplee 1 semana.
- Priorización: Debería tenerlo.
- Criterios de aceptación: Los usuarios podrán ver el nombre del resto de participantes de la partida.

Sprint 2

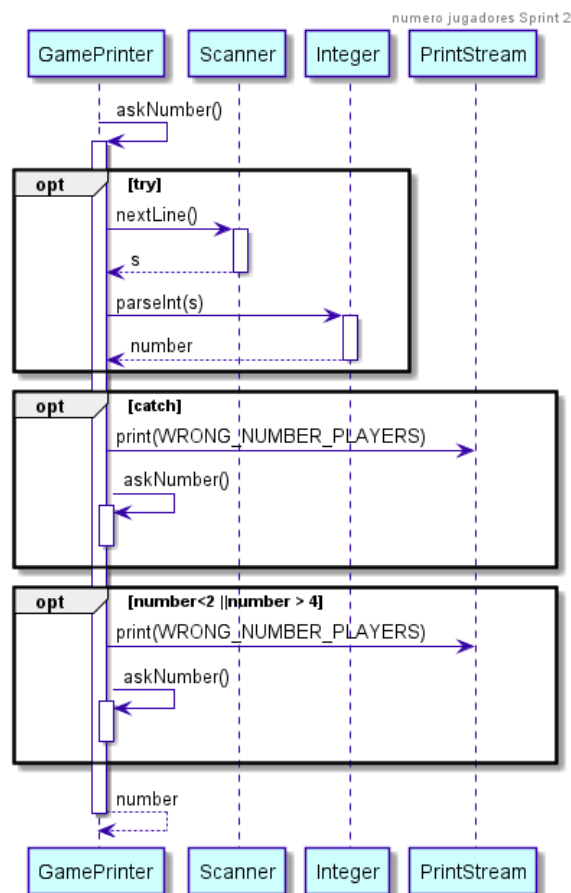
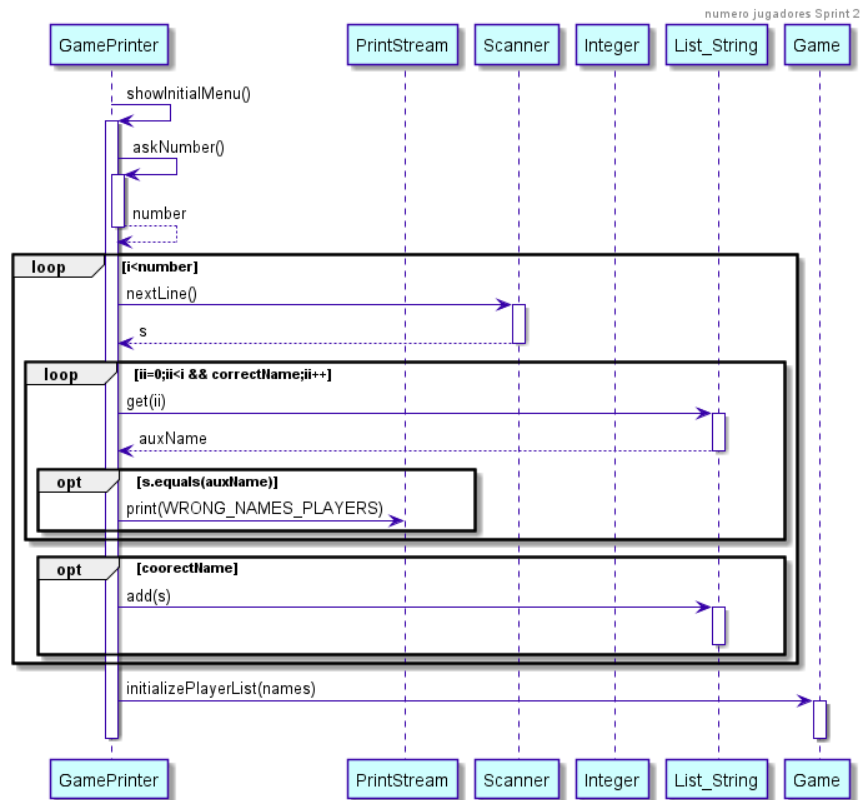
Creamos esta historia en el Sprint 2, pues anteriormente nos parecía que al jugar en consola, si la única manera de identificarse es por número de jugador podría resultar confuso. Sin embargo al crearla e implementarla en nuestro código, nos damos cuenta de la similitud que tiene con la historia de usuario 2: “Como usuario quiero seleccionar el número de jugadores de la partida”, por lo que como hemos dicho al hablar de esta, consideramos que era mejor fusionarlas, ya que el código de ambas iría de la mano.

Los usuarios dejan de identificarse por número, y pasan a identificarse por un identificador que introducen al comienzo de la partida.



Como hemos dicho anteriormente ahora el *showInitialMenu()* se encarga de además de establecer el número de jugadores, que el usuario pueda identificarse a través de un nombre que escribe por pantalla. Además nos aseguramos de que los nombres que se introduzcan no estén repetidos, la función encargada de crear los jugadores con cada uno de sus nombres es *inicializaPlayerList(names)*.

A continuación, se muestra el diagrama secuencial de la función en cuestión.



Sprint 5

Posteriormente, a pesar de su primera unión, decidimos fusionarla con la historia de usuario 9: “Como usuario quiero poder jugar contra la máquina en diferentes dificultades”. Dentro de esta historia ya tenemos la opción de elegir el número de jugadores, el tipo (bot o humano), y el nombre, por esto, consideramos que esta historia puede ser eliminada, ya que el objetivo general está descrito en la 9.

6. Como usuario quiero poder guardar una partida en curso.

Durante toda la partida existe la opción de *SAVE GAME*, con la que se podrá guardar la partida que se está jugando en ese momento.

Todas las partidas se almacenan individualmente en un fichero que es generado al pulsar el botón descrito anteriormente, y posteriormente todas ellas almacenan su identificador en un fichero general.

- Estimación: Se estima que en realizar esta historia se emplearán 2 semanas.
- Priorización: Debería tenerlo.
- Criterios de aceptación: El usuario podrá guardar la partida que está jugando en el momento en el que él desee.

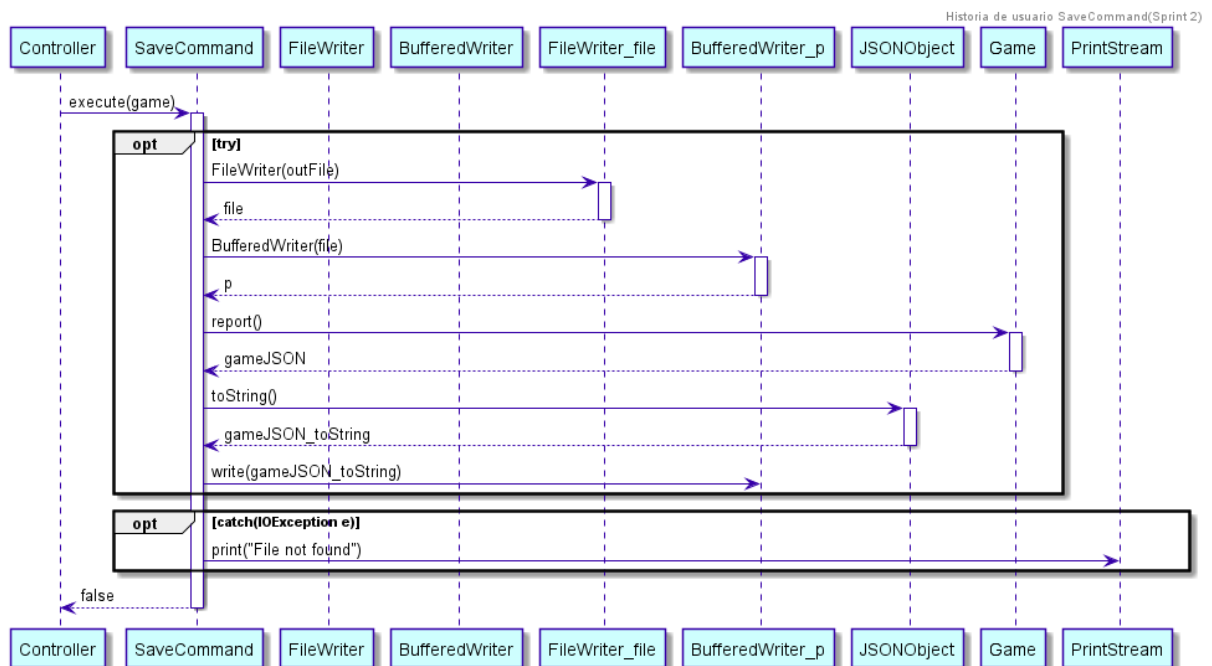
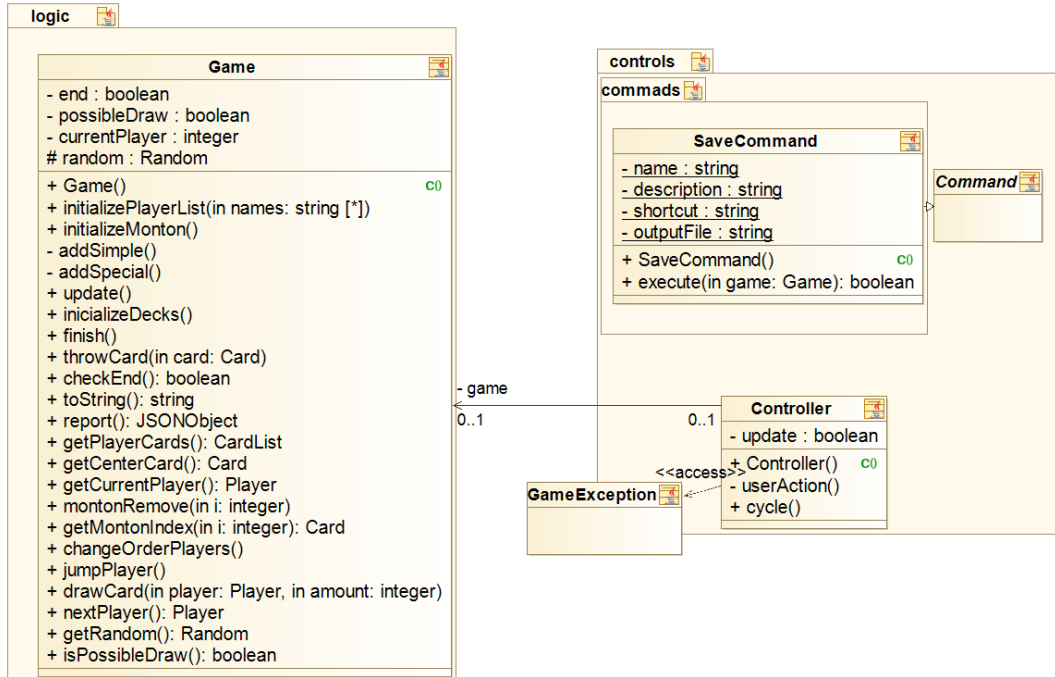
Vemos a continuación un ejemplo del archivo general, donde hay una única partida guardada, con identificador *Game1*, y toda la información de la partida queda registrada en el archivo “save0.json”.

```
{"savefiles":[{"filename":"saves/save0.json","id":"Game1"}]}
```

Las partidas se guardarán en formato `JSONObject`, como se ha comentado anteriormente.

Sprint 2

Esta historia de usuario es implementada en el segundo sprint, para permitir a un jugador guardar la partida en la que está con la posterior intención de que esta se pueda jugar más tarde. En este sprint, como la única manera de jugar era mediante la consola, se crea la acción como un comando.



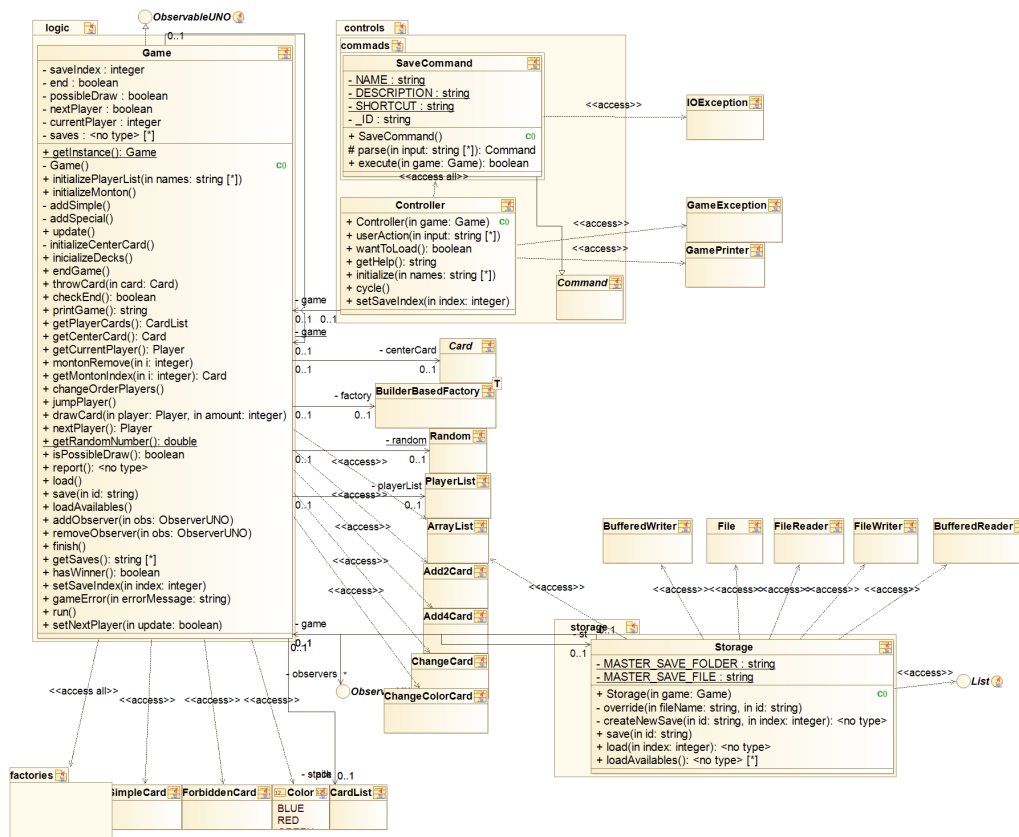
Sprint 3

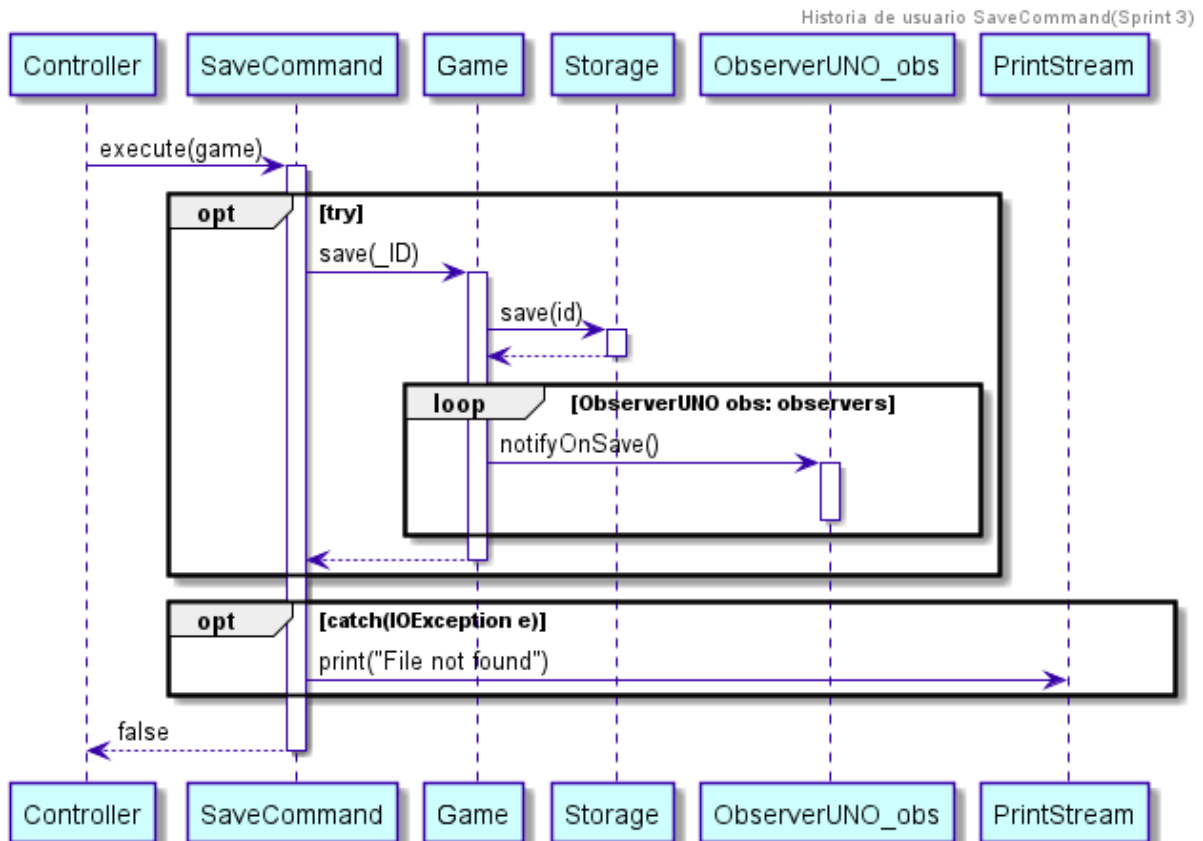
En este Sprint creamos la clase *Storage* que se encarga de la comunicación entre el sistema del usuario y el modelo del programa, es decir, se encarga de tomar y crear archivos de guardado en el ordenador del jugador.

La idea perseguida al crear esta clase es conseguir abstraer la comunicación entre el modelo y el sistema de archivos con el objetivo de hacerlo genérico. De esta forma, si en algún momento se quisiese cambiar la forma de almacenamiento o el formato de los archivos de guardado la tarea se haría de manera trivial.

El guardado del juego está diseñado para permitir varios archivos distintos simultáneos permitiendo al usuario elegir qué partida desea reanudar. Para conseguir este objetivo nos decidimos por utilizar un archivo maestro que contiene una lista con los identificadores de las partidas registradas y el archivo donde se localizan. De esta forma es simple mostrar al usuario aquello que ha guardado para que pueda elegir lo que más le convenga (*loadAvailables* sirve para esta misma funcionalidad). La clase también permite la sobrescritura de los archivos en el caso de que la partida debe volver a ser pausada y reanudada en otro momento.

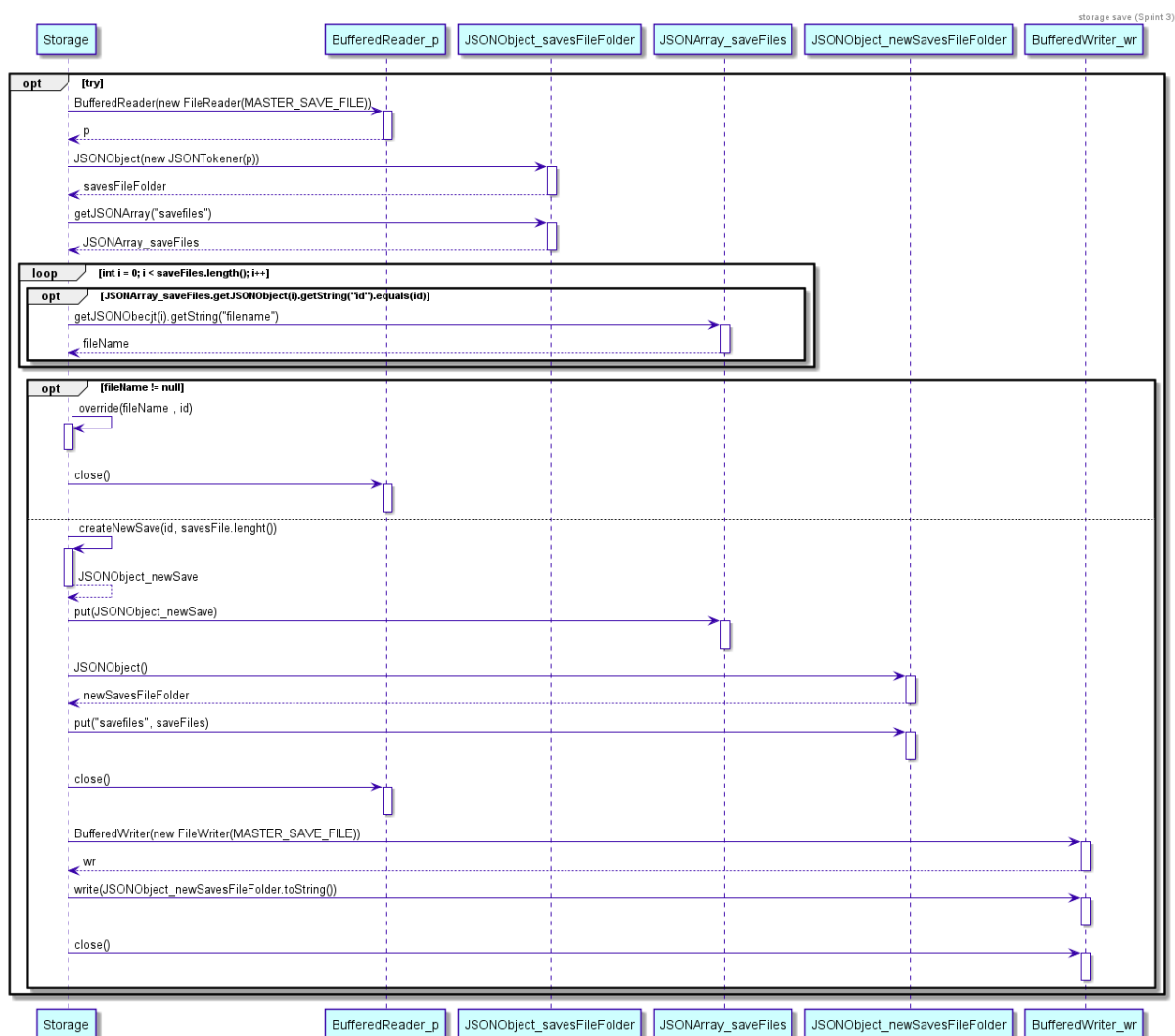
Por último, la clase tiene una estructura estática, es decir, todos los métodos son estáticos. El razonamiento detrás de esta decisión es que el acceso al sistema no debe depender del estado de ningún objeto.



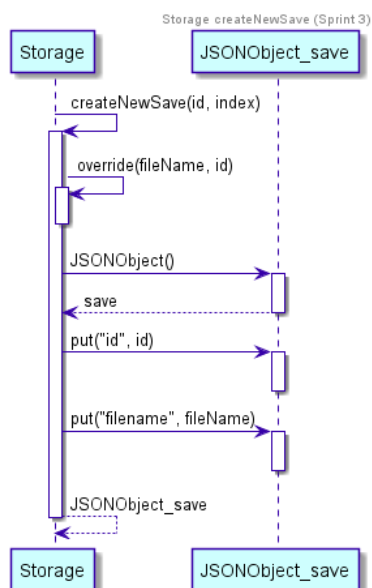


Se crea un `BufferedReader` que lea el fichero donde tenemos guardados los `JSONObject`s de todos los ficheros de partidas guardadas. Si alguno de los `id` de los `JSONObject`s guardados coincide con el nombre que ha introducido el usuario para guardar su partida entonces se llamará a la función `override` que cambiará la partida guardada anteriormente en ese fichero por la nueva.

Si no se encuentra el nombre del fichero introducido, el juego creará un nuevo fichero donde se guarde la partida.



Como se puede observar a continuación la clase *Storage* contiene un método encargado de crear el *JSONObject* que identificará el nuevo fichero donde se guardará la partida.



7. Como usuario quiero poder cargar una partida que he guardado con anterioridad.

El usuario podrá cargar y así reanudar una partida que ha guardado con anterioridad. Todas las partidas se encuentran guardadas en ficheros, de los cuales se extraen los datos para dejar esta misma exactamente por el punto en el que se abandonó.

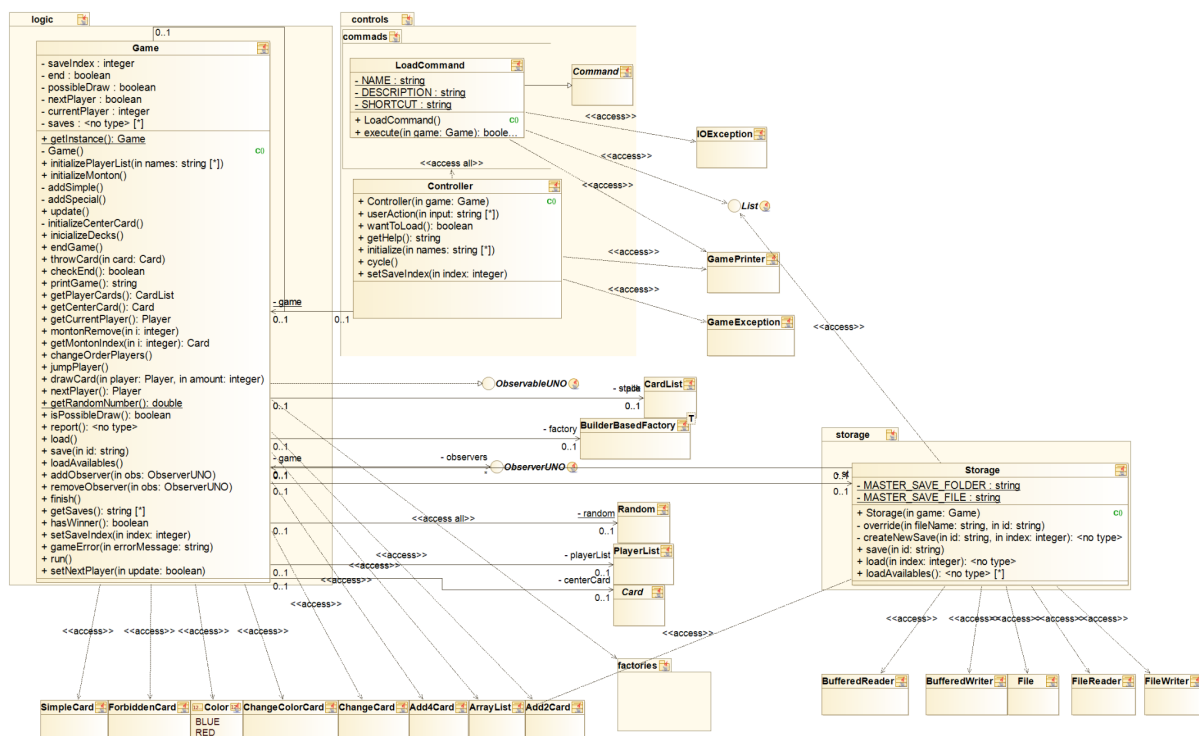
- Estimación: Se estima que en realizar esta tarea se emplearán 2 semanas.
- Priorización: Debería tenerlo.
- Criterios de aceptación: Se podrá cargar y por lo tanto reanudar una partida que ha sido previamente guardada.

Sprint 2

En este sprint se plantea el objetivo de esta historia de usuario, que es básicamente poder construir una partida del juego mediante los datos guardados en un fichero anteriormente. Sin embargo no se llegó a implementar esta funcionalidad hasta el Sprint siguiente.

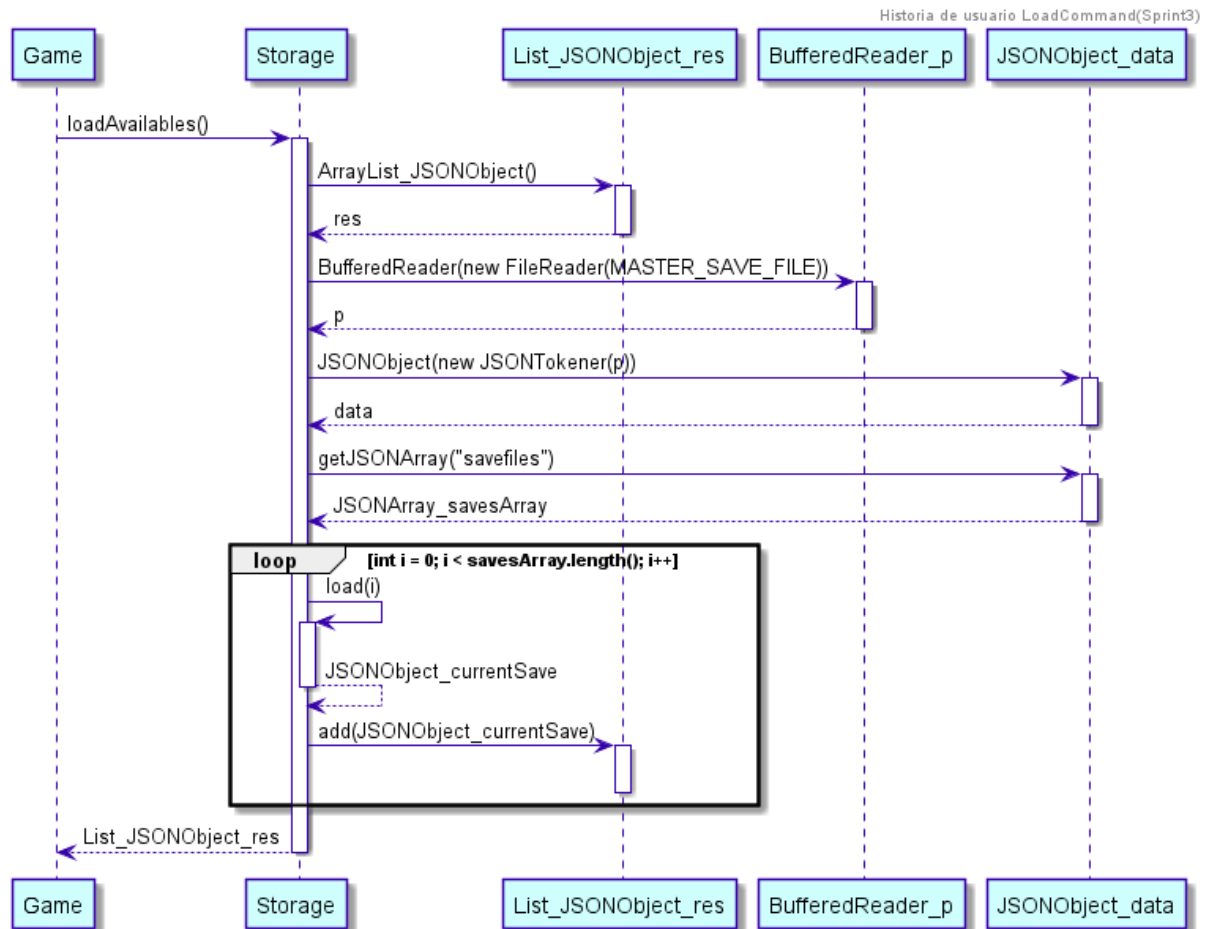
Sprint 3

Ya se consigue implementar el *LoadCommand*, el cual si el usuario lo introduce por consola se cargará una partida. En un principio el *LoadCommand* estaba incluido en la lista de *Commands* por lo que si el usuario estaba jugando una partida y en la mitad el usuario introducía el *LoadCommand* se borraba la partida que estaba en curso y se cargaba una nueva. A lo largo del Sprint nos dimos cuenta de que esto no tenía mucho sentido pues el cargar una partida solo debería ser posible al comienzo del juego, sin tener posibilidad de introducirlo posteriormente. Por lo tanto, lo quitamos de la lista de *Commands* de forma que si el usuario volvía a introducir el *LoadCommand* el juego informaba de que ese *Command* no era válido.

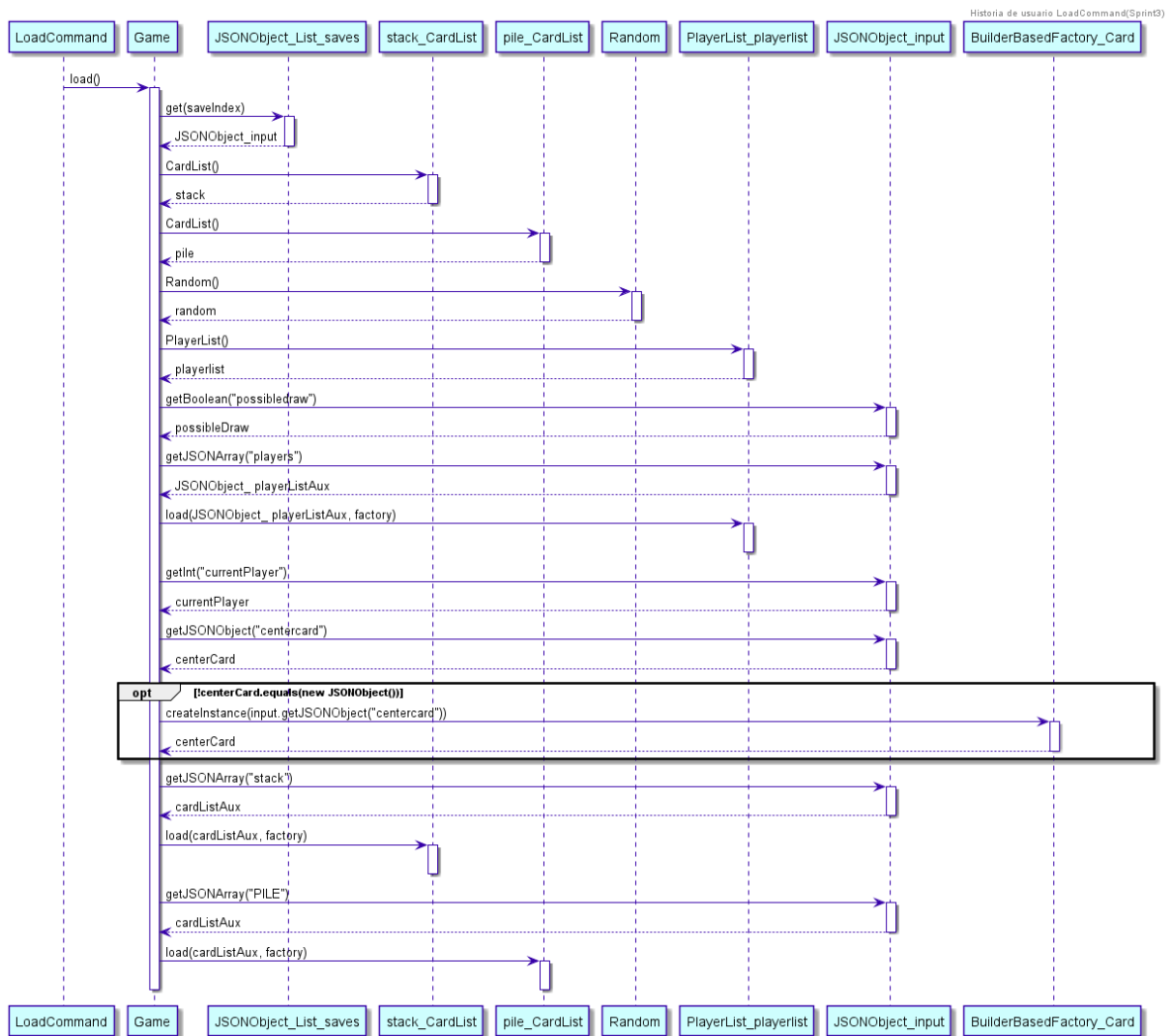


Como se puede observar el *execute(game)* del *LoadCommand* era bastante simple, primero se identificaban qué partidas eran posibles cargar y posteriormente se cargaba si se había encontrado el *fileName* correspondiente.

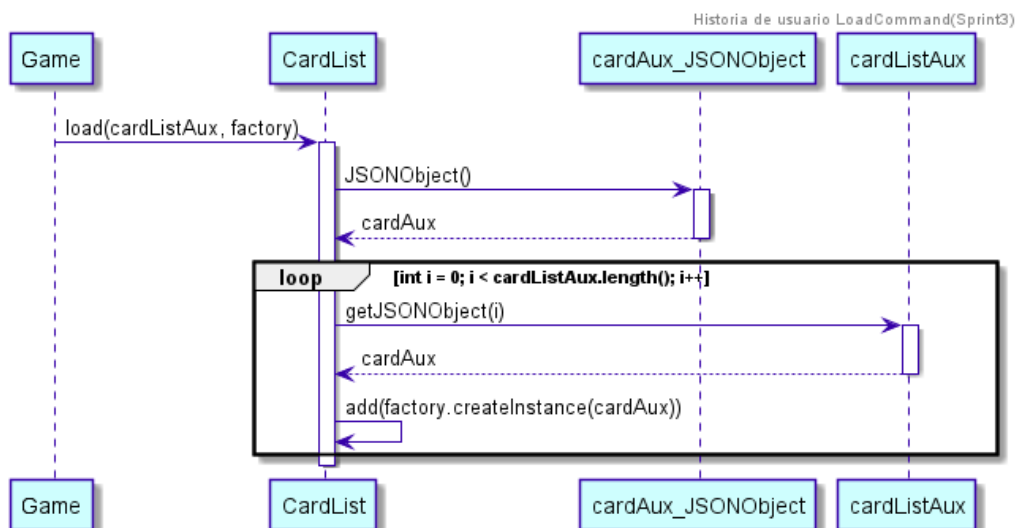
El *loadAvailables()* de *Game* llama al *loadAvailables()* de *Storage* clase encargada de toda la lógica de guardar y cargar partidas. Se lee el fichero donde están las partidas guardadas y se guardan en una lista de *JSONObject*s que devuelve el método y se convierten en el nuevo *saves* de *Game*. Un parámetro de tipo lista de *JSONObject*s que contiene las partidas guardadas.



La función `load(index)` de `Game` se encarga de ir desglosando y guardando como parámetros del `Game` todos los `JSONObject`s pertenecientes al fichero para poder cargar la partida. En el diagrama se puede ver cómo se van cargando el *pile*, *stack* y *playerList*.

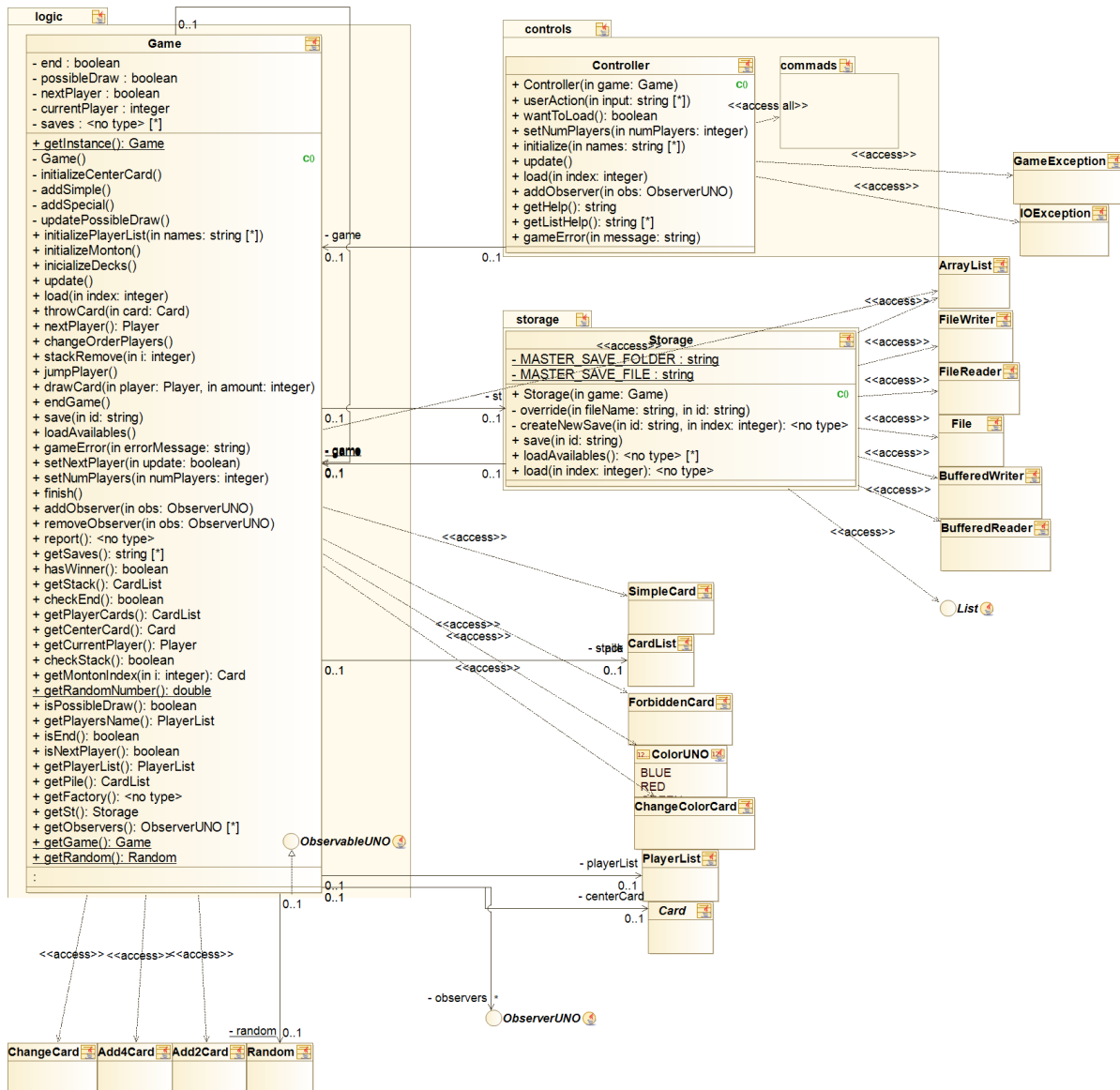


Hemos incluido además el diagrama del método *load* de *CardList* en el que se puede observar cómo se obtiene cada *Card* del *JSONArray* y mediante la factoría se crea la instancia de *Card* que se añade en la *CardList*. La funcionalidad de *load* de *PlayerList* es completamente análoga.



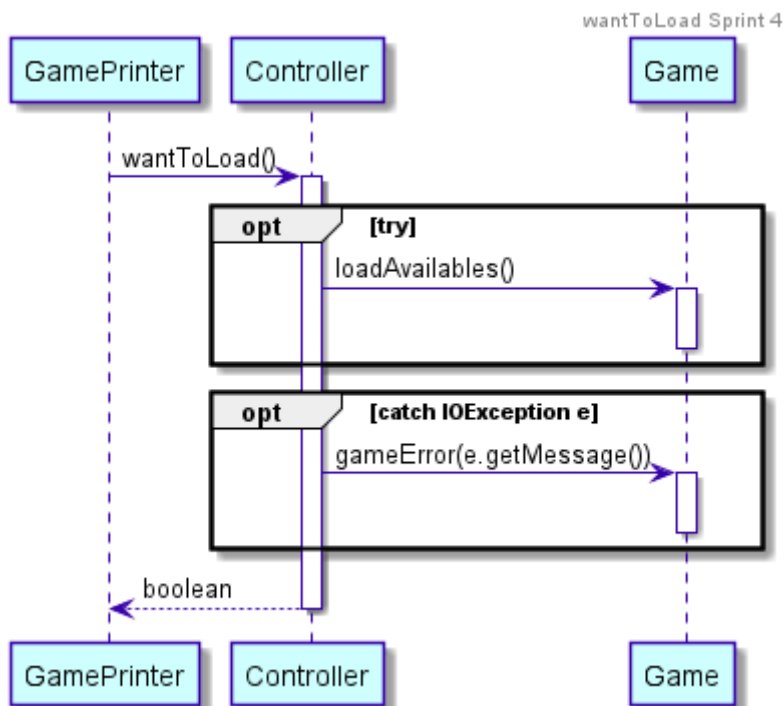
Sprint 4

Se eliminó el comando *Load* ya que no era necesario pues esta acción solo se puede hacer al principio de la partida por lo que se da la opción al usuario de cargar una partida directamente, sin necesidad de tener un comando exclusivamente para ello.

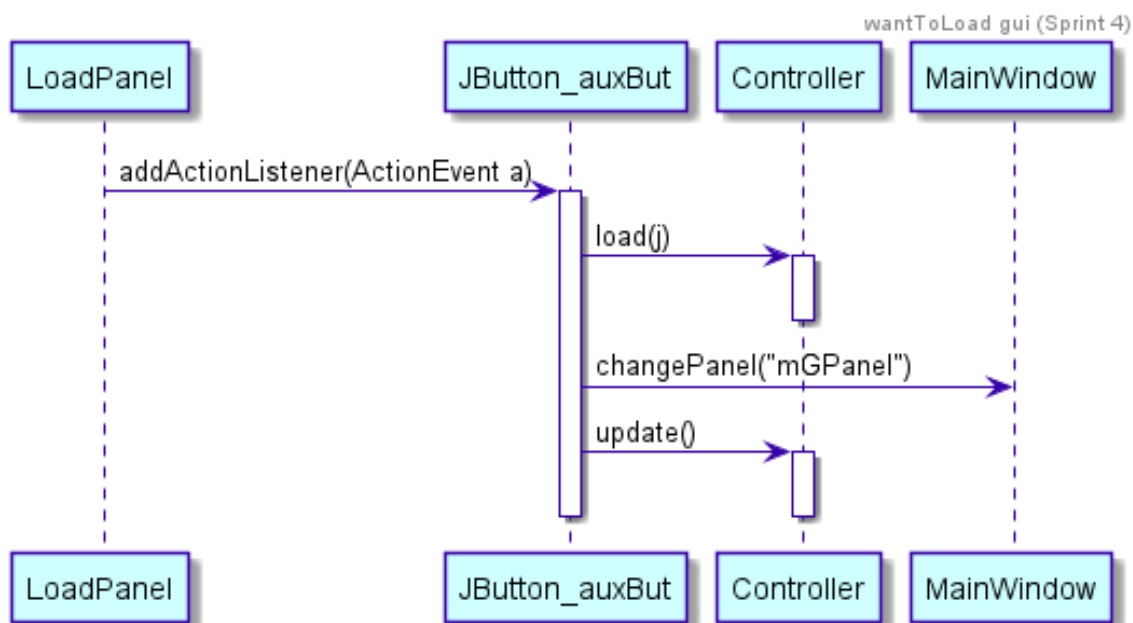


Si el juego se está jugando a través de la consola, al comienzo de la partida, se preguntará al usuario si desea cargar una partida, desde el *showInitialMenu()* del *GamePrinter*, si el usuario quiere cargarla, se llama al controller que llama al método *loadAvailables()* de *Game* cuya estructura y funcionalidad es igual a la del Sprint

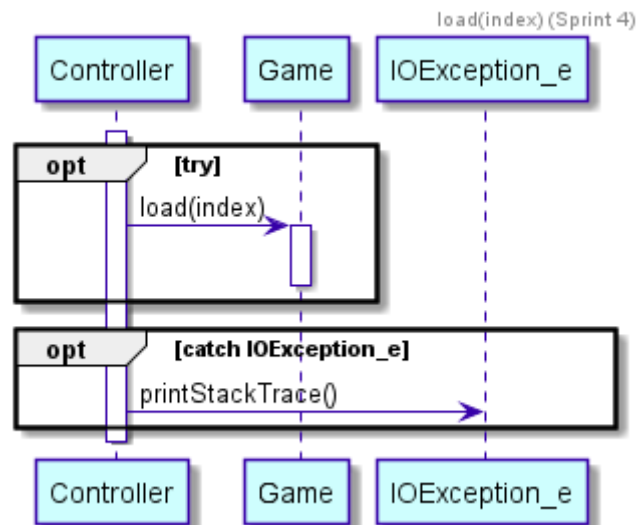
anterior. Si es posible cargarla se devuelve true y se carga la partida, si no ha sido posible se lanzará una excepción.



Si por el contrario el juego se está jugando a través de la interfaz gráfica, será el *LoadPanel* el encargado de cargar la partida. Si en el *InitialPanel* el usuario pulsa el botón *Load Game* se mostrará en pantalla el *Load Panel*, en el cual se mostrarán los nombres que el usuario haya introducido para identificar las distintas partidas previamente guardadas. El usuario pulsará la partida que desee cargar y esta se cargará a través del método `load(index)` de *Controller*.



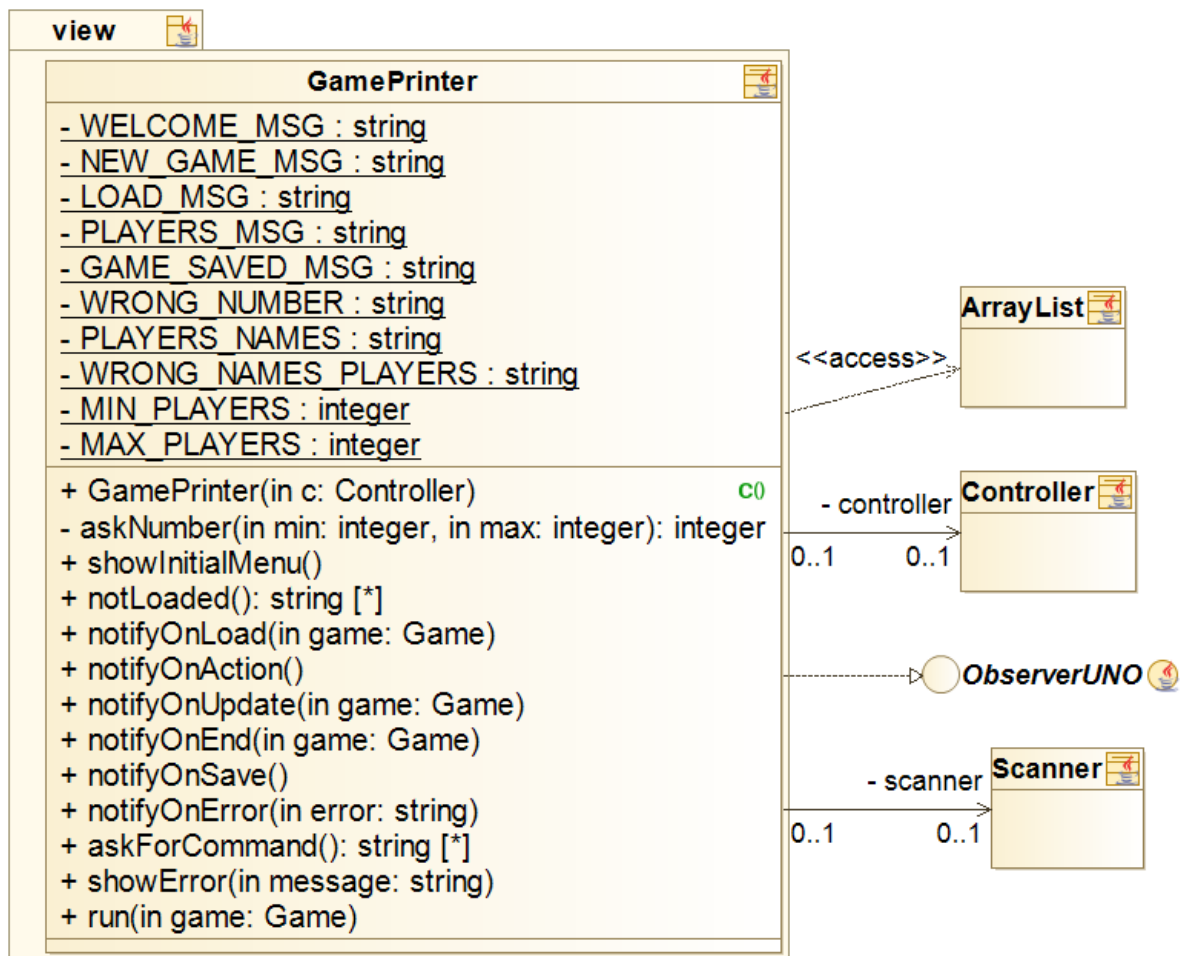
A continuación se muestra el método *load(index)* de *Controller* este llama al *load(index)* de *Game* cuya funcionalidad y estructura es igual a la mostrada en el anterior Sprint.

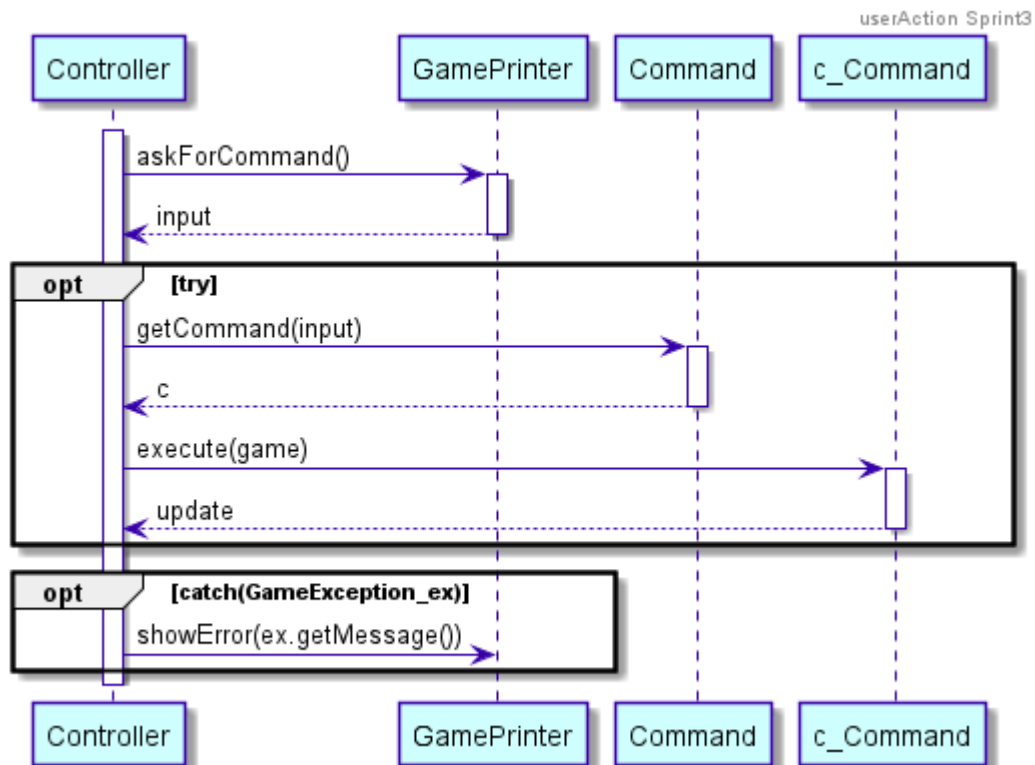


8. Como usuario quiero poder introducir mediante teclado un comando y que se ejecute en la partida. (ELIMINADA)

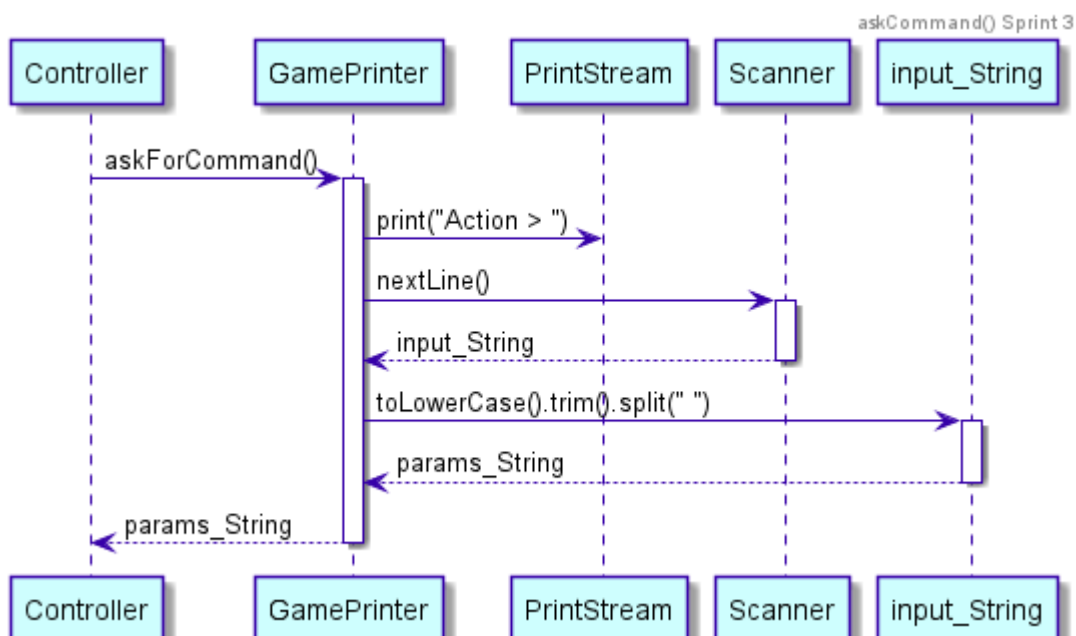
Sprint 3

Esta historia de usuario es introducida en este sprint aunque realmente ya estaba implementada por la primera historia.





El *Controller* a través del método `askForCommand()` de *GamePrinter* recibe la entrada introducida por el usuario en la consola. La entrada la gestionamos convirtiéndola en minúsculas, ya que de esta forma aunque el usuario haya metido el *Command* en mayúsculas o intercalando minúsculas y mayúsculas no de error y el juego lo identifique de manera correcta.



Sprint 5

Nos damos cuenta de que la historia como tal, a pesar de ser necesaria, solo añade líneas de documentación que realmente no aportan nada sustancial al proyecto, por eso consideramos que debía ser eliminada.

9. Como usuario quiero poder jugar contra la máquina en diferentes dificultades.

Tras elegir el número de jugadores de la partida, el usuario podrá elegir para cada uno de ellos si quiere que sea una persona humana, jugando igual que él, o un bot. De este último tipo de jugador, se podrá elegir el nivel de dificultad que tendrá. Las tres dificultades son:

Fácil

En este modo, la máquina lanzará la primera carta que pueda de su lista. En caso de ser una carta de las encargadas de los cambios de color (*Add4Card* o *ChangeColorCard*), se seleccionará de manera aleatoria el color a elegir, sin tener en cuenta el resto de cartas que tiene en su mano. Este bot no será inteligente, y se le podrá ganar con facilidad.

Medio

Para cada turno de juego creamos una lista con todas las cartas de su actual lista que puedan ser lanzadas en ese momento, y contamos el número de cartas de cada color, así, cuando el bot lance una carta, elija entre el montón de las disponibles, la del color más repetido.

En el caso de que solo haya dos jugadores, el bot intentará lanzar un prohibido (*ForbiddenCard*), y así repetir turno.

El orden de prioridad de las cartas a lanzar entre las cartas del montón disponibles es:

1. *Add4Card*
2. *Add2Card*
3. *ForbiddenCard*
4. *ChangeCard*
5. *SimpleCard*
6. *ChangeColorCard*

Difícil

La estrategia de juego para este bot tiene un cambio crucial con respecto a la anterior, que es que la máquina, sabrá no sólo el número de cartas del siguiente jugador sino también todas y cada una de ellas. De esta manera resultará casi imposible al humano ganar, ya que el bot echará en todo momento la carta que menos convenga al usuario.

Si se trata de solo dos jugadores, como en la estrategia media, se intentará lanzar un prohibido para que el usuario repita turno. Luego, comprobaremos si para alguna de nuestras cartas, el otro jugador no puede tirar, obligándoles a robar. En el caso de que

el otro contrincante pueda lanzar con cualquier carta que el bot escoja, este se encargará de que sea la que peor le convenga: intentará cambiar al color del cual posea menos cartas o si está a punto de ganar lanzará el *Add4Card*.

El nuevo orden de prioridades de las cartas es:

1. *Add4Card*
2. *ChangeColorCard*
3. *ForbiddenCard*
4. *Add2Card*
5. *ChangeCard*
6. *SimpleCard*

La partida sin embargo sigue su curso normal y con las mismas reglas ya definidas anteriormente.

- Estimación: Se estima que en realizar esta tarea se emplearán 2 semanas.
- Priorización: Debería tenerlo.
- Criterios de aceptación: Se podrá jugar contra un bot al que previamente se le ha seleccionado su dificultad de juego.

Sprint 4

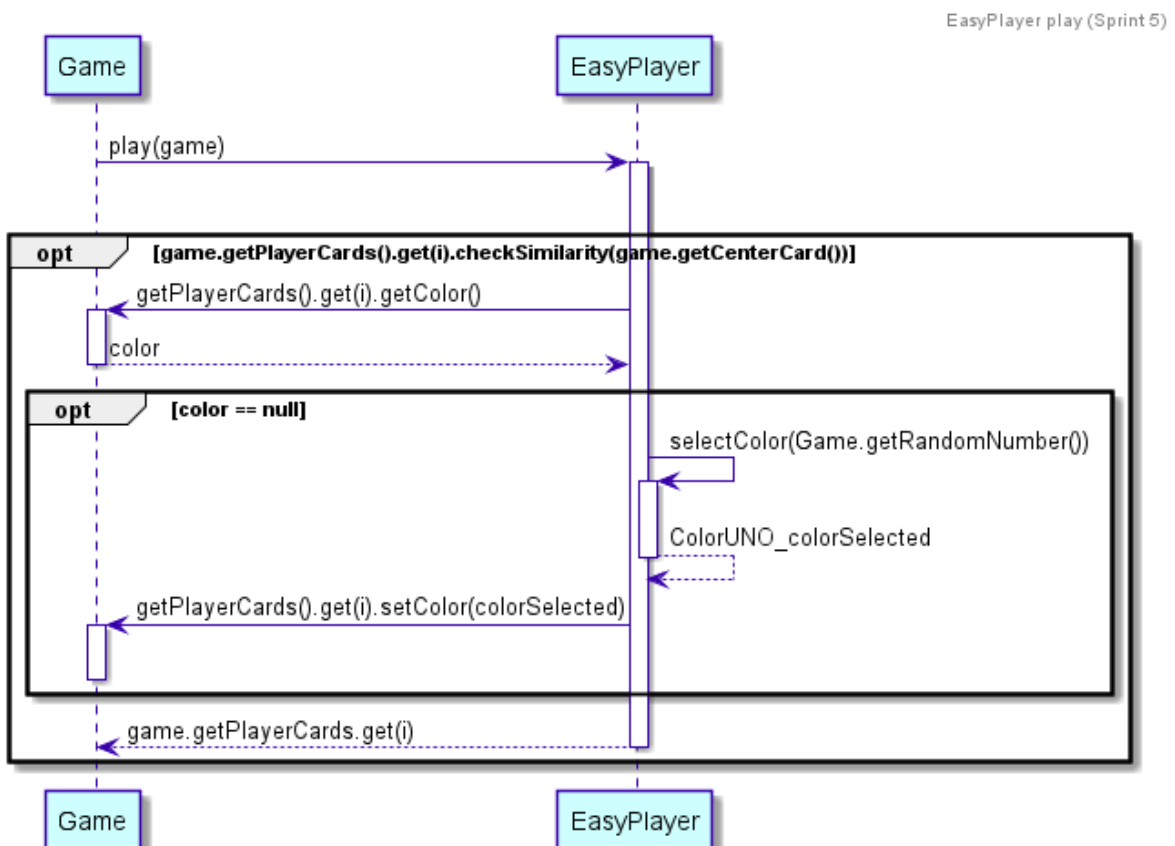
En este sprint aún no se crea la historia de usuario, sin embargo ante la demanda de una funcionalidad que permitiese jugar contra un bot automático en diferentes dificultades, se comienzan a definir las diferentes estrategias que pudiesen complicar en mayor o menor medida la partida para los jugadores humanos.

Sprint 5

La historia es ya creada e implementada, es también ligeramente modificada respecto a lo que se definió en el sprint anterior, pues estos cambios facilitaron la programación de las mismas. Para la realización de esta historia de usuario hemos hecho uso del patrón estrategia que nos permite crear los tres algoritmos definidos anteriormente para poder ser utilizados por diferentes bots *player* que utilizarán cualquiera de ellas según desee en cada partida.

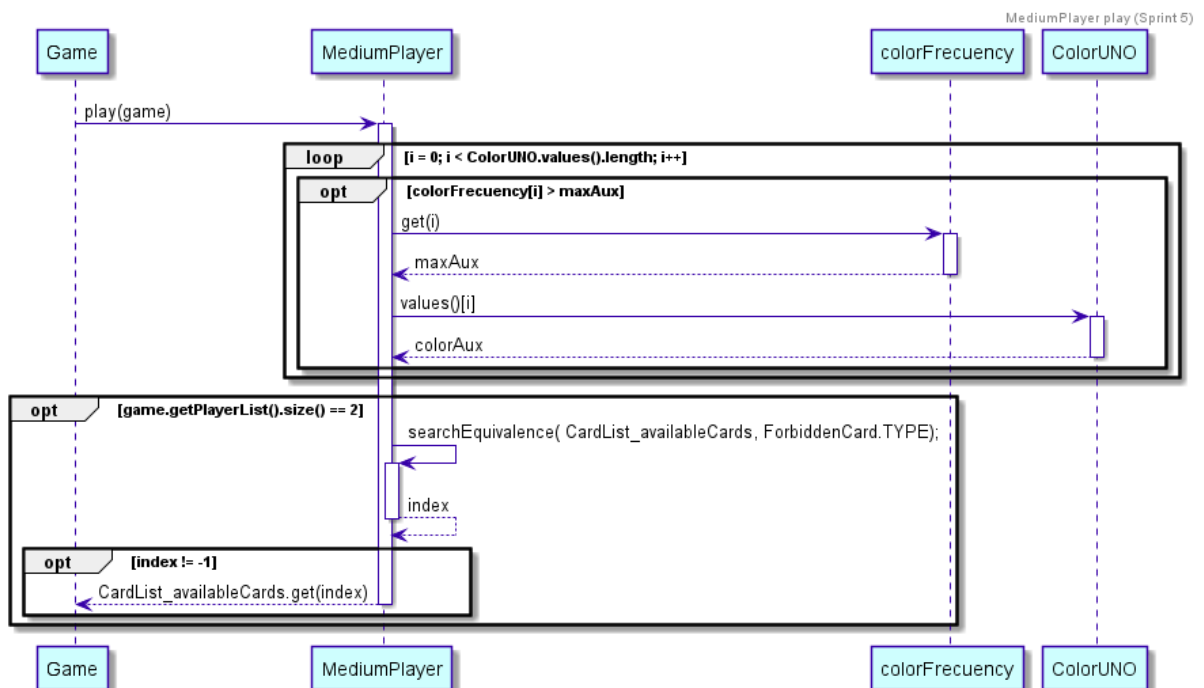
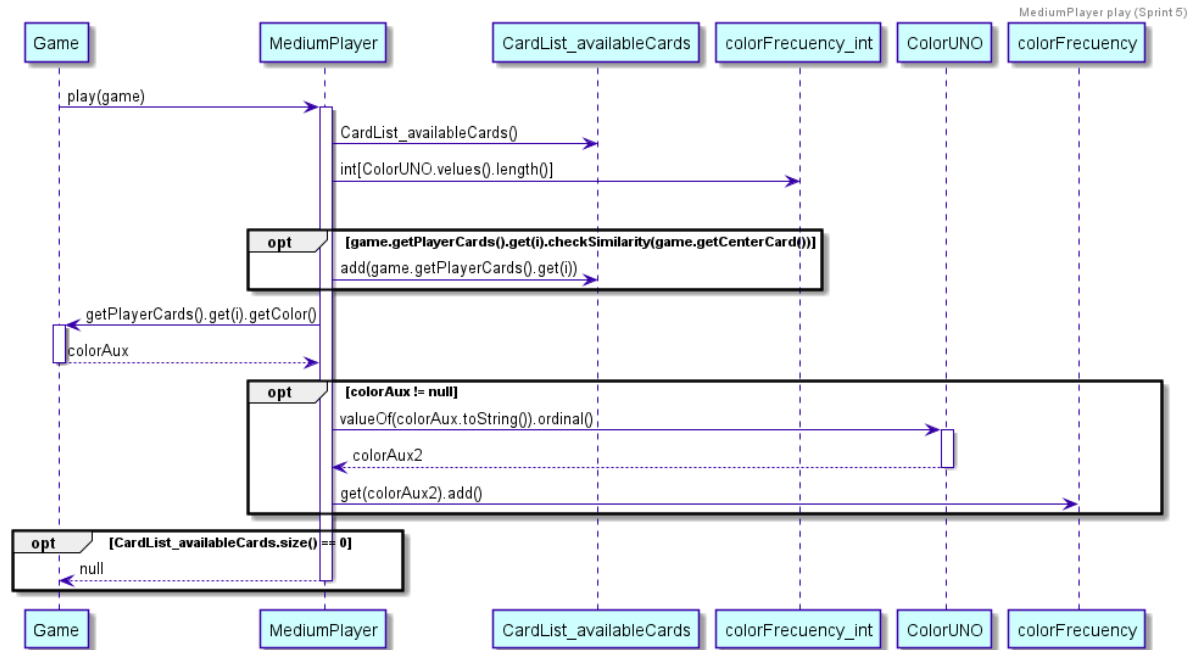
A continuación mostramos los diagramas de secuencia de los métodos *play()* de cada estrategia.

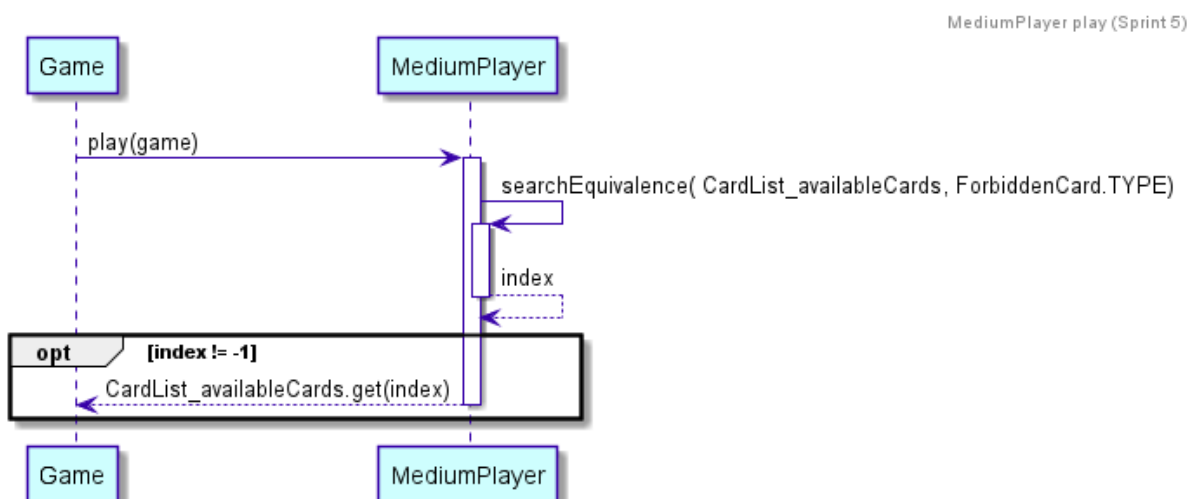
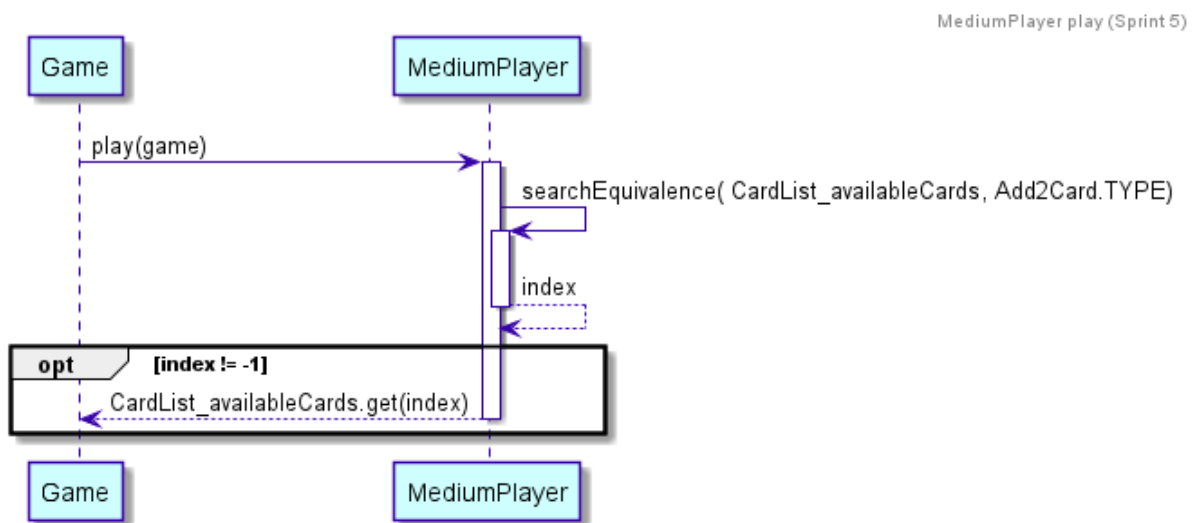
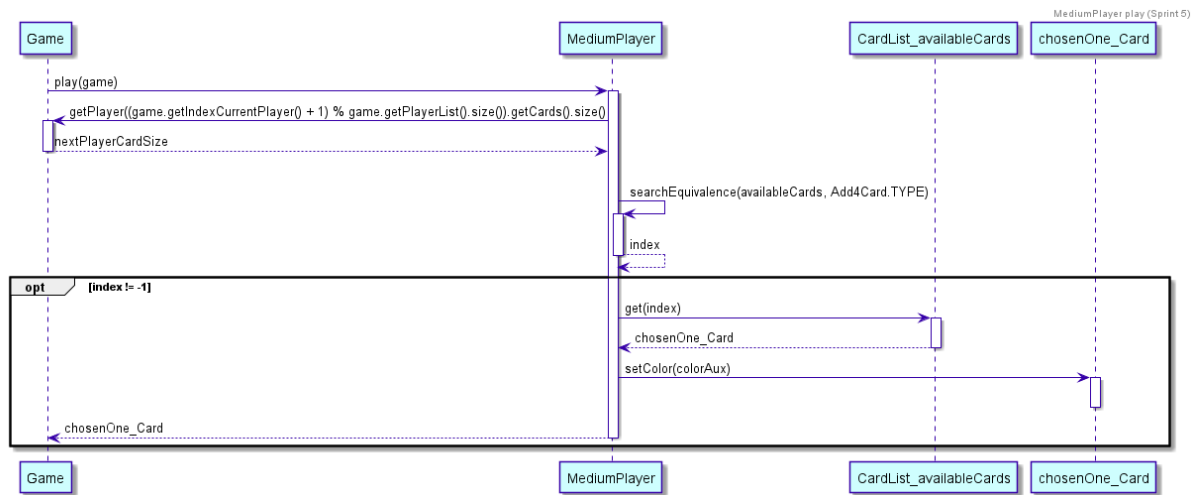
Easy Player



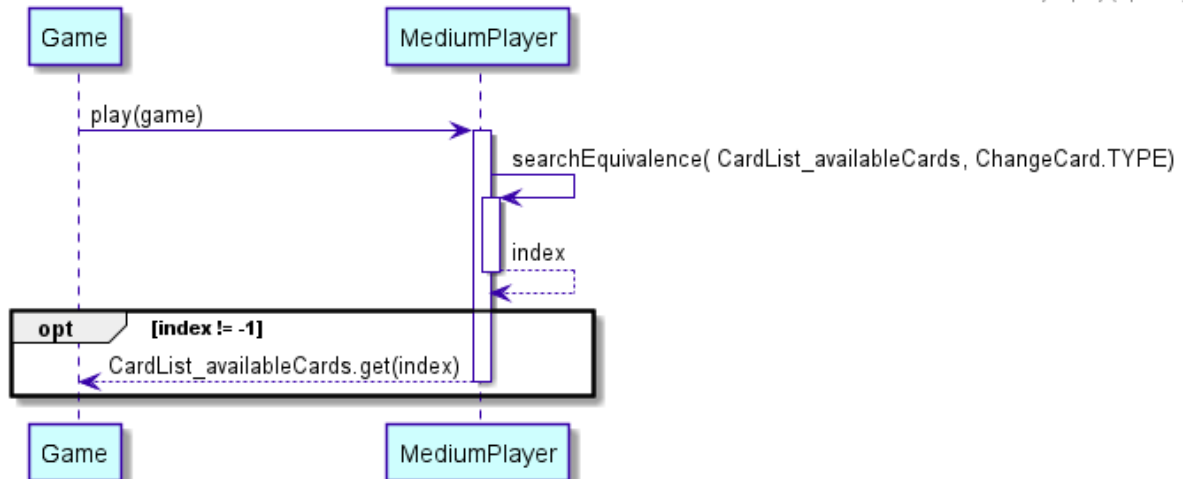
Tanto en el *MediumPlayer* como en el *HardPlayer*, debido a la gran cantidad de datos y la dimensión del diagrama nos vimos obligados a partirlos en diferentes diagramas, pero todos representan distintas partes del mismo método.

MediumPlayer

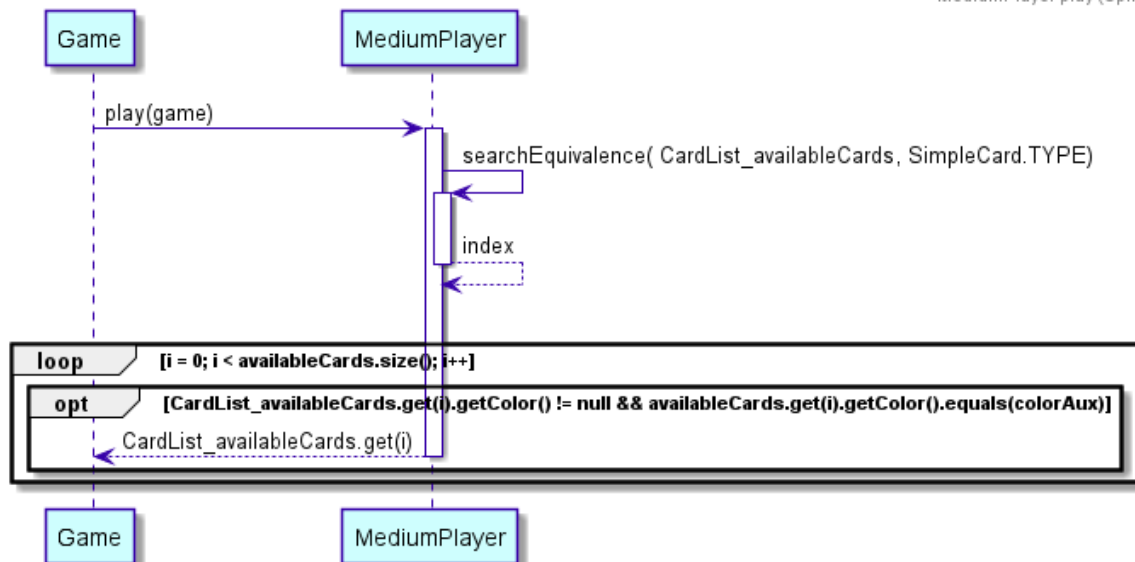




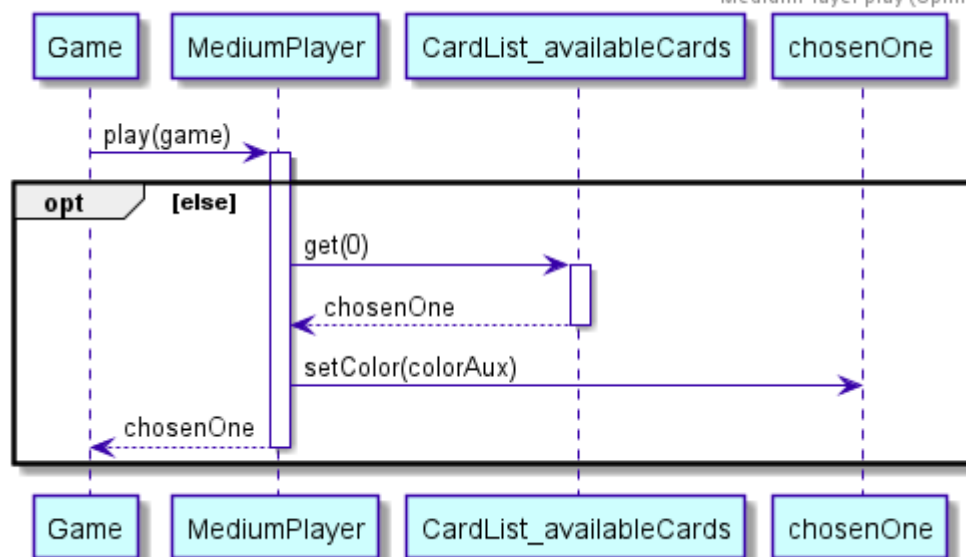
MediaPlayer play (Sprint 5)



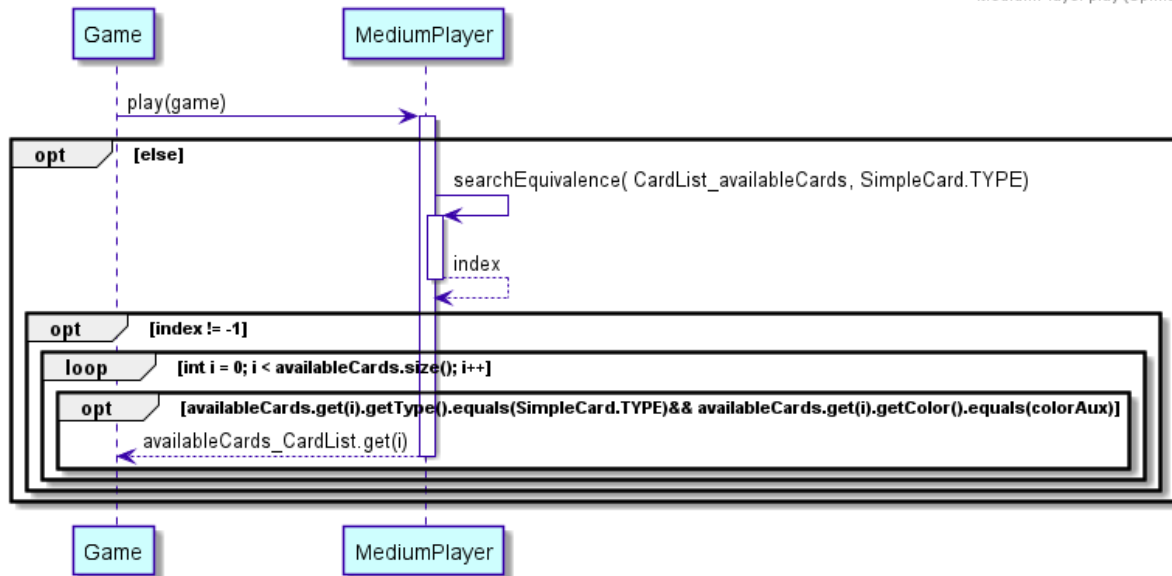
MediaPlayer play (Sprint 5)



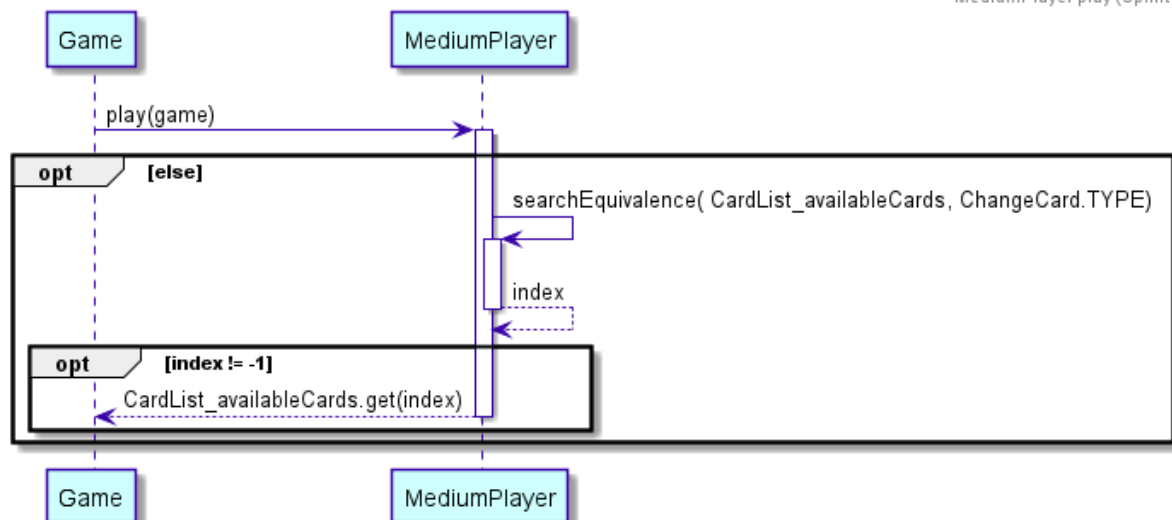
MediaPlayer play (Sprint 5)



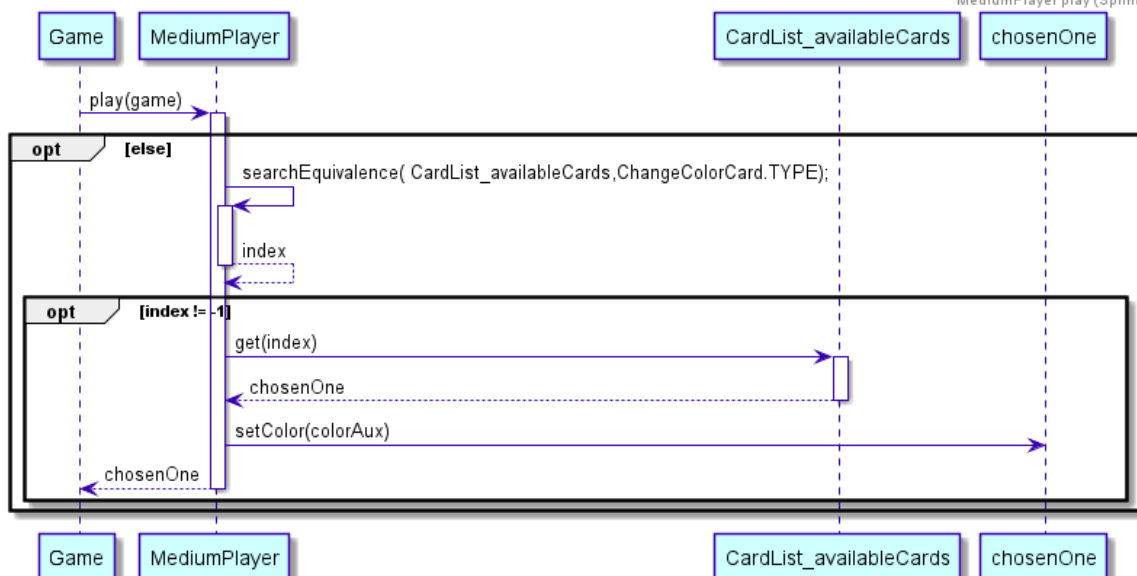
MediaPlayer play (Sprint 5)



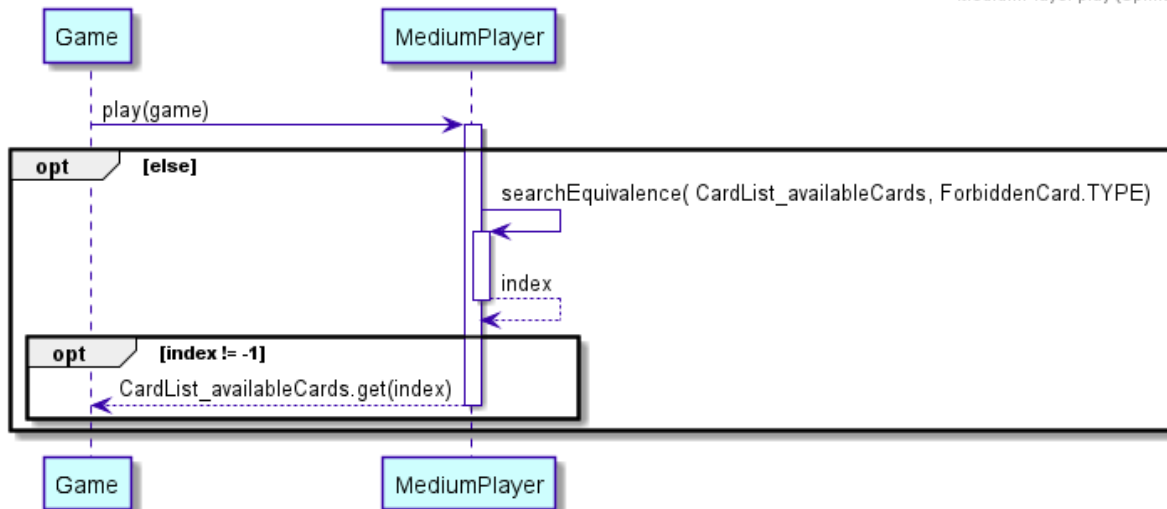
MediaPlayer play (Sprint 5)



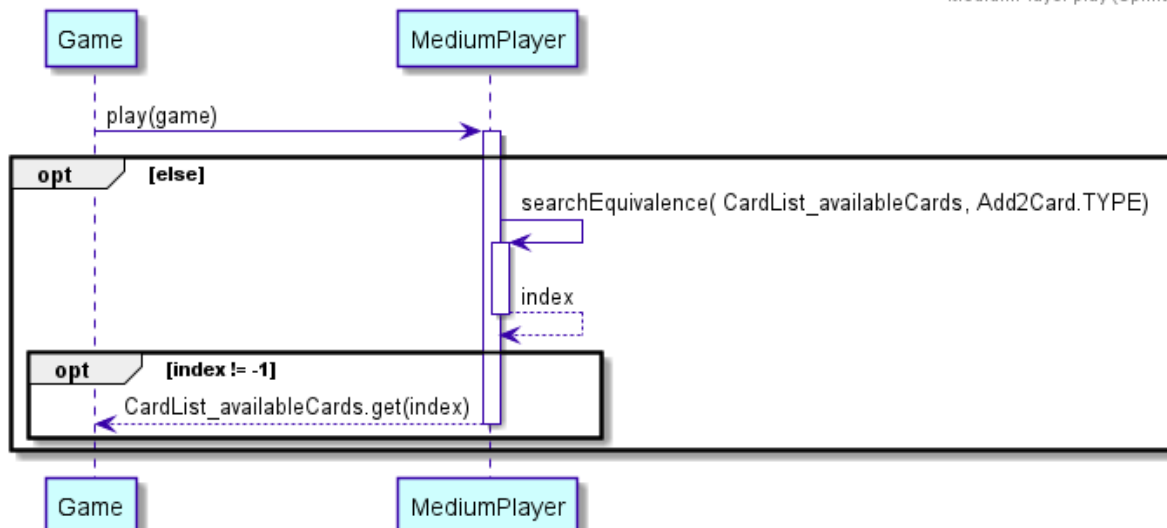
MediaPlayer play (Sprint 5)



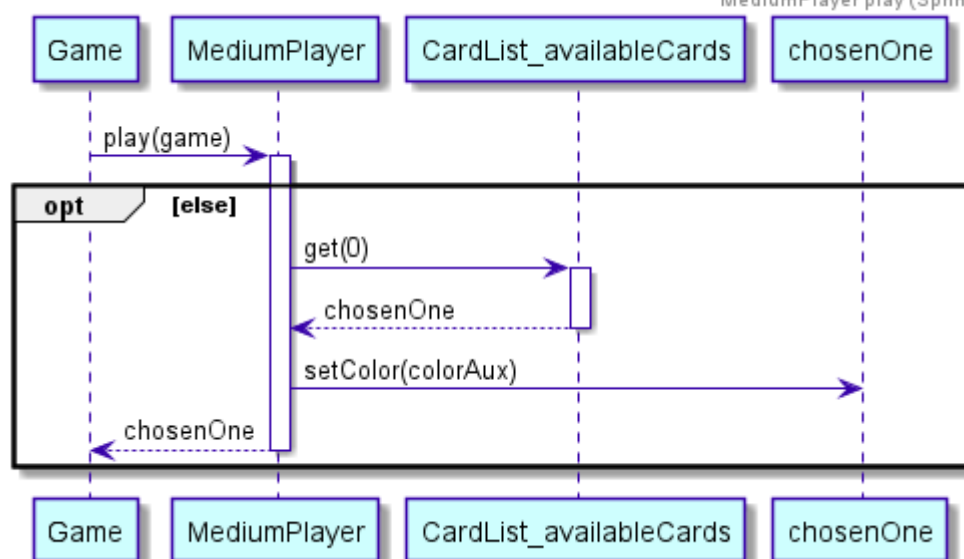
MediumPlayer play (Sprint 5)



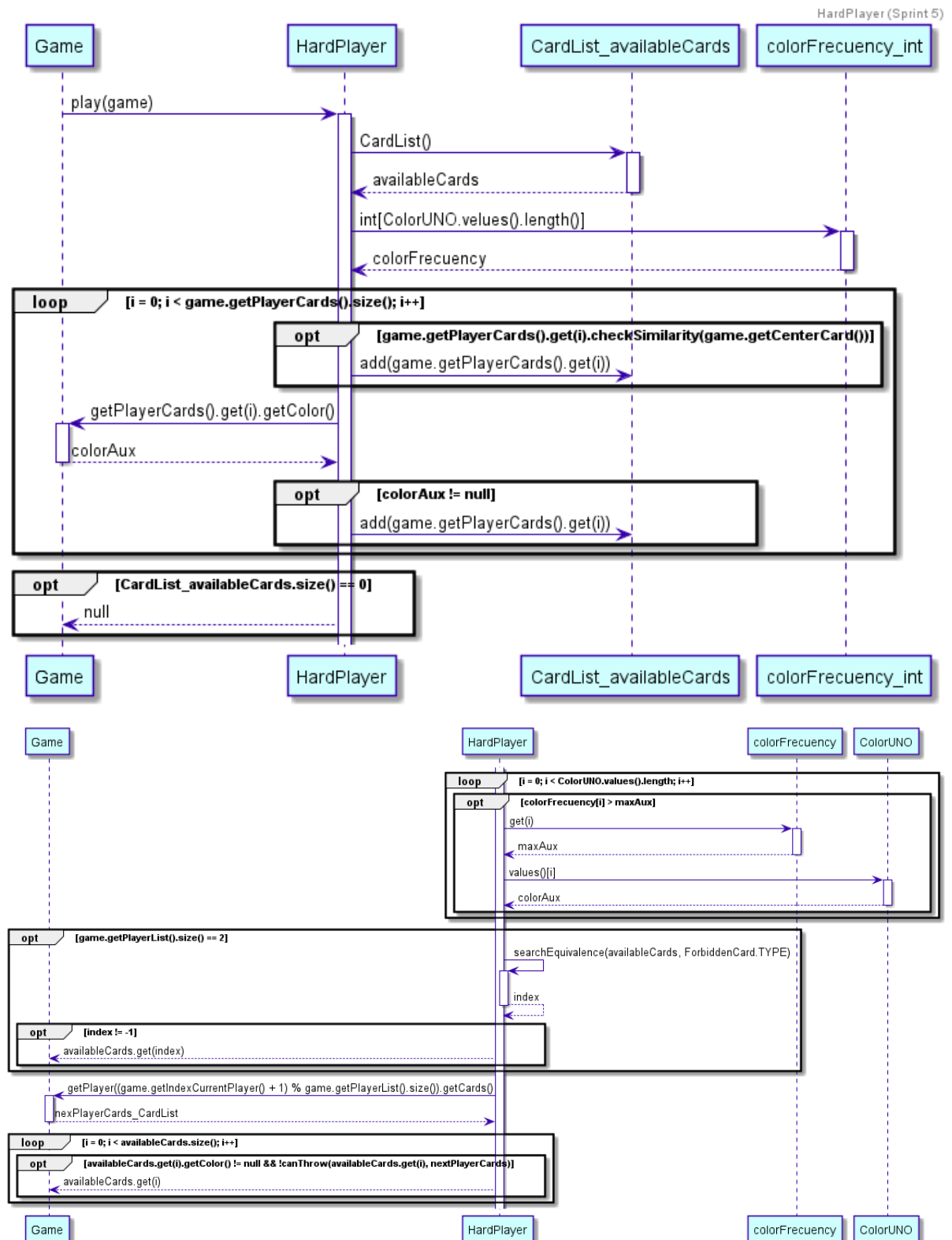
MediumPlayer play (Sprint 5)

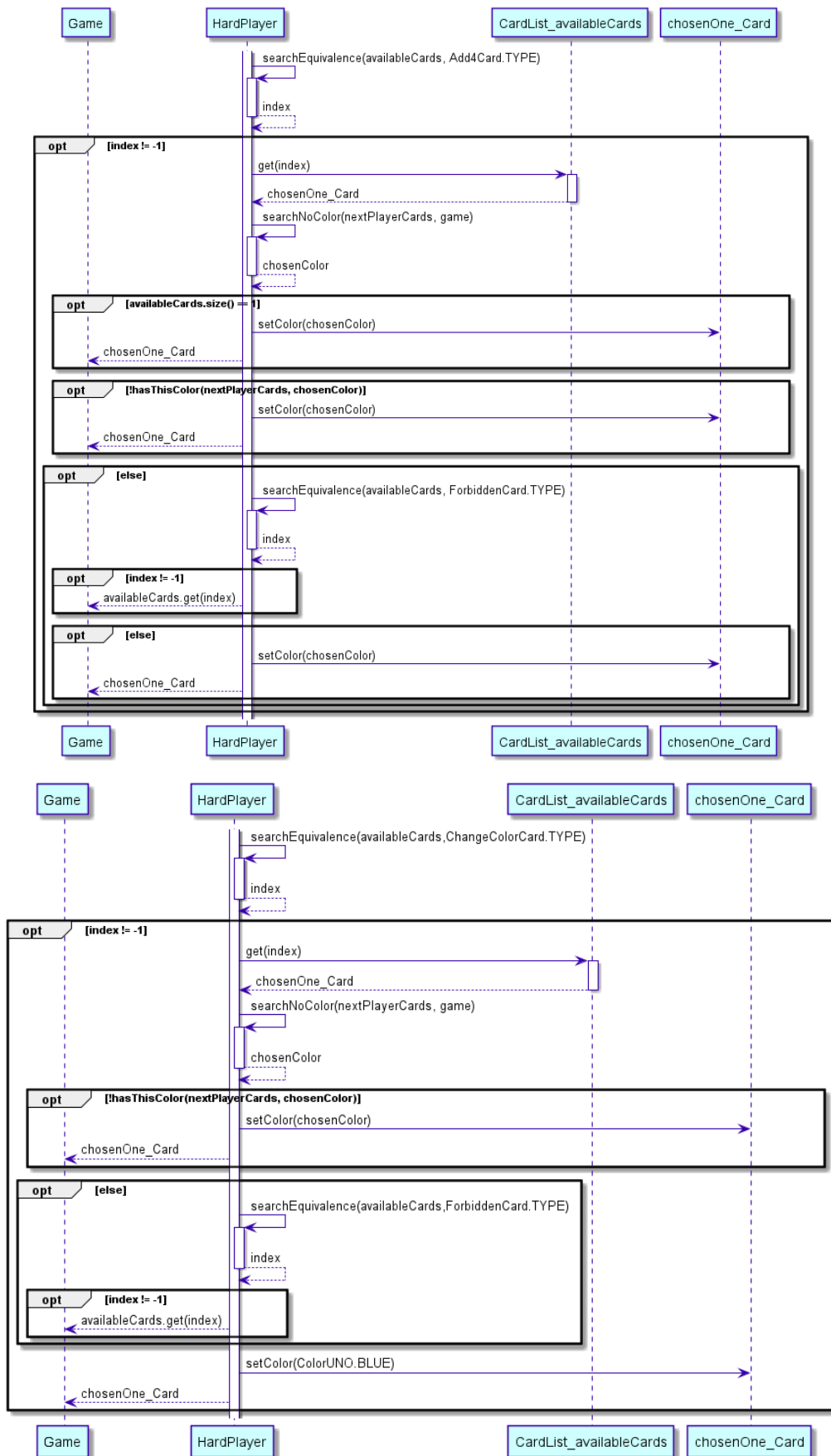


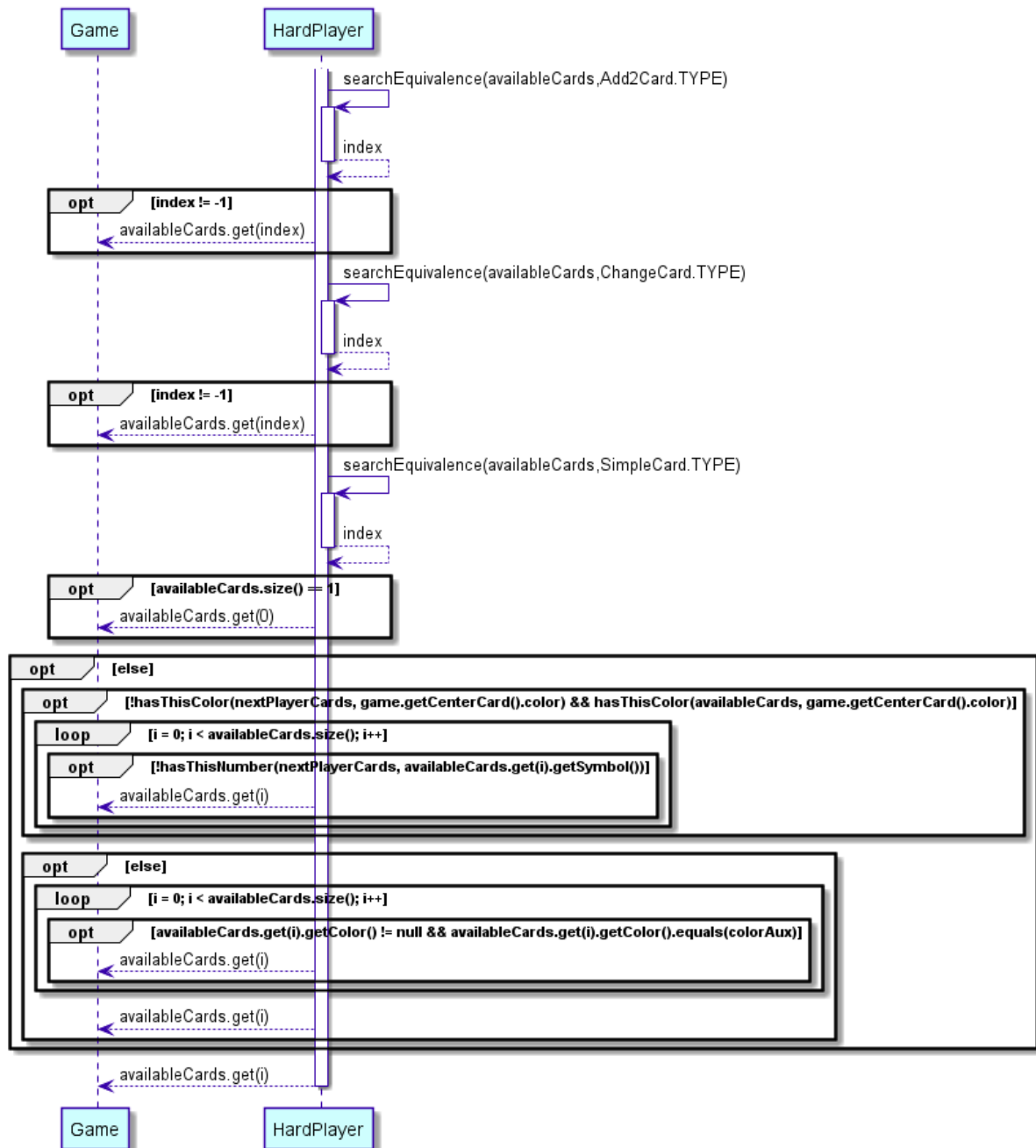
MediumPlayer play (Sprint 5)



HardPlayer







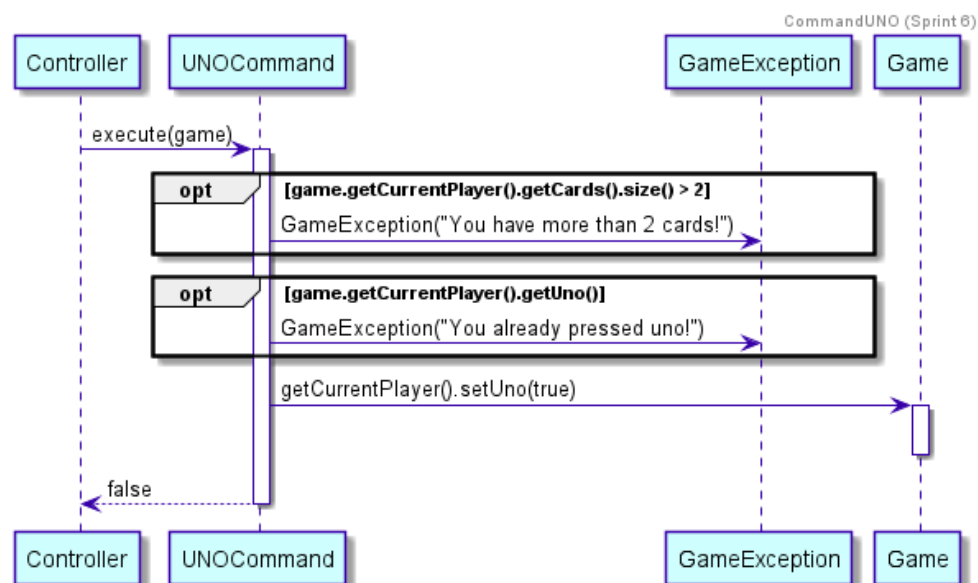
Sprint 6

Añadimos toda la lógica del botón UNO a las estrategias. Tras meditarlo hemos decidido que la mejor manera de implementar todo su funcionamiento, sin que afecte al trabajo ya realizado, es que siga el siguiente comportamiento:

Cuando un jugador tiene 2 cartas, deberá darle al botón antes de lanzar alguna de las dos, si lo hace el juego seguirá su curso, pero en cambio, si se olvida de pulsarlo tendrá que robar una carta como castigo, la cual aparecerá en su mano en el siguiente turno. En lo que incumbe a las estrategias, hemos decidido implementarlo mediante una serie de probabilidades elegidas específicamente para el nivel de juego:

	Probabilidad de pulsar	Probabilidad de olvidarse
<i>Easy</i>	0.2	0.8
<i>Medium</i>	0.7	0.3
<i>Hard</i>	1	0

De esta manera, en el modo *Easy*, es muy probable que el bot se olvide de pulsar el botón, por lo que difícilmente podría ganar alguna partida, ya que a la hora de estar cerca de ganar, tendría que volver a robar. Por el contrario en la estrategia *Hard* nunca robará por lo que será más fácil ganar para este bot. El modo *Medium* se encuentra en un término medio pero dando más probabilidades a que sí lo pulse para así representar lo que podría pasar con una persona humana.

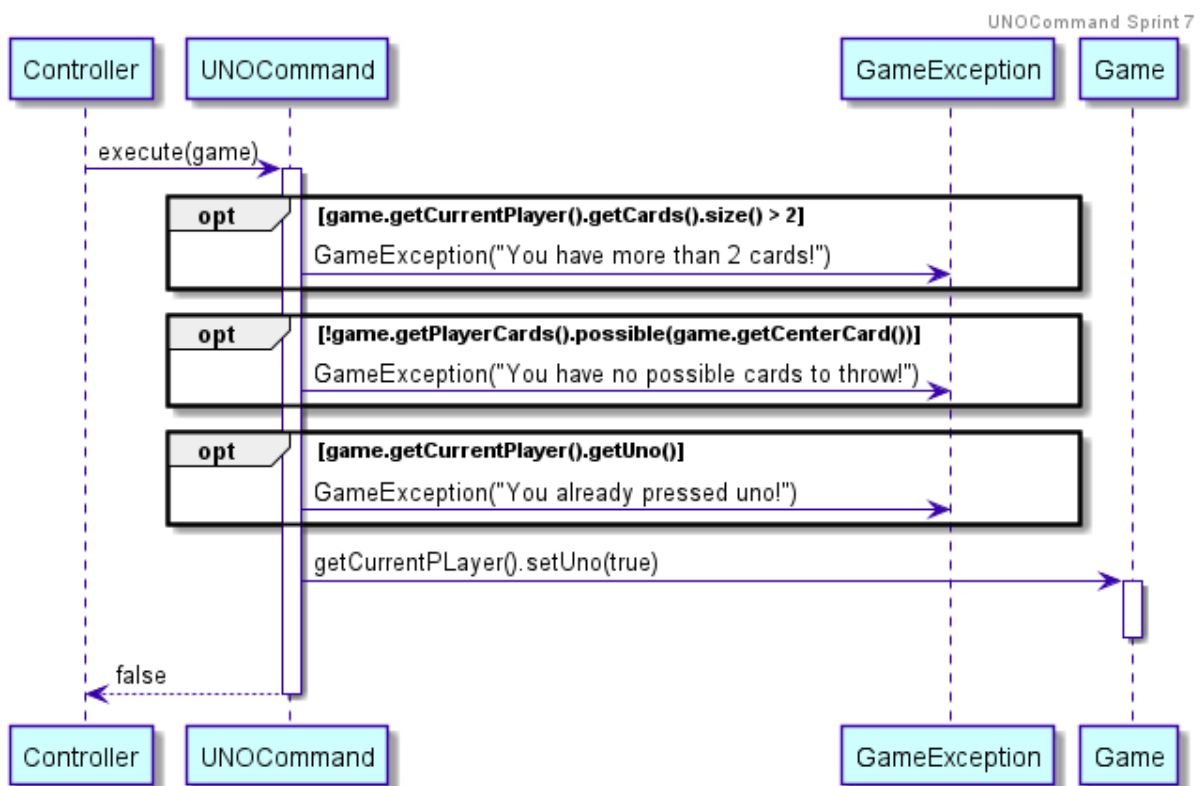


Sprint 7

Hemos añadido la comprobación de que el usuario pueda tirar una de las dos cartas de las que dispone, en cuyo caso, debería pulsar el botón del UNO dependiendo de su nivel de dificultad.

En los jugadores de inteligencia artificial, hemos arreglado el pulsar la funcionalidad de pulsar el botón del UNO, tras darnos cuenta de que en el sprint anterior no estaba bien implementado y no funcionaba correctamente.

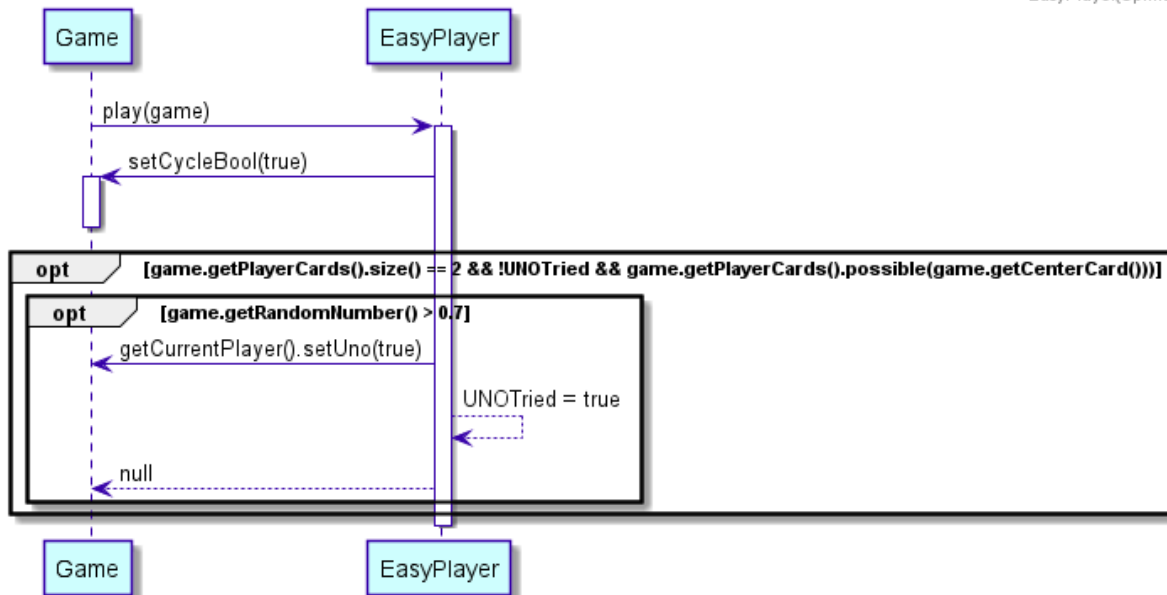
Además, tras las modificaciones en el método update de Game, decidimos que la carta seleccionada por el bot, se lance desde el método play() directamente.



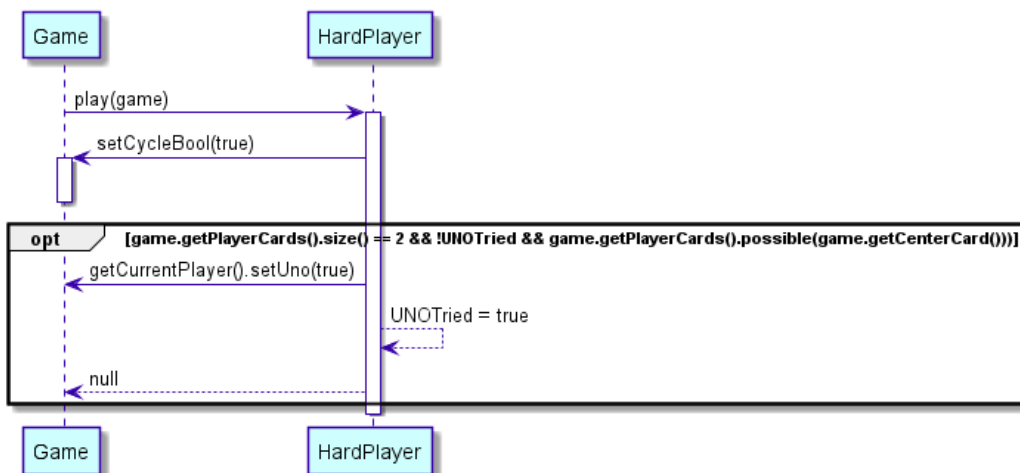
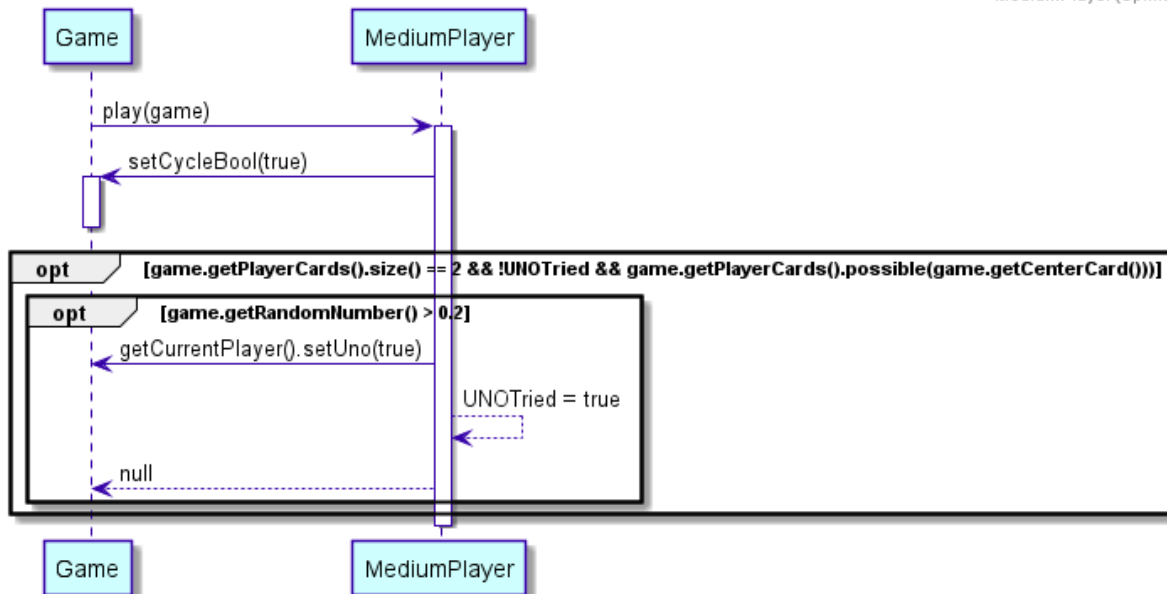
Como se puede observar se ha añadido el siguiente código al comienzo de cada método `play()` para poder conseguir lo mencionado previamente.

A continuación mostramos diagramas de secuencia de cómo se lleva a cabo la lógica de pulsar el botón del UNO.

EasyPlayer(Sprint 7)

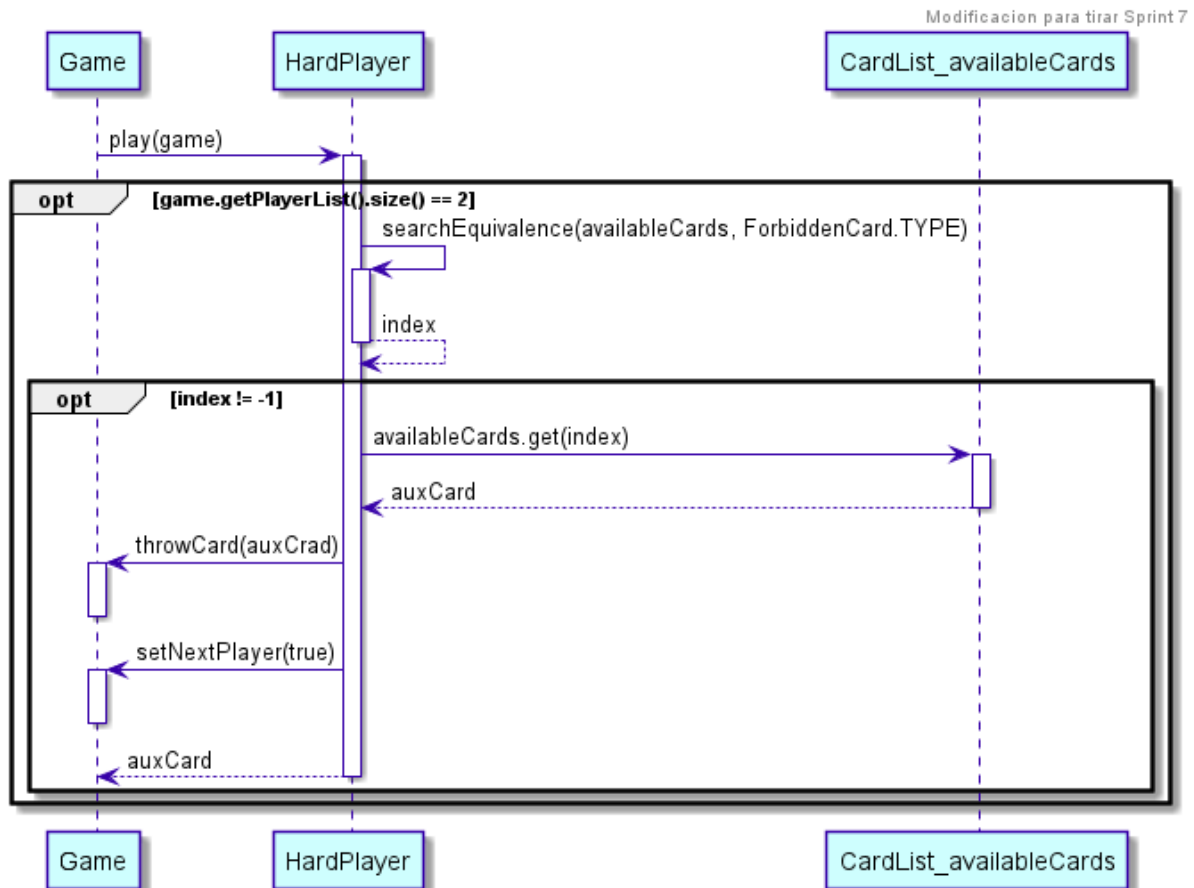


MediumPlayer (Sprint 7)

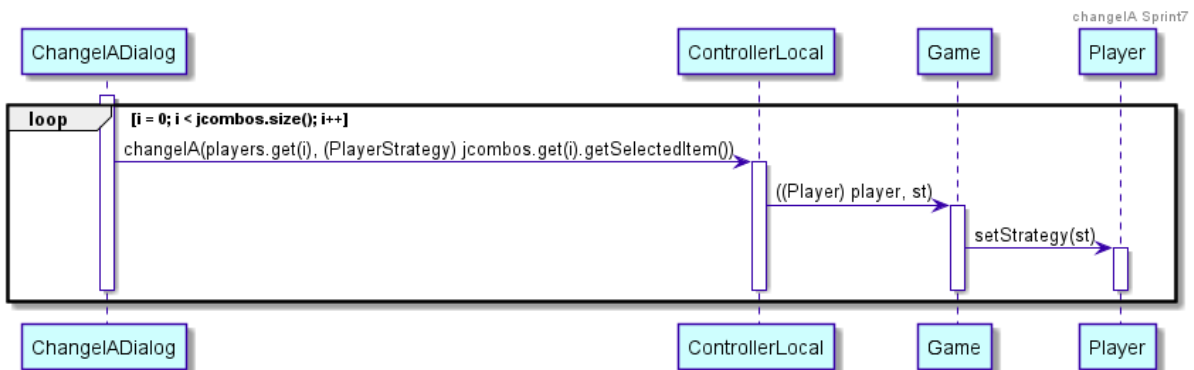


Para ejemplificar la funcionalidad de que ahora la carta seleccionada por el bot se lanza en el método *play()*, usamos el método del *HardPlayer*, análogo a las otras estrategias.

Antes de cada return del método *play()* visto en el Sprint 5 se hará el método *throwCard* y *setNextPlayer*.



Además cambiamos el modo de establecer las estrategias, siendo solo posible si el usuario no está jugando en red.



10. Como usuario quiero poder jugar una partida en red.

Una de las opciones que se le presenta al usuario al abrir el juego, es jugar una partida en red, permitiendo que varias personas jueguen online.

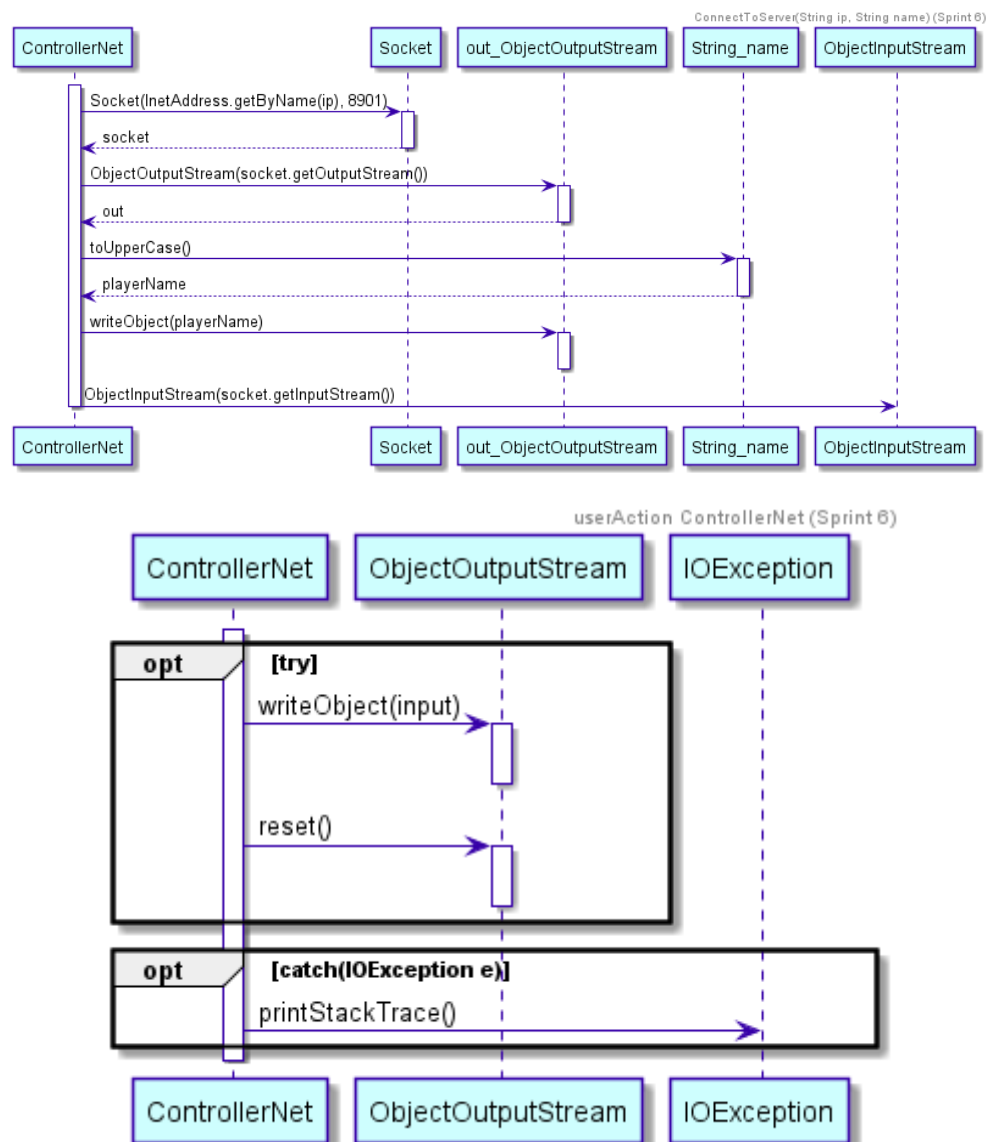
- Estimación: Se estima que en realizar esta tarea se emplearán 2 semanas.
- Priorización: Debería tenerlo.
- Criterios de aceptación: Se podrá jugar contra otros usuarios sin estar juntos, mediante conexión a la red local.

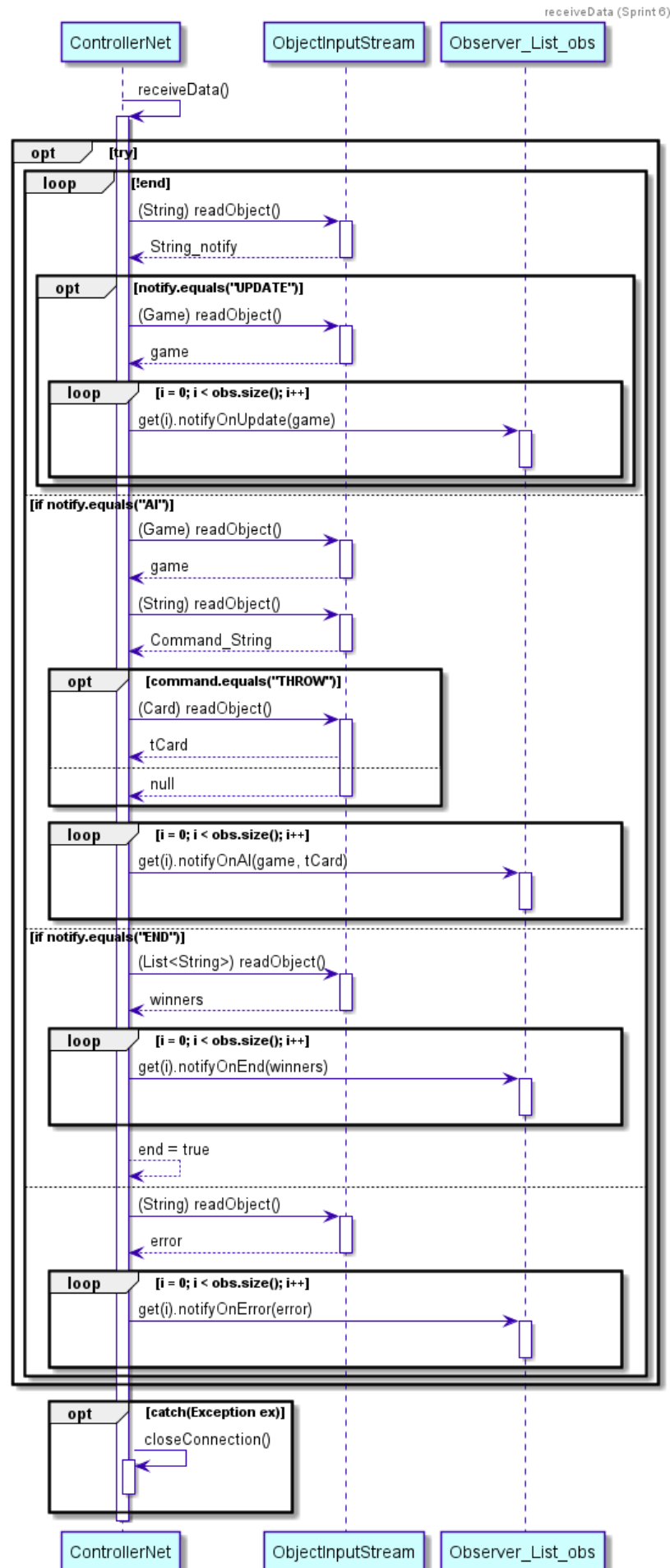
Sprint 6

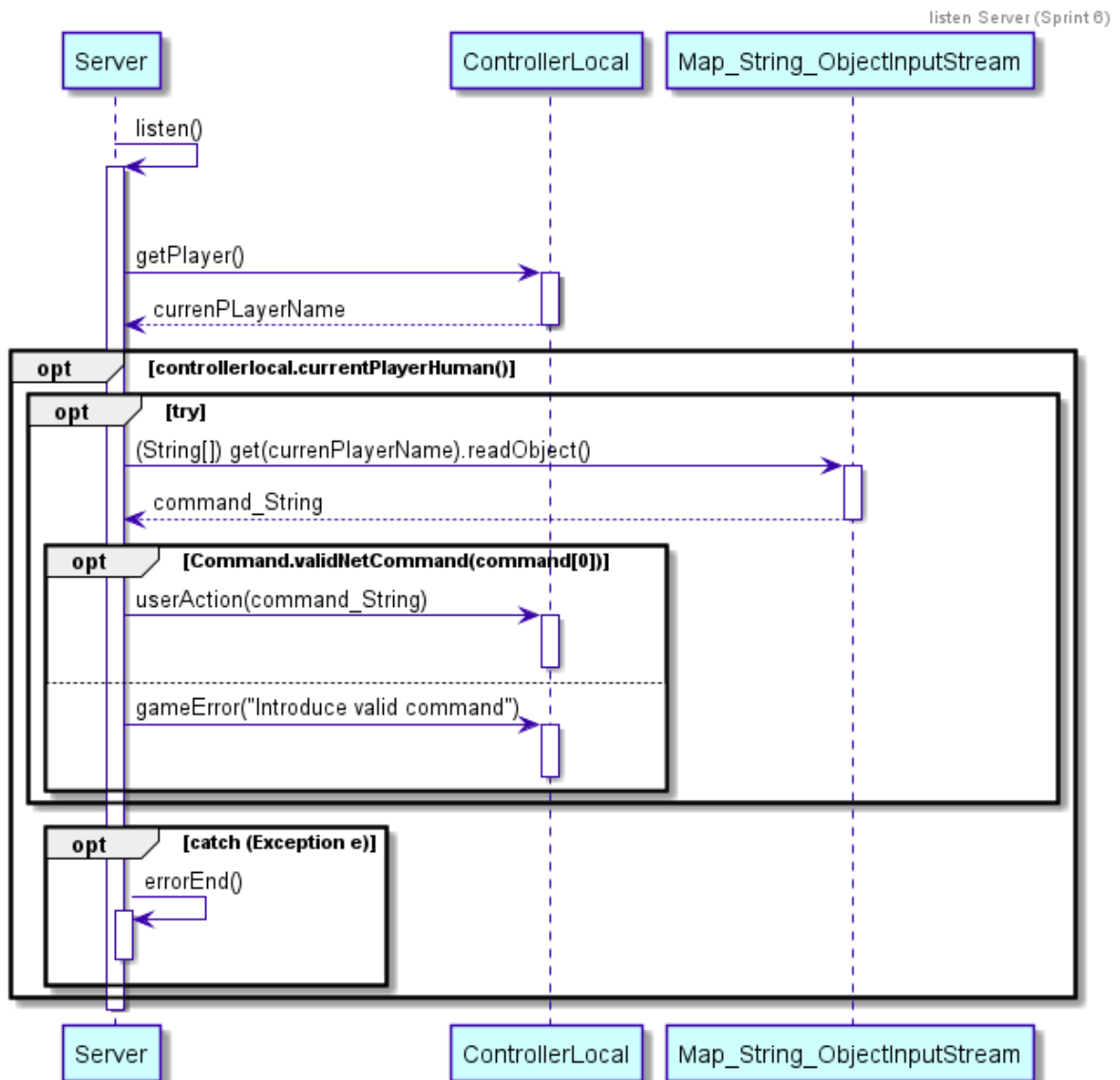
Esta historia de usuario es creada e implementada ante la demanda de esta funcionalidad.

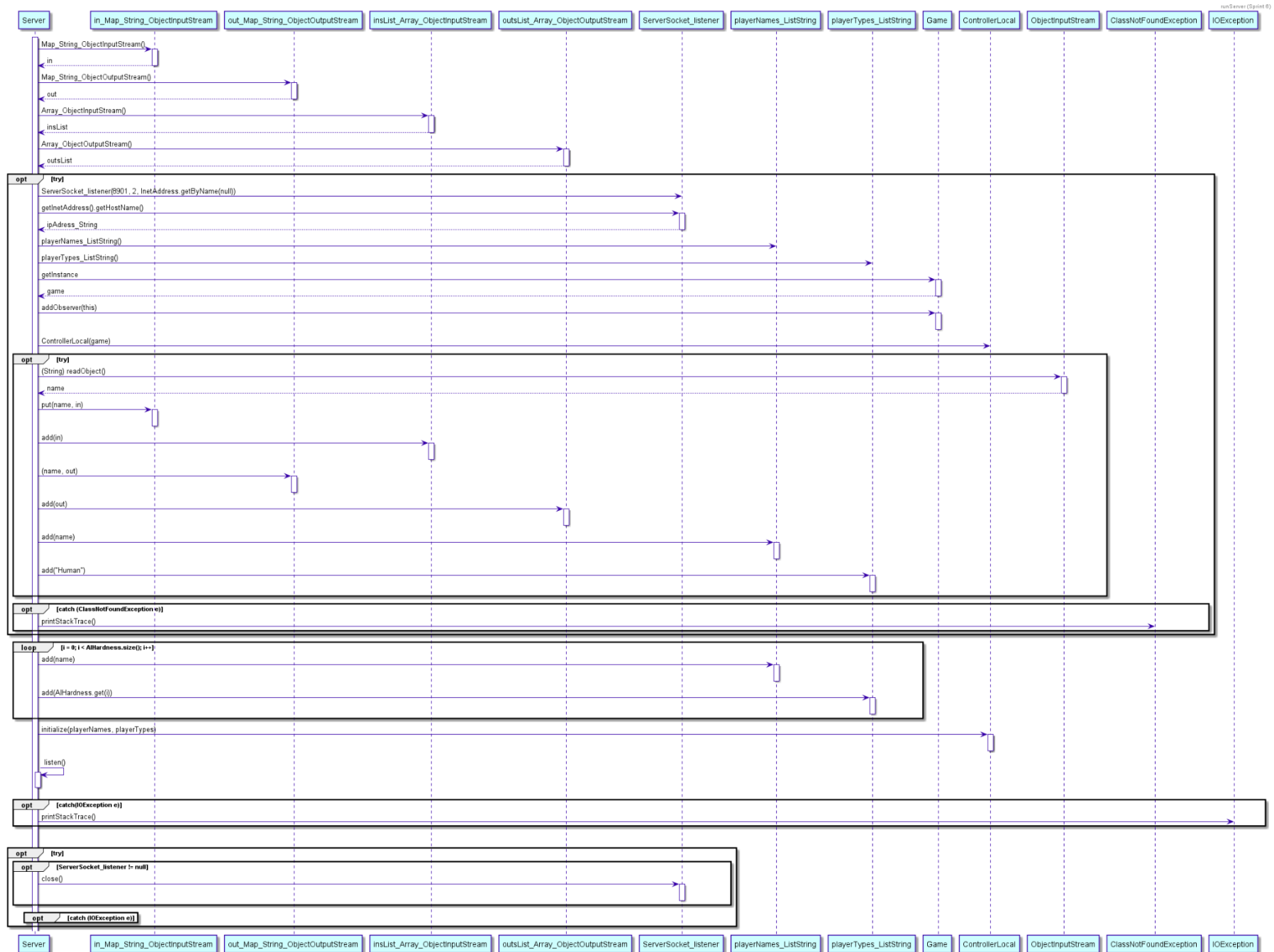
El funcionamiento que siguió la conexión en red ya ha sido definido, pero de manera resumida se aplicó el patrón cliente-servidor aprovechando las características de los sockets de java.

La comunicación entre el cliente y el servidor es bidireccional, el cliente recibe del servidor los cambios que se van aplicando sobre el juego mientras que en el otro sentido los clientes indican al nodo compartido cuál es la acción que ha llevado a cabo un usuario. Las transferencias de datos han sido realizadas a través de objetos serializables para facilitar la codificación.

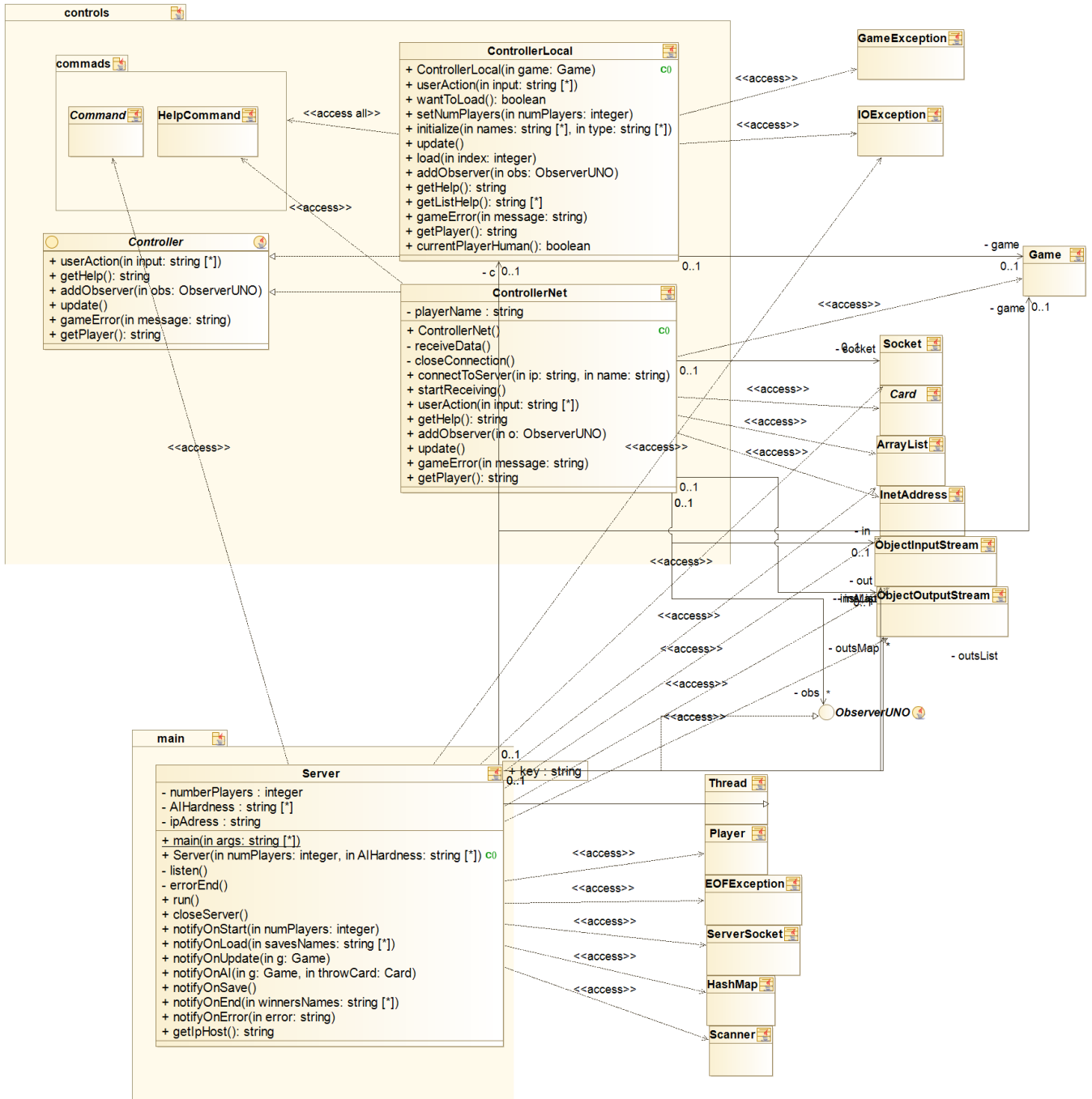








Diseño y evolución de las historias de usuario



Sprint 7

En este sprint implementamos que en el modo de juego en red, los jugadores no puedan tener los mismos nombres, como en el modo en consola y en el modo local. Esta idea solo se trata de una simple comprobación.

Además, en vez de utilizar el método *getInstance()* en el *Server*, decidimos usar el *reset()* para asegurarnos de que funcionará correctamente si una vez terminada una partida online, se vuelve a iniciar otra online.

Por otro lado, solucionamos un error que ocurría en el *notifyOnError* que se deba en el caso de que se intentase notificar a una IA puesto que estas no tienen clientes esto era imposible y causaba que el juego terminase abruptamente.

Por último, hicimos que el servidor se cerrase correctamente en el caso de que algún cliente abandonase de repente la partida.

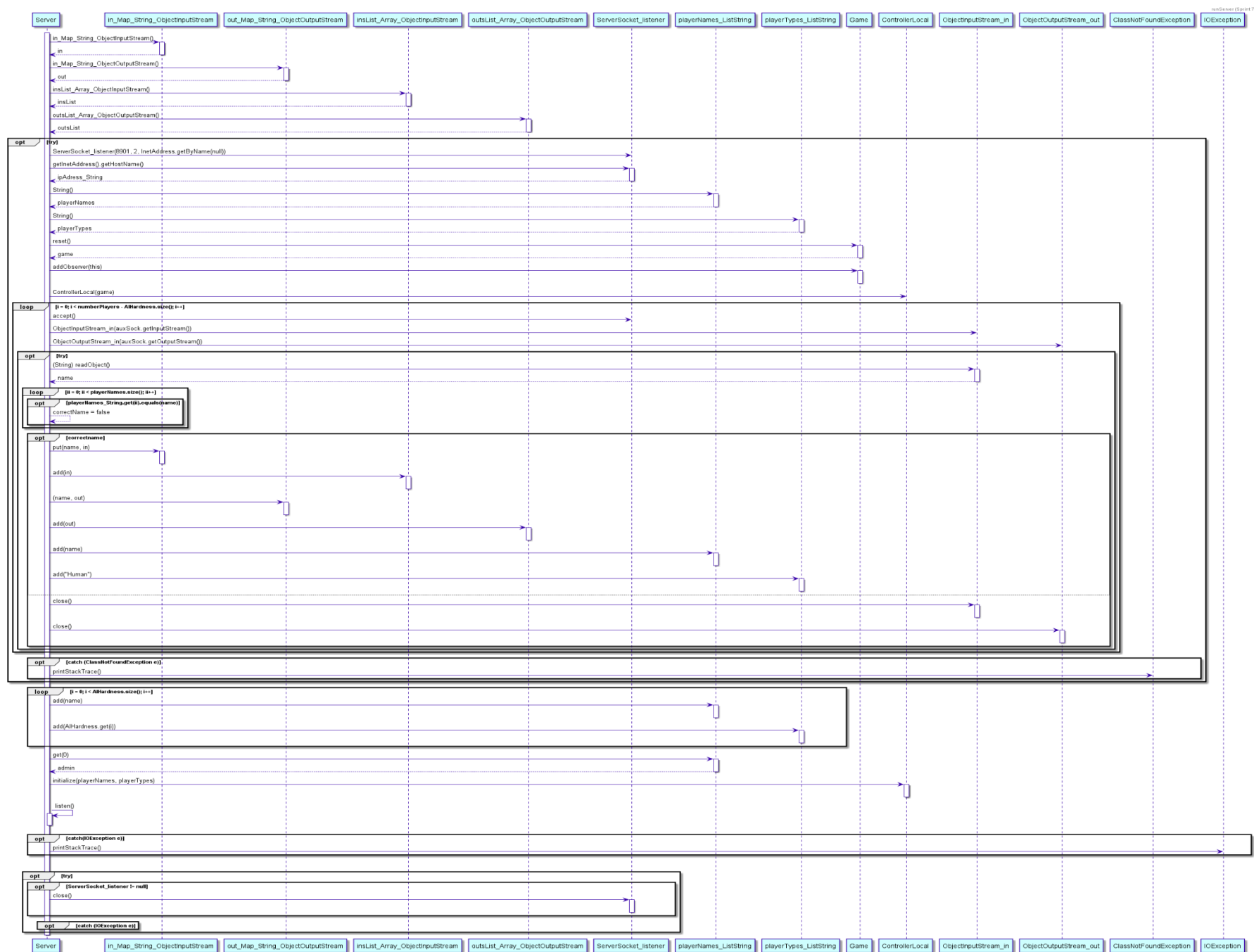


Diagrama de componentes

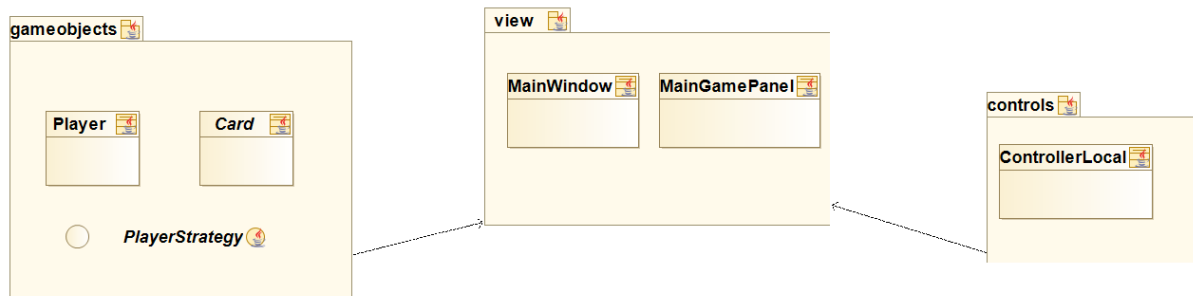


Diagrama de componentes Partida Local

Mostramos un diagrama de componentes a alto nivel reflejando las conexiones entre las principales componentes del diseño de una partida en local. *Gameobjects* que contiene la lógica y objetos del juego, *view* para la representación, y *Controller*, que controla el juego.

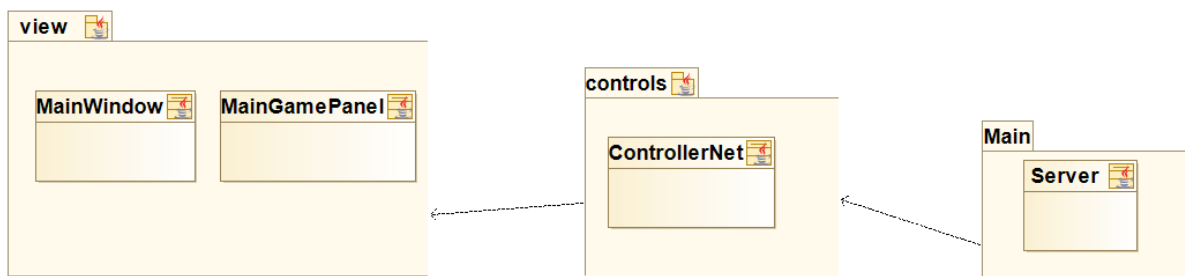


Diagrama de componentes Partida Online

Se trata de otro diagrama de componentes a alto nivel, pero este refleja los principales componentes del diseño del juego en red. La vista se mantiene igual, pero ahora el controlador del juego es *ControllerNet*, y el main de la aplicación es *Server*.

Patrones de diseño utilizados en el proyecto

Patrón MVC

Este patrón separa los datos y la lógica de su representación y el módulo encargado de gestionar todas las acciones a realizar, así como las comunicaciones.

Aplicación del patrón MVC en nuestro proyecto

MODELO:

El modelo de nuestro programa se encuentra contenido en el paquete **logic** y en sus sucesivos subpaquetes que son:

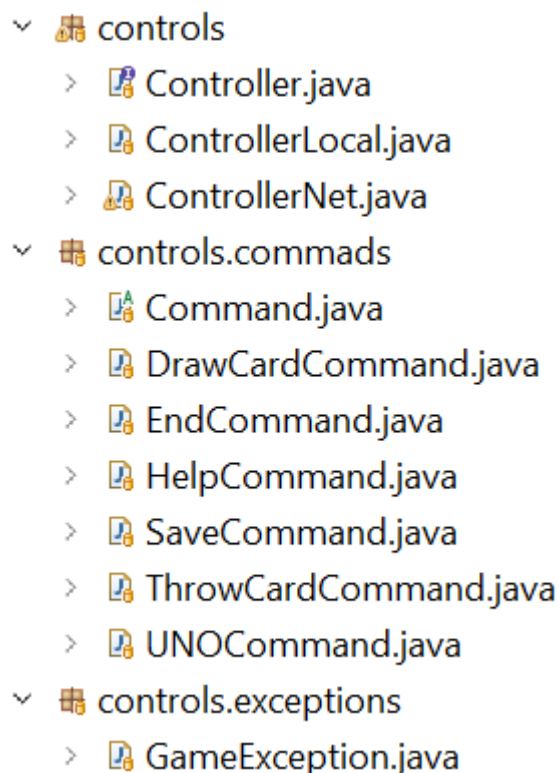
- **logic:** este es el paquete raíz en él se encuentran la clase game, encargada de mantener el estado y de la realización de los cambios en el juego propiamente dicho, así como las interfaces *ObservableUNO* y *ObserverUNO* para la implementación del ya nombrado patrón observador.
- **logic.factories:** en este subpaquete se encuentran las factorías de las cartas y de las estrategias de los distintos tipos de jugadores. Estas factorías se basan en el patrón factoría abstracta.
- **logic.gameobjects:** aquí se encuentran las clases que representan a los distintos objetos del juego (modelo), es decir, las cartas, jugador, estrategias de jugadores y las listas de cartas y jugadores.

▼  logic >  Game.java >  GameStatus.java >  ObservableUNO.java >  ObserverUNO.java ▼  logic.factories >  Add2CardBuilder.java >  Add4CardBuilder.java >  Builder.java >  BuilderBasedFactory.java >  CardBuilder.java >  ChangeCardBuilder.java >  ChangeColorCardBuilder.java >  EasyPlayerBuilder.java >  Factory.java >  ForbiddenCardBuilder.java >  HardPlayerBuilder.java >  HumanPlayerBuilder.java >  MediumPlayerBuilder.java >  PlayerBuilder.java >  SimpleCardBuilder.java	▼  logic.gameobjects >  Add2Card.java >  Add4Card.java >  Card.java >  CardList.java >  CardStatus.java >  ChangeCard.java >  ChangeColorCard.java >  ColorUNO.java >  EasyPlayer.java >  ForbiddenCard.java >  HardPlayer.java >  HumanPlayer.java >  MediumPlayer.java >  Player.java >  PlayerList.java >  PlayerStatus.java >  PlayerStrategy.java >  SimpleCard.java >  SpecialCard.java
--	---

CONTROLADOR:

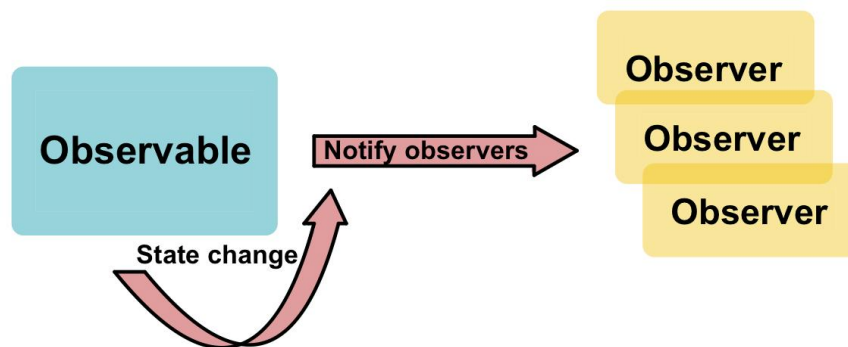
El controlador de nuestro programa se encuentra contenido en el paquete **controls** y en sus sucesivos subpaquetes, que son:

- **controls:** este es el paquete raíz en él se encuentran la interfaz **Controller** y sus implementaciones *ControllerLocal* y *ControllerNet*. Su función consiste en tomar un input y transformarlo en una acción en el modelo. En el caso del local el cambio se transmite directamente al modelo mientras que el Net se encarga de recibir y enviar información con el servidor.
- **controls.commands:** este subpaquete se utiliza para la implementación del patrón comando. Con una clase abstracta comando añadimos una serie de implementaciones de este para así poder realizar cambios sobre el modelo de manera genérica con una entrada que mantiene una estructura similar
- **controls.exceptions:** aquí únicamente nos encontramos con una clase excepción que se utiliza para diferenciar las excepciones que ocurren debido a fallos del juego y no de otro tipo.



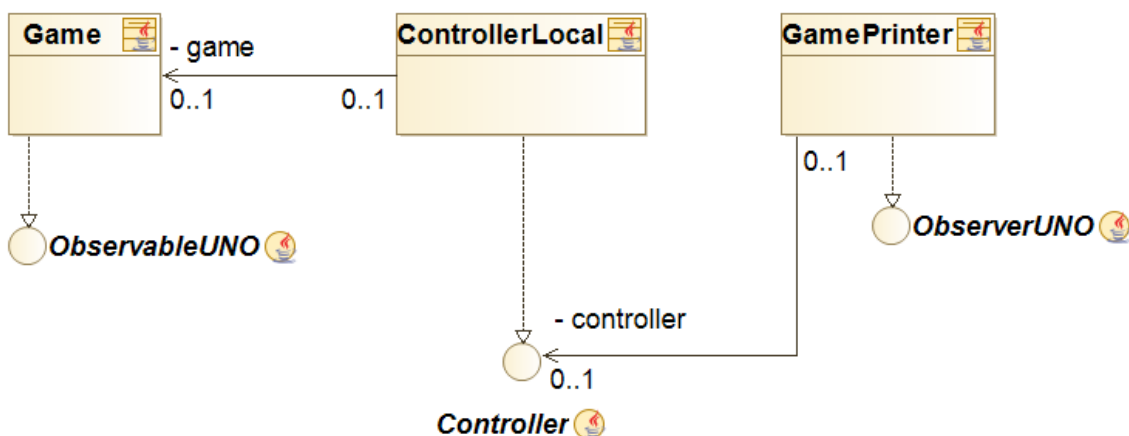
Patrón Observer

El patrón de diseño observador se utiliza para problemas en los que se requiere tener clases que sufren cambios (*Observable*), y clases que necesitan conocer todos esos cambios a tiempo real (*Observers*) de manera abstracta y genérica. Así el primero de estos tiene una lista de los segundos y notifica a todos en cada cambio con un *update()*.



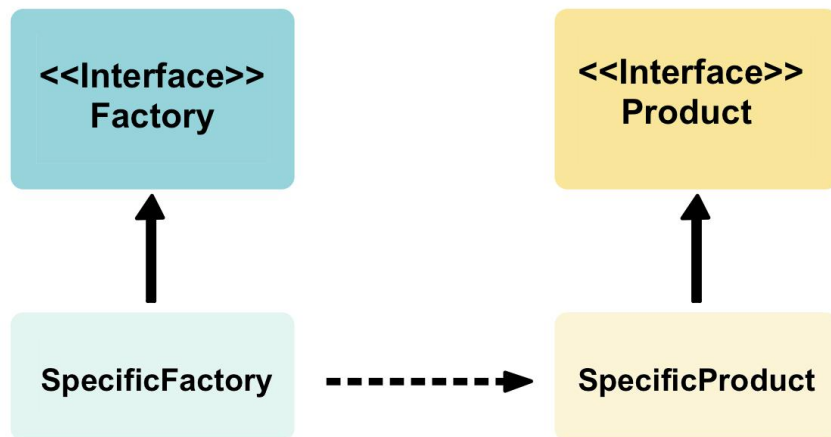
Aplicación del patrón Observador en nuestro proyecto

Aplicamos este patrón entre Modelo y Vista, como ya hemos explicado en el patrón Modelo-Vista-Controlador. En nuestro caso utilizamos los nombres *ObservableUNO* y *ObserverUNO*.



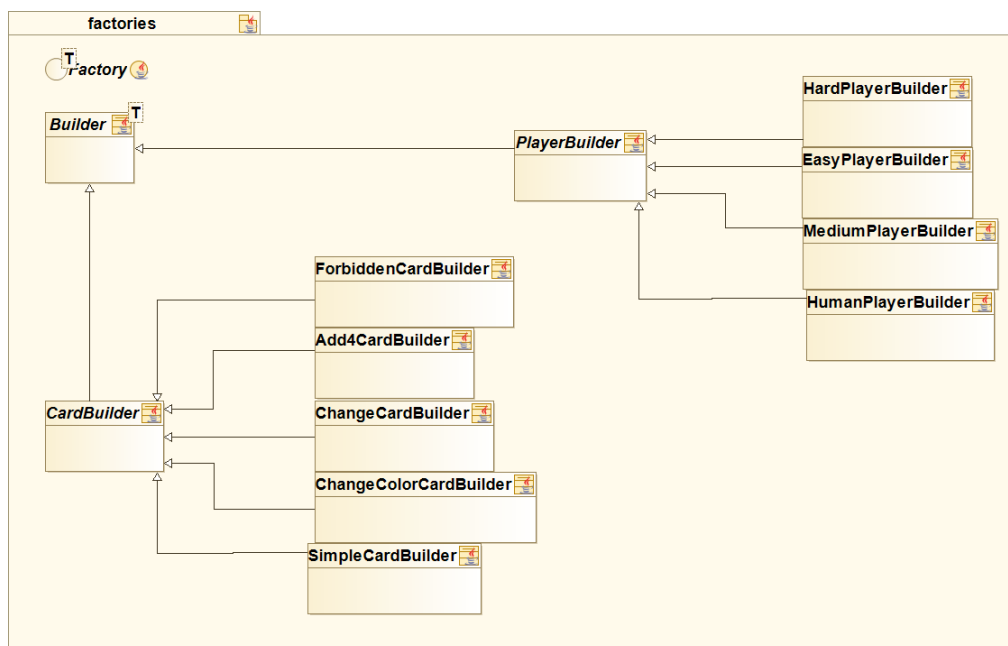
Patrón Factory Method

Se trata de un patrón de creación que utiliza clases Factory para la creación de objetos, cuando implican más que una simple instanciación. Se define una interfaz para crear un objeto y así liberar al objeto creado de responsabilidades que no le corresponden.



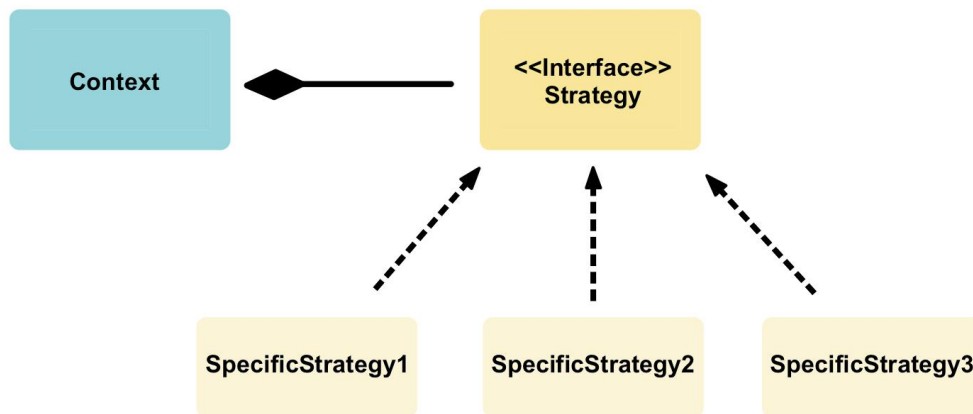
Aplicación del patrón Factory Method en nuestro proyecto

Aplicamos este Patrón en la creación de todos los objetos del juego, tanto las cartas como las distintas estrategias de los jugadores.



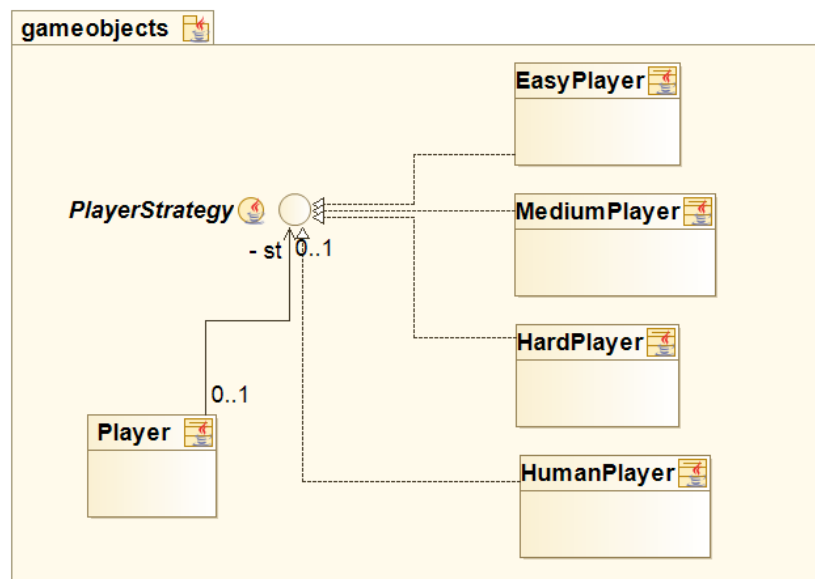
Patrón Strategy

El patrón Strategy permite la creación de diferentes algoritmos que realizan la tarea de darle un comportamiento “humano” a la máquina en diferentes rangos que podrán ser intercambiados según los deseos y necesidades del cliente.



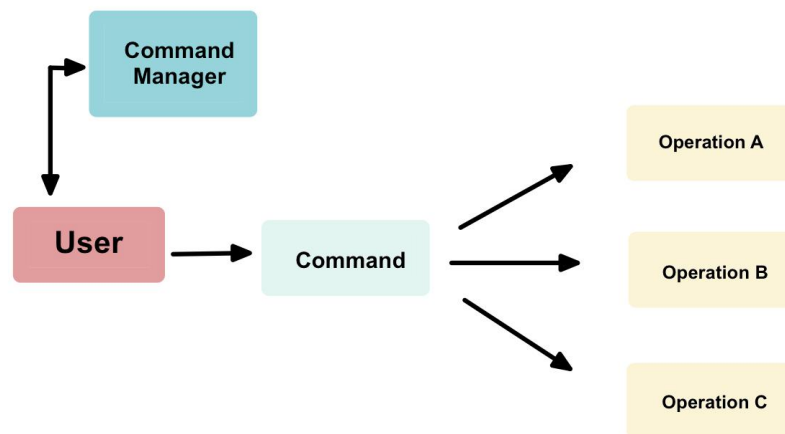
Aplicación del patrón Strategy en nuestro proyecto

En nuestro caso el **Contexto** son los diferentes tipos de jugadores creados como **EasyPlayer**, **MediumPlayer**, **HardPlayer** y **HumanPlayer**, cada uno de los cuales contiene una estrategia diferente que se basan en aumentar o disminuir el nivel de dificultad (excepto **HumanPlayer** que simplemente realizará los movimientos que quiera el usuario) pero todos ellos realizando una tarea similar.



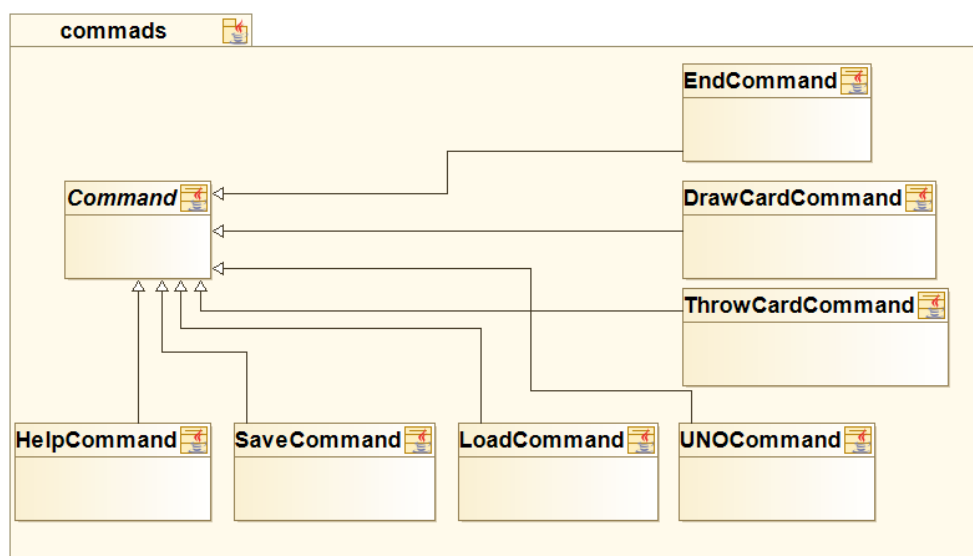
Patrón Command

Encapsula las peticiones del usuario en objetos, permitiendo así parametrizar a los clientes con diferentes peticiones, hacer cola o llevar un registro de todas las peticiones que se han recibido, y así poder deshacer las operaciones.



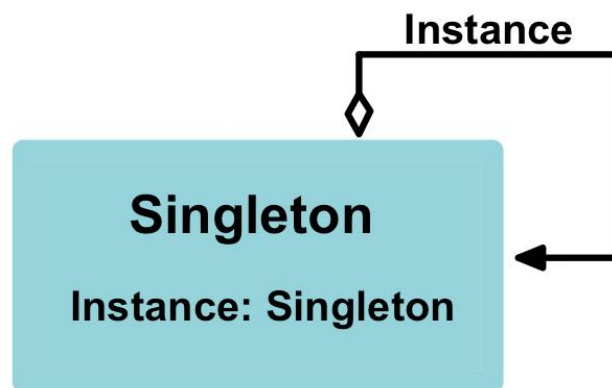
Aplicación del patrón Command en nuestro proyecto

Todas las acciones realizadas durante el proceso de la partida están recogidas en los comandos, ya sea tirar una carta (*ThrowCardCommand*), guardar una partida (*SaveCommand*), etc. La clase *Command* los recoge a todos ellos y las acciones deseadas se ejecutan en game.



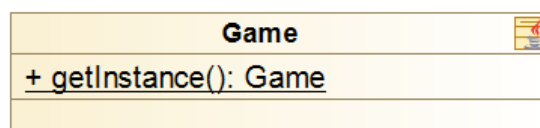
Patrón Singleton

El patrón Singleton se encarga de garantizar que una clase sólo tenga una instancia, y proporciona un punto de acceso global a ella.



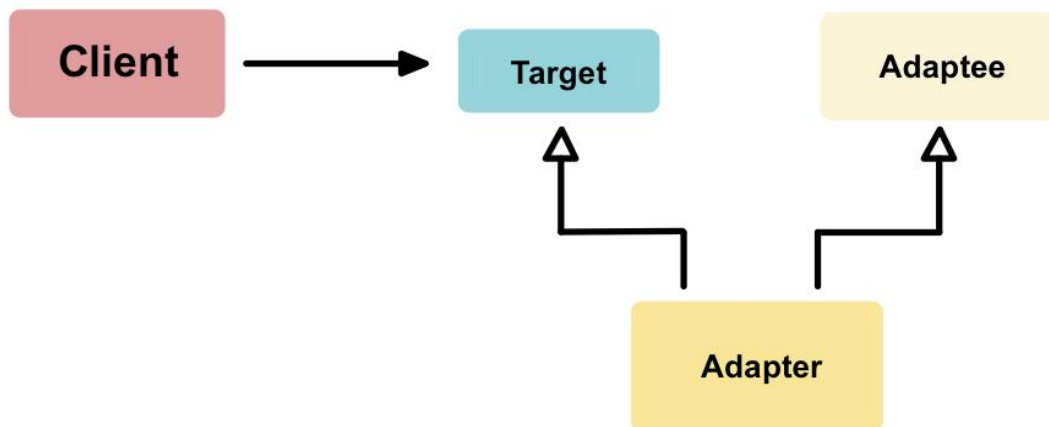
Aplicación del patrón Singleton en nuestro proyecto

En nuestro proyecto utilizamos este patrón para asegurarnos de que únicamente haya una instancia de *Game*, por lo que solo se creará un *game* nuevo si este resulta ser *null*.



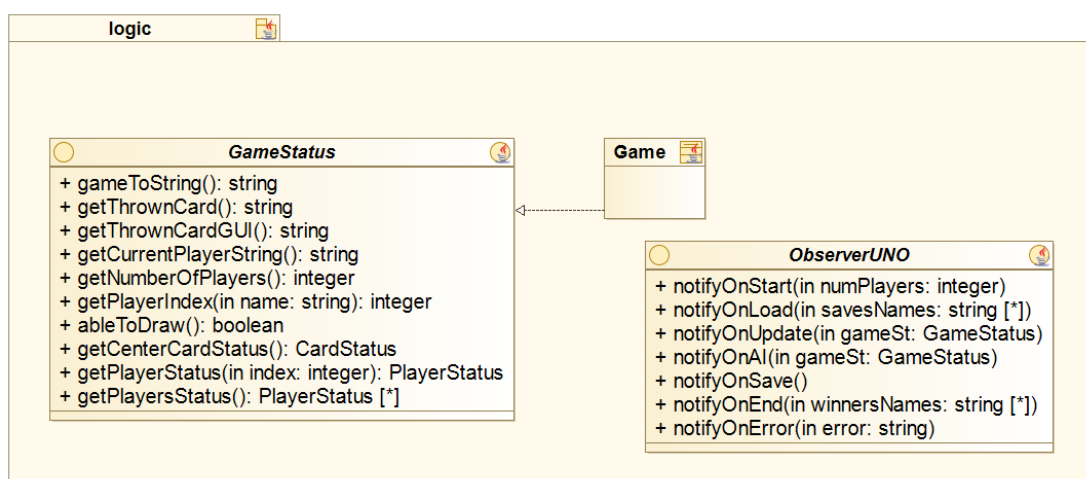
Patrón Adapter

Este patrón es utilizado para convertir la interfaz de una clase en otra interfaz que es la que esperan los clientes. Permite que cooperen clases que de otra manera no tendrían interfaces compatibles.

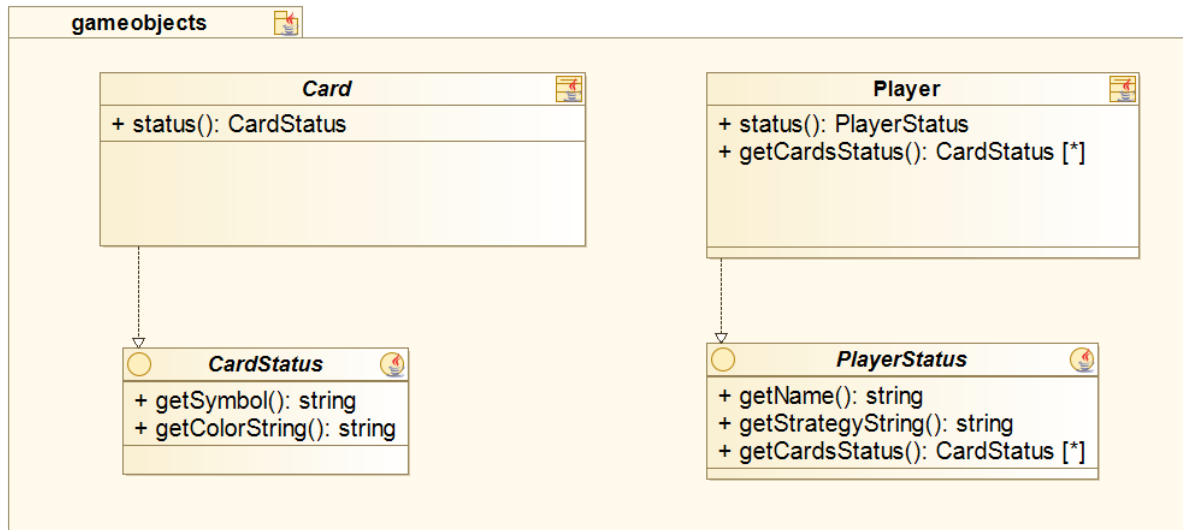


Aplicación del patrón Adapter en nuestro proyecto

Creamos la interfaz GameStatus para que sea la encargada de realizar las llamadas a todos los métodos expuestos en la imagen siguiente, de esta manera, Game implementará GameStatus y al realizar las notificaciones de cambios en los observadores, se utilizará un GameStatus que contiene únicamente lo que se necesita en vez de pasarle todo el Game.

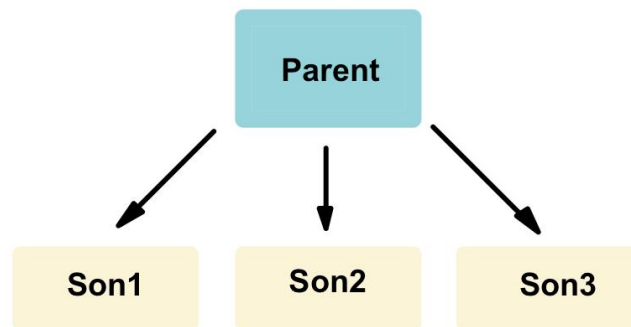


De igual manera en el proyecto utilizamos el patrón *adapter* para conocer el estado de las cartas y los jugadores.



Patrón Polimorfismo

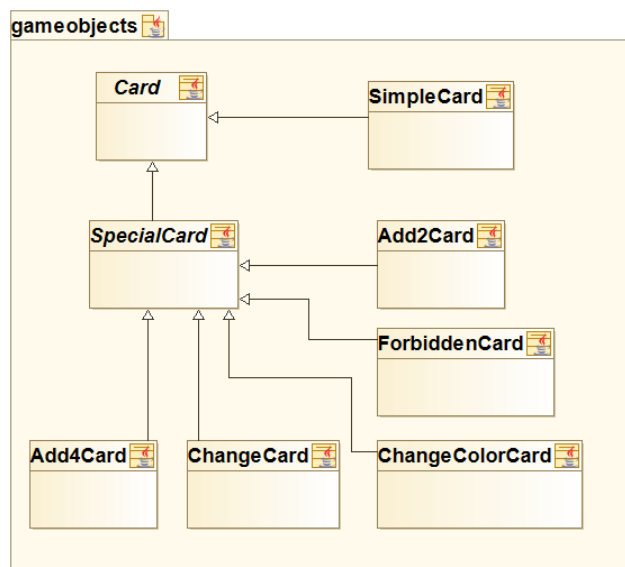
Para poder manejar las alternativas basadas en el tipo se utiliza el patrón polimorfismo, explicado de una manera básica, el polimorfismo son todas las herencias que se utilizan cuando varias clases van a tener una lógica muy similar.



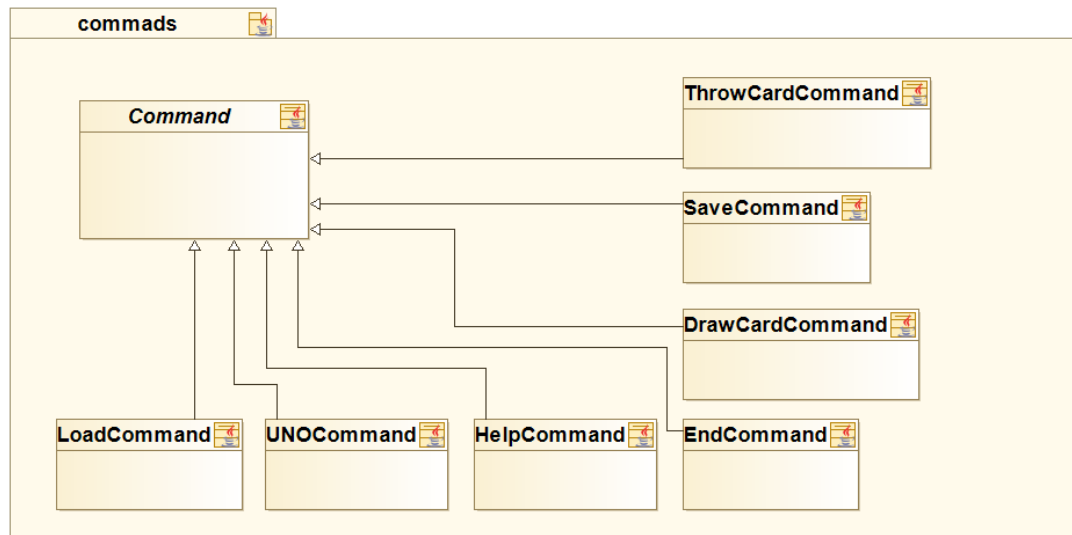
Aplicación del patrón Polimorfismo en nuestro proyecto

Durante todo el proyecto utilizamos este patrón en varias ocasiones, aquí mostramos dos clases en las que se puede apreciar a la perfección la intencionalidad del patrón polimorfismo:

- *Card*: todas las cartas parten de un clase inicial carta, luego estas se diferencian entre las cartas normales y todas las demás que tienen una funcionalidad especial, sin embargo todas ellas se comportan de forma similar ante las diferentes acciones que puedan realizar los jugadores.



- *Command*: todos los comandos utilizados para implementar las acciones que realicen los jugadores, todos ellos mantienen en común las ideas principales, como pueden ser sus atributos (name, description y shortcut), así como diferentes funciones, por ejemplo para comprobar la validez de los argumentos recibidos.



Patrón Bajo Acoplamiento

El patrón Bajo Acoplamiento es un principio a tener en mente en todas las decisiones de diseño. El patrón impulsa la asignación de responsabilidades de manera que su localización no incremente el acoplamiento hasta un nivel que lleve a los resultados negativos que puede producir un acoplamiento alto. No puede considerarse en forma independiente de otros patrones como Experto o Alta Cohesión, sino que más bien ha de incluirse como uno de los principios del diseño que influyen en la decisión de asignar responsabilidades.

Patrón de Alta Cohesión

El patrón Alta Cohesión nos indica que los datos y responsabilidades de una entidad están fuertemente ligados a la misma, en un sentido lógico. En otras palabras, la información que maneja una entidad software debe estar fuertemente ligada a lo que la entidad software representa lógicamente.

Sistema resultante

